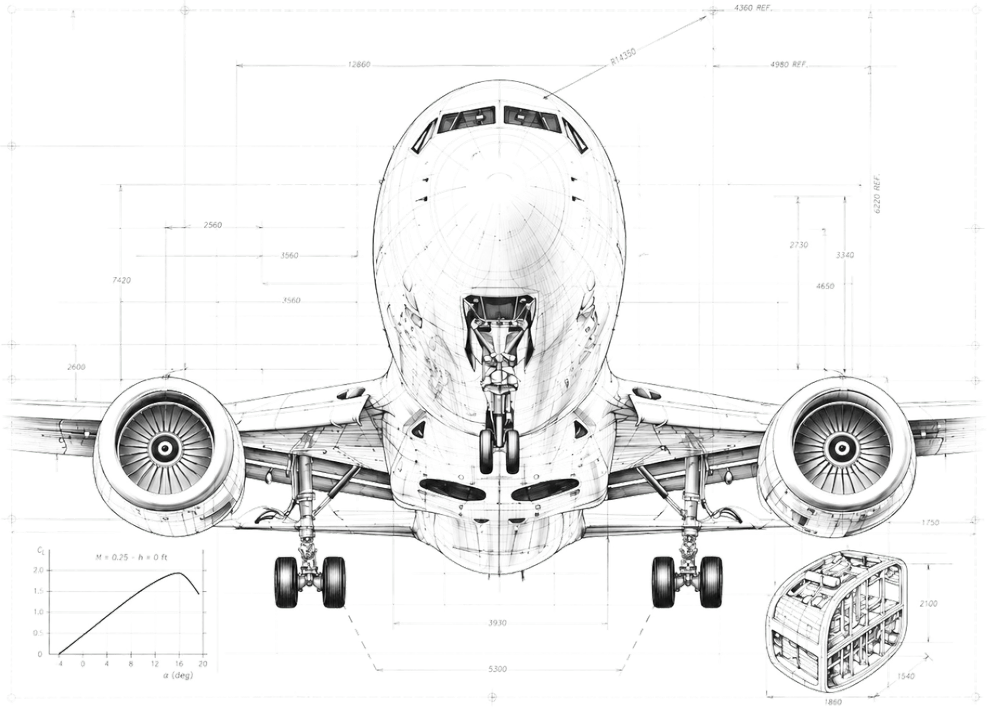
Costruzioni Aeronautiche

ITIS “E. Majorana” — Fluidodinamica numerica

Flussi non viscosi

Dal potenziale di velocità alla formazione dell’urto: problemi ai limiti, equazioni ellittiche e iperboliche



Monografia didattica di fluidodinamica numerica
Modulo 2: flussi non viscosi

Prof. Ing. Biagio Raucci

Esempi numerici in Python (NumPy, SciPy, Matplotlib)

Indice

Presentazione **vi**

1 Flussi potenziali incomprimibili **1**

1.1 Perché si può trascurare la viscosità 1
1.2 Irrotazionalità e potenziale di velocità 2
1.3 Incomprimibilità ed equazione di Laplace 2
1.4 La pressione: equazione di Eulero e teorema di Bernoulli 3
1.5 La funzione di corrente 4
1.6 Sorgenti e pozzi distribuiti: l'equazione di Poisson 4
1.7 Un primo esperimento numerico: il reticolo di flusso 5

2 Problemi ai limiti per equazioni differenziali ordinarie **9**

2.1 Dai valori iniziali ai valori al contorno 9
2.2 Le differenze finite 9
2.3 L'equazione alle differenze e il sistema tridiagonale 11
2.4 L'eliminazione di Gauss per sistemi tridiagonali 12
2.5 Il flusso radiale prodotto da sorgenti distribuite 13
2.6 Condizioni al contorno sulla derivata 18

3 Il metodo inverso: sovrapposizione di flussi elementari **19**

3.1 Metodo diretto e metodo inverso 19
3.2 I quattro flussi elementari 19
3.3 Corpi generati per sovrapposizione 21
3.4 La ricerca dei punti di una linea di corrente su una griglia 23
3.5 Programma 2.2: corrente uniforme e cinque doppiette 24
3.6 Quando la sovrapposizione non genera un corpo 28
3.7 Il metodo delle immagini 28

4 Corpi di rivoluzione: il metodo di von Kármán **30**

4.1 Dal metodo inverso al metodo diretto 30
4.2 Flussi assialsimmetrici e funzione di corrente di Stokes 30
4.3 Il metodo di von Kármán 31
4.4 Programma 2.3: la sfera 33

4.5 Un corpo “a dirigibile”: l’ellissoide allungato 36

5 Il metodo inverso II: trasformazione conforme e profili alari 41
5.1 Il potenziale complesso 41
5.2 Le trasformazioni conformi 42
5.3 La trasformazione di Joukowski 43
5.4 Il profilo di Joukowski e la condizione di Kutta 43
5.5 Programma 2.4: il flusso attorno al profilo di Joukowski 45
5.6 Dal profilo di Joukowski ai profili reali: il metodo dei pannelli 50

6 Classificazione delle equazioni alle derivate parziali e metodi per le equazioni ellittiche 58
6.1 Subsonico e supersonico: due modi di propagare i disturbi 58
6.2 La classificazione delle equazioni del secondo ordine 59
6.3 L’equazione di Poisson discretizzata 60
6.4 Soluzione diretta o iterativa? 61
6.5 I metodi di Jacobi e di Gauss–Seidel 61
6.6 Il sovrarilassamento successivo (SOR) 63
6.7 Programma 2.5: un vortice in un recinto rettangolare 64
6.8 Jacobi, Gauss–Seidel e SOR a confronto 67

7 Flussi in condotti e attorno a corpi: contorni irregolari e condizioni di derivata 72
7.1 Il problema dei contorni 72
7.2 Programma 2.6: flusso in una camera con una parete inclinata 72
7.3 Contorni curvi: la formula a bracci disuguali 76
7.4 La condizione di derivata e i nodi fittizi 79
7.5 Programma 2.7: il cilindro nel canale 79
7.6 Il flusso assialsimmetrico in un condotto 83

8 Equazioni iperboliche: propagazione e riflessione di onde di piccola ampiezza 84
8.1 L’equazione delle onde 84
8.2 Lo schema alle differenze centrate 85
8.3 La stabilità: l’analisi di von Neumann 86
8.4 La soluzione di d’Alembert e il significato della condizione CFL 87
8.5 La partenza del calcolo 88
8.6 Programma 2.8: un’ondina in un tubo chiuso e aperto 89
8.7 Il numero di Courant alla prova 94

9 Onde di ampiezza finita e formazione dell’urto 98
9.1 Perché un’onda di ampiezza finita si deforma 98
9.2 Le equazioni per la velocità del gas e la velocità del suono 98
9.3 Caratteristiche e invarianti di Riemann 99

9.4 Il metodo delle caratteristiche . . . 99
9.5 Lo schema di Courant, Isaacson e Rees . . . 100
9.6 Programma 2.9: nascita di un'onda d'urto . . . 101

10 Convezione non lineare: gli schemi di MacCormack e di Beam–Warming 108
10.1 L'equazione di convezione e la forma conservativa . . . 108
10.2 Lo schema esplicito di MacCormack . . . 109
10.3 Programma 2.10: un salto che si propaga . . . 110
10.4 Lo schema implicito di Beam–Warming . . . 113
10.5 Programma 2.11: Beam–Warming con e senza dissipazione . . . 114
10.6 L'equazione di Burgers viscosa . . . 117

11 Un'applicazione biomedica: il flusso pulsante in un tubo elastico 121
11.1 Il modello monodimensionale . . . 121
11.2 L'equazione di stato e la velocità delle onde . . . 122
11.3 La forma conservativa e lo schema numerico . . . 122
11.4 Risultati . . . 124

A Simboli, listati di supporto e istruzioni 129
A.1 Simboli principali . . . 129
A.2 Listati di supporto . . . 130
A.3 Come riprodurre i risultati . . . 136
A.4 Codici in rete e QR code . . . 137
A.5 Come sostituire le figure . . . 137

B Esercizi svolti 139
B.1 Esercizio 1 – Combinazioni di flussi elementari . . . 139
B.2 Esercizio 2 – Il metodo delle immagini . . . 141
B.3 Esercizio 3 – Un corpo vicino a una parete . . . 142
B.4 Esercizio 4 – Il metodo di von Kármán con sorgenti puntiformi . . . 144
B.5 Esercizio 5 – Progetto: il metodo di von Kármán per un profilo NACA 0009 . . . 145
B.6 Esercizio 6 – La lastra piana senza circolazione . . . 147
B.7 Esercizio 7 – La lastra piana con la condizione di Kutta . . . 148
B.8 Esercizio 8 – SOR contro Liebmann . . . 149
B.9 Esercizio 9 – Due vortici nel rettangolo . . . 150
B.10 Esercizio 10 – Un cilindro a sezione rettangolare in un canale . . . 151
B.11 Esercizio 11 – Flusso assialsimmetrico in un tubo con diaframmi . . . 152
B.12 Esercizio 12 – Progetto: il tubo d'urto linearizzato . . . 154
B.13 Esercizio 13 – Riflessione di onde di ampiezza finita . . . 156
B.14 Esercizio 14 – La soluzione esatta dell'equazione di Burgers . . . 157
B.15 Esercizio 15 – Burgers con lo schema di MacCormack . . . 158

B.16 Esercizio 16 – Burgers con lo schema di Beam–Warming 160
B.17 Esercizio 17 – Il tubo elastico: frequenza, ampiezza, attrito 161
B.18 Listato completo 164

Bibliografia **178**

Elenco dei listati

1.1 `prog2_0_reticolo.py` – reticolo di flusso del semicorpo di Rankine 7
2.1 Programma 2.1 – flusso radiale da sorgenti distribuite 16
3.1 Programma 2.2 – corrente uniforme e cinque doppiette 26
4.1 Programma 2.3 – metodo di von Kármán per la sfera 34
4.2 `prog2_3b_ellissoide.py` – ellissoide prolato, confronto con la soluzione esatta 38
5.1 Programma 2.4 – profilo di Joukowski per trasformazione conforme 48
5.2 `prog2_4c_pannelli.py` – metodo dei pannelli di vortice 53
6.1 Programma 2.5 – vortice in un recinto rettangolare, metodo di Liebmann 65
6.2 `prog2_5b_convergenza.py` – confronto fra Jacobi, Gauss–Seidel e SOR . . . 69
7.1 Programma 2.6 – flusso in una camera con parete inclinata 75
7.2 Programma 2.7 – canale con un cilindro circolare 80
8.1 Programma 2.8 – leapfrog: ondina in un tubo chiuso–aperto 90
8.2 `prog2_8b_courant.py` – esperimenti sul numero di Courant 96
9.1 Programma 2.9 – onda di ampiezza finita e formazione dell'urto 105
10.1 Programma 2.10 – convezione non lineare, schema di MacCormack 112
10.2 Programma 2.11 – convezione non lineare, schema di Beam–Warming 115
10.3 `prog2_12_burgers.py` – equazione di Burgers 118
11.1 `prog2_13_tubo_elastico.py` – flusso pulsante in un tubo elastico 126
A.1 `metodi_numerici.py` – modulo comune 130
A.2 `stile_grafici.py` – impostazioni di Matplotlib 132
A.3 `prog2_2b_flussi_elementari.py` – flussi elementari e cilindro 133
A.4 `prog2_4b_mappe.py` – esempi di trasformazioni conformi 135
B.1 `esercizi_svolti.py` (esercizi 1–17) 164

Presentazione

Di che cosa parla questa monografia

Nel primo modulo del corso l'incognita era una funzione del solo tempo: la posizione di una sfera che cade, l'angolo di un pendolo, la traiettoria di un aliante. Bastava integrare un sistema di equazioni differenziali *ordinarie* a partire dalle condizioni iniziali. In questo secondo modulo l'incognita diventa un *campo*: la velocità e la pressione in ogni punto di una regione di spazio, attorno a un profilo alare, dentro un condotto, lungo un tubo percorso da un'onda di pressione. Le equazioni del moto diventano equazioni *alle derivate parziali* (PDE, *partial differential equations*) e, accanto alle condizioni iniziali, compaiono le *condizioni al contorno*.

Il filo conduttore è l'ipotesi di fluido **non viscoso**. Non è un'astrazione da manuale: per un velivolo in volo il numero di Reynolds è dell'ordine di 10^6–10^8, e fuori dal sottile strato limite aderente alla parete gli effetti della viscosità sono davvero trascurabili. Con questa ipotesi la teoria del potenziale predice con ottima precisione la portanza e la distribuzione di pressione su un'ala; con la stessa ipotesi, ma senza linearizzare, si descrive come un'onda di pressione si irripidisce fino a diventare un'onda d'urto. Il modulo percorre entrambe le strade:

- i **flussi potenziali incomprimibili**, governati dalle equazioni di Laplace e di Poisson, che sono *ellittiche*: la soluzione in un punto dipende da tutto il contorno, e il calcolo si riduce a un problema ai limiti;
- la **propagazione delle onde** nei gas, governata da equazioni *iperboliche*: l'informazione viaggia a velocità finita lungo le linee caratteristiche, e il calcolo procede nel tempo come nel primo modulo, ma con vincoli di stabilità che legano il passo temporale a quello spaziale.

Questa monografia costituisce il secondo modulo del corso di fluidodinamica numerica. I temi seguono una progressione classica della letteratura, qui rielaborata con dimostrazioni complete, programmi in Python corredati di grafici ed esercizi interamente svolti; a ogni passo i risultati numerici vengono confrontati con una soluzione esatta, perché un programma di cui non si conosce l'errore non è uno strumento di progetto. La bibliografia raccoglie le fonti di riferimento: fra queste, i testi classici di meccanica dei fluidi computazionale [1, 2], Chapra e Canale [3] per l'algebra lineare numerica e i metodi iterativi, Ferziger e Perić [4] e Tannehill, Anderson e Pletcher [5] per la discretizzazione delle equazioni alle derivate parziali, LeVeque [6] per le leggi di conservazione iperboliche, Kuethe e Chow [11] e Katz e Plotkin [12] per l'aerodinamica del potenziale.

Come è organizzata

Capitolo 1 – Flussi potenziali incomprimibili.
Perché si può trascurare la viscosità; potenziale di velocità e funzione di corrente; le equazioni di Laplace e di Poisson; il teorema di Bernoulli e il coefficiente di pressione.

Capitolo 2 – Problemi ai limiti per equazioni ordinarie.
Le differenze finite, il sistema tridiagonale, l'algoritmo di Thomas; il flusso radiale prodotto da una distribuzione di sorgenti.

Capitolo 3 – Il metodo inverso: sovrapposizione di flussi elementari.
Corrente uniforme, sorgente, vortice, doppietta; corpi generati per sovrapposizione, cilindro con circolazione e teorema di Kutta–Joukowski; la ricerca delle linee di corrente su una griglia; il metodo delle immagini.

Capitolo 4 – Corpi di rivoluzione: il metodo di von Kármán.
Sorgenti distribuite sull'asse di un corpo assegnato; la sfera e l'ellissoide "a dirigibile"; il mal condizionamento del metodo.

Capitolo 5 – Trasformazione conforme e profili alari.
Potenziale complesso, trasformazione di Joukowski, condizione di Kutta, portanza; dal profilo di Joukowski ai profili NACA con il metodo dei pannelli di vortice.

Capitolo 6 – Classificazione delle PDE ed equazioni ellittiche.
Equazioni ellittiche, paraboliche, iperboliche; i metodi di Jacobi, Gauss–Seidel e del sovrarilassamento; il vortice in un recinto rettangolare.

Capitolo 7 – Contorni irregolari e condizioni di derivata.
Il flusso in una camera con parete inclinata e attorno a un cilindro in un canale.

Capitolo 8 – Onde di piccola ampiezza.
L'equazione delle onde, lo schema *leapfrog*, la stabilità di von Neumann e la condizione di Courant–Friedrichs–Lewy; riflessione del suono all'estremo chiuso e all'estremo aperto di un tubo.

Capitolo 9 – Onde di ampiezza finita e formazione dell'urto.
Caratteristiche e invarianti di Riemann; lo schema di Courant, Isaacson e Rees; l'istante in cui nasce l'urto.

Capitolo 10 – Convezione non lineare.
Gli schemi di MacCormack e di Beam–Warming, la dissipazione artificiale, l'equazione di Burgers.

Capitolo 11 – Il flusso pulsante in un tubo elastico.
Un sistema iperbolico non lineare completo: il modello monodimensionale di un'arteria.

Appendici.
Simboli e listati di supporto; i diciassette esercizi svolti passo per passo.

Ogni capitolo segue lo stesso schema del primo modulo: prima l'*intuizione fisica*, poi il *modello matematico*, quindi l'*algoritmo* e il *programma*, infine i *risultati* discussi criticamente. I riquadri colorati hanno il significato consueto:

- **teoria** (viola): definizioni, enunciati, dimostrazioni;

- **fasi** (blu): procedure e algoritmi passo per passo;
- **ipotesi** (giallo): le assunzioni semplificative del modello;
- **dati** (grigio): valori numerici usati nei calcoli;
- **esempi** (arancio): applicazioni pratiche e analogie aeronautiche;
- **risultati** (verde): i numeri ottenuti e il loro significato;
- **avvisi** (rosso): errori frequenti e limiti del modello.

I programmi Python

Tutti i programmi sono scritti in Python 3 con le librerie `numpy`, `scipy` e `matplotlib`; l'esercizio 14 usa anche `sympy` per il calcolo simbolico. Il codice privilegia la leggibilità: le formule compaiono nel programma con gli stessi simboli del testo e, dove la vettorizzazione di NumPy renderebbe l'algoritmo meno riconoscibile (i metodi iterativi di Gauss–Seidel e del sovrarilassamento, che aggiornano un nodo alla volta), si è preferito il ciclo esplicito.

I programmi sono disponibili in rete all'indirizzo `https://braucci.github.io/CFD/monografia2/codici/`. Accanto a ogni listato, nel margine destro, un codice QR rimanda al file corrispondente: si inquadra con la fotocamera del telefono oppure, nella versione PDF, si clicca direttamente.

codici/

Struttura della cartella `codice/`

`metodi_numerici.py` strumenti comuni: eliminazione tridiagonale e algoritmo di Thomas, ricerca delle linee di corrente su una griglia, un'iterazione dei metodi di Jacobi, Gauss–Seidel e SOR, fattore di sovrarilassamento ottimo.

`stile_grafici.py` impostazioni comuni di Matplotlib (palette della casa).

`prog2_0` ... `prog2_13` i programmi dei capitoli; il numero ricalca quello del programma nel testo (`prog2_1` è il Programma 2.1, e così via); le lettere `b`, `c` indicano gli approfondimenti.

`esercizi_svolti.py` il codice dei diciassette esercizi dell'Appendice B.

Ogni script si esegue dalla cartella `codice/` con `python3 nome_script.py`; stampa a video le tabelle riportate nel testo e salva le figure in formato PDF nella cartella `figure/`, da cui il documento LaTeX le preleva.

Prerequisiti

Si danno per noti i contenuti del primo modulo (integrazione nel tempo, ordine di accuratezza, stabilità) e le nozioni di base di analisi (derivate parziali, sviluppo in serie di Taylor, numeri complessi), di algebra lineare (sistemi di equazioni, matrici) e di aerodinamica elementare (numero di Reynolds e di Mach, coefficienti di portanza e di resistenza, teorema di Kutta–Joukowski).

Capitolo 1

Flussi potenziali incomprimibili

1.1 Perché si può trascurare la viscosità

Tutti i fluidi reali sono viscosi. La viscosità nasce dalla ridistribuzione della quantità di moto fra molecole vicine per effetto degli urti: dove esiste un gradiente di velocità, uno strato di fluido più veloce trascina quello più lento e ne è frenato. Sulla superficie di un elemento fluido nasce così uno sforzo, tangenziale come nello strato limite di un'ala, oppure normale come all'interno di un'onda d'urto.

L'importanza relativa degli sforzi viscosi rispetto alle forze d'inerzia è misurata dal numero di Reynolds,

$$Re = \frac{\rho V L}{\mu} = \frac{V L}{\nu} \sim \frac{\text{forza d'inerzia}}{\text{forza viscosa}}, \tag{1.1}$$

dove V e L sono una velocità e una lunghezza caratteristiche. Per l'aria in condizioni standard $\nu \simeq 1{,}5 \cdot 10^{-5}$ m^2/s: un'ala di corda $c = 1{,}5$ m che vola a 60 m/s ha $Re = 6 \cdot 10^6$. Nei flussi di interesse aeronautico le forze viscose sono dunque, *in media*, milioni di volte più piccole di quelle d'inerzia.

Nel 1904 Prandtl [19] propose di sfruttare questo fatto in modo preciso: per una corrente ad alto numero di Reynolds attorno a un corpo affusolato da cui la corrente non si separa, l'influenza della viscosità è confinata in uno *strato limite* molto sottile a contatto con la parete; fuori da esso il fluido si comporta come se fosse privo di viscosità. Lo spessore dello strato limite laminare su una lastra cresce come $\delta \simeq 5x/\sqrt{Re_x}$: sull'ala dell'esempio, al bordo d'uscita, è di pochi millimetri, a fronte di una corda di un metro e mezzo.

Il modello di Prandtl e le sue conseguenze

1. Fuori dallo strato limite il fluido è **non viscoso**; poiché lo strato è sottile, la corrente esterna "vede" il corpo con la sua forma geometrica e deve essere **tangente alla superficie**.
2. La distribuzione di velocità della corrente esterna lungo la parete diventa la condizione al contorno esterna dello strato limite, che a sua volta rispetta la condizione di aderenza (velocità nulla sulla parete).
3. La teoria non viscosa predice molto bene la *portanza* e la distribuzione di pressione su un profilo alare finché la corrente non si separa (cioè al di sotto dello stallo); la *resistenza*, invece, richiede la soluzione delle equazioni dello strato limite.

In questa monografia lo strato limite viene ignorato del tutto: ci occupiamo della corrente esterna, che determina la pressione e quindi la portanza.

1.2 Irrotazionalità e potenziale di velocità

Le vorticità nascono dagli sforzi viscosi tangenziali: lo strato limite è una regione *rotazionale*. Al contrario, in assenza di forze viscose (e di altre forze capaci di mettere in rotazione le particelle) una corrente che a monte, lontano dal corpo, è irrotazionale rimane tale in tutta la regione esterna allo strato limite. Se $\mathbf{V}$ indica il campo di velocità in questa regione, la condizione di irrotazionalità afferma che la vorticità è nulla:

$$\nabla \times \mathbf{V} = \mathbf{0}. \tag{1.2}$$

Perché la corrente resta irrotazionale: il teorema di Kelvin

Si consideri una linea chiusa $\mathcal{C}$ che si muove con il fluido e la circolazione $\Gamma = \oint_{\mathcal{C}} \mathbf{V} \cdot \mathrm{d}\mathbf{s}$. Per un fluido non viscoso, a densità costante (o barotropico) e soggetto solo a forze di massa conservative, il teorema di Kelvin afferma che $\mathrm{d}\Gamma/\mathrm{d}t = 0$: la circolazione lungo una linea materiale non cambia nel tempo. Lontano a monte la corrente è uniforme e la circolazione su qualunque linea è nulla; trasportata dal fluido fino alla zona del corpo, resta nulla. Per il teorema di Stokes $\Gamma = \iint (\nabla \times \mathbf{V}) \cdot \mathbf{n}\, \mathrm{d}S$, e poiché ciò vale per qualunque superficie appoggiata a una qualunque linea materiale, la vorticità deve essere nulla ovunque. La vorticità può essere creata solo dove l'ipotesi cade: nello strato limite, a causa della viscosità.

La (1.2) è soddisfatta automaticamente se la velocità è il gradiente di una funzione scalare ϕ, detta **potenziale di velocità**:

$$\mathbf{V} = \nabla \phi, \tag{1.3}$$

perché il rotore di un gradiente è identicamente nullo. Per questo le correnti irrotazionali si chiamano anche *correnti potenziali*. Il guadagno è notevole: le tre componenti della velocità sono sostituite da una sola funzione scalare.

1.3 Incomprimibilità ed equazione di Laplace

Se inoltre il fluido è incomprimibile (lo è con buona approssimazione l'aria per numeri di Mach inferiori a circa 0,3), il campo di velocità deve rispettare l'equazione di continuità

$$\nabla \cdot \mathbf{V} = 0, \tag{1.4}$$

che con la (1.3) diventa l'**equazione di Laplace**

$$\nabla^2 \phi = 0, \tag{1.5}$$

dove $\nabla^2 \equiv \nabla \cdot \nabla$ è l'operatore laplaciano. Per correnti piane, in coordinate cartesiane e polari,

$$u = \frac{\partial \phi}{\partial x}, \quad v = \frac{\partial \phi}{\partial y}, \qquad \frac{\partial^2 \phi}{\partial x^2} + \frac{\partial^2 \phi}{\partial y^2} = 0, \tag{1.6}$$

$$u_r = \frac{\partial \phi}{\partial r}, \quad u_\theta = \frac{1}{r}\frac{\partial \phi}{\partial \theta}, \qquad \frac{\partial^2 \phi}{\partial r^2} + \frac{1}{r}\frac{\partial \phi}{\partial r} + \frac{1}{r^2}\frac{\partial^2 \phi}{\partial \theta^2} = 0. \tag{1.7}$$

Il problema del flusso attorno a un corpo assegnato

Secondo il modello di Prandtl, il flusso incomprimibile attorno a un corpo affusolato ad alto numero di Reynolds si calcola in due tempi:

1. si cerca una soluzione ϕ dell'equazione di Laplace (1.5) nella regione esterna al corpo che soddisfi la condizione all'infinito (corrente uniforme) e la condizione di tangenza sulla superficie, $\partial\phi/\partial n = 0$;
2. nota la velocità, si calcola la pressione con il teorema di Bernoulli.

L'equazione è *lineare*: la somma di due soluzioni è ancora una soluzione. Su questa proprietà si fondano i metodi dei Capitoli 3–5.

1.4 La pressione: equazione di Eulero e teorema di Bernoulli

Una volta determinato il campo di velocità con considerazioni puramente cinematiche, la pressione p si ottiene da una relazione dinamica, l'**equazione di Eulero**, cioè l'equazione della quantità di moto per un fluido non viscoso (le forze di massa sono omesse):

$$\rho\frac{\mathrm{D}\mathbf{V}}{\mathrm{D}t} = -\nabla p, \qquad \frac{\mathrm{D}}{\mathrm{D}t} \equiv \frac{\partial}{\partial t} + \mathbf{V}\cdot\nabla, \tag{1.8}$$

dove $\mathrm{D}/\mathrm{D}t$ è la *derivata sostanziale*, che dà la rapidità di variazione seguendo la particella fluida.

Dall'equazione di Eulero al teorema di Bernoulli

Vale l'identità vettoriale $(\mathbf{V}\cdot\nabla)\mathbf{V} = \nabla\left(\frac{1}{2}V^2\right) - \mathbf{V}\times(\nabla\times\mathbf{V})$. In una corrente irrotazionale l'ultimo termine è nullo e $\mathbf{V} = \nabla\phi$; con ρ costante la (1.8) diventa

$$\nabla\left(\rho\frac{\partial\phi}{\partial t} + p + \tfrac{1}{2}\rho V^2\right) = \mathbf{0}.$$

Un vettore che ha gradiente nullo dappertutto è costante nello spazio (può dipendere solo dal tempo, e la dipendenza si assorbe in ϕ). Si ottiene così il **teorema di Bernoulli** per correnti incomprimibili e irrotazionali:

$$\rho\frac{\partial\phi}{\partial t} + p + \tfrac{1}{2}\rho V^2 = H, \tag{1.9}$$

valido fra due punti *qualsiasi* del campo, non solo lungo una linea di corrente. In regime stazionario la somma della pressione statica p e della pressione dinamica $\frac{1}{2}\rho V^2$ è la stessa ovunque; la costante H è la *pressione di ristagno*, quella misurata dove il fluido è fermo (la presa di un tubo di Pitot).

Se p_∞ e V_∞ sono pressione e velocità della corrente indisturbata, conviene esprimere la pressione con il **coefficiente di pressione**

$$C_p \equiv \frac{p - p_\infty}{\frac{1}{2}\rho V_\infty^2} = 1 - \left(\frac{V}{V_\infty}\right)^2, \tag{1.10}$$

dove la seconda uguaglianza vale per la corrente stazionaria e segue subito dalla (1.9). $C_p = 1$ nei punti di ristagno, $C_p = 0$ dove la velocità è quella all'infinito, $C_p < 0$ (depressione) dove il fluido accelera: sul dorso di un'ala portante.

1.5 La funzione di corrente

Invece del potenziale di velocità, per correnti piane o assialsimmetriche incomprimibili si usa spesso la **funzione di corrente** ψ. L'equazione di continuità (1.4) suggerisce di scrivere la velocità come rotore di un potenziale vettore diretto lungo l'asse z perpendicolare al piano del moto,

$$\mathbf{V} = \nabla \times (\mathbf{i}_z \psi), \qquad \text{cioè} \qquad u = \frac{\partial \psi}{\partial y}, \quad v = -\frac{\partial \psi}{\partial x}; \tag{1.11}$$

la divergenza di un rotore è nulla e la continuità è soddisfatta identicamente. L'irrotazionalità (1.2) si scrive allora

$$\nabla \times \nabla \times (\mathbf{i}_z \psi) = \mathbf{0} \quad \Longrightarrow \quad \frac{\partial^2 \psi}{\partial x^2} + \frac{\partial^2 \psi}{\partial y^2} = 0, \tag{1.12}$$

e in coordinate polari

$$u_r = \frac{1}{r}\frac{\partial \psi}{\partial \theta}, \quad u_\theta = -\frac{\partial \psi}{\partial r}, \qquad \frac{\partial^2 \psi}{\partial r^2} + \frac{1}{r}\frac{\partial \psi}{\partial r} + \frac{1}{r^2}\frac{\partial^2 \psi}{\partial \theta^2} = 0. \tag{1.13}$$

Il significato fisico della funzione di corrente

1. **Le linee ψ = cost sono linee di corrente.** Lungo una curva ψ = cost si ha $\mathrm{d}\psi = \psi_x \mathrm{d}x + \psi_y \mathrm{d}y = -v\,\mathrm{d}x + u\,\mathrm{d}y = 0$, cioè $\mathrm{d}y/\mathrm{d}x = v/u$: la curva è tangente alla velocità in ogni punto.
2. **La differenza di ψ è la portata.** Siano A e B due punti e ℓ una curva qualsiasi che li congiunge. La portata volumetrica (per unità di apertura) che attraversa ℓ è $\int_\ell \mathbf{V}\cdot\mathbf{n}\,\mathrm{d}s$; con $\mathbf{n}\,\mathrm{d}s = (\mathrm{d}y, -\mathrm{d}x)$ si ottiene

$$\int_\ell (u\,\mathrm{d}y - v\,\mathrm{d}x) = \int_\ell (\psi_y \mathrm{d}y + \psi_x \mathrm{d}x) = \psi_B - \psi_A,$$

indipendentemente dalla curva scelta. Fra due linee di corrente scorre sempre la stessa portata: dove le linee si avvicinano il fluido accelera, dove si allontanano rallenta. È la lettura "a vista" di tutti i diagrammi di questa monografia.

Come il potenziale, anche la funzione di corrente delle correnti piane soddisfa l'equazione di Laplace; il termine non stazionario del teorema di Bernoulli, però, non si esprime in forma semplice con ψ. Le linee ϕ = cost (*equipotenziali*) e ψ = cost sono ortogonali in ogni punto in cui la velocità non è nulla, perché $\nabla\phi\cdot\nabla\psi = u(-v) + v\,u = 0$: insieme formano il *reticolo di flusso*.

1.6 Sorgenti e pozzi distribuiti: l'equazione di Poisson

L'equazione di continuità (1.4) non vale dove il fluido viene creato o distrutto. Generalizziamo la formulazione a un campo in cui siano presenti sorgenti e pozzi distribuiti con intensità $q(x,y,z,t)$, il volume di fluido creato nell'unità di tempo dall'unità di volume.

Si consideri il volume di controllo elementare $\Delta x\,\Delta y\,\Delta z$ fisso nello spazio di Fig. 1.1. Attraverso la faccia di sinistra entra nell'unità di tempo il volume $u\,\Delta y\,\Delta z$, da quella di destra esce $(u + \partial u/\partial x\,\Delta x)\Delta y\,\Delta z$; il bilancio netto nella direzione x è $(\partial u/\partial x)\Delta x\,\Delta y\,\Delta z$ e analogamente

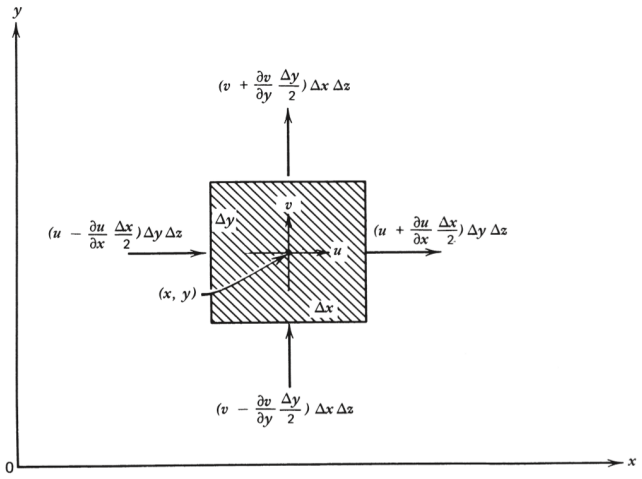


Figura 1.1: Flusso attraverso un volume di controllo elementare.

nelle altre due direzioni. Per un fluido incomprimibile il volume uscente deve uguagliare quello prodotto dalle sorgenti contenute, $q\,\Delta x\,\Delta y\,\Delta z$:

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} + \frac{\partial w}{\partial z} = q(x, y, z, t), \qquad \nabla \cdot \mathbf{V} = q. \tag{1.14}$$

I pozzi corrispondono a $q < 0$; per $q = 0$ si ritrova la (1.4). Con $\mathbf{V} = \nabla\phi$ si ottiene l'**equazione di Poisson**

$$\nabla^2\phi = q. \tag{1.15}$$

Allo stesso tipo appartiene l'equazione della funzione di corrente in presenza di vorticità distribuita, $\nabla^2\psi = -\zeta$, che risolveremo nel Cap. 6.

Lineare non vuol dire facile

Le equazioni di Laplace e di Poisson sono lineari, ma la soluzione analitica del flusso attorno a un corpo di forma qualsiasi non è affatto semplice: la difficoltà sta tutta nelle condizioni al contorno su una superficie di forma arbitraria. I capitoli che seguono introducono tecniche numeriche e seminumeriche di complessità crescente: la sovrapposizione di soluzioni elementari (Cap. 3), le distribuzioni di singolarità (Cap. 4 e 5), la trasformazione conforme (Cap. 5), infine la discretizzazione diretta dell'equazione su una griglia (Cap. 6 e 7).

1.7 Un primo esperimento numerico: il reticolo di flusso

Per fissare le idee consideriamo la corrente uniforme U a cui si sovrappone una sorgente piana di intensità σ (portata per unità di apertura) posta nell'origine. Anticipando i risultati del Cap. 3, potenziale e funzione di corrente sono

$$\phi = Ux + \frac{\sigma}{2\pi}\ln r, \qquad \psi = Uy + \frac{\sigma}{2\pi}\theta. \tag{1.16}$$

La sorgente spinge il fluido in tutte le direzioni; la corrente uniforme lo porta a valle. Sull'asse x, a monte della sorgente, le due velocità si annullano nel punto di ristagno $x_s = -\sigma/(2\pi U)$; la linea di corrente che vi passa, $\psi = \pm\sigma/2$, separa il fluido emesso dalla sorgente da quello

della corrente esterna e ha la forma di un corpo aperto a valle, il *semicorpo di Rankine*, il cui spessore tende a σ/U (il fluido della sorgente, portata σ, deve defluire a velocità U).

Il programma `prog2_0_reticolo.py` (Listato 1.1) traccia il reticolo di flusso (Fig. 1.2) e controlla numericamente che il potenziale soddisfi l'equazione di Laplace. Per farlo sostituisce le derivate seconde con le differenze finite che studieremo nel prossimo capitolo, $\partial^2\phi/\partial x^2 \simeq (\phi_{i+1,j} - 2\phi_{i,j} + \phi_{i-1,j})/h^2$ e analogamente in y, e calcola il "laplaciano discreto" in un punto del campo per passi h decrescenti.

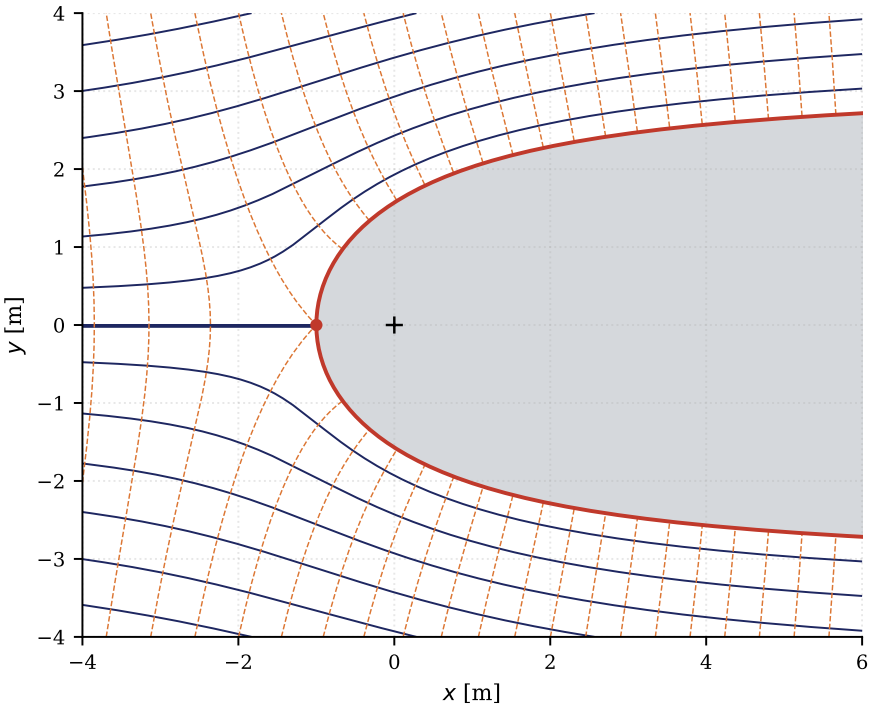


Figura 1.2: Reticolo di flusso del semicorpo di Rankine ($U = 1$ m/s, $\sigma = 2\pi$ m^2/s): linee di corrente (continue) ed equipotenziali (tratteggiate), ortogonali in ogni punto tranne il ristagno ($x_s = -1$ m). In rosso il contorno del corpo, $\psi = \pm\sigma/2$.

Risultati

- Punto di ristagno $x_s = -1{,}000$ m; semispessore asintotico $\sigma/(2U) = 3{,}142$ m.
- Il laplaciano discreto di ϕ nel punto $(1;\, 1{,}5)$ vale $1{,}07 \cdot 10^{-2}$ per $h = 0{,}4$ m e si riduce di un fattore 4,00 a ogni dimezzamento di h, fino a $2{,}6 \cdot 10^{-6}$ per $h = 0{,}00625$ m (Fig. 1.3): ϕ è armonica e la formula a cinque punti ha un errore di troncamento proporzionale a h^2.
- Il coefficiente di pressione minimo sul corpo è $C_p = -0{,}587$, nel punto $x = 1{,}05$ m: il fluido che aggira il "muso" accelera fino a $1{,}26\,U$.

Dove si incontra un semicorpo di Rankine

La forma del semicorpo è quella, idealizzata, della parte anteriore di molti corpi affusolati: l'ogiva di una fusoliera, il naso di un missile, il pilone che sostiene un motore. Il risultato $C_p^{\min} = -0{,}587$ dice quanto accelera la corrente nella zona in cui il muso si raccorda alla parte cilindrica; è la zona in cui si posizionano le prese statiche dei sistemi

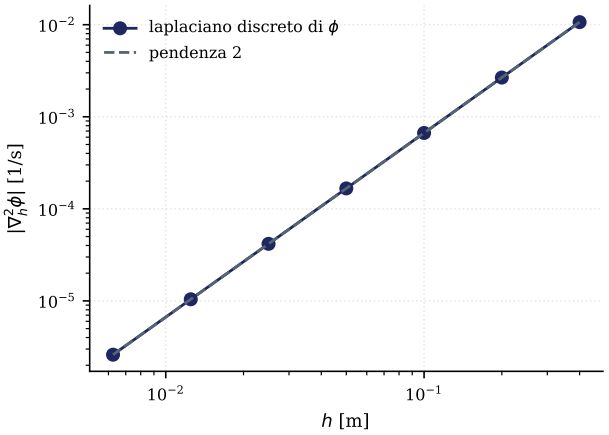


Figura 1.3: Il laplaciano discreto del potenziale (1.16) tende a zero come h^2.

anemometrici solo dopo averne valutato l'errore di posizione, perché lì la pressione locale è sensibilmente diversa da quella della corrente indisturbata.

prog2_0_reticolo.py

```python
"""
prog2_0_reticolo.py -- il reticolo di flusso (linee phi = cost e psi = cost)
di un semicorpo di Rankine (corrente uniforme + sorgente) e la verifica
numerica che phi soddisfa l'equazione di Laplace.

Uscite: figure prog2_0_reticolo.pdf e prog2_0_laplaciano.pdf, tabella a video.
"""
import numpy as np
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, ROSSO, GRIGIO, VERDE

U = 1.0                  # velocita' della corrente uniforme  [m/s]
SIGMA = 2.0 * np.pi      # intensita' della sorgente [m^2/s] (portata per unita' di
    apertura)


def phi(x, y):
    """Potenziale di velocita':  U x + sigma/(2 pi) ln r."""
    return U * x + SIGMA / (2 * np.pi) * np.log(np.hypot(x, y))


def psi(x, y):
    """Funzione di corrente:  U y + sigma/(2 pi) theta,  theta in (-pi, pi]."""
    return U * y + SIGMA / (2 * np.pi) * np.arctan2(y, x)


def velocita(x, y):
    r2 = x**2 + y**2
    u = U + SIGMA / (2 * np.pi) * x / r2
    v = SIGMA / (2 * np.pi) * y / r2
    return u, v


# --- grandezze caratteristiche del semicorpo ---------------------------------
xs = -SIGMA / (2 * np.pi * U)          # punto di ristagno
semilarghezza = SIGMA / (2 * U)        # semispessore a valle infinita
print(f"punto di ristagno       x_s = {xs:.4f} m")
print(f"semispessore asintotico h   = {semilarghezza:.4f} m  (spessore
```

```
    {2*semilarghezza:.4f} m)")

# --- reticolo di flusso ------------------------------------------------------
x = np.linspace(-4, 6, 501)
y = np.linspace(-4, 4, 401)
X, Y = np.meshgrid(x, y, indexing="ij")
with np.errstate(divide="ignore"):
    PHI, PSI = phi(X, Y), psi(X, Y)
# i punti interni al corpo (|psi| < sigma/2) sono mascherati: li' il campo non
# interessa e il taglio di arctan2 sul semiasse x<0 produrrebbe un salto di psi
interno = np.abs(PSI) < SIGMA / 2
PSIm = np.where(interno, np.nan, PSI)
PHIm = np.where(interno, np.nan, PHI)

fig, ax = plt.subplots(figsize=(6.4, 4.6))
ax.contour(X, Y, PSIm, levels=np.arange(-7.0, 7.01, 0.5), colors=BLU,
    linewidths=0.8,
           linestyles="solid")
ax.contour(X, Y, PHIm, levels=np.arange(-6.0, 9.01, 0.5), colors=ARANCIO,
           linewidths=0.6, linestyles="--")
# linea di corrente divisoria psi = +-sigma/2: contorno del semicorpo
# r(theta) = sigma (pi - |theta|) / (2 pi U |sin theta|)
t = np.linspace(-np.pi + 1e-3, np.pi - 1e-3, 800)      # theta del contorno
rc = SIGMA / (2 * U) * (np.pi - np.abs(t)) / (np.pi * np.abs(np.sin(t)) + 1e-15)
ax.fill(rc * np.cos(t), rc * np.sin(t), color=GRIGIO, alpha=0.25, lw=0)
ax.plot(rc * np.cos(t), rc * np.sin(t), color=ROSSO, lw=1.6)
ax.plot([xs], [0], "o", color=ROSSO, ms=4)
ax.plot([0], [0], "+", color="k", ms=7)
ax.set_xlim(-4, 6); ax.set_ylim(-4, 4); ax.set_aspect("equal")
ax.set_xlabel("$x$ [m]"); ax.set_ylabel("$y$ [m]")
salva(fig, "prog2_0_reticolo")

# --- verifica: il laplaciano discreto di phi tende a zero come h^2 -----------
x0, y0 = 1.0, 1.5                  # punto di prova nel campo di moto
print("\n      h        lapl. discreto       |errore|     rapporto")
hh = 0.4 / 2.0**np.arange(0, 7)
errori = []
for h in hh:
    lap = (phi(x0 + h, y0) + phi(x0 - h, y0) + phi(x0, y0 + h) + phi(x0, y0 - h)
           - 4 * phi(x0, y0)) / h**2
    errori.append(abs(lap))
for k, h in enumerate(hh):
    rapp = errori[k - 1] / errori[k] if k else np.nan
    print(f"  {h:8.5f}   {errori[k]:+.6e}   {errori[k]:.3e}   {rapp:6.2f}")

fig, ax = plt.subplots(figsize=(4.6, 3.3))
ax.loglog(hh, errori, "o-", color=BLU, label="laplaciano discreto di $\\phi$")
ax.loglog(hh, errori[0] * (hh / hh[0])**2, "--", color=GRIGIO, label="pendenza 2")
ax.set_xlabel("$h$ [m]"); ax.set_ylabel(r"$|\nabla_h^2\phi|$ [1/s]")
ax.legend()
salva(fig, "prog2_0_laplaciano")

# --- coefficiente di pressione sul contorno del corpo (Bernoulli) ------------
uc, vc = velocita(rc * np.cos(t), rc * np.sin(t))
cp = 1 - (uc**2 + vc**2) / U**2
k = np.argmin(cp)
print(f"\nC_p minimo sul corpo = {cp[k]:.4f}  in x = {rc[k]*np.cos(t[k]):.3f} m")
```

Listato 1.1: `prog2_0_reticolo.py`: reticolo di flusso del semicorpo di Rankine e verifica numerica dell'equazione di Laplace.

Capitolo 2

Problemi ai limiti per equazioni differenziali ordinarie

2.1 Dai valori iniziali ai valori al contorno

Nel primo modulo le equazioni differenziali ordinarie erano corredate di condizioni assegnate tutte nello stesso istante iniziale: posizione e velocità a $t = 0$. Si marciava in avanti un passo alla volta e ogni nuovo valore dipendeva solo da quelli già calcolati. In un'altra classe di problemi fluidodinamici la soluzione deve soddisfare l'equazione in un *intervallo* della variabile indipendente e, in più, condizioni assegnate *agli estremi* dell'intervallo: si parla di **problemi ai limiti** (o ai valori al contorno). La temperatura in una barra con gli estremi a temperatura fissata, la deformata di una trave appoggiata, il potenziale di un flusso radiale fra due cilindri coassiali sono esempi tipici.

La differenza è sostanziale. Nel problema ai valori iniziali l'informazione si propaga in una sola direzione, dal passato al futuro; nel problema ai limiti ogni punto interno "sente" entrambi gli estremi, e i valori incogniti devono essere determinati *tutti insieme*, risolvendo un sistema di equazioni algebriche. È la stessa natura delle equazioni ellittiche del Cap. 6, di cui questo capitolo è la versione monodimensionale.

Consideriamo l'equazione differenziale ordinaria lineare del secondo ordine

$$\frac{\mathrm{d}^2 f}{\mathrm{d}x^2} + A(x)\frac{\mathrm{d}f}{\mathrm{d}x} + B(x)\, f = D(x), \qquad x_{\min} \le x \le x_{\max}, \tag{2.1}$$

con le condizioni al contorno che assegnano i valori di f agli estremi, $f(x_{\min})$ e $f(x_{\max})$ (condizioni di *Dirichlet*). Il metodo verrà poi esteso alle condizioni sulla derivata (condizioni di *Neumann*).

2.2 Le differenze finite

Il primo passo del metodo consiste nel sostituire l'equazione differenziale con un'equazione alle differenze finite. Come nei problemi ai valori iniziali, l'asse della variabile indipendente si divide in intervalli di ampiezza costante h. Se l'intervallo $[x_{\min}, x_{\max}]$ è diviso in $n+1$ parti e i nodi sono numerati come in Fig. 2.1 a partire da $x_0 = x_{\min}$, si ha

$$x_i = x_0 + i\,h, \qquad i = 0, 1, \ldots, n+1, \qquad x_{n+1} = x_{\max}, \tag{2.2}$$

e si indica $f_i \equiv f(x_i)$.

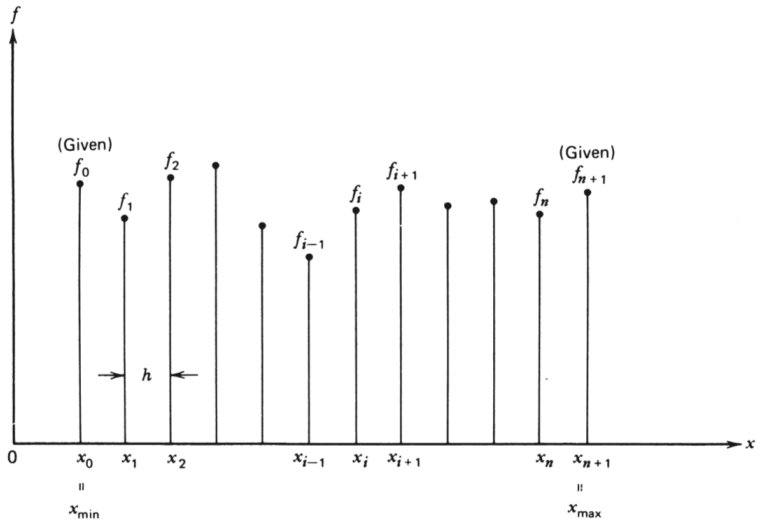


Figura 2.1: Discretizzazione di un problema ai limiti monodimensionale.

Si vuole approssimare la derivata di f nel nodo x_i con una combinazione dei valori di f nei nodi vicini. Sia $x_j = x_i + mh$ un nodo vicino (m intero positivo o negativo). Per h piccolo la funzione si sviluppa in serie di Taylor attorno a x_i:

$$f_j = f(x_i + mh) = f_i + mh\, f_i' + \frac{(mh)^2}{2!} f_i'' + \frac{(mh)^3}{3!} f_i''' + \cdots \tag{2.3}$$

dove l'apice indica la derivata rispetto a x. Con $m = -1$ e $m = +1$:

$$f_{i-1} = f_i - h f_i' + \tfrac{1}{2}h^2 f_i'' - \tfrac{1}{6}h^3 f_i''' + \tfrac{1}{24}h^4 f_i^{\mathrm{iv}} - \cdots \tag{2.4}$$

$$f_{i+1} = f_i + h f_i' + \tfrac{1}{2}h^2 f_i'' + \tfrac{1}{6}h^3 f_i''' + \tfrac{1}{24}h^4 f_i^{\mathrm{iv}} + \cdots \tag{2.5}$$

Le formule fondamentali e il loro errore di troncamento

Differenza in avanti. Dalla (2.5), $f_i' = (f_{i+1} - f_i)/h - \frac{1}{2}h f_i'' - \cdots$; trascurando i termini in h:

$$f_i' \simeq \frac{f_{i+1} - f_i}{h}, \qquad \text{errore } \mathcal{O}(hf'') . \tag{2.6}$$

Differenza all'indietro. Dalla (2.4), con un errore dello stesso ordine ma di segno opposto:

$$f_i' \simeq \frac{f_i - f_{i-1}}{h}, \qquad \text{errore } \mathcal{O}(hf'') . \tag{2.7}$$

Differenza centrata. Sottraendo la (2.4) dalla (2.5) i termini pari si cancellano: $f_{i+1} - f_{i-1} = 2hf_i' + \frac{1}{3}h^3 f_i''' + \cdots$, quindi

$$f_i' \simeq \frac{f_{i+1} - f_{i-1}}{2h}, \qquad \text{errore } -\tfrac{1}{6}h^2 f_i''' = \mathcal{O}(h^2 f''') . \tag{2.8}$$

Derivata seconda centrata. Sommando le due serie si cancellano i termini dispari: $f_{i+1} + f_{i-1} = 2f_i + h^2 f_i'' + \frac{1}{12}h^4 f_i^{\mathrm{iv}} + \cdots$, quindi

$$f_i'' \simeq \frac{f_{i+1} - 2f_i + f_{i-1}}{h^2}, \qquad \text{errore } -\tfrac{1}{12}h^2 f_i^{\mathrm{iv}} = \mathcal{O}(h^2 f^{\mathrm{iv}}) . \tag{2.9}$$

Le formule centrate sono *del secondo ordine*: dimezzando il passo l'errore si riduce a un quarto. Formule di accuratezza qualsiasi si ottengono assegnando a m altri valori nella (2.3) e combinando gli sviluppi in modo da eliminare i termini indesiderati.

Derivare dati di galleria del vento

Un profilo di velocità misurato con un rastrello di tubi di Pitot a passo $h = 2$ mm, o la distribuzione di pressione letta su una fila di prese statiche, si deriva proprio con queste formule: nei punti interni con la (2.8), agli estremi con una formula non centrata. Il confronto fra le formule (2.6) e (2.8) è istruttivo anche sperimentalmente: con dati affetti da rumore di misura ε, l'errore dovuto al rumore cresce come ε/h, mentre quello di troncamento diminuisce con h; esiste quindi un passo ottimo, e infittire le prese oltre un certo limite non migliora la stima del gradiente.

2.3 L'equazione alle differenze e il sistema tridiagonale

Torniamo all'equazione (2.1). Per avere errori dello stesso ordine nelle due derivate si usano entrambe le formule centrate (2.8) e (2.9). Nel nodo x_i:

$$\frac{f_{i+1} - 2f_i + f_{i-1}}{h^2} + A_i \frac{f_{i+1} - f_{i-1}}{2h} + B_i f_i = D_i,$$

dove A_i, B_i, D_i indicano $A(x_i)$, $B(x_i)$, $D(x_i)$. Moltiplicando per h^2 e raccogliendo:

$$C_{i1} f_{i-1} + C_{i2} f_i + C_{i3} f_{i+1} = C_{i4}, \tag{2.10}$$

$$C_{i1} = 1 - \frac{h}{2} A_i, \qquad C_{i2} = -2 + h^2 B_i, \qquad C_{i3} = 1 + \frac{h}{2} A_i, \qquad C_{i4} = h^2 D_i. \tag{2.11}$$

I coefficienti sono costanti note in ogni nodo interno. Scritta per $i = 1, 2, \ldots, n$, la (2.10) fornisce n equazioni algebriche lineari nelle n incognite f_i. Nella prima e nell'ultima compaiono i valori al contorno f_0 e f_{n+1}, che sono noti: i termini corrispondenti passano a secondo membro,

$$C_{14} \leftarrow C_{14} - C_{11} f_0, \qquad C_{n4} \leftarrow C_{n4} - C_{n3} f_{n+1}, \tag{2.12}$$

dopo di che C_{11} e C_{n3} si possono porre uguali a zero. Il sistema assume la forma matriciale

$$\begin{bmatrix} C_{12} & C_{13} & & & & \\ C_{21} & C_{22} & C_{23} & & & \\ & C_{31} & C_{32} & C_{33} & & \\ & & \ddots & \ddots & \ddots & \\ & & C_{n-1,1} & C_{n-1,2} & C_{n-1,3} & \\ & & & C_{n1} & C_{n2} & \end{bmatrix} \begin{pmatrix} f_1 \\ f_2 \\ f_3 \\ \vdots \\ f_{n-1} \\ f_n \end{pmatrix} = \begin{pmatrix} C_{14} \\ C_{24} \\ C_{34} \\ \vdots \\ C_{n-1,4} \\ C_{n4} \end{pmatrix}. \tag{2.13}$$

La matrice dei coefficienti è **tridiagonale**: i suoi elementi non nulli formano una banda larga tre elementi lungo la diagonale principale. È una conseguenza diretta della formula a tre punti: ogni equazione lega un nodo solo ai suoi due vicini.

2.4 L'eliminazione di Gauss per sistemi tridiagonali

Un sistema tridiagonale si risolve con l'eliminazione di Gauss, sfruttando il fatto che sotto la diagonale c'è un solo elemento non nullo per riga. Si moltiplica la seconda equazione per C_{12}, la prima per C_{21} e si sottrae, eliminando f_1:

$$(C_{22}C_{12} - C_{21}C_{13})f_2 + C_{23}C_{12}f_3 = C_{24}C_{12} - C_{21}C_{14}.$$

Se si chiamano di nuovo C_{22}, C_{23}, C_{24} i coefficienti di questa equazione e la si sostituisce alla seconda, il sistema conserva la forma (2.13) con C_{21} sostituito da zero. Lavorando con la nuova seconda equazione e con la terza si elimina C_{31}, e così via fino a $C_{n-1,1}$. In sintesi, l'eliminazione consiste nelle riassegnazioni

$$\left.\begin{aligned} C_{i2} &\leftarrow C_{i2}C_{i-1,2} - C_{i1}C_{i-1,3} \\ C_{i3} &\leftarrow C_{i3}C_{i-1,2} \\ C_{i4} &\leftarrow C_{i4}C_{i-1,2} - C_{i1}C_{i-1,4} \end{aligned}\right\} \qquad i = 2, 3, \ldots, n-1. \tag{2.14}$$

A questo punto tutte le equazioni, tranne l'ultima, hanno due soli termini a primo membro. Le ultime due si risolvono insieme:

$$f_n = \frac{C_{n4}C_{n-1,2} - C_{n1}C_{n-1,4}}{C_{n2}C_{n-1,2} - C_{n1}C_{n-1,3}}, \tag{2.15}$$

e le altre incognite si ottengono all'indietro con la formula ricorrente

$$f_j = \frac{C_{j4} - C_{j3}f_{j+1}}{C_{j2}}, \qquad j = n-1, n-2, \ldots, 1. \tag{2.16}$$

L'algoritmo di Thomas e il costo del calcolo

Le (2.14) non dividono per il pivot: i coefficienti vengono moltiplicati riga dopo riga e il loro valore assoluto cresce come il prodotto dei pivot precedenti (circa 2^i per l'equazione (2.1)). Per qualche decina di nodi non è un problema; per migliaia di nodi si rischia il traboccamento (*overflow*) della rappresentazione in virgola mobile. La variante usata in pratica, l'**algoritmo di Thomas**, normalizza ogni riga dividendola per il pivot. Scritto il sistema come $a_i x_{i-1} + b_i x_i + c_i x_{i+1} = d_i$, si calcolano in avanti

$$c'_1 = \frac{c_1}{b_1}, \quad d'_1 = \frac{d_1}{b_1}; \qquad c'_i = \frac{c_i}{b_i - a_i c'_{i-1}}, \quad d'_i = \frac{d_i - a_i d'_{i-1}}{b_i - a_i c'_{i-1}}, \quad i = 2, \ldots, n,$$

e all'indietro $x_n = d'_n$, $x_i = d'_i - c'_i x_{i+1}$. Le operazioni sono circa $8n$, contro le circa $\frac{2}{3}n^3$ dell'eliminazione di Gauss su una matrice piena: per $n = 1000$, ottomila operazioni invece di settecento milioni. Il procedimento è stabile, senza bisogno di scambiare le righe, quando la matrice è *diagonalmente dominante*, $|b_i| \geq |a_i| + |c_i|$: è il caso dell'equazione (2.1) con $B \leq 0$ e $h|A|/2 \leq 1$, perché allora $|C_{i2}| = 2 - h^2 B_i \geq C_{i1} + C_{i3} = 2$.

Le due procedure sono raccolte nel modulo comune `metodi_numerici.py` (Listato A.1, in appendice); ecco le due funzioni.

```python
def trid(C):
    C = np.array(C, dtype=float)          # copia di lavoro
    n = C.shape[0]
    for i in range(1, n - 1):             # eliminazione di C[i,0], i = 1..n-2
        c2 = C[i, 1] * C[i - 1, 1] - C[i, 0] * C[i - 1, 2]
        c3 = C[i, 2] * C[i - 1, 1]
        c4 = C[i, 3] * C[i - 1, 1] - C[i, 0] * C[i - 1, 3]
        C[i, 1], C[i, 2], C[i, 3] = c2, c3, c4
    f = np.zeros(n)
    f[n - 1] = ((C[n - 1, 3] * C[n - 2, 1] - C[n - 1, 0] * C[n - 2, 3]) /
                (C[n - 1, 1] * C[n - 2, 1] - C[n - 1, 0] * C[n - 2, 2]))
    for j in range(n - 2, -1, -1):        # sostituzione all'indietro
        f[j] = (C[j, 3] - C[j, 2] * f[j + 1]) / C[j, 1]
    return f

def thomas(a, b, c, d):
    n = len(d)
    cp = np.zeros(n); dp = np.zeros(n)
    cp[0] = c[0] / b[0]; dp[0] = d[0] / b[0]
    for i in range(1, n):                 # eliminazione in avanti
        den = b[i] - a[i] * cp[i - 1]
        cp[i] = c[i] / den if i < n - 1 else 0.0
        dp[i] = (d[i] - a[i] * dp[i - 1]) / den
    x = np.zeros(n); x[-1] = dp[-1]
    for i in range(n - 2, -1, -1):        # sostituzione all'indietro
        x[i] = dp[i] - cp[i] * x[i + 1]
    return x
```

metodi_numerici.py

In Python l'indice parte da zero: la riga `C[i]` contiene i quattro coefficienti $C_{i+1,1}\dots C_{i+1,4}$ del testo.

Riepilogo del metodo

1. Si scrive l'equazione nella forma (2.1) e si identificano $A(x)$, $B(x)$, $D(x)$.
2. Si calcolano i coefficienti (2.11) in ogni nodo interno.
3. Si trasferiscono a secondo membro i valori al contorno con le (2.12).
4. Si risolve il sistema tridiagonale con le (2.14)–(2.16) (funzione `trid`) o con l'algoritmo di Thomas.
5. Se serve la derivata f', la si calcola con la differenza centrata (2.8), usando i valori al contorno nei nodi 1 e n.

2.5 Il flusso radiale prodotto da sorgenti distribuite

Applichiamo il metodo a un problema di cui si conosce la soluzione esatta, così da misurarne l'accuratezza. Si consideri un flusso radiale, a simmetria cilindrica, nella corona $r_0 \le r \le 4r_0$, al cui interno è distribuita una densità di sorgenti che cresce linearmente con r, $q = q_0\, r/r_0$. Per simmetria ϕ dipende solo da r e l'equazione di Poisson (1.15) in coordinate polari diventa

$$\left(\frac{\mathrm{d}^2}{\mathrm{d}r^2} + \frac{1}{r}\frac{\mathrm{d}}{\mathrm{d}r}\right)\phi = q_0\frac{r}{r_0}, \qquad u_r = \frac{\mathrm{d}\phi}{\mathrm{d}r}. \tag{2.17}$$

Conviene lavorare in forma adimensionale. Con una velocità caratteristica u_0 si pone

$$R = \frac{r}{r_0}, \qquad \Phi = \frac{\phi}{r_0 u_0}, \qquad U = \frac{u_r}{u_0}, \tag{2.18}$$

e nel dominio $1 \le R \le 4$ si ottiene

$$\left(\frac{\mathrm{d}^2}{\mathrm{d}R^2} + \frac{1}{R}\frac{\mathrm{d}}{\mathrm{d}R}\right)\Phi = \alpha R, \qquad U = \frac{\mathrm{d}\Phi}{\mathrm{d}R}, \tag{2.19}$$

con $\alpha = q_0 r_0 / u_0$ intensità adimensionale delle sorgenti.

La soluzione esatta

L'operatore a primo membro è $\frac{1}{R}\frac{\mathrm{d}}{\mathrm{d}R}\left(R\frac{\mathrm{d}\Phi}{\mathrm{d}R}\right)$. Integrando due volte:

$$\frac{1}{R}\frac{\mathrm{d}}{\mathrm{d}R}\left(R\Phi'\right) = \alpha R \;\Rightarrow\; R\Phi' = \tfrac{1}{3}\alpha R^3 + \beta \;\Rightarrow\; \Phi = \tfrac{1}{9}\alpha R^3 + \beta \ln R + \gamma.$$

Il primo termine è la soluzione particolare dovuta alle sorgenti distribuite; i due termini della soluzione omogenea rappresentano una *sorgente lineare* nell'origine, di intensità proporzionale a β, e una costante, che non influisce sulla velocità e si può porre uguale a zero. Con $\alpha = 9$ e $\beta = -24$ (una distribuzione di sorgenti più un pozzo nell'origine):

$$\Phi = R^3 - 24\ln R, \qquad U = 3R^2 - \frac{24}{R}, \tag{2.20}$$

soluzione esatta dell'equazione $\Phi'' + \Phi'/R = 9R$ con le condizioni al contorno

$$\Phi(1) = 1, \qquad \Phi(4) = 64 - 24\ln 4. \tag{2.21}$$

La velocità si annulla in $R = 2$: a sinistra il pozzo prevale e il fluido va verso l'interno, a destra prevalgono le sorgenti distribuite.

Il confronto con la forma generale (2.1) dà

$$A(R) = \frac{1}{R}, \qquad B(R) = 0, \qquad D(R) = 9R. \tag{2.22}$$

Il Programma 2.1 (Listato 2.1) divide l'intervallo $1 \le R \le 4$ in 30 parti ($h = 0{,}1$), calcola Φ nei 29 nodi interni e ne ricava la velocità con la differenza centrata $U_i = (\Phi_{i+1} - \Phi_{i-1})/(2h)$.

Tabella 2.1: Programma 2.1: confronto fra soluzione numerica (30 intervalli) ed esatta in alcuni nodi.

R	Φ numerico	Φ esatto	U numerica	U esatta
1,10	−0,954 30	−0,956 44	−18,219 79	−18,188 18
1,50	−6,349 59	−6,356 16	−9,257 86	−9,250 00
2,00	−8,627 73	−8,635 53	−0,000 05	0,000 00
2,50	−6,358 95	−6,365 98	9,152 12	9,150 00
3,00	0,638 53	0,633 31	19,002 74	19,000 00
3,50	12,811 51	12,808 69	29,895 71	29,892 86
3,90	26,656 15	26,655 56	39,478 96	39,476 15

Risultati

- L'errore massimo è $7{,}8 \cdot 10^{-3}$ su Φ e $3{,}2 \cdot 10^{-2}$ su U (Tab. 2.1 e Fig. 2.2). L'errore relativo su U resta sotto lo 0,2%; quello su Φ è inferiore all'1% quasi ovunque, ma

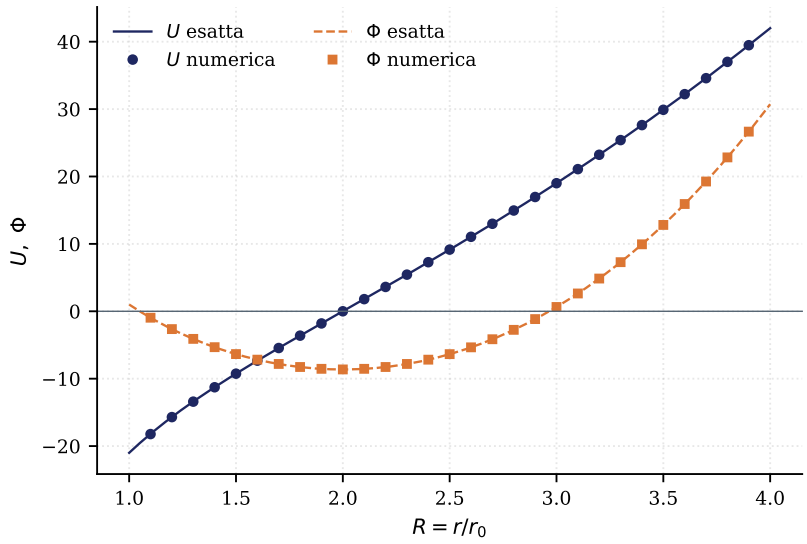


Figura 2.2: Programma 2.1: velocità radiale U e potenziale Φ numerici (simboli) ed esatti (linee).

sale allo 0,8% vicino a $R = 3$, dove Φ stesso passa per lo zero.

- Lo studio di convergenza (Fig. 2.3 e tabella seguente) conferma che il metodo è del secondo ordine: dimezzando h l'errore su Φ si riduce esattamente di un fattore 4.

intervalli	h	$\max \lvert e_\Phi \rvert$	$\max \lvert e_U \rvert$	ordine Φ	ordine U
15	0,200 00	$3{,}110 \times 10^{-2}$	$8{,}970 \times 10^{-2}$	–	–
30	0,100 00	$7{,}803 \times 10^{-3}$	$3{,}160 \times 10^{-2}$	1,99	1,50
60	0,050 00	$1{,}953 \times 10^{-3}$	$9{,}420 \times 10^{-3}$	2,00	1,75
120	0,025 00	$4{,}882 \times 10^{-4}$	$2{,}575 \times 10^{-3}$	2,00	1,87
240	0,012 50	$1{,}221 \times 10^{-4}$	$6{,}736 \times 10^{-4}$	2,00	1,93
480	0,006 25	$3{,}052 \times 10^{-5}$	$1{,}723 \times 10^{-4}$	2,00	1,97
960	0,003 13	$7{,}630 \times 10^{-6}$	$4{,}356 \times 10^{-5}$	2,00	1,98

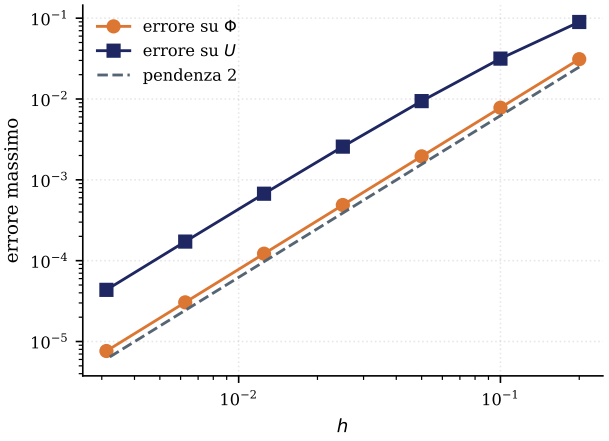


Figura 2.3: Errore massimo su Φ e su U al variare del passo h.

Due osservazioni sull'errore

1. **L'errore relativo può ingannare.** Vicino a uno zero della soluzione l'errore relativo diventa grande anche se quello assoluto è piccolo e uniforme: per giudicare un calcolo si guarda l'errore assoluto riferito alla scala della grandezza (qui, ad esempio, $\max|\Phi| \simeq 26$).
2. **L'ordine della velocità converge più lentamente.** L'ordine osservato su U parte da 1,5 e tende a 2 solo per h piccolo: la soluzione varia rapidamente vicino a $R = 1$ (il pozzo nell'origine), e le derivate terze che compaiono nell'errore della (2.8) sono grandi. Il comportamento asintotico, quello previsto dall'analisi di Taylor, si manifesta solo quando il passo è abbastanza piccolo da "risolvere" la scala di variazione della soluzione.

prog2_1_flusso_radiale.py

```
"""
prog2_1_flusso_radiale.py -- PROGRAMMA 2.1
Problema ai limiti per un'equazione differenziale ordinaria del secondo ordine:
flusso radiale prodotto da una distribuzione di sorgenti e da una sorgente
lineare nell'origine.

    Phi'' + Phi'/R = 9 R ,   1 <= R <= 4
    Phi(1) = 1 ,  Phi(4) = 64 - 24 ln 4
    soluzione esatta:  Phi = R^3 - 24 ln R ,  U = Phi' = 3 R^2 - 24/R

Differenze centrate del secondo ordine + sistema tridiagonale (funzione trid).
"""
import numpy as np
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, GRIGIO, ROSSO
from metodi_numerici import trid, thomas

# ----- coefficienti dell'equazione nella forma f'' + A f' + B f = D ---------
A = lambda R: 1.0 / R
B = lambda R: 0.0 * R
D = lambda R: 9.0 * R
phi_esatta = lambda R: R**3 - 24.0 * np.log(R)
u_esatta = lambda R: 3.0 * R**2 - 24.0 / R

RMIN, RMAX = 1.0, 4.0
PHI0, PHINP1 = phi_esatta(RMIN), phi_esatta(RMAX)     # = 1 e 64 - 24 ln 4


def risolvi(intervalli, solutore="trid"):
    """Restituisce R, Phi, U nei punti interni per un numero dato di intervalli."""
    n = intervalli - 1                    # numero di incognite
    h = (RMAX - RMIN) / intervalli
    R = RMIN + h * np.arange(1, n + 1)
    C = np.zeros((n, 4))
    C[:, 0] = 1.0 - 0.5 * h * A(R)        # coefficiente di f_{i-1}
    C[:, 1] = -2.0 + h * h * B(R)         # coefficiente di f_i
    C[:, 2] = 1.0 + 0.5 * h * A(R)        # coefficiente di f_{i+1}
    C[:, 3] = h * h * D(R)                # termine noto
    # condizioni al contorno: i termini noti passano a secondo membro
    C[0, 3] -= C[0, 0] * PHI0
    C[0, 0] = 0.0
    C[-1, 3] -= C[-1, 2] * PHINP1
    C[-1, 2] = 0.0
    if solutore == "trid":
        phi = trid(C)
```

```
    else:
        phi = thomas(C[:, 0], C[:, 1], C[:, 2], C[:, 3])
    # velocita' radiale con la differenza centrata
    ff = np.concatenate([[PHI0], phi, [PHINP1]])
    U = (ff[2:] - ff[:-2]) / (2.0 * h)
    return R, phi, U


if __name__ == "__main__":
    R, phi, U = risolvi(30)
    print("   I     R       PHI        PHI esatto    err.%      U          U esatto")
    for i in range(len(R)):
        ep = phi_esatta(R[i])
        print(f"  {i+1:2d}  {R[i]:4.2f}  {phi[i]:10.5f}  {ep:10.5f}  "
              f"{100*abs(phi[i]-ep)/abs(ep):6.3f}  {U[i]:10.5f}  {u_esatta(R[i]):10.5f}")

    # ----- grafico della soluzione -----------------------------------------
    Rf = np.linspace(RMIN, RMAX, 300)
    fig, ax = plt.subplots(figsize=(5.4, 3.6))
    ax.plot(Rf, u_esatta(Rf), color=BLU, lw=1.0, label="$U$ esatta")
    ax.plot(R, U, "o", color=BLU, ms=3.5, label="$U$ numerica")
    ax.plot(Rf, phi_esatta(Rf), "--", color=ARANCIO, lw=1.0, label=r"$\Phi$ esatta")
    ax.plot(R, phi, "s", color=ARANCIO, ms=3.2, label=r"$\Phi$ numerica")
    ax.axhline(0, color=GRIGIO, lw=0.6)
    ax.set_xlabel("$R = r/r_0$"); ax.set_ylabel(r"$U,\ \Phi$")
    ax.legend(ncol=2)
    salva(fig, "prog2_1_flusso_radiale")

    # ----- studio di convergenza (algoritmo di Thomas) ---------------------
    print("\n intervalli     h        max|err Phi|    max|err U|   ordine Phi  ordine U")
    NN = [15, 30, 60, 120, 240, 480, 960]
    hh, ep, eu = [], [], []
    for nint in NN:
        R, phi, U = risolvi(nint, solutore="thomas")
        hh.append(3.0 / nint)
        ep.append(np.max(np.abs(phi - phi_esatta(R))))
        eu.append(np.max(np.abs(U - u_esatta(R))))
    for k in range(len(NN)):
        op = np.log2(ep[k-1] / ep[k]) if k else np.nan
        ou = np.log2(eu[k-1] / eu[k]) if k else np.nan
        print(f"    {NN[k]:5d}   {hh[k]:8.5f}   {ep[k]:.4e}    {eu[k]:.4e}    {op:5.2f}     {ou:5.2f}")

    fig, ax = plt.subplots(figsize=(4.6, 3.3))
    hh = np.array(hh)
    ax.loglog(hh, ep, "o-", color=ARANCIO, label=r"errore su $\Phi$")
    ax.loglog(hh, eu, "s-", color=BLU, label="errore su $U$")
    ax.loglog(hh, 0.8 * ep[0] * (hh / hh[0])**2, "--", color=GRIGIO, label="pendenza 2")
    ax.set_xlabel("$h$"); ax.set_ylabel("errore massimo")
    ax.legend()
    salva(fig, "prog2_1_convergenza")

    # ----- confronto trid / thomas e crescita dei coefficienti -------------
    R, p1, _ = risolvi(30, "trid")
    R, p2, _ = risolvi(30, "thomas")
    print(f"\n differenza massima trid/thomas (30 intervalli):
```

```
    {np.max(np.abs(p1-p2)):.2e}")
```

Listato 2.1: Programma 2.1 – flusso radiale da sorgenti distribuite (`prog2_1_flusso_radiale.py`).

2.6 Condizioni al contorno sulla derivata

Spesso a un estremo non è noto il valore di f ma quello della sua derivata, ad esempio $f'(x_{\max}) = g$: una parete impermeabile per il potenziale, una sezione di uscita in cui la corrente è parallela all'asse per la funzione di corrente, l'estremo aperto di un tubo per la velocità di un'onda acustica. In questo caso f_{n+1} diventa un'incognita e l'equazione (2.10) va scritta anche nel nodo di contorno $i = n + 1$, dove però coinvolge il nodo f_{n+2}, che sta *fuori* dal dominio. La tecnica classica introduce un **nodo fittizio** (*ghost node*) e ne ricava il valore dalla condizione al contorno, approssimata con la differenza centrata:

$$\frac{f_{n+2} - f_n}{2h} = g \quad \Longrightarrow \quad f_{n+2} = f_n + 2hg. \tag{2.23}$$

Sostituendo nell'equazione del nodo $n + 1$, il nodo fittizio scompare e il sistema resta tridiagonale, con un'equazione in più. L'accuratezza complessiva rimane del secondo ordine, perché anche la condizione al contorno è approssimata con un errore $\mathcal{O}(h^2)$. Useremo questa tecnica nel Cap. 7 per la sezione di uscita di un canale, nel Cap. 8 per l'estremo aperto di un tubo e negli esercizi 10–12.

Capitolo 3

Il metodo inverso: sovrapposizione di flussi elementari

3.1 Metodo diretto e metodo inverso

Nel problema del flusso uniforme attorno a un corpo di forma assegnata bisogna trovare una soluzione dell'equazione di Laplace che soddisfi la condizione di corrente uniforme all'infinito e la condizione di tangenza sulla superficie del corpo. Risolvere il problema in questo modo, *direttamente*, non è semplice, nonostante la linearità dell'equazione: un metodo numerico diretto sarà introdotto nel Cap. 6. Qui seguiamo la strada opposta, il **metodo inverso**.

L'equazione di Laplace per la funzione di corrente ha una forma così semplice che se ne trovano facilmente alcune soluzioni elementari, ciascuna delle quali rappresenta una corrente fisicamente possibile. Poiché una combinazione lineare di soluzioni di un'equazione lineare è ancora una soluzione, se ne possono generare infinite altre. Alcune di queste combinazioni simulano il flusso attorno a corpi di varie forme. Nel metodo inverso, invece di cercare il flusso attorno a un corpo prescritto, si costruiscono e si catalogano i flussi attorno a corpi diversi ottenuti da combinazioni di flussi elementari; da questo "catalogo" si sceglie poi la combinazione che produce un corpo vicino alla forma desiderata. La stessa idea si usa in elettromagnetismo e nella conduzione del calore, governati dalle stesse equazioni di Laplace e di Poisson.

3.2 I quattro flussi elementari

I flussi elementari bidimensionali più comuni sono la corrente uniforme, la sorgente (o il pozzo), il vortice e la doppietta. Le loro funzioni di corrente, con il centro della singolarità nel punto (x_0, y_0), r e θ coordinate polari rispetto a quel punto, sono:

(a) corrente uniforme
$$\psi = U\,(y\cos\alpha - x\sin\alpha), \tag{3.1}$$

(b) sorgente lineare
$$\psi = \frac{\sigma}{2\pi}\arctan\frac{y-y_0}{x-x_0} = \frac{\sigma}{2\pi}\,\theta, \tag{3.2}$$

(c) vortice lineare
$$\psi = \frac{\Gamma}{2\pi}\ln\sqrt{(x-x_0)^2+(y-y_0)^2} = \frac{\Gamma}{2\pi}\ln r, \tag{3.3}$$

(d) doppietta
$$\psi = -\frac{\kappa}{2\pi}\,\frac{y-y_0}{(x-x_0)^2+(y-y_0)^2} = -\frac{\kappa}{2\pi}\,\frac{\sin\theta}{r}. \tag{3.4}$$

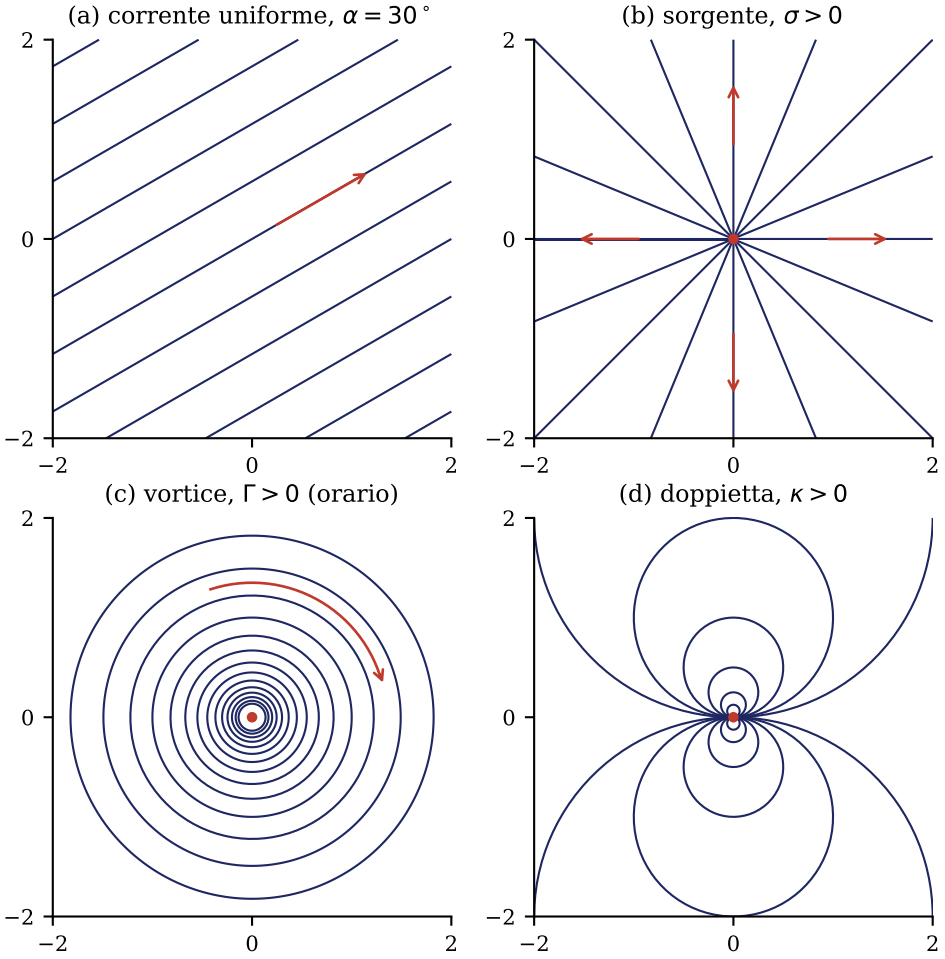


Figura 3.1: Linee di corrente dei quattro flussi elementari (`prog2_2b_flussi_elementari.py`).

Proprietà dei flussi elementari

Corrente uniforme. Dalla (1.11): $u = U\cos\alpha$, $v = U\sin\alpha$. È una corrente di velocità U inclinata di α sull'asse x; per $\alpha = 0$ è orizzontale.

Sorgente. Dalla (1.13): $u_r = \sigma/(2\pi r)$, $u_\theta = 0$. Il fluido esce radialmente; la portata attraverso una circonferenza qualsiasi è $2\pi r\, u_r = \sigma$, il volume emesso nell'unità di tempo da un tratto unitario della sorgente lineare. $\sigma < 0$ è un pozzo. La velocità tende all'infinito per $r \to 0$.

Vortice. $u_r = 0$, $u_\theta = -\Gamma/(2\pi r)$. Le linee di corrente sono circonferenze; il prodotto della lunghezza di una linea per la velocità lungo di essa è costante e uguale alla *circolazione* Γ, che misura l'intensità del vortice. Con la convenzione dell'aerodinamica, adottata in tutta la monografia, la circolazione *oraria* è positiva (il segno meno in u_θ dice che, con $\Gamma > 0$, il fluido ruota in senso orario).

Doppietta. Una sorgente σ in $(x_0 - \Delta x, y_0)$ e un pozzo $-\sigma$ in $(x_0 + \Delta x, y_0)$ si avvicinano mantenendo costante il prodotto $2\sigma\Delta x = \kappa$. Detto $\theta(\xi) = \arctan[(y - y_0)/(x - \xi)]$ l'angolo sotto cui il punto (x, y) è visto da un punto ξ dell'asse, si ha $\partial\theta/\partial\xi =$

$(y - y_0)/r^2$ e quindi

$$\psi = \frac{\sigma}{2\pi}\big[\theta(x_0 - \Delta x) - \theta(x_0 + \Delta x)\big] \simeq -\frac{\sigma}{2\pi}\, 2\Delta x\, \frac{\partial\theta}{\partial\xi} = -\frac{\kappa}{2\pi}\frac{y - y_0}{(x - x_0)^2 + (y - y_0)^2},$$

che è la (3.4). Le linee di corrente sono circonferenze passanti per il centro con il centro sull'asse $x = x_0$. Lo stesso campo si ottiene con due vortici di segno opposto che si avvicinano lungo l'asse y. κ è l'intensità della doppietta.

Sorgente, vortice e doppietta hanno in comune la velocità infinita nel centro: per questo si chiamano **singolarità**. Non creano difficoltà matematiche se ogni punto singolare cade in una regione che si considera occupata dal corpo: il flusso esterno al corpo ne è libero.

3.3 Corpi generati per sovrapposizione

In generale le sorgenti e i pozzi generano corpi di estensione infinita, come il semicorpo di Rankine del §1.7; le doppiette, o gruppi di sorgenti e pozzi di intensità totale nulla, generano corpi chiusi. L'esempio più importante è il **cilindro circolare**: alla corrente uniforme orizzontale si aggiunge una doppietta nell'origine,

$$\psi = Uy - \frac{\kappa}{2\pi}\frac{y}{x^2 + y^2} = Ur\sin\theta\left(1 - \frac{a^2}{r^2}\right), \qquad a = \sqrt{\frac{\kappa}{2\pi U}}. \tag{3.5}$$

La linea $\psi = 0$ comprende l'asse x e la circonferenza $r = a$: la circonferenza è una linea di corrente e può essere sostituita da una parete solida. Sulla superficie $u_r = 0$ e

$$u_\theta = -\frac{\partial\psi}{\partial r}\Big|_{r=a} = -2U\sin\theta, \qquad C_p = 1 - 4\sin^2\theta. \tag{3.6}$$

Il cilindro con circolazione e il teorema di Kutta–Joukowski

Aggiungendo un vortice di circolazione Γ nell'origine la circonferenza resta una linea di corrente (il vortice ha linee di corrente circolari):

$$\psi = Ur\sin\theta\left(1 - \frac{a^2}{r^2}\right) + \frac{\Gamma}{2\pi}\ln\frac{r}{a}, \qquad u_\theta\big|_{r=a} = -2U\sin\theta - \frac{\Gamma}{2\pi a}. \tag{3.7}$$

Il termine $\ln(r/a)$ invece di $\ln r$ aggiunge solo una costante e mantiene $\psi = 0$ sulla superficie. I punti di ristagno si spostano sulla parte inferiore del cilindro, dove $\sin\theta_s = -\Gamma/(4\pi U a)$; per $\Gamma > 4\pi U a$ lasciano la superficie e si fondono in un unico ristagno libero nel fluido (Fig. 3.2). Con la circolazione oraria la velocità è maggiore sul dorso, la pressione minore, e il cilindro è spinto verso l'alto.

Il programma `prog2_2b_flussi_elementari.py` (Listato A.3 in appendice) integra numericamente la pressione C_p sulla superficie e calcola i coefficienti di forza riferiti al diametro $2a$, $C_D = -\frac{1}{2}\int_0^{2\pi} C_p\cos\theta\,\mathrm{d}\theta$ e $C_L = -\frac{1}{2}\int_0^{2\pi} C_p\sin\theta\,\mathrm{d}\theta$.

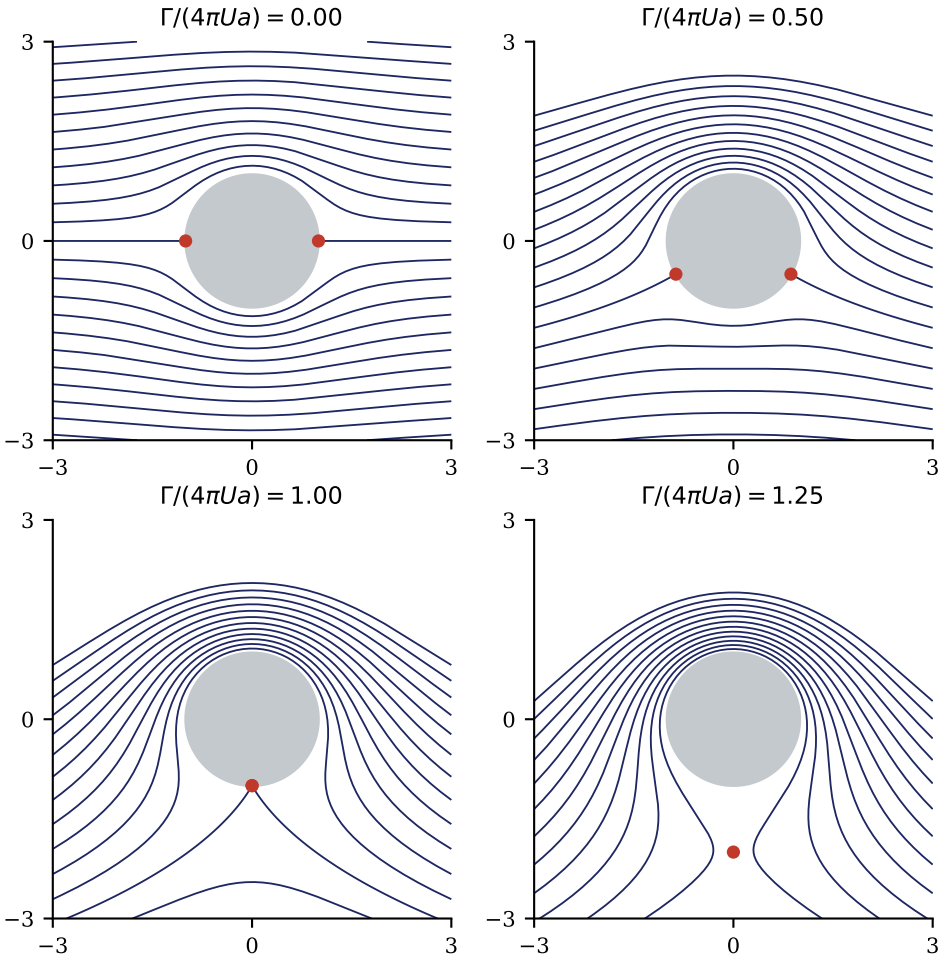


Figura 3.2: Cilindro circolare con circolazione crescente. I punti rossi sono i ristagni.

Risultato: la portanza è quella di Kutta–Joukowski, la resistenza è nulla

$\Gamma/(Ua)$	$\sin\theta_s$	C_D (integrale)	C_L (integrale)	$\Gamma/(Ua)$
0,0000	0,000	$-5{,}6\times10^{-17}$	0,000 00	0,000 00
6,2832	0,500	0,0	6,283 19	6,283 19
12,5664	1,000	$3{,}1\times10^{-16}$	12,566 37	12,566 37
15,7080	1,250	$-4{,}4\times10^{-16}$	15,707 96	15,707 96

La portanza per unità di apertura vale $L' = \rho U\Gamma$ (**teorema di Kutta–Joukowski**), cioè $C_L = L'/(\frac{1}{2}\rho U^2\, 2a) = \Gamma/(Ua)$, e la resistenza è nulla a meno degli errori di arrotondamento.

Il paradosso di d'Alembert

Un corpo che si muove di moto uniforme in un fluido ideale non subisce resistenza: la distribuzione di pressione (3.6) è simmetrica fra la metà anteriore e quella posteriore. Il risultato, in netto contrasto con l'esperienza, non è un errore della teoria: è la conseguenza dell'avere eliminato la viscosità, e quindi lo strato limite e la scia. La teoria non viscosa serve per la portanza e per la distribuzione di pressione dei corpi affusolati, non per la resistenza (§1.1).

L'effetto Magnus: dalla palla "tagliata" al rotore Flettner

Un cilindro che ruota in una corrente trascina per viscosità il fluido circostante e genera una circolazione: è l'effetto Magnus, che incurva la traiettoria di una palla da tennis colpita con effetto. Negli anni Venti Anton Flettner lo sfruttò per propellere navi con grandi cilindri rotanti al posto delle vele; oggi i rotori Flettner tornano sulle navi mercantili come ausilio per ridurre i consumi. Il cilindro reale, però, produce molta meno portanza del valore $\rho U \Gamma$ calcolato con la circolazione "teorica" $\Gamma = 2\pi a \cdot \omega a$, perché la corrente si separa: è il limite della teoria non viscosa per corpi non affusolati.

3.4 La ricerca dei punti di una linea di corrente su una griglia

Il metodo di sovrapposizione è semplice in linea di principio; per vedere la forma del corpo e il campo di moto bisogna però tracciare alcune linee di corrente rappresentative di una funzione $\psi = f(x, y)$ assegnata. Lo spazio rettangolare compreso fra $x_{\min}$ e $x_{\max}$ e fra $y_{\min}$ e $y_{\max}$ si suddivide con rette verticali a distanza Δx e orizzontali a distanza Δy (non necessariamente uguali). I nodi della griglia hanno coordinate (x_i, y_j), $i = 1, \ldots, m$, $j = 1, \ldots, n$, e in essi si calcola

$$\psi_{i,j} = f(x_i, y_j). \tag{3.8}$$

Si cercano poi le intersezioni delle verticali della griglia con la linea $\psi = \psi_a$: se le verticali sono abbastanza fitte, i punti trovati descrivono la forma della linea.

Segnaposto figura

file atteso: `immagini/fig_2_4_3.png`

Valori di ψ lungo la verticale x_i rappresentati come segmenti verticali; la linea $\psi = \psi_a$ attraversa l'intervallo in cui le differenze P e Q hanno segno opposto (Fig. 2.4.3 della fonte).

Figura 3.3: Distribuzione della funzione di corrente lungo una verticale della griglia.

Algoritmo di ricerca (subroutine SEARCH)

1. Per ogni verticale x_i e per ogni intervallo $[y_j, y_{j+1}]$ si calcolano le differenze fra la funzione agli estremi e il valore cercato:

$$P = \psi_{i,j} - \psi_a, \qquad Q = \psi_{i,j+1} - \psi_a.$$

2. La linea attraversa l'intervallo se $PQ < 0$ (i due valori stanno da parti opposte di ψ_a).
3. Approssimando ψ con una retta nell'intervallo (Fig. 3.3), l'ordinata dell'intersezione è

$$y = y_{j+1} - \Delta y \, \frac{|Q|}{|P| + |Q|}. \tag{3.9}$$

Infatti, per la similitudine dei triangoli, la distanza del punto di intersezione dall'estremo superiore sta a Δy come $|Q|$ sta a $|P| + |Q|$.

4. Esaminati tutti gli intervalli di una verticale, si passa alla successiva; i punti trovati si numerano con un contatore k.

Per linee quasi parallele all'asse y conviene cercare le intersezioni con le orizzontali: la funzione `cerca_xy` esegue entrambe le ricerche.

```python
def _attraversamenti(s, v, va):
    P = v[:-1] - va
    Q = v[1:] - va
    ok = np.isfinite(P) & np.isfinite(Q) & (P * Q <= 0.0) & ((P != 0) | (Q != 0))
    k = np.nonzero(ok)[0]
    ds = s[k + 1] - s[k]
    return s[k + 1] - ds * np.abs(Q[k]) / (np.abs(P[k]) + np.abs(Q[k]))

def cerca_x(x, y, psi, psia):
    xx, yy = [], []
    for i in range(len(x)):
        ys = _attraversamenti(y, psi[i, :], psia)
        xx.extend([x[i]] * len(ys))
        yy.extend(ys)
    return np.array(xx), np.array(yy)
```

metodi_numerici.py

La versione Python esamina tutti gli intervalli di una verticale in un colpo solo con le operazioni vettoriali di NumPy; il test $PQ \le 0$ con almeno un valore non nullo include i casi in cui la linea passa esattamente per un nodo, e i nodi marcati con `NaN` (esterni al campo di moto) vengono ignorati. Matplotlib offre la funzione `contour`, che fa la stessa cosa in due dimensioni con un algoritmo più raffinato; nelle figure di questo capitolo le due rappresentazioni sono sovrapposte, così da confrontare l'approssimazione dell'interpolazione lineare con le curve "esatte".

3.5 Programma 2.2: corrente uniforme e cinque doppiette

Il Programma 2.2 (Listato 3.1) studia la corrente ottenuta aggiungendo a una corrente uniforme orizzontale cinque doppiette di intensità diverse, poste sull'asse x nei punti $x/l = -1,\ -0{,}5,\ 0,\ 0{,}5,\ 1$, con l lunghezza di riferimento. Dalle (3.1) e (3.4):

$$\psi = Uy - \sum_{i=1}^{5} \frac{\kappa_i}{2\pi} \frac{y}{(x - d_i l)^2 + y^2}, \tag{3.10}$$

e in forma adimensionale, con $\Psi = \psi/(Ul)$, $X = x/l$, $Y = y/l$, $c_i = \kappa_i/(2\pi l^2 U)$:

$$\Psi = Y - \sum_{i=1}^{5} \frac{c_i Y}{(X - d_i)^2 + Y^2}. \tag{3.11}$$

Dati del Programma 2.2

Posizioni $d_i = -1,\ -0{,}5,\ 0,\ 0{,}5,\ 1$; intensità $c_i = 0{,}15,\ 0{,}30,\ 0{,}20,\ 0{,}10,\ 0{,}05$; griglia $-3 \le X \le 3$, $-2 \le Y \le 2$ con $\Delta X = \Delta Y = 0{,}2$; linee cercate $\Psi = 0,\ \pm 0{,}5,\ \pm 1$.

Una doppietta è una singolarità con velocità e funzione di corrente infinite nel centro, e i centri delle doppiette coincidono con cinque nodi della griglia. La difficoltà si evita spostando le doppiette verso destra di una distanza piccolissima, 10^{-6}: l'effetto sul campo di moto è invisibile, ma nessun nodo cade più esattamente su una singolarità.

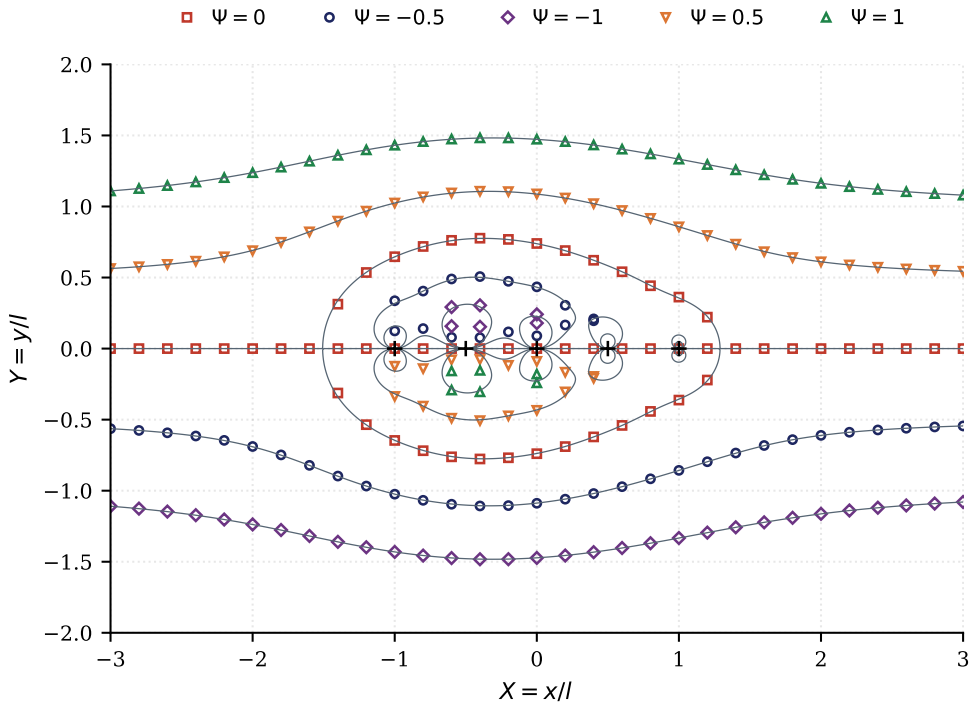


Figura 3.4: Programma 2.2: corrente uniforme e cinque doppiette (croci). Simboli: punti trovati dall'algoritmo di ricerca; linee grigie: curve tracciate su una griglia fine.

La Fig. 3.4 mostra un corpo a forma di uovo che racchiude tutte le singolarità. All'interno compaiono linee chiuse ($\Psi = \pm 0{,}5, \pm 1$) che descrivono un flusso fittizio fra le doppiette: dentro il corpo il campo non ha significato fisico, perché lo spazio è occupato dal solido. Il programma calcola anche gli estremi del corpo, che sono i punti di ristagno sull'asse, e lo spessore, cercando per ogni X la radice $Y > 0$ di $\Psi(X, Y) = 0$ con il metodo di Brent (`scipy.optimize.brentq`). La velocità sul contorno si ottiene derivando analiticamente la (3.11),

$$u = \frac{\partial \Psi}{\partial Y} = 1 - \sum_i c_i \frac{(X - d_i)^2 - Y^2}{[(X - d_i)^2 + Y^2]^2}, \qquad v = -\frac{\partial \Psi}{\partial X} = -\sum_i c_i \frac{2(X - d_i) Y}{[(X - d_i)^2 + Y^2]^2},$$

e la pressione con il teorema di Bernoulli (1.10).

Risultati

- Punti di ristagno in $X = -1{,}507$ e $X = 1{,}289$: lunghezza del corpo $2{,}797\, l$.
- Spessore massimo $1{,}551\, l$ in $X = -0{,}372$, cioè nella zona delle doppiette più intense: rapporto di finezza (lunghezza/spessore) 1,80.
- Coefficiente di pressione minimo $C_p = -1{,}73$ in $X = -0{,}505$ (Fig. 3.5).

Il corpo è più largo dove si trovano le doppiette più forti: questa osservazione è la guida per costruire, con il metodo inverso, corpi simmetrici rispetto all'asse x, ad esempio profili simmetrici a incidenza nulla.

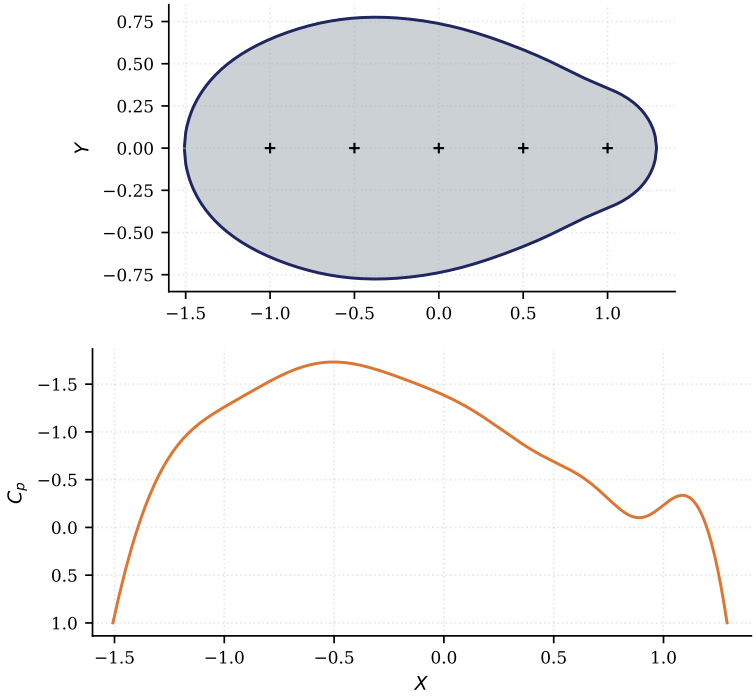


Figura 3.5: Forma del corpo generato dalle cinque doppiette e coefficiente di pressione sul suo contorno.

prog2_2_doppiette.py

```
"""
prog2_2_doppiette.py -- PROGRAMMA 2.2
Metodo inverso di sovrapposizione: corrente uniforme + cinque doppiette
sull'asse x. Si calcola la funzione di corrente adimensionale sui nodi di una
griglia rettangolare, si cercano i punti di cinque linee di corrente lungo le
verticali della griglia (ricerca con interpolazione lineare) e si disegnano.
Si calcola poi il coefficiente di pressione sul corpo (Bernoulli).

  Psi = Y - sum_i c_i Y / ((X - d_i)^2 + Y^2)
"""
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import brentq
from stile_grafici import salva, BLU, ARANCIO, VERDE, VIOLA, ROSSO, GRIGIO
from metodi_numerici import cerca_x

D = np.array([-1.0, -0.5, 0.0, 0.5, 1.0])      # posizioni d_i delle doppiette
C = np.array([0.15, 0.30, 0.20, 0.10, 0.05])   # intensita' adimensionali c_i
EPS = 1e-6        # spostamento delle doppiette: i loro centri non cadono sui nodi


def Psi(X, Y, alfa=0.0, d=D, c=C):
    """Funzione di corrente: corrente uniforme inclinata di alfa + doppiette."""
    p = Y * np.cos(alfa) - X * np.sin(alfa)
    for di, ci in zip(d, c):
        p = p - ci * Y / ((X - di - EPS)**2 + Y**2)
    return p


def velocita(X, Y, d=D, c=C):
    """u = dPsi/dY, v = -dPsi/dX (derivate analitiche, corrente con alfa = 0)."""
    u = np.ones_like(X)
```

```
    v = np.zeros_like(X)
    for di, ci in zip(d, c):
        xr = X - di - EPS
        r2 = xr**2 + Y**2
        u -= ci * (xr**2 - Y**2) / r2**2
        v -= ci * 2.0 * xr * Y / r2**2
    return u, v


# ----- griglia e funzione di corrente nei nodi -----------------------------
DX = DY = 0.2
x = np.arange(-3.0, 3.0 + 1e-9, DX)
y = np.arange(-2.0, 2.0 + 1e-9, DY)
X, Y = np.meshgrid(x, y, indexing="ij")
PSI = Psi(X, Y)

COSTANTI = [0.0, -0.5, -1.0, 0.5, 1.0]
SIMBOLI = ["s", "o", "D", "v", "^"]
COLORI = [ROSSO, BLU, VIOLA, ARANCIO, VERDE]

fig, ax = plt.subplots(figsize=(6.0, 4.0))
for psia, simb, col in zip(COSTANTI, SIMBOLI, COLORI):
    xx, yy = cerca_x(x, y, PSI, psia)
    ax.plot(xx, yy, simb, ms=3.2, color=col, mfc="none", label=fr"$\Psi={psia:g}$")
    if psia == 0.0:
        print(f"Psi = 0: {len(xx)} punti trovati (compresi quelli sull'asse x)")
# confronto: contorno 'esatto' tracciato su una griglia molto fine
xf = np.linspace(-3, 3, 601); yf = np.linspace(-2, 2, 401)
XF, YF = np.meshgrid(xf, yf, indexing="ij")
ax.contour(XF, YF, Psi(XF, YF), levels=sorted(COSTANTI), colors=GRIGIO,
    linewidths=0.5,
           linestyles="solid")
ax.plot(D, 0 * D, "k+", ms=6)
ax.set_aspect("equal"); ax.set_xlim(-3, 3); ax.set_ylim(-2, 2)
ax.set_xlabel("$X = x/l$"); ax.set_ylabel("$Y = y/l$")
ax.legend(loc="upper center", ncol=5, bbox_to_anchor=(0.5, 1.13), handletextpad=0.2)
salva(fig, "prog2_2_doppiette")

# ----- forma del corpo: estremi e spessore ---------------------------------
# sull'asse il corpo inizia e finisce nei punti di ristagno u(X,0) = 0
u_asse = lambda X: velocita(np.array([X]), np.array([0.0]))[0][0]
x_ant = brentq(u_asse, -3.0, -1.01)
x_post = brentq(u_asse, 1.01, 3.0)
# semispessore: per ogni X la radice Y>0 di Psi(X,Y)=0
Xc = np.linspace(x_ant + 1e-4, x_post - 1e-4, 400)
Yc = np.array([brentq(lambda yy: Psi(xc, yy), 1e-6, 2.0) for xc in Xc])
k = np.argmax(Yc)
print(f"punti di ristagno: X = {x_ant:.4f} e X = {x_post:.4f}; "
      f"lunghezza = {x_post - x_ant:.4f} l")
print(f"spessore massimo = {2*Yc[k]:.4f} l in X = {Xc[k]:.3f}; "
      f"rapporto di finezza = {(x_post-x_ant)/(2*Yc[k]):.3f}")

# ----- coefficiente di pressione sul corpo (Bernoulli) ---------------------
u, v = velocita(Xc, Yc)
cp = 1.0 - (u**2 + v**2)
print(f"C_p minimo = {cp.min():.4f} in X = {Xc[np.argmin(cp)]:.3f}")
fig, ax = plt.subplots(2, 1, figsize=(5.6, 5.4),
                       gridspec_kw={"height_ratios": [1.0, 1.0]})
ax[0].fill_between(Xc, -Yc, Yc, color=GRIGIO, alpha=0.3)
ax[0].plot(Xc, Yc, color=BLU); ax[0].plot(Xc, -Yc, color=BLU)
```

```
ax[0].plot(D, 0 * D, "k+", ms=5)
ax[0].set_xlim(-1.6, 1.4); ax[0].set_ylim(-0.85, 0.85)
ax[0].set_aspect("equal"); ax[0].set_ylabel("$Y$")
ax[1].set_xlim(-1.6, 1.4)
ax[1].plot(Xc, cp, color=ARANCIO)
ax[1].invert_yaxis()
ax[1].set_xlabel("$X$"); ax[1].set_ylabel("$C_p$")
salva(fig, "prog2_2_cp_corpo")

# ----- una sola doppietta e corrente inclinata di 30 gradi ------------------
alfa = np.radians(30.0)
PSI30 = Psi(X, Y, alfa, d=[0.0], c=[1.0])
fig, ax = plt.subplots(figsize=(6.0, 4.0))
for psia, simb, col in zip(COSTANTI, SIMBOLI, COLORI):
    xx, yy = cerca_x(x, y, PSI30, psia)
    ax.plot(xx, yy, simb, ms=3.2, color=col, mfc="none", label=fr"$\Psi={psia:g}$")
ax.contour(XF, YF, Psi(XF, YF, alfa, d=[0.0], c=[1.0]), levels=sorted(COSTANTI),
           colors=GRIGIO, linewidths=0.5, linestyles="solid")
ax.set_aspect("equal"); ax.set_xlim(-3, 3); ax.set_ylim(-2, 2)
ax.set_xlabel("$X$"); ax.set_ylabel("$Y$")
ax.legend(loc="upper center", ncol=5, bbox_to_anchor=(0.5, 1.13), handletextpad=0.2)
salva(fig, "prog2_2_doppietta30")
```

Listato 3.1: Programma 2.2 – corrente uniforme e cinque doppiette (`prog2_2_doppiette.py`).

3.6 Quando la sovrapposizione non genera un corpo

La sovrapposizione di flussi elementari non produce sempre il flusso attorno a un corpo rigido. Se nell'esempio del cilindro la corrente uniforme orizzontale viene sostituita da una corrente inclinata di 30° sull'asse x, la doppietta (il cui asse resta orizzontale) e la corrente non sono più "allineate": il campo risultante (Fig. 3.6, ultima parte del Programma 2.2) non contiene alcuna linea di corrente chiusa che racchiuda la doppietta. Per ottenere il cilindro investito da una corrente inclinata bisogna ruotare anche l'asse della doppietta, come si vedrà con il potenziale complesso nel Cap. 5.

3.7 Il metodo delle immagini

Si combinino un vortice di circolazione Γ in $(0, d)$ e un vortice di circolazione $-\Gamma$ in $(0, -d)$. Per simmetria la componente verticale della velocità è nulla sull'asse x, che è quindi una linea di corrente: la metà superiore del campo descrive il flusso di un vortice in presenza di una parete piana. Analogamente, il campo di una sorgente in $(-d, 0)$ vicina a una parete verticale coincidente con l'asse y si ottiene sommando la sorgente e la sua immagine speculare, una sorgente di uguale intensità in $(d, 0)$. È il **metodo delle immagini**: il flusso in presenza di una parete piana si costruisce aggiungendo alle singolarità reali le loro immagini speculari rispetto alla parete (con lo stesso segno per sorgenti e doppiette, con segno opposto per i vortici). Lo applicheremo negli esercizi 2 e 3.

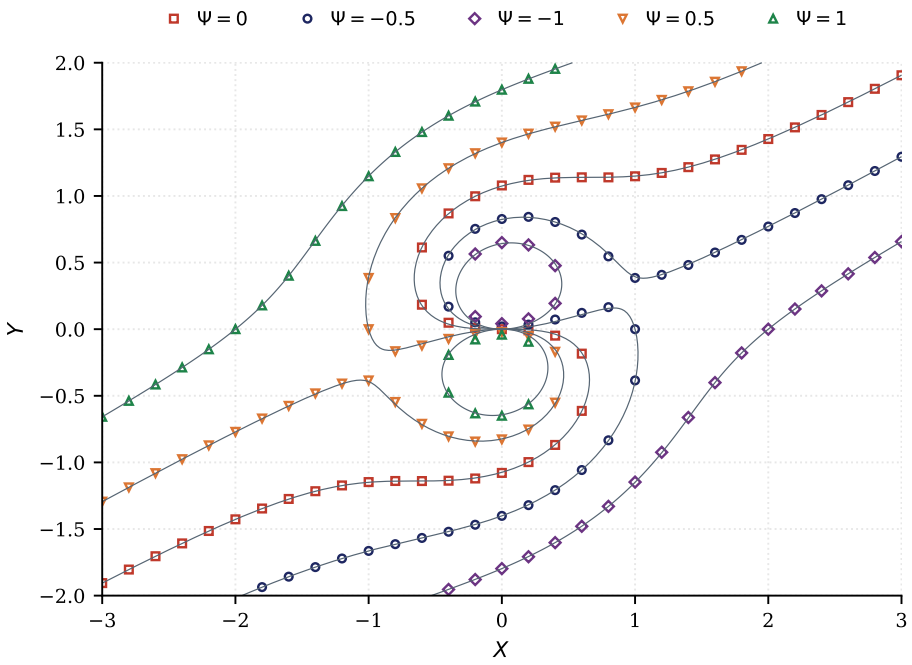


Figura 3.6: Doppietta nell'origine e corrente uniforme inclinata di 30°: nessun corpo chiuso. Simboli come in Fig. 3.4.

L'effetto suolo

In decollo e in atterraggio un'ala vola a una quota dal suolo paragonabile alla propria apertura. Il suolo si modella con l'immagine speculare del sistema di vortici dell'ala: l'immagine del vortice aderente e dei vortici liberi riduce la velocità indotta (il *downwash*) sull'ala, e quindi la resistenza indotta, e aumenta la pendenza della retta di portanza. È l'effetto suolo, che fa "galleggiare" un velivolo in fase di richiamata e che i grandi ekranoplani sovietici sfruttavano per volare a pochi metri dall'acqua. Lo stesso metodo spiega perché un vortice di estremità generato vicino al suolo si allontana lateralmente dalla pista invece di scendere: la sua immagine lo trasporta parallelamente alla parete (esercizio 2).

Capitolo 4

Corpi di rivoluzione: il metodo di von Kármán

4.1 Dal metodo inverso al metodo diretto

La sovrapposizione di flussi elementari genera corpi di forme svariate, ma usarla per ottenere il flusso attorno a un corpo *assegnato* richiede di aggiustare per tentativi posizioni e intensità delle singolarità finché la forma risultante non è abbastanza vicina a quella voluta. Nel 1927 von Kármán [15] propose un procedimento sistematico: le singolarità si collocano in posizioni fissate all'interno del corpo, lungo l'asse, e le loro intensità si calcolano imponendo che la superficie del corpo sia una linea (anzi, una superficie) di corrente. Il metodo fu ideato per la carena assialsimmetrica dei dirigibili; vale anche per i corpi piani (esercizio 5). Lo ricaviamo e lo mettiamo alla prova su due problemi di cui è nota la soluzione esatta: la sfera e l'ellissoide di rivoluzione allungato.

4.2 Flussi assialsimmetrici e funzione di corrente di Stokes

Per i flussi assialsimmetrici si adottano le coordinate cilindriche (r, ϑ, z) con l'asse z coincidente con l'asse di simmetria (Fig. 4.1); per simmetria nulla dipende da ϑ e la velocità ha due componenti, u_r e u_z. Per un flusso stazionario incomprimibile si definisce ancora una funzione di corrente, detta *di Stokes*, legata alle componenti della velocità da

$$u_r = -\frac{1}{r}\frac{\partial \psi}{\partial z}, \qquad u_z = \frac{1}{r}\frac{\partial \psi}{\partial r}. \tag{4.1}$$

L'equazione di continuità in coordinate cilindriche, $\partial(r u_r)/\partial r + \partial(r u_z)/\partial z = 0$, è soddisfatta identicamente. Il valore di ψ in un punto è proporzionale alla portata che attraversa il cerchio di raggio r passante per quel punto: $\int_0^r 2\pi r' u_z \, \mathrm{d}r' = 2\pi\psi$.

L'equazione di ψ non è un'equazione di Laplace

L'unica componente non nulla della vorticità è quella azimutale, $\omega_\vartheta = \partial u_r/\partial z - \partial u_z/\partial r$. Sostituendo le (4.1):

$$\omega_\vartheta = -\frac{1}{r}\frac{\partial^2 \psi}{\partial z^2} - \frac{\partial}{\partial r}\left(\frac{1}{r}\frac{\partial \psi}{\partial r}\right) = -\frac{1}{r}\left(\frac{\partial^2 \psi}{\partial r^2} - \frac{1}{r}\frac{\partial \psi}{\partial r} + \frac{\partial^2 \psi}{\partial z^2}\right).$$

La condizione di irrotazionalità è quindi

$$\frac{\partial^2\psi}{\partial r^2} - \frac{1}{r}\frac{\partial\psi}{\partial r} + \frac{\partial^2\psi}{\partial z^2} = 0, \tag{4.2}$$

che differisce dall'equazione di Laplace per il segno del termine del primo ordine: il laplaciano in coordinate cilindriche avrebbe $+\frac{1}{r}\partial/\partial r$. L'equazione è comunque lineare, e il principio di sovrapposizione resta valido.

Le soluzioni elementari si costruiscono come nel piano. Con $R = \sqrt{r^2+z^2}$ distanza dall'origine:

corrente uniforme lungo z : $\psi = \frac{1}{2}Ur^2$, (4.3)

sorgente puntiforme nell'origine: $\psi = -\dfrac{mz}{R}$, (4.4)

doppietta nell'origine: $\psi = -\dfrac{\mu r^2}{R^3}$. (4.5)

Dalle (4.1) la sorgente dà $u_z = mz/R^3$ e $u_r = mr/R^3$: la velocità è radiale, uscente dall'origine, di modulo m/R^2. La costante $m = VR^2$ è proporzionale al volume emesso nell'unità di tempo ($4\pi m$) e si chiama intensità della sorgente puntiforme; $m < 0$ è un pozzo. La doppietta si ottiene avvicinando lungo l'asse z una sorgente e un pozzo e mantenendo costante il prodotto μ dell'intensità per la distanza.

La sfera: corrente uniforme più doppietta

Con $\mu = \frac{1}{2}Ua^3$ la somma delle (4.3) e (4.5) è

$$\psi = \tfrac{1}{2}Ur^2\left(1 - \frac{a^3}{R^3}\right). \tag{4.6}$$

La superficie $\psi = 0$ è formata dall'asse z ($r = 0$) e dalla sfera $R = a$. In coordinate sferiche ($r = R\sin\varphi$) la velocità tangenziale sulla sfera è

$$V = \frac{1}{R\sin\varphi}\left|\frac{\partial\psi}{\partial R}\right|_{R=a} = \frac{1}{a\sin\varphi}\,\tfrac{1}{2}U\sin^2\varphi\left(2a + \frac{a^3}{a^2}\right) = \tfrac{3}{2}U\sin\varphi,$$

cioè

$$|\mathbf{V}| = \frac{3}{2}U\frac{r}{a}, \qquad C_p = 1 - \frac{9}{4}\left(\frac{r}{a}\right)^2. \tag{4.7}$$

La velocità massima, all'equatore, è $1{,}5\,U$: meno del $2U$ del cilindro, perché attorno alla sfera il fluido ha anche la terza dimensione per aggirare l'ostacolo.

4.3 Il metodo di von Kármán

Per approssimare il flusso di una corrente assiale di velocità U attorno a un corpo di rivoluzione assegnato, si distribuiscono sorgenti e pozzi lungo l'asse, all'interno del corpo, in n segmenti opportunamente scelti (Fig. 4.1). Il segmento j ha estremi $P'_j(0, z'_j)$ e $P'_{j+1}(0, z'_{j+1})$, lunghezza $s_j = z'_{j+1} - z'_j$, e contiene sorgenti di intensità costante q_j per unità di lunghezza. I segmenti possono avere lunghezze diverse; il loro numero dipende dall'accuratezza desiderata.

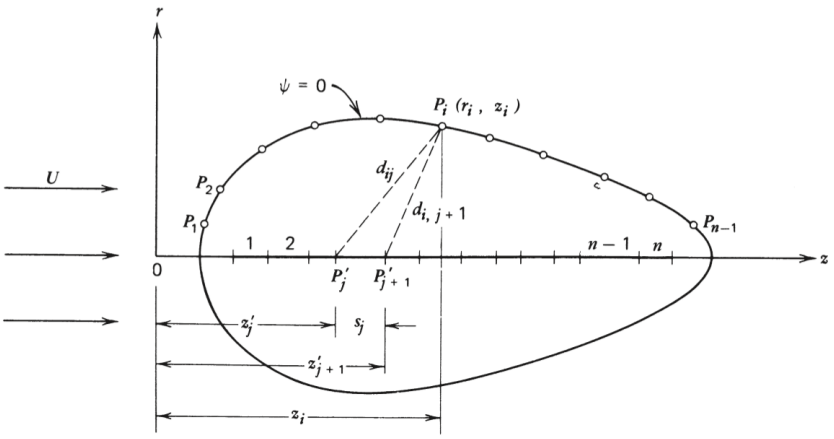


Figura 4.1: Schema del metodo di von Kármán.

Le sorgenti contenute in un tratto $\mathrm{d}\zeta$ del segmento j, nel punto $(0,\zeta)$, inducono nel punto (r,z) la funzione di corrente $\mathrm{d}\psi = -q_j(z-\zeta)\,\mathrm{d}\zeta/\sqrt{r^2+(z-\zeta)^2}$. Poiché $\mathrm{d}\sqrt{r^2+(z-\zeta)^2}/\mathrm{d}\zeta = -(z-\zeta)/\sqrt{r^2+(z-\zeta)^2}$, l'integrazione da z'_j a z'_{j+1} è immediata e la funzione di corrente complessiva, corrente uniforme più segmenti, è

$$\psi(r,z) = \tfrac{1}{2}Ur^2 + \sum_{j=1}^{n} q_j \left[\sqrt{r^2+(z-z'_{j+1})^2} - \sqrt{r^2+(z-z'_j)^2}\right]. \tag{4.8}$$

Si scelgono ora $n-1$ punti $P_1,\ldots,P_{n-1}$ sulla superficie del corpo. In ciascuno $\psi = 0$, perché la superficie è una superficie di corrente (la stessa dell'asse a monte e a valle). Le due radici nella (4.8) calcolate in $P_i(r_i,z_i)$ sono le distanze $d_{i,j+1}$ e d_{ij} di P_i dagli estremi del segmento j. Con la notazione

$$\Delta d_{ij} = d_{ij} - d_{i,j+1} = \sqrt{r_i^2+(z_i-z'_j)^2} - \sqrt{r_i^2+(z_i-z'_{j+1})^2} \tag{4.9}$$

e chiamando $\tilde{q}_j = q_j/U$ le intensità adimensionali, la condizione $\psi(P_i) = 0$ diventa

$$\sum_{j=1}^{n} \Delta d_{ij}\,\tilde{q}_j = \tfrac{1}{2}r_i^2, \qquad i = 1,\ldots,n-1. \tag{4.10}$$

Manca un'equazione. Poiché q_j è proporzionale al volume prodotto nell'unità di tempo dall'unità di lunghezza del segmento, perché il corpo sia chiuso la portata totale delle sorgenti e dei pozzi deve essere nulla:

$$\sum_{j=1}^{n} s_j\tilde{q}_j = 0. \tag{4.11}$$

Le $n-1$ equazioni (4.10) e la (4.11) formano il sistema lineare

$$\begin{bmatrix} \Delta d_{11} & \Delta d_{12} & \cdots & \Delta d_{1n} \\ \vdots & \vdots & & \vdots \\ \Delta d_{n-1,1} & \Delta d_{n-1,2} & \cdots & \Delta d_{n-1,n} \\ s_1 & s_2 & \cdots & s_n \end{bmatrix} \begin{pmatrix} \tilde{q}_1 \\ \vdots \\ \tilde{q}_{n-1} \\ \tilde{q}_n \end{pmatrix} = \begin{pmatrix} r_1^2/2 \\ \vdots \\ r_{n-1}^2/2 \\ 0 \end{pmatrix}, \tag{4.12}$$

questa volta con matrice *piena*: ogni segmento influenza tutti i punti della superficie. Si risolve con l'eliminazione di Gauss (`numpy.linalg.solve`); la formula $\tilde{\mathbf{q}} = \mathbf{d}^{-1}\mathbf{r}$ con l'inversa esplicita è equivalente ma più costosa e meno accurata, e non va usata.

Note le intensità, la (4.8) è determinata e le (4.1) danno le componenti della velocità:

$$u_r = -\frac{U}{r}\sum_{j=1}^{n}\tilde{q}_j\left[\frac{z-z'_{j+1}}{\sqrt{r^2+(z-z'_{j+1})^2}}-\frac{z-z'_j}{\sqrt{r^2+(z-z'_j)^2}}\right], \tag{4.13}$$

$$u_z = U\left\{1+\sum_{j=1}^{n}\tilde{q}_j\left[\frac{1}{\sqrt{r^2+(z-z'_{j+1})^2}}-\frac{1}{\sqrt{r^2+(z-z'_j)^2}}\right]\right\}. \tag{4.14}$$

Le velocità indotte dalle sorgenti si attenuano lontano dal corpo. La pressione si ricava dal teorema di Bernoulli stazionario: indicando con p_∞ la pressione lontano dal corpo,

$$C_p = \frac{p-p_\infty}{\frac{1}{2}\rho U^2} = 1-\left(\frac{u_r}{U}\right)^2-\left(\frac{u_z}{U}\right)^2. \tag{4.15}$$

4.4 Programma 2.3: la sfera

Per mettere alla prova il metodo lo applichiamo alla sfera, di cui la (4.7) dà la soluzione esatta. Lunghezze e velocità si rendono adimensionali con a e U (nel programma $a = U = 1$).

Dati del Programma 2.3

Dieci segmenti uguali di lunghezza $0{,}16a$ fra $z = -0{,}8a$ e $z = 0{,}8a$; nove punti equispaziati sulla semicirconferenza superiore, individuati dall'angolo φ_i fra l'asse z e il raggio:

$$\varphi_i = (10-i)\frac{\pi}{10}, \qquad r_i = a\sin\varphi_i, \qquad z_i = a\cos\varphi_i, \qquad i = 1,\dots,9.$$

Tabella 4.1: Programma 2.3: velocità e coefficiente di pressione nei nove punti di controllo sulla sfera.

r/a	z/a	V/U	V/U esatta	C_p	C_p esatto
0,3090	−0,9511	0,463 36	0,463 53	0,785 29	0,785 14
0,5878	−0,8090	0,881 72	0,881 68	0,222 57	0,222 64
0,8090	−0,5878	1,213 52	1,213 53	−0,472 63	−0,472 64
0,9511	−0,3090	1,426 59	1,426 58	−1,035 15	−1,035 14
1,0000	0,0000	1,500 00	1,500 00	−1,250 00	−1,250 00

Risultati

- Velocità e pressione coincidono con i valori esatti fino alla quarta cifra decimale (Tab. 4.1, simmetrica rispetto all'equatore); l'errore massimo su C_p è $1{,}5\cdot10^{-4}$, nei punti più vicini ai poli.
- Il metodo impone $\psi = 0$ nei punti di controllo, non la tangenza della velocità. Verifica indipendente: la componente normale della velocità è $|\mathbf{V}\cdot\mathbf{n}|/U \le 4{,}8\cdot10^{-4}$ nei punti di controllo e $\le 3{,}0\cdot10^{-3}$ su tutta la superficie.
- Le intensità dei segmenti, invece, hanno un andamento sorprendente:

$$\tilde{q}_j = 0{,}012;\ -0{,}142;\ 0{,}873;\ -4{,}385;\ 29{,}20;\ -29{,}20;\ 4{,}385;\ -0{,}873;\ 0{,}142;\ -0{,}012,$$

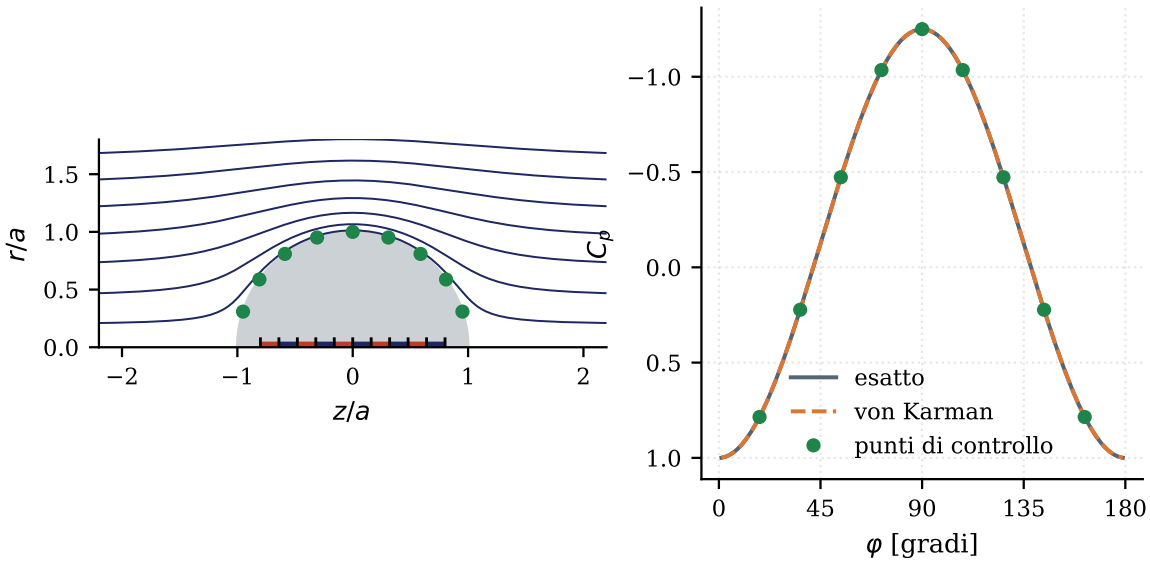


Figura 4.2: Programma 2.3. (a) Segmenti di sorgenti (rosso $\tilde{q}_j > 0$, blu $\tilde{q}_j < 0$), punti di controllo e linee di corrente. (b) Coefficiente di pressione sulla sfera: soluzione esatta, metodo di von Kármán su tutta la superficie e nei punti di controllo.

con segni alterni e valori enormi nei due segmenti centrali. Il numero di condizionamento della matrice (4.12) è $3 \cdot 10^5$.

Il risultato si spiega con la soluzione esatta: il flusso attorno alla sfera è generato da una *doppietta puntiforme* nell'origine, che i segmenti uniformi possono solo imitare con una coppia sorgente–pozzo molto intensa al centro e piccole correzioni alterne ai lati. Le singolarità "giuste" per la sfera non stanno distribuite su un segmento: il sistema trova comunque una combinazione che rende $\psi = 0$ nei punti assegnati, ma la trova con coefficienti grandi e alterni. Il prossimo esempio mostra quando questa situazione diventa pericolosa.

prog2_3_karman_sfera.py

```
"""
prog2_3_karman_sfera.py -- PROGRAMMA 2.3
Metodo di von Karman per il flusso assialsimmetrico attorno a un corpo di
rivoluzione: n segmenti di sorgenti uniformi sull'asse, n-1 punti di
controllo sulla superficie + condizione di chiusura (portata totale nulla).
Verifica sulla sfera di raggio a, per cui la soluzione esatta e'
   V = 1,5 U r/a ,   C_p = 1 - (9/4)(r/a)^2.
Le funzioni karman_* sono riutilizzate da prog2_3b_ellissoide.py.
"""
import numpy as np
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, ROSSO, GRIGIO, VERDE


def karman_risolvi(zz, r, z):
    """zz: n+1 estremi dei segmenti; (r, z): n-1 punti sulla superficie.
    Restituisce le intensita' adimensionali q_j/U e la matrice del sistema."""
    n = len(zz) - 1
    d = np.sqrt(r[:, None]**2 + (z[:, None] - zz[None, :])**2)    # (n-1) x (n+1)
    M = np.zeros((n, n))
    M[:-1, :] = d[:, :-1] - d[:, 1:]              # Delta d_ij = d_ij - d_i,j+1
    M[-1, :] = np.diff(zz)                        # chiusura: sum s_j q_j = 0
    b = np.concatenate([0.5 * r**2, [0.0]])
    q = np.linalg.solve(M, b)
```

```
    return q, M


def karman_velocita(q, zz, r, z, U=1.0):
    """Componenti (u_r, u_z) della velocita' nei punti (r, z), r > 0."""
    d = np.sqrt(r[:, None]**2 + (z[:, None] - zz[None, :])**2)
    g = (z[:, None] - zz[None, :]) / d
    ur = -U / r * ((g[:, 1:] - g[:, :-1]) @ q)
    uz = U * (1.0 + (1.0 / d[:, 1:] - 1.0 / d[:, :-1]) @ q)
    return ur, uz


def karman_psi(q, zz, R, Z, U=1.0):
    """Funzione di corrente (2D array) per il disegno delle linee di corrente."""
    psi = 0.5 * U * R**2
    for j in range(len(q)):
        psi = psi + U * q[j] * (np.sqrt(R**2 + (Z - zz[j + 1])**2) -
                                np.sqrt(R**2 + (Z - zz[j])**2))
    return psi


if __name__ == "__main__":
    A, U = 1.0, 1.0
    zz = -0.8 * A + 0.16 * A * np.arange(11)       # 10 segmenti tra -0,8a e 0,8a
    i = np.arange(1, 10)
    fi = (10 - i) * np.pi / 10                     # 9 punti sulla superficie
    r, z = A * np.sin(fi), A * np.cos(fi)

    q, M = karman_risolvi(zz, r, z)
    np.set_printoptions(precision=4, suppress=True, linewidth=110)
    print("Matrice del sistema (10 x 10) e termine noto:")
    print(np.column_stack([M, np.concatenate([0.5 * r**2, [0]])]))
    print("\nIntensita' q_j/U dei segmenti:")
    print(q)
    print(f"numero di condizionamento della matrice: {np.linalg.cond(M):.3e}")

    ur, uz = karman_velocita(q, zz, r, z, U)
    V = np.hypot(ur, uz)
    Vex = 1.5 * U * r / A
    cp, cpex = 1 - (V / U)**2, 1 - (Vex / U)**2
    print("\n   r/a     z/a      V/U      V/U esatta    C_p       C_p esatto
    |errore C_p|")
    for k in range(9):
        print(f"  {r[k]:.4f}  {z[k]:+.4f}  {V[k]:.5f}   {Vex[k]:.5f}   {cp[k]:+.5f}
      "
              f"{cpex[k]:+.5f}    {abs(cp[k]-cpex[k]):.1e}")

    # tangenza della velocita' (controllo indipendente): V . n = 0 sulla sfera
    print(f"\nmax |V.n|/U nei punti di controllo: "
          f"{np.max(np.abs(ur*np.sin(fi) + uz*np.cos(fi))):.2e}")

    # --- C_p su tutta la superficie (anche fra i punti di controllo) -------
    fic = np.linspace(0.02, np.pi - 0.02, 300)
    rc, zc = A * np.sin(fic), A * np.cos(fic)
    urc, uzc = karman_velocita(q, zz, rc, zc, U)
    vn = urc * np.sin(fic) + uzc * np.cos(fic)
    print(f"max |V.n|/U su tutta la superficie: {np.max(np.abs(vn)):.2e}")
    cpc = 1 - (urc**2 + uzc**2) / U**2

    fig, ax = plt.subplots(1, 2, figsize=(6.6, 3.0), gridspec_kw={"width_ratios":
```

```
    [1.15, 1]})
    # (a) geometria e linee di corrente
    zg = np.linspace(-2.2, 2.2, 441); rg = np.linspace(1e-3, 2.4, 241)
    ZG, RG = np.meshgrid(zg, rg, indexing="ij")
    P = karman_psi(q, zz, RG, ZG, U)
    P[np.hypot(ZG, RG) < A] = np.nan
    ax[0].contour(ZG, RG, P, levels=[0.02, 0.1, 0.25, 0.45, 0.7, 1.0, 1.35],
                  colors=BLU, linewidths=0.7, linestyles="solid")
    t = np.linspace(0, np.pi, 200)
    ax[0].fill_between(A * np.cos(t), 0, A * np.sin(t), color=GRIGIO, alpha=0.3)
    for j in range(10):
        col = ROSSO if q[j] > 0 else BLU
        ax[0].plot(zz[j:j + 2], [0.012, 0.012], color=col, lw=3.0,
    solid_capstyle="butt")
    ax[0].plot(zz, 0 * zz + 0.012, "|", color="k", ms=6)
    ax[0].plot(z, r, "o", color=VERDE, ms=4)
    ax[0].set_aspect("equal"); ax[0].set_xlabel("$z/a$"); ax[0].set_ylabel("$r/a$")
    ax[0].set_ylim(0, 1.8); ax[0].grid(False)
    # (b) C_p
    ax[1].plot(np.degrees(fic), 1 - 2.25 * np.sin(fic)**2, color=GRIGIO,
    label="esatto")
    ax[1].plot(np.degrees(fic), cpc, "--", color=ARANCIO, label="von Karman")
    ax[1].plot(np.degrees(fi), cp, "o", color=VERDE, ms=4, label="punti di
    controllo")
    ax[1].invert_yaxis(); ax[1].set_xlabel(r"$\varphi$ [gradi]");
    ax[1].set_ylabel("$C_p$")
    ax[1].set_xticks([0, 45, 90, 135, 180]); ax[1].legend()
    salva(fig, "prog2_3_sfera")
```

Listato 4.1: Programma 2.3 – metodo di von Kármán per la sfera (`prog2_3_karman_sfera.py`).

4.5 Un corpo “a dirigibile”: l’ellissoide allungato

Nel lavoro originale di von Kármán la carena del dirigibile ZR III (il futuro USS *Los Angeles*) era generata da un gruppo di sorgenti verso la prua e da un gruppo di pozzi verso la poppa, nell’ipotesi che i contributi delle sorgenti di prua alla pressione sulla parte poppiera, e viceversa, fossero trascurabili; il C_p calcolato concordava bene con quello misurato, salvo che nelle vicinanze della gondola. Un banco di prova moderno, con soluzione esatta nota, è l’**ellissoide di rivoluzione allungato** di semiassi a (lungo l’asse) e $b < a$, la forma più semplice “a sigaro”.

La soluzione esatta per l’ellissoide allungato

Sia $e = \sqrt{1 - b^2/a^2}$ l’eccentricità; i fuochi sono in $z = \pm ae$. La teoria classica del potenziale [16, 17] dà la velocità sulla superficie, per una corrente assiale U:

$$\frac{V}{U} = (1+k)\sqrt{\frac{1 - z^2/a^2}{1 - e^2 z^2/a^2}}, \qquad 1 + k = \frac{2}{2 - \alpha_0}, \qquad \alpha_0 = \frac{2(1-e^2)}{e^3}\left(\tfrac{1}{2}\ln\frac{1+e}{1-e} - e\right), \tag{4.16}$$

dove k è il coefficiente di massa aggiunta longitudinale. Per $e \to 0$ si ha $\alpha_0 \to \tfrac{2}{3}$, $1 + k \to \tfrac{3}{2}$ e si ritrova la sfera. Il campo è generato *esattamente* da una distribuzione lineare di sorgenti fra i due fuochi, $q(\zeta) = -K\zeta$ per $|\zeta| < ae$: sorgenti nella metà anteriore, pozzi in quella posteriore, intensità totale nulla.

La costante K si ricava imponendo il ristagno nel naso $z = a$. Sull'asse, a monte dei fuochi ($z > ae$, dove $r = 0$), la (4.14) con la distribuzione continua diventa

$$u_z = U - K \int_{-ae}^{ae} \frac{\zeta \, d\zeta}{(z-\zeta)^2} = U - K \left[\frac{2zae}{z^2 - a^2e^2} - \ln \frac{z+ae}{z-ae} \right].$$

Ponendo $u_z(a) = 0$ e osservando che $a^2 - a^2e^2 = b^2$:

$$K = \frac{U}{2e\, a^2/b^2 - \ln \dfrac{1+e}{1-e}}. \tag{4.17}$$

Il programma `prog2_3b_ellissoide.py` (Listato 4.2) applica il metodo di von Kármán, con le stesse funzioni del Programma 2.3, a un ellissoide di finezza $a/b = 4$, tipica di una carena di dirigibile. Per questo corpo $e = 0{,}968$, $\alpha_0 = 0{,}151$, $1 + k = 1{,}082$ e $K = 0{,}0372\,U$. Si usano n segmenti uguali fra $-c$ e $+c$ e $n-1$ punti di controllo equispaziati lungo l'asse sulla superficie; si confrontano due scelte: segmenti estesi esattamente fra i fuochi ($c = ae$) e segmenti un po' più corti ($c = 0{,}9\,ae$).

Risultati: dove si mettono le singolarità conta

segmenti	n	cond(M)	$\max \lvert V - V_{\text{es}} \rvert / U$	$\max \lvert \tilde{q}_j \rvert$
fra i fuochi	6	$1{,}62 \times 10^{1}$	$2{,}13 \times 10^{-2}$	$3{,}13 \times 10^{-2}$
($c = ae$)	10	$1{,}06 \times 10^{2}$	$1{,}15 \times 10^{-2}$	$3{,}33 \times 10^{-2}$
	20	$7{,}93 \times 10^{3}$	$3{,}62 \times 10^{-3}$	$3{,}48 \times 10^{-2}$
	40	$3{,}02 \times 10^{7}$	$6{,}85 \times 10^{-4}$	$3{,}54 \times 10^{-2}$
più corti	6	$1{,}88 \times 10^{1}$	$3{,}37 \times 10^{-1}$	$4{,}69 \times 10^{-2}$
($c = 0{,}9\,ae$)	10	$1{,}41 \times 10^{2}$	$2{,}70 \times 10^{-1}$	$7{,}36 \times 10^{-2}$
	20	$1{,}78 \times 10^{4}$	$2{,}15 \times 10^{-1}$	$3{,}93$
	40	$3{,}85 \times 10^{9}$	$2{,}23$	$6{,}61 \times 10^{5}$

Con i segmenti fra i fuochi l'errore diminuisce regolarmente con n (da 10 a 40 segmenti si riduce di un fattore 17) e le intensità riproducono la distribuzione lineare esatta $-K\zeta$ (Fig. 4.4, a sinistra); con dieci segmenti la velocità superficiale è già indistinguibile da quella esatta sulla scala del grafico (Fig. 4.3). Con i segmenti più corti il metodo *non converge*: aumentando n le intensità oscillano con ampiezze sempre maggiori (Fig. 4.4, a destra) e l'errore cresce.

Un problema mal posto

Il sistema (4.12) è la versione discreta di un'equazione integrale *di prima specie*: l'incognita $q(\zeta)$ compare solo sotto il segno di integrale, moltiplicata per un nucleo regolare. Equazioni di questo tipo sono intrinsecamente mal condizionate: componenti molto oscillanti di q producono effetti piccolissimi sulla superficie, perché vengono "mediate" dall'integrale, e quindi il sistema non le sa distinguere dagli errori di arrotondamento. Se la distribuzione esatta esiste e cade nella regione in cui si sono messe le singolarità (fra i fuochi per l'ellissoide), il metodo la ritrova; se non esiste, il sistema ne fabbrica una oscillante e instabile. Per questo i metodi moderni non distribuiscono le singolarità

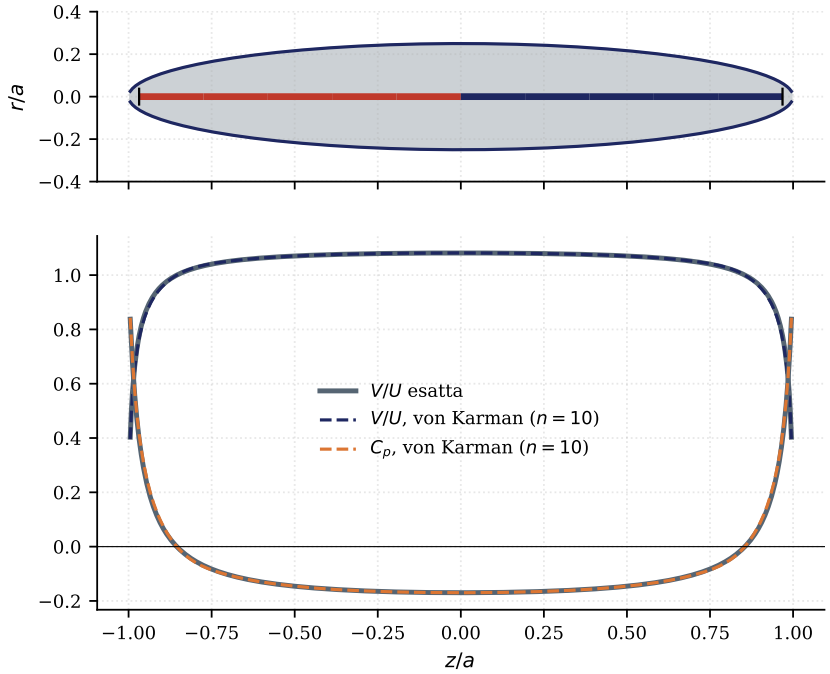


Figura 4.3: Ellissoide di finezza 4: segmenti di sorgenti fra i fuochi (in alto) e velocità e C_p sulla superficie, esatti (grigio) e con il metodo di von Kármán a dieci segmenti (tratteggio).

sull'asse ma *sulla superficie* del corpo, dove l'equazione integrale diventa di seconda specie e ben condizionata: sono i metodi a pannelli di Hess e Smith [13], di cui il Cap. 5 presenta la versione bidimensionale con vortici.

prog2_3b_ellissoide.py

```
"""
prog2_3b_ellissoide.py -- il metodo di von Karman applicato a un corpo
"a dirigibile": l'ellissoide di rivoluzione allungato (semiassi a, b; finezza
a/b = 4), per il quale la soluzione esatta e' nota.

  velocita' sulla superficie:  V/U = (1+k) sqrt[(1 - z^2/a^2)/(1 - e^2 z^2/a^2)]
  con e = sqrt(1 - b^2/a^2),  1 + k = 2/(2 - alfa0),
  alfa0 = 2(1-e^2)/e^3 [ (1/2) ln((1+e)/(1-e)) - e ].
  La distribuzione esatta di sorgenti e' lineare fra i fuochi z = -ae, +ae:
  q(zeta) = -K zeta,  K = U / [2 e a^2/b^2 - ln((1+e)/(1-e))].
"""
import numpy as np
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, ROSSO, GRIGIO, VERDE, VIOLA
from prog2_3_karman_sfera import karman_risolvi, karman_velocita

U, A, B = 1.0, 1.0, 0.25
e = np.sqrt(1 - (B / A)**2)
alfa0 = 2 * (1 - e**2) / e**3 * (0.5 * np.log((1 + e) / (1 - e)) - e)
uno_piu_k = 2 / (2 - alfa0)
K = U / (2 * e * A**2 / B**2 - np.log((1 + e) / (1 - e)))
print(f"eccentricita' e = {e:.5f}, fuochi in z = +-{A*e:.4f}")
print(f"alfa0 = {alfa0:.5f}, 1 + k = {uno_piu_k:.5f}, K = {K:.5f}")


def V_esatta(z):
    return U * uno_piu_k * np.sqrt((1 - z**2 / A**2) / (1 - e**2 * z**2 / A**2))

```

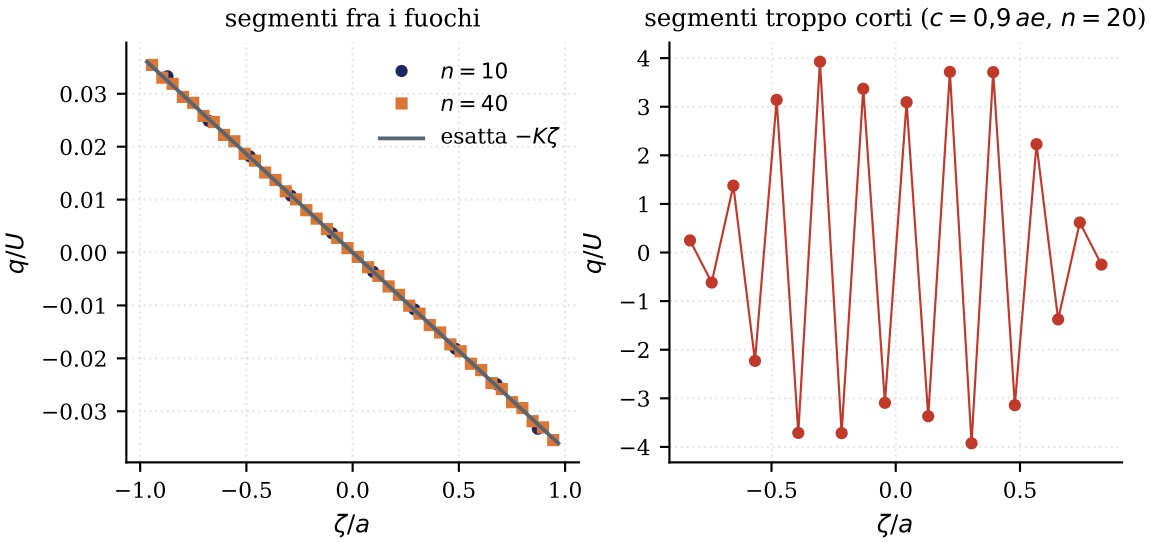


Figura 4.4: Intensità dei segmenti. A sinistra, segmenti fra i fuochi: i valori seguono la distribuzione esatta $-K\zeta$. A destra, segmenti troppo corti: oscillazioni di segno alterno.

```
29
30 def calcola(n, frazione=1.0):
31     """n segmenti uguali fra -c e +c (c = frazione * a e); n-1 punti di
32     controllo equispaziati in z sulla superficie."""
33     c = frazione * A * e
34     zz = np.linspace(-c, c, n + 1)
35     zi = np.linspace(-A, A, n + 1)[1:-1]
36     ri = B * np.sqrt(1 - zi**2 / A**2)
37     q, M = karman_risolvi(zz, ri, zi)
38     return zz, q, M, zi, ri
39
40
41 zt = np.linspace(-0.995, 0.995, 600) * A          # punti di verifica
42 rt = B * np.sqrt(1 - zt**2 / A**2)
43 print("\n frazione   n    cond(M)      max|V-Vex|/U   max|q|/U")
44 risultati = {}
45 for frazione in (1.0, 0.9):
46     for n in (6, 10, 20, 40):
47         zz, q, M, zi, ri = calcola(n, frazione)
48         ur, uz = karman_velocita(q, zz, rt, zt, U)
49         err = np.max(np.abs(np.hypot(ur, uz) - V_esatta(zt)))
50         risultati[(frazione, n)] = (zz, q, ur, uz)
51         print(f"   {frazione:.1f}    {n:3d}   {np.linalg.cond(M):.2e}     {err:.2e}
            {np.max(np.abs(q)):.2e}")
52
53 # ----- figura 1: forma, V/U e C_p per n = 10 ---------------------------------
54 zz, q, ur, uz = risultati[(1.0, 10)]
55 V = np.hypot(ur, uz)
56 fig, ax = plt.subplots(2, 1, figsize=(5.8, 4.8), sharex=True,
57                        gridspec_kw={"height_ratios": [1, 2.2]})
58 ax[0].fill_between(zt, -rt, rt, color=GRIGIO, alpha=0.3)
59 ax[0].plot(zt, rt, color=BLU); ax[0].plot(zt, -rt, color=BLU)
60 for j in range(len(q)):
61     ax[0].plot(zz[j:j + 2], [0, 0], color=ROSSO if q[j] > 0 else BLU, lw=3,
62                solid_capstyle="butt")
63 ax[0].plot([-A * e, A * e], [0, 0], "k|", ms=8)
64 ax[0].set_ylim(-0.4, 0.4); ax[0].set_ylabel("$r/a$")
65 ax[1].plot(zt, V_esatta(zt), color=GRIGIO, lw=2.2, label="$V/U$ esatta")
```

```
ax[1].plot(zt, V, "--", color=BLU, label="$V/U$, von Karman ($n=10$)")
ax[1].plot(zt, 1 - V_esatta(zt)**2, color=GRIGIO, lw=2.2)
ax[1].plot(zt, 1 - V**2, "--", color=ARANCIO, label="$C_p$, von Karman ($n=10$)")
ax[1].axhline(0, color="k", lw=0.5)
ax[1].set_xlabel("$z/a$"); ax[1].legend(loc="center")
salva(fig, "prog2_3b_ellissoide")

# ----- figura 2: intensita' dei segmenti ------------------------------------
fig, ax = plt.subplots(1, 2, figsize=(6.6, 2.8))
for n, col, mk in ((10, BLU, "o"), (40, ARANCIO, "s")):
    zz, q, _, _ = risultati[(1.0, n)]
    ax[0].plot(0.5 * (zz[1:] + zz[:-1]), q, mk, color=col, ms=3.5, label=f"$n={n}$")
zl = np.linspace(-A * e, A * e, 10)
ax[0].plot(zl, -K * zl, color=GRIGIO, label=r"esatta $-K\zeta$")
ax[0].set_xlabel(r"$\zeta/a$"); ax[0].set_ylabel("$q/U$"); ax[0].legend()
ax[0].set_title("segmenti fra i fuochi")
zz, q, _, _ = risultati[(0.9, 20)]
ax[1].plot(0.5 * (zz[1:] + zz[:-1]), q, "o-", color=ROSSO, ms=3.5, lw=0.8)
ax[1].set_xlabel(r"$\zeta/a$"); ax[1].set_ylabel("$q/U$")
ax[1].set_title(r"segmenti troppo corti ($c=0{,}9\,ae$, $n=20$)")
salva(fig, "prog2_3b_intensita")
```

Listato 4.2: `prog2_3b_ellissoide.py`: metodo di von Kármán per l'ellissoide di rivoluzione allungato.

Il metodo di von Kármán oggi

La distribuzione di singolarità sull'asse sopravvive nella *teoria dei corpi snelli* (*slender body theory*), usata nel progetto preliminare di fusoliere, missili e serbatoi esterni: per un corpo snello la densità di sorgenti è, in prima approssimazione, proporzionale alla derivata dell'area della sezione, $q \simeq U\,\mathrm{d}A/\mathrm{d}z/(4\pi)$, senza bisogno di risolvere alcun sistema. Su questa idea si basa la *regola delle aree* di Whitcomb per la resistenza d'onda transonica: due velivoli con la stessa distribuzione longitudinale delle aree delle sezioni hanno, in prima approssimazione, la stessa resistenza d'onda, da cui la fusoliera "a vita di vespa" dei caccia degli anni Cinquanta.

Capitolo 5

Il metodo inverso II: trasformazione conforme e profili alari

5.1 Il potenziale complesso

Per le correnti piane stazionarie le componenti della velocità derivano sia dal potenziale sia dalla funzione di corrente, ed entrambe le funzioni soddisfano l'equazione di Laplace:

$$u = \frac{\partial \phi}{\partial x} = \frac{\partial \psi}{\partial y}, \qquad v = \frac{\partial \phi}{\partial y} = -\frac{\partial \psi}{\partial x}. \tag{5.1}$$

Introduciamo la variabile complessa $z = x + \mathrm{i}y$, con $\mathrm{i} = \sqrt{-1}$ unità immaginaria, e costruiamo con potenziale e funzione di corrente la funzione complessa

$$w(z) = \phi(x, y) + \mathrm{i}\,\psi(x, y). \tag{5.2}$$

Le condizioni di Cauchy–Riemann

Se w è una funzione di z e la sua derivata $\mathrm{d}w/\mathrm{d}z$ esiste in tutta la regione occupata dal fluido, il valore della derivata non può dipendere dalla direzione lungo cui l'incremento $\Delta z = \Delta x + \mathrm{i}\Delta y$ tende a zero [18]. Scegliendo $\Delta y = 0$:

$$\frac{\mathrm{d}w}{\mathrm{d}z} = \lim_{\Delta x \to 0} \frac{\phi(x + \Delta x, y) - \phi(x, y) + \mathrm{i}[\psi(x + \Delta x, y) - \psi(x, y)]}{\Delta x} = \frac{\partial \phi}{\partial x} + \mathrm{i}\frac{\partial \psi}{\partial x};$$

scegliendo $\Delta x = 0$, con $\Delta z = \mathrm{i}\Delta y$:

$$\frac{\mathrm{d}w}{\mathrm{d}z} = \lim_{\Delta y \to 0} \frac{\phi(x, y + \Delta y) - \phi(x, y) + \mathrm{i}[\psi(x, y + \Delta y) - \psi(x, y)]}{\mathrm{i}\Delta y} = -\mathrm{i}\frac{\partial \phi}{\partial y} + \frac{\partial \psi}{\partial y}.$$

Uguagliando parti reali e immaginarie si ottengono le **condizioni di Cauchy–Riemann**,

$$\frac{\partial \phi}{\partial x} = \frac{\partial \psi}{\partial y}, \qquad \frac{\partial \phi}{\partial y} = -\frac{\partial \psi}{\partial x}, \tag{5.3}$$

necessarie per l'esistenza della derivata. Sono esattamente le (5.1): in una corrente potenziale piana $\phi + \mathrm{i}\psi$ è una **funzione analitica** di z. Viceversa, eliminando ψ o ϕ fra le (5.3) si ottiene l'equazione di Laplace per l'altra funzione: parte reale e parte immaginaria di *qualunque* funzione analitica sono il potenziale e la funzione di corrente

di una corrente fisicamente possibile. La (5.2) si chiama **potenziale complesso** e la sua derivata è la *velocità complessa coniugata*:

$$\frac{dw}{dz} = u - iv, \qquad |\mathbf{V}| = \left|\frac{dw}{dz}\right|. \tag{5.4}$$

Il potenziale complesso più semplice è $w = Uz\,e^{-i\alpha}$: separandolo in parte reale e immaginaria si ottiene $w = U(x\cos\alpha + y\sin\alpha) + iU(y\cos\alpha - x\sin\alpha)$, la cui parte immaginaria è la funzione di corrente (3.1) della corrente uniforme inclinata di α; la (5.4) dà $u = U\cos\alpha$, $v = U\sin\alpha$. In modo analogo si verifica che

$$w = \frac{\sigma}{2\pi}\ln(z - z_0), \qquad w = \frac{i\Gamma}{2\pi}\ln(z - z_0), \qquad w = \frac{\kappa}{2\pi(z - z_0)} \tag{5.5}$$

sono i potenziali complessi della sorgente, del vortice (orario per $\Gamma > 0$) e della doppietta delle (3.2)–(3.4), con $z_0 = x_0 + iy_0$. Si possono generare tante correnti quante sono le funzioni analitiche che si sanno scrivere, e il principio di sovrapposizione continua a valere: $w = Uz + \kappa/(2\pi z)$ è il potenziale del cilindro di raggio $\sqrt{\kappa/(2\pi U)}$ del §3.3.

5.2 Le trasformazioni conformi

Esiste un altro metodo potente per generare nuove correnti: la **trasformazione conforme**, che porta un flusso noto in un piano complesso $\zeta = \xi + i\eta$ in un flusso nuovo nel piano $z = x + iy$. Sia $w(\zeta)$ il potenziale complesso, noto, del flusso nel piano ζ, e sia

$$z = f(\zeta) \tag{5.6}$$

una funzione analitica. Attraverso la (5.6) w diventa una funzione di z, ed è ancora analitica, perché la derivata

$$\frac{dw}{dz} = \frac{dw/d\zeta}{dz/d\zeta} \tag{5.7}$$

esiste dove esistono le due derivate a secondo membro e $dz/d\zeta \neq 0$. Le parti reale e immaginaria di w, espresse nelle nuove coordinate, sono dunque il potenziale e la funzione di corrente di un flusso nel piano z: una linea di corrente $\psi = c$ e un'equipotenziale $\phi = k$ del piano ζ si trasformano in una linea di corrente e in un'equipotenziale del piano z con gli stessi valori, deformate ma ancora perpendicolari. Per questo la trasformazione si dice *conforme*: conserva gli angoli. Fanno eccezione i **punti critici**, dove $dz/d\zeta = 0$: lì gli angoli non si conservano e, se il flusso nel piano ζ non ha un ristagno nel punto corrispondente, la velocità nel piano z diventa infinita.

Per il modulo della velocità la (5.7) dà

$$V_z = \frac{V_\zeta}{|dz/d\zeta|}\,: \tag{5.8}$$

dove la trasformazione "stira" le distanze la velocità diminuisce, dove le "comprime" aumenta.

Il flusso in un angolo retto

La corrente uniforme $w = U\zeta$ del semipiano superiore, attraverso la trasformazione

$\zeta \mapsto \zeta^2$, diventa

$$w = Uz^2 = U(x^2 - y^2) + \mathrm{i}\,2Uxy :$$

le linee di corrente sono le iperboli $2xy = \text{cost}$ e le equipotenziali le iperboli $x^2 - y^2 = \text{cost}$ (Fig. 5.1a). È il flusso nell'angolo formato da due pareti perpendicolari, e localmente il flusso nei pressi di qualunque punto di ristagno su una parete piana: è la soluzione da cui parte la teoria dello strato limite nel ristagno (flusso di Hiemenz) e che si usa per stimare lo scambio termico sul bordo d'attacco di un'ala o di una paletta di turbina.

5.3 La trasformazione di Joukowski

In generale la funzione a secondo membro della (5.6) si può sviluppare in serie di potenze positive e negative di ζ; trattenendo due termini di forma particolare si ottiene la **trasformazione di Joukowski**

$$z = \zeta + \frac{b^2}{\zeta}. \tag{5.9}$$

I punti critici sono $\zeta = \pm b$, dove $\mathrm{d}z/\mathrm{d}\zeta = 1 - b^2/\zeta^2 = 0$; lontano dall'origine la trasformazione è quasi l'identità, e quindi una corrente uniforme all'infinito resta la stessa corrente uniforme. Il suo uso più semplice si ha con $b = a$: il potenziale della corrente uniforme del piano z, $w = Uz$, si scrive nel piano ζ come $U(\zeta + a^2/\zeta)$, il flusso attorno al cilindro di raggio a, ritrovato senza sovrapporre la doppietta.

Con $b < a$ la stessa trasformazione porta il cerchio $|\zeta| = a$ in un'ellisse di semiassi $a + b^2/a$ e $a - b^2/a$ (Fig. 5.1b): si ottiene il flusso attorno a un cilindro ellittico tracciando le linee di corrente del cilindro circolare nel piano ζ e trasformandone i punti. Più interessante è il caso dei cerchi che passano per il punto critico $\zeta = b$ (Fig. 5.1c, d): un cerchio con centro sull'asse immaginario e passante per $\pm b$ diventa un **arco di cerchio**, cioè una lamina curva; un cerchio con centro fuori dagli assi, passante per b e contenente $-b$, diventa un **profilo di Joukowski**, arrotondato davanti e con il bordo d'uscita a cuspide nell'immagine $z = 2b$ del punto critico.

Perché si calcola numericamente

Per trovare il flusso attorno all'ellisse o al profilo si potrebbe invertire la (5.9), $\zeta = \frac{1}{2}\left(z \pm \sqrt{z^2 - 4b^2}\right)$, e sostituire nel potenziale del cilindro. Separare poi la funzione risultante in parte reale e immaginaria è un lavoro analitico lungo e soggetto a errori, e bisogna scegliere con attenzione il ramo della radice. Conviene seguire la strada opposta: si calcolano i punti di alcune linee di corrente nel piano ζ, dove il flusso è semplice e noto, e li si trasforma *direttamente* con la (5.9), che non richiede inversioni.

5.4 Il profilo di Joukowski e la condizione di Kutta

Consideriamo una corrente uniforme U, inclinata dell'angolo d'incidenza α, attorno al profilo di Joukowski, immagine di un cerchio di raggio a con centro nel punto $P = (\xi_P, \eta_P)$ del piano ζ e passante per il punto critico $Q = (b, 0)$. Dalla geometria $a^2 = \eta_P^2 + (b - \xi_P)^2$; assegnati a, b e η_P,

$$\xi_P = b - \sqrt{a^2 - \eta_P^2}. \tag{5.10}$$

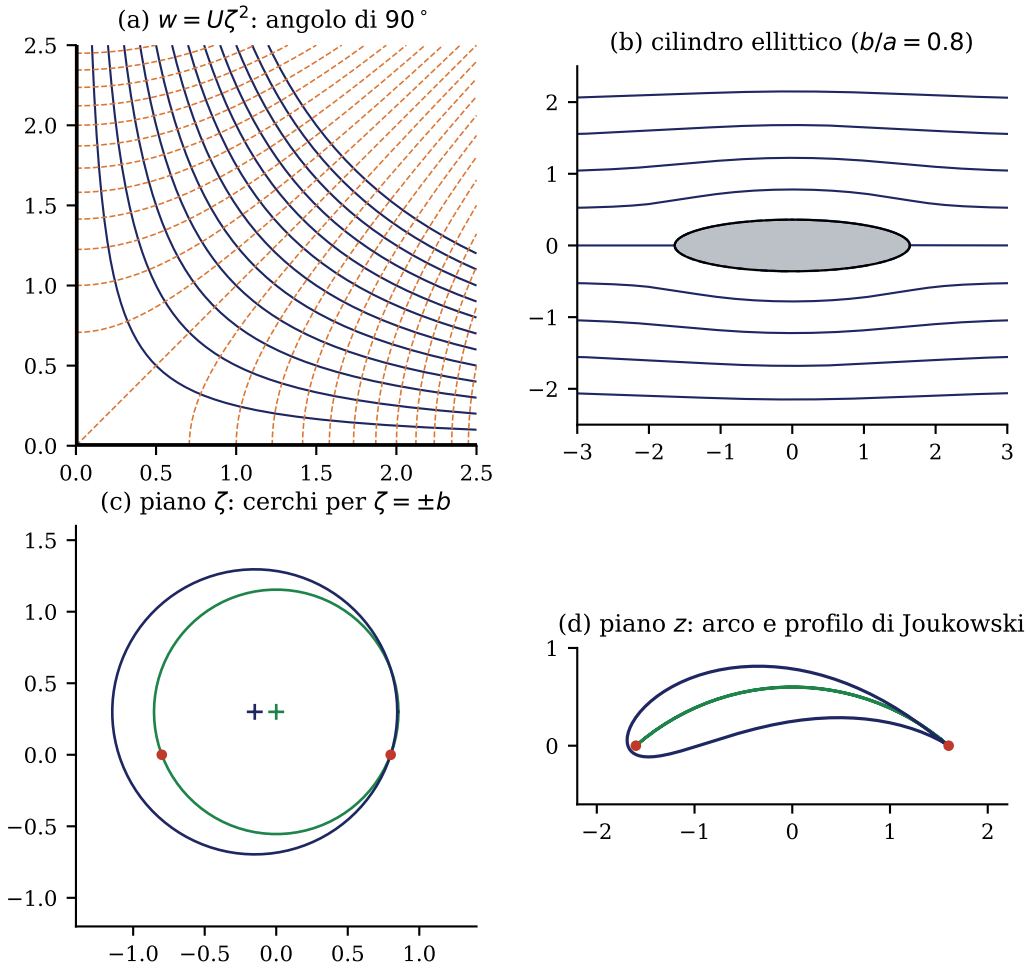


Figura 5.1: Esempi di trasformazioni conformi (`prog2_4b_mappe.py`). (a) Flusso nell'angolo retto: linee di corrente (continue) ed equipotenziali (tratteggiate). (b) Cilindro ellittico ottenuto dal cerchio $|\zeta| = a$ con $b/a = 0{,}8$. (c) Due cerchi del piano ζ passanti per i punti critici $\pm b$: con centro sull'asse immaginario (verde) e fuori dagli assi (blu). (d) Le loro immagini: un arco di cerchio e un profilo di Joukowski.

ξ_P deve essere negativo perché il profilo abbia il bordo affilato a valle; la corda è circa $4b$, lo spessore dipende da quanto il cerchio "sporge" oltre $-b$ (cioè da $a - b$), la curvatura (la *freccia*) da η_P. Per $\eta_P = 0$ il profilo è simmetrico.

La (5.8) con la (5.9) dà

$$V_z = \frac{V_\zeta}{|1 - (b/\zeta)^2|} : \tag{5.11}$$

nel bordo d'uscita, immagine del punto critico Q, la velocità è infinita a meno che non sia nulla V_ζ in Q. Fisicamente una velocità infinita attorno a uno spigolo vivo non è ammissibile: l'osservazione dei corpi con bordo d'uscita affilato mostra che la corrente lascia il bordo d'uscita in modo regolare.

La condizione di Kutta e la circolazione del profilo

Condizione di Kutta: la velocità nel bordo d'uscita deve essere nulla se l'angolo del bordo d'uscita è finito, finita se l'angolo è nullo (cuspide). Per il profilo di Joukowski la condizione è soddisfatta solo se Q è un punto di ristagno sul cerchio: il profilo deve

generare attorno a sé la circolazione oraria Γ appena sufficiente a portare in Q il ristagno posteriore.
Nel sistema con origine in P, $\zeta' = \zeta - \zeta_P$, il flusso attorno al cerchio è corrente inclinata, doppietta e vortice:

$$w = U\left(\zeta' e^{-i\alpha} + \frac{a^2 e^{i\alpha}}{\zeta'}\right) + \frac{i\Gamma}{2\pi}\ln\frac{\zeta'}{a}. \tag{5.12}$$

Sul cerchio, $\zeta' = ae^{i\varphi}$, la velocità è tangenziale e vale $u_\varphi = -2U\sin(\varphi - \alpha) - \Gamma/(2\pi a)$. Il punto Q è visto da P sotto l'anomalia $\varphi_Q = -\beta$, con $\sin\beta = \eta_P/a$; imponendo $u_\varphi(\varphi_Q) = 0$:

$$\Gamma = 4\pi a U \sin(\alpha + \beta). \tag{5.13}$$

Per $\alpha = 0$, poiché $\sin\beta = \eta_P/a$, si ha semplicemente $\Gamma = 4\pi\eta_P U$. Per il teorema di Kutta–Joukowski la portanza per unità di apertura è $L' = \rho U\Gamma$ e, riferita alla corda c,

$$C_L = \frac{2\Gamma}{Uc} = \frac{8\pi a}{c}\sin(\alpha + \beta). \tag{5.14}$$

La portanza si annulla per $\alpha = -\beta$: l'angolo di portanza nulla di un profilo curvo è negativo. Per un profilo sottile ($a \simeq b$, $c \simeq 4b$) e piccoli angoli si ritrova $C_L \simeq 2\pi(\alpha + \beta)$, la pendenza 2π per radiante della teoria dei profili sottili.

La velocità nel bordo d'uscita

Nel bordo d'uscita del profilo di Joukowski la (5.11) è una forma $0/0$: numeratore e denominatore si annullano insieme e la velocità ha un limite finito e *non nullo*, in accordo con la condizione di Kutta per un bordo a cuspide (angolo nullo). Nel programma il valore nel bordo d'uscita si calcola come media dei valori in due punti vicinissimi, a cavallo di Q; calcolarlo direttamente darebbe `nan`.

5.5 Programma 2.4: il flusso attorno al profilo di Joukowski

Dati del Programma 2.4

$U = 1$ m/s, $a = 1$ m, $b = 0{,}8$ m, $\eta_P = 0{,}199$ m, da cui $\xi_P = -0{,}180$ m e $\beta = 11{,}48°$; incidenza $\alpha = 0$. Griglia nel piano ζ: $-3 \le \xi, \eta \le 3$ con passo 0,05 m; linee di corrente $\psi = -2; -1{,}5; \ldots; 2{,}5$.

Le fasi del Programma 2.4

1. Si calcola $\psi = \operatorname{Im} w$ con la (5.12) nei nodi della griglia del piano ζ; i nodi interni al cerchio si scartano (valore `NaN`).
2. Con la funzione `cerca_x` (§3.4) si trovano i punti delle linee di corrente lungo le verticali della griglia.
3. I punti si trasformano con la (5.9); si conservano quelli che cadono nella finestra di disegno (è il compito della subroutine MAPPNG del programma originale).
4. Per la pressione si usano punti *esatti* del contorno: $\zeta' = ae^{i\varphi}$, con φ che parte da

$-\beta$ (il bordo d'uscita); la velocità si calcola con la (5.7),

$$V = U \left| \frac{e^{-i\alpha} - (a/\zeta')^2 e^{i\alpha} + i\Gamma/(2\pi U\zeta')}{1 - (b/\zeta)^2} \right|, \tag{5.15}$$

e il coefficiente di pressione con la (1.10). In questo modo si ottengono le posizioni esatte dei punti del profilo (la ricerca darebbe solo posizioni approssimate) e si escludono automaticamente i punti della linea $\psi = 0$ che non stanno sul profilo.

5. Si integra la pressione lungo il contorno per verificare il teorema di Kutta–Joukowski e il paradosso di d'Alembert.

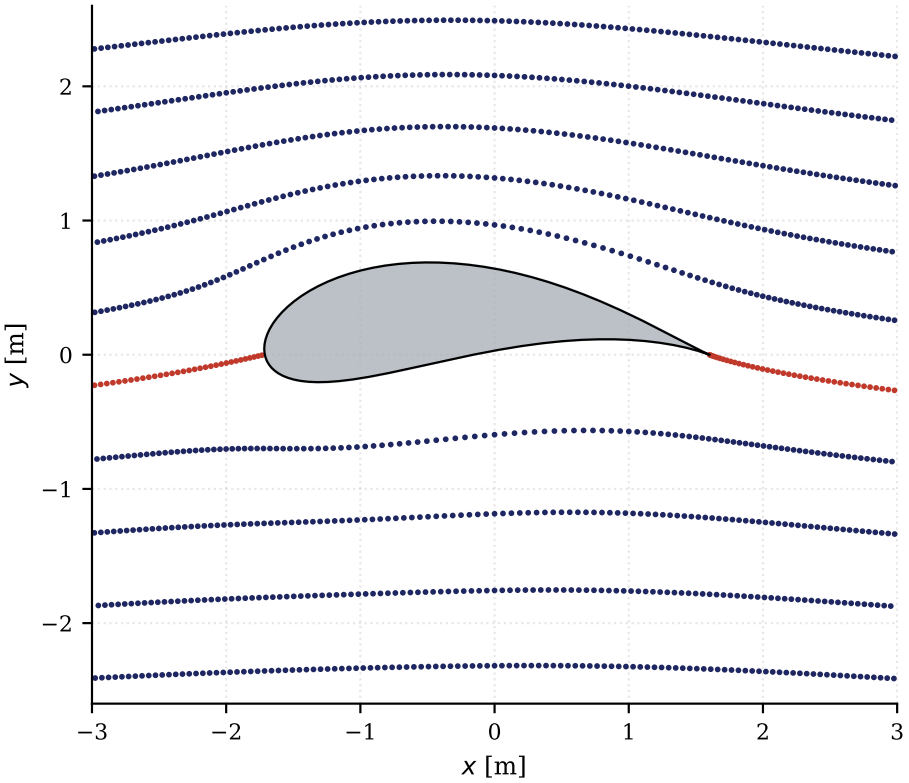


Figura 5.2: Programma 2.4: linee di corrente attorno al profilo di Joukowski ($\alpha = 0$), ottenute trasformando i punti trovati nel piano del cerchio. In rosso la linea $\psi = 0$.

Risultati

- Geometria: bordo d'attacco in $x = -1{,}716$ m, bordo d'uscita in $x = 2b = 1{,}600$ m, corda $c = 3{,}316$ m (un po' più di $4b$). Spessore massimo 24,3% c al 25% della corda; freccia massima 10,1% c a metà corda. È un profilo spesso e molto curvo.
- Circolazione $\Gamma = 4\pi\eta_P U = 2{,}5007$ m^2/s; $C_L = 2\Gamma/(Uc) = 1{,}5083$. L'integrale della pressione sul contorno dà $C_L = 1{,}5083$ e $C_D = -4 \cdot 10^{-9}$.
- Nella Fig. 5.2 lo spazio fra il profilo e la linea $\psi = 0{,}5$ è più stretto di quello fra il profilo e la linea $\psi = -0{,}5$: la velocità è più alta sul dorso. Il ramo inferiore della curva di Fig. 5.3 (con l'asse dei C_p rovesciato, come d'uso) appartiene al dorso, quello superiore al ventre; l'area racchiusa dalla curva è proporzionale alla portanza.

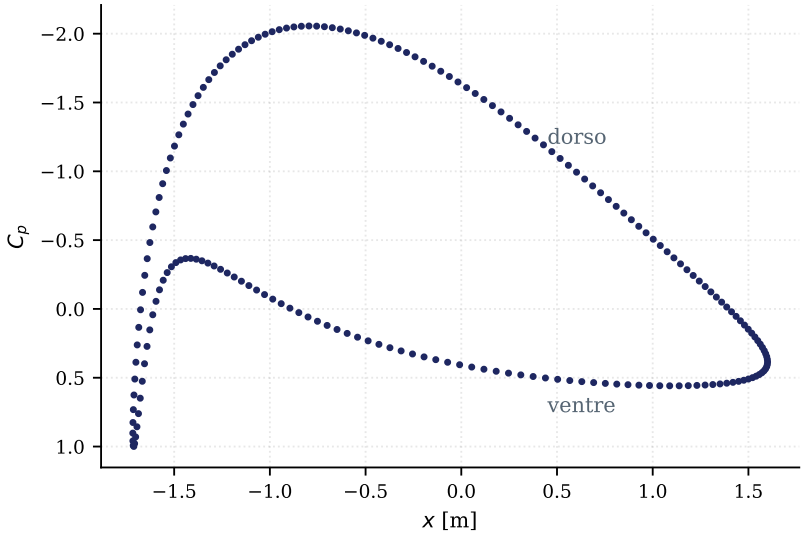


Figura 5.3: Programma 2.4: distribuzione del coefficiente di pressione sui duecento punti del profilo.

- Nel ristagno anteriore $C_p = 1$; il minimo è $C_p = -2{,}056$; nel bordo d'uscita $C_p = 0{,}385$, cioè $V = 0{,}78\,U$: la velocità è finita ma non nulla, pur essendo l'immagine del ristagno Q.

Il programma generalizza il calcolo all'incidenza $\alpha \neq 0$, che nella (5.12) compare sia nella corrente sia nella circolazione di Kutta. La Fig. 5.4 mostra come cresce il picco di depressione sul dorso vicino al bordo d'attacco e come varia C_L: la curva è quasi una retta, con angolo di portanza nulla $\alpha_{0L} = -\beta = -11{,}5°$, e si discosta lentamente dalla legge lineare $8\pi a(\alpha+\beta)/c$ per la presenza del seno nella (5.14) ($C_L = 3{,}020$ contro 3,106 a $\alpha = 12°$).

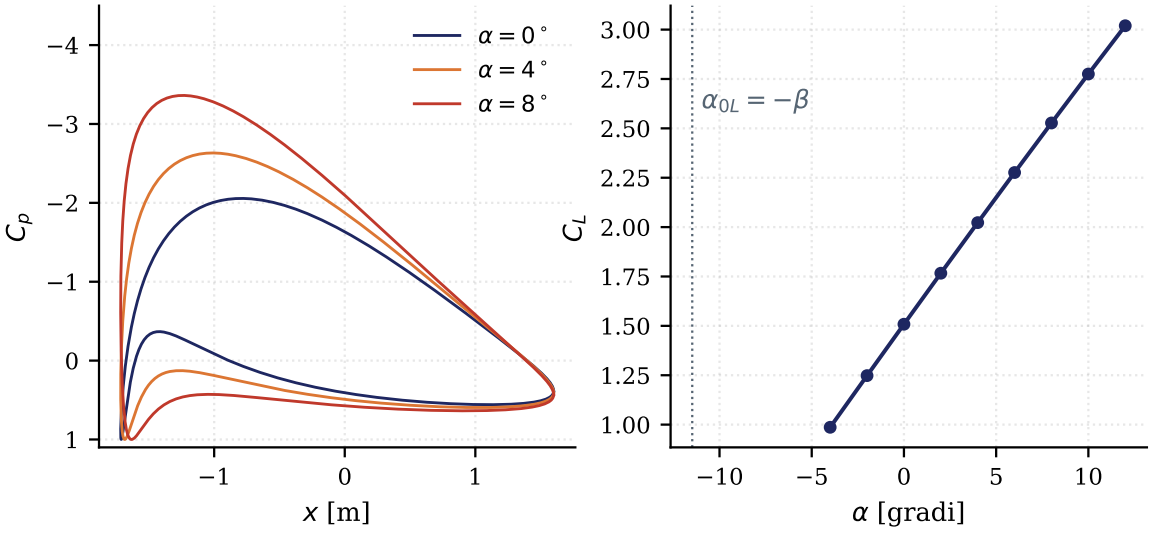


Figura 5.4: Profilo di Joukowski a incidenza variabile: distribuzioni di C_p e curva $C_L(\alpha)$.

prog2_4_joukowski.py

```
"""
prog2_4_joukowski.py -- PROGRAMMA 2.4
Dal flusso attorno a un cilindro circolare al flusso attorno a un profilo di
Joukowski mediante la trasformazione conforme  z = zeta + b^2/zeta.

Piano zeta (del cerchio): cerchio di raggio a centrato in P = (x_P, y_P), con
x_P = b - sqrt(a^2 - y_P^2) cosi' che il cerchio passi per Q = (b, 0).
Potenziale complesso (corrente U inclinata di alfa, condizione di Kutta in Q):
   w = U [ zeta' e^{-i alfa} + a^2 e^{i alfa}/zeta' ] + i Gamma/(2 pi) ln(zeta'/a),
   zeta' = zeta - zeta_P,   Gamma = 4 pi a U sin(alfa + beta),  sin beta = y_P/a.
(Per alfa = 0 si ritrova w = U [zeta' + a^2/zeta' + i 2 y_P ln(zeta'/a)].)
"""
import numpy as np
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, ROSSO, GRIGIO, VERDE
from metodi_numerici import cerca_x

U, A, B, YP = 1.0, 1.0, 0.8, 0.199
XP = B - np.sqrt(A**2 - YP**2)
ZP = XP + 1j * YP
BETA = np.arcsin(YP / A)


def joukowski(zeta):
    return zeta + B**2 / zeta


def circolazione(alfa):
    """Circolazione (oraria positiva) che soddisfa la condizione di Kutta."""
    return 4 * np.pi * A * U * np.sin(alfa + BETA)


def w_complesso(zeta, alfa=0.0):
    zl = zeta - ZP
    gam = circolazione(alfa)
    return (U * (zl * np.exp(-1j * alfa) + A**2 * np.exp(1j * alfa) / zl)
            + 1j * gam / (2 * np.pi) * np.log(zl / A))


def _velocita_cerchio(fi, alfa):
    """|dw/dz| nel punto del cerchio di anomalia fi (fi misurata da P)."""
    zl = A * np.exp(1j * fi)                       # zeta' sul cerchio
    zeta = zl + ZP
    gam = circolazione(alfa)
    dw_dzeta = U * (np.exp(-1j * alfa) - A**2 * np.exp(1j * alfa) / zl**2) \
        + 1j * gam / (2 * np.pi * zl)
    dz_dzeta = 1 - (B / zeta)**2
    return np.abs(dw_dzeta / dz_dzeta)


def superficie(npunti=200, alfa=0.0):
    """Punti esatti del profilo e velocita' (formula 2.6.25 generalizzata).
    Il primo punto e' il bordo d'uscita (immagine di Q, fi = -beta): li'
    numeratore e denominatore si annullano e la velocita' e' il limite,
    calcolato come media dei valori a fi = -beta +- 1e-5."""
    fi = -BETA + np.linspace(0.0, 2 * np.pi, npunti, endpoint=False)
    z = joukowski(A * np.exp(1j * fi) + ZP)
    with np.errstate(divide="ignore", invalid="ignore"):
        V = _velocita_cerchio(fi, alfa)
    V[0] = 0.5 * (_velocita_cerchio(fi[0] - 1e-5, alfa) +
```

```
                    _velocita_cerchio(fi[0] + 1e-5, alfa))
    return fi, z, V


if __name__ == "__main__":
    print(f"centro del cerchio P = ({XP:.4f}, {YP:.4f}),  beta =
    {np.degrees(BETA):.3f} gradi")

    # ---------------- geometria del profilo --------------------------------
    fi, z, V = superficie(2000)
    x, y = z.real, z.imag
    i_le, i_te = np.argmin(x), np.argmax(x)
    corda = x[i_te] - x[i_le]
    # dorso e ventre: si interpola su una stessa ascissa
    xs = np.linspace(x[i_le], x[i_te], 400)
    up = np.interp(xs, *zip(*sorted(zip(x[fi < fi[i_le]], y[fi < fi[i_le]]))))
    lo = np.interp(xs, *zip(*sorted(zip(x[fi > fi[i_le]], y[fi > fi[i_le]]))))
    spess = up - lo
    curv = 0.5 * (up + lo)
    print(f"bordo d'attacco x = {x[i_le]:.4f}, bordo d'uscita x = {x[i_te]:.4f}")
    print(f"corda c = {corda:.4f} (4b = {4*B:.4f})")
    print(f"spessore massimo = {spess.max():.4f} = {100*spess.max()/corda:.2f}% c "
          f"a x/c = {(xs[np.argmax(spess)]-x[i_le])/corda:.3f}")
    print(f"freccia massima  = {curv.max():.4f} = {100*curv.max()/corda:.2f}% c "
          f"a x/c = {(xs[np.argmax(curv)]-x[i_le])/corda:.3f}")

    # ---------------- linee di corrente: ricerca nel piano zeta + mappa ----
    DX1 = 0.05
    x1 = np.arange(-3.0, 3.0 + 1e-9, DX1)
    y1 = np.arange(-3.0, 3.0 + 1e-9, DX1)
    X1, Y1 = np.meshgrid(x1, y1, indexing="ij")
    with np.errstate(divide="ignore", invalid="ignore"):
        PSI = w_complesso(X1 + 1j * Y1).imag
    PSI[np.abs(X1 + 1j * Y1 - ZP) < A] = np.nan    # interno del cerchio
    COST = -2.0 + 0.5 * np.arange(10)
    fig, ax = plt.subplots(figsize=(6.2, 4.6))
    for k, c in enumerate(COST):
        xx, yy = cerca_x(x1, y1, PSI, c)
        zm = joukowski(xx + 1j * yy)
        tieni = (np.abs(zm.real) <= 3) & (np.abs(zm.imag) <= 3)
        ax.plot(zm.real[tieni], zm.imag[tieni], ".", ms=2.2,
                color=ROSSO if c == 0 else BLU)
    ax.fill(x, y, color=GRIGIO, alpha=0.4)
    ax.plot(x, y, color="k", lw=0.8)
    ax.set_aspect("equal"); ax.set_xlim(-3, 3); ax.set_ylim(-2.6, 2.6)
    ax.set_xlabel("$x$ [m]"); ax.set_ylabel("$y$ [m]")
    salva(fig, "prog2_4_linee")

    # ---------------- distribuzione di pressione ---------------------------
    fi, z, V = superficie(200)
    cp = 1 - (V / U)**2
    gam = circolazione(0.0)
    # portanza dall'integrale della pressione (contorno chiuso, normale esterna)
    fid, zd, Vd = superficie(20000)
    cpd = 1 - (Vd / U)**2
    zc = 0.5 * (zd + np.roll(zd, -1))
    cpc = 0.5 * (cpd + np.roll(cpd, -1))
    cpc[~np.isfinite(cpc)] = 0.0
    dz = np.roll(zd, -1) - zd                    # contorno percorso in senso
    antiorario
```

```
    # forza = -oint p n ds; per contorno antiorario n ds = -i dz  ->  F = i oint p dz
    F = 1j * np.sum(cpc * dz)                    # in unita' di (1/2) rho U^2
    CL_p, CD_p = F.imag / corda, F.real / corda
    print(f"\nGamma = 4 pi y_P U = {gam:.5f} m^2/s")
    print(f"C_L (Kutta-Joukowski) = 2 Gamma/(U c) = {2*gam/(U*corda):.5f}")
    print(f"C_L (integrale di C_p) = {CL_p:.5f},   C_D (integrale) = {CD_p:+.2e}")
    print(f"C_p al bordo d'uscita = {cp[0]:.4f};  C_p minimo = {cp.min():.4f}")

    fig, ax = plt.subplots(figsize=(5.4, 3.6))
    ax.plot(z.real, cp, ".", color=BLU, ms=4)
    ax.invert_yaxis()
    ax.set_xlabel("$x$ [m]"); ax.set_ylabel("$C_p$")
    ax.annotate("ventre", (0.45, 0.75), color=GRIGIO)
    ax.annotate("dorso", (0.45, -1.2), color=GRIGIO)
    salva(fig, "prog2_4_cp")

    # ---------------- estensione: incidenza variabile -----------------------
    print("\n alfa [gradi]   C_L esatto   8 pi a (alfa+beta)/c  (seno linearizzato)")
    alfas = np.radians(np.arange(-4, 13, 2))
    CL = [2 * circolazione(al) / (U * corda) for al in alfas]
    for al, cl in zip(alfas, CL):
        print(f"   {np.degrees(al):5.1f}       {cl:7.4f}       {8*np.pi*A*(al+BETA)/corda:7.4f}")
    fig, ax = plt.subplots(1, 2, figsize=(6.6, 2.8))
    for al, col in zip(np.radians([0, 4, 8]), [BLU, ARANCIO, ROSSO]):
        fi, z, V = superficie(400, al)
        ax[0].plot(z.real, 1 - V**2, color=col, lw=1.0,
                   label=fr"$\alpha={np.degrees(al):.0f}^\circ$")
    ax[0].set_ylim(1.1, -4.5); ax[0].set_xlabel("$x$ [m]"); ax[0].set_ylabel("$C_p$")
    ax[0].legend()
    ax[1].plot(np.degrees(alfas), CL, "o-", color=BLU, ms=3.5)
    ax[1].axvline(-np.degrees(BETA), color=GRIGIO, ls=":", lw=0.8)
    ax[1].annotate(r"$\alpha_{0L}=-\beta$", (-np.degrees(BETA) + 0.5, 2.6), color=GRIGIO)
    ax[1].set_xlabel(r"$\alpha$ [gradi]"); ax[1].set_ylabel("$C_L$")
    salva(fig, "prog2_4_incidenza")
```

Listato 5.1: Programma 2.4 – profilo di Joukowski per trasformazione conforme (`prog2_4_joukowski.py`).

5.6 Dal profilo di Joukowski ai profili reali: il metodo dei pannelli

I profili di Joukowski hanno avuto grande importanza storica, ma non sono i profili che si usano sulle ali: hanno il bordo d'uscita a cuspide, troppo sottile per essere costruito, e la loro famiglia ha solo tre parametri. Per un profilo di forma qualsiasi, ad esempio un NACA a quattro cifre [14], serve un metodo *diretto*. Il più diffuso è il **metodo dei pannelli**: il contorno si approssima con N segmenti rettilinei (i pannelli) e su ciascuno si distribuisce una singolarità, la cui intensità si determina imponendo la tangenza della corrente in un punto di controllo per pannello. È l'erede del metodo di von Kármán, con le singolarità sulla superficie invece che sull'asse (§4.5).

Pannelli di vortice a intensità lineare

1. Il contorno è percorso in senso orario: dal bordo d'uscita lungo il ventre fino al bordo d'attacco, poi lungo il dorso di nuovo al bordo d'uscita. I nodi (X_j, Y_j), $j = 1, \dots, N+1$, delimitano N pannelli di lunghezza S_j e inclinazione θ_j; il primo e l'ultimo nodo coincidono nel bordo d'uscita. I punti di controllo (x_i, y_i) sono i punti medi dei pannelli.
2. Su ogni pannello la densità di vortici γ varia linearmente fra i valori γ_j e γ_{j+1} nei nodi: la distribuzione è continua lungo tutto il contorno, e le incognite sono gli $N+1$ valori nodali.
3. La velocità indotta nel punto i da un pannello con γ lineare ha un'espressione in forma chiusa (integrale della legge di Biot–Savart lungo il segmento [11]), che si scrive come $\gamma_j\, C_{ij}^{(1)} + \gamma_{j+1}\, C_{ij}^{(2)}$ con coefficienti di influenza che dipendono solo dalla geometria. Sommando i contributi dei due pannelli che condividono il nodo j si ottengono le matrici A_{ij}^{n} (componente normale) e A_{ij}^{t} (componente tangenziale).
4. *Condizione di tangenza* negli N punti di controllo: con $\gamma'_j = \gamma_j/(2\pi U)$,
$$\sum_{j=1}^{N+1} A_{ij}^{n}\gamma'_j = \sin(\theta_i - \alpha), \qquad i = 1, \dots, N.$$
5. *Condizione di Kutta*: il bordo d'uscita è affilato e la densità di vortici vi si annulla; poiché il primo e l'ultimo nodo coincidono e il contorno vi è percorso in versi opposti, la condizione si scrive $\gamma'_1 + \gamma'_{N+1} = 0$. È l'equazione $N+1$.
6. Risolto il sistema, la velocità tangenziale nei punti di controllo è $V_i/U = \cos(\theta_i - \alpha) + \sum_j A_{ij}^{t}\gamma'_j$, da cui $C_p = 1 - (V_i/U)^2$. La portanza si ottiene integrando la pressione oppure dalla circolazione totale $\Gamma = \sum_j \tfrac{1}{2}(\gamma_j + \gamma_{j+1})S_j$; i due valori devono coincidere, e il loro confronto è un controllo del calcolo.

Il programma `prog2_4c_pannelli.py` (Listato 5.2) costruisce i coefficienti di influenza in forma vettoriale (matrici $N \times N$ di NumPy) e li mette alla prova in tre passi.

1. Validazione sul profilo di Joukowski. I nodi si prendono sul profilo del Programma 2.4, ottenuti trasformando $N+1$ punti equispaziati in anomalia sul cerchio (si addensano spontaneamente vicino ai bordi, dove la curvatura è maggiore); la soluzione dei pannelli si confronta con quella esatta della trasformazione conforme.

N	C_L pannelli	C_L esatto	$\max\lvert C_p - C_p^{\text{es}}\rvert$, $x/c < 0{,}9$
40	1,437 74	1,508 26	$1{,}06 \times 10^{-1}$
80	1,557 79	1,508 26	$9{,}71 \times 10^{-2}$
160	1,517 25	1,508 26	$3{,}46 \times 10^{-2}$
320	1,513 55	1,508 26	$1{,}78 \times 10^{-2}$

Il coefficiente di portanza converge al valore esatto (0,35% di scarto con 320 pannelli) e l'errore sulla pressione si dimezza a ogni raddoppio di N (Fig. 5.5). Vicino al bordo d'uscita l'accordo è peggiore: la cuspide, dove dorso e ventre sono tangenti, è la situazione più sfavorevole per un metodo a pannelli, perché pannelli quasi sovrapposti si influenzano con coefficienti quasi uguali.

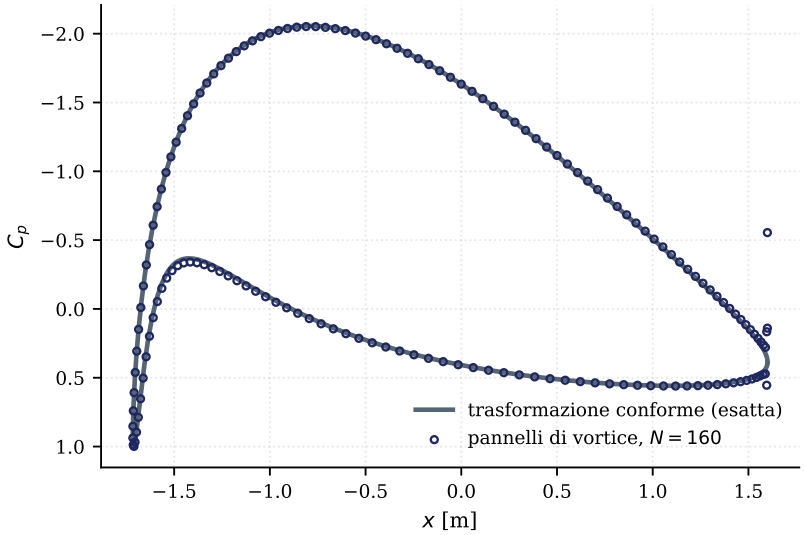


Figura 5.5: Validazione del metodo dei pannelli sul profilo di Joukowski: C_p esatto (trasformazione conforme) e con 160 pannelli.

2. Confronto con dati di riferimento. Per il profilo simmetrico NACA 0009 a incidenza nulla, con 160 pannelli, la velocità sul dorso differisce dai valori tabulati [14] al più di $0{,}008\,U$ (in $x/c = 0{,}05$), e in genere di 2–3 millesimi:

x/c	0,05	0,1	0,2	0,3	0,4	0,5	0,6	0,7	0,8	0,9
V/U tabella	1,140	1,144	1,137	1,119	1,100	1,082	1,061	1,043	1,018	0,982
V/U pannelli	1,132	1,143	1,134	1,117	1,098	1,080	1,061	1,041	1,018	0,986

3. Applicazione ai profili NACA a quattro cifre. Per i profili NACA 0012, 2412 e 4415, a incidenze da $-4°$ a $10°$, il programma calcola C_L e il coefficiente di momento rispetto al punto a un quarto della corda, $C_{m,c/4}$ (positivo se cabrante), e li confronta con la teoria dei profili sottili.

Risultati: NACA 0012, 2412 e 4415 ($N = 160$)

		α_{0L}		$C_{m,c/4}$	
profilo	$dC_L/d\alpha$ [1/rad]	pannelli	profili sottili	pannelli	profili sottili
0012	6,85	0°	0°	−0,004	0
2412	6,84	−2,16°	−2,08°	−0,060	−0,053
4415	6,98	−4,35°	−4,15°	−0,118	−0,106

La pendenza della retta di portanza è circa il 9–11% maggiore di 2π: è l'effetto dello spessore, che la teoria dei profili sottili trascura. L'angolo di portanza nulla e il momento dipendono soprattutto dalla curvatura della linea media e sono vicini ai valori della teoria dei profili sottili. A $\alpha = 4°$ il NACA 2412 ha $C_L = 0{,}737$ (Fig. 5.7).

Che cosa manca alla teoria non viscosa

Le misure in galleria [14] danno per il NACA 2412 una pendenza di circa 0,105 per grado (6,0/rad) e $\alpha_{0L} \simeq -2{,}1°$: l'angolo di portanza nulla è previsto bene, la pendenza è

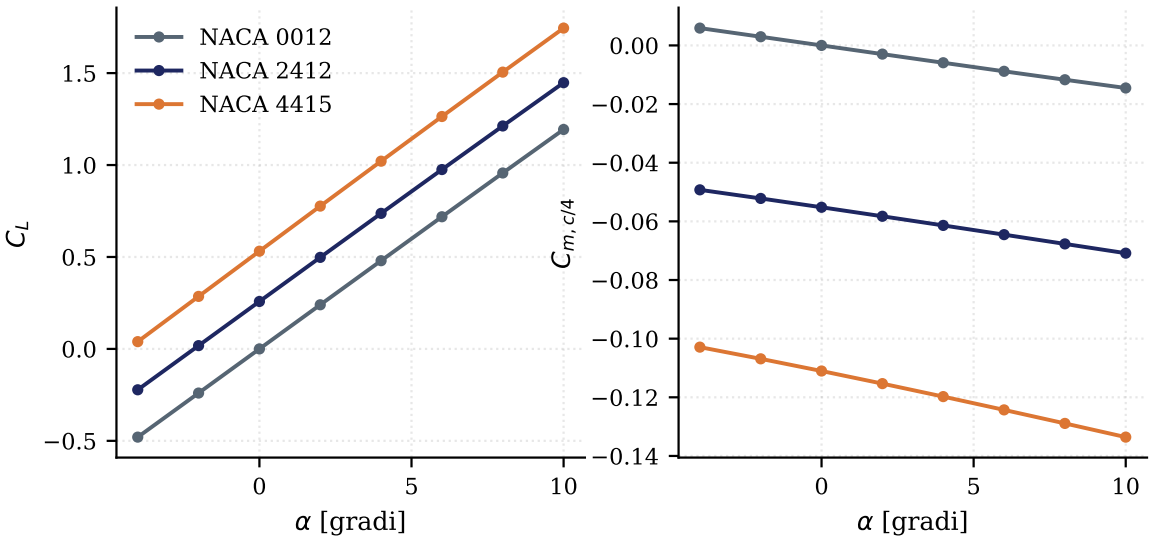


Figura 5.6: Metodo dei pannelli: coefficiente di portanza e di momento per tre profili NACA.

sovrastimata del 13% circa. La differenza è dovuta allo strato limite, che ispessisce il profilo "visto" dalla corrente esterna e riduce la circolazione rispetto a quella imposta dalla condizione di Kutta. Ancora più evidente è il limite verso le alte incidenze: il metodo dei pannelli non conosce lo stallo e la sua retta di portanza prosegue indefinitamente. I codici di progetto (ad esempio XFOIL) accoppiano il metodo dei pannelli a un calcolo dello strato limite, proprio secondo lo schema di Prandtl del §1.1.

prog2_4c_pannelli.py

```
"""
prog2_4c_pannelli.py -- metodo dei pannelli di vortice a intensita' lineare
per profili di forma qualsiasi (metodo diretto).

Il contorno e' approssimato da N pannelli rettilinei percorsi in senso orario
(bordo d'uscita -> ventre -> bordo d'attacco -> dorso -> bordo d'uscita).
Su ogni pannello la vorticita' gamma varia linearmente; le N+1 incognite sono i
valori di gamma nei nodi. Equazioni: velocita' normale nulla nei N punti di
controllo (punti medi) + condizione di Kutta  gamma_1 + gamma_{N+1} = 0.
Coefficienti di influenza in forma chiusa (formulazione classica a vortici
lineari; le incognite sono gamma' = gamma / (2 pi V_inf)).

1) Validazione: profilo di Joukowski del Programma 2.4 (soluzione esatta nota).
2) Verifica con dati: NACA 0009 a incidenza nulla (velocita' sul dorso).
3) Applicazione: NACA 2412 e 4415, C_L e C_m,c/4 in funzione di alfa.
"""
import numpy as np
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, ROSSO, GRIGIO, VERDE, VIOLA
import prog2_4_joukowski as jk


def naca4(codice, n_pannelli=100, corda=1.0):
    """Nodi di un profilo NACA a 4 cifre (spaziatura a coseno, bordo d'uscita
    chiuso), ordinati in senso orario a partire dal bordo d'uscita."""
    m = int(codice[0]) / 100.0
    p = int(codice[1]) / 10.0
    t = int(codice[2:]) / 100.0
    nlato = n_pannelli // 2
    x = 0.5 * (1 - np.cos(np.linspace(0, np.pi, nlato + 1)))
```

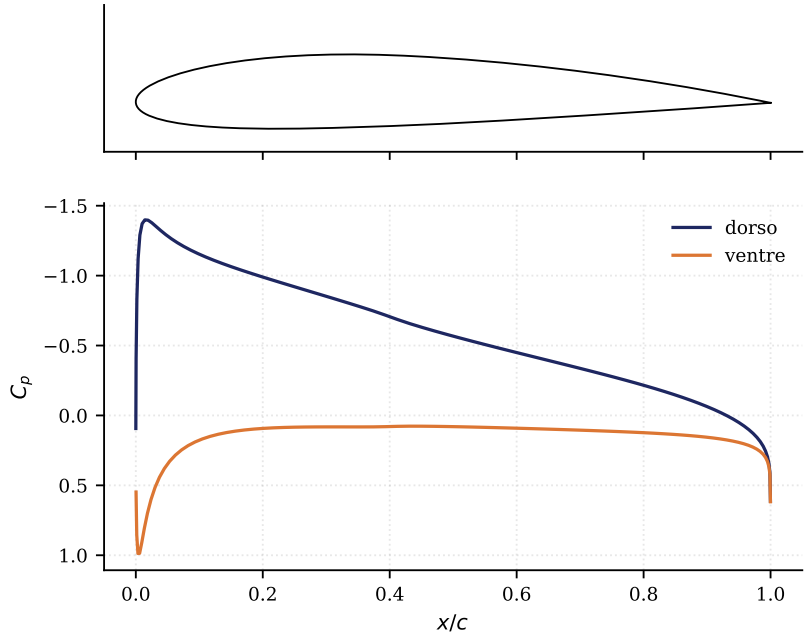


Figura 5.7: Distribuzione di C_p sul NACA 2412 a $\alpha = 4°$ (160 pannelli).

```
    yt = 5 * t * (0.2969 * np.sqrt(x) - 0.1260 * x - 0.3516 * x**2
                  + 0.2843 * x**3 - 0.1036 * x**4)          # -0.1036: b.u. chiuso
    yc = np.zeros_like(x); dyc = np.zeros_like(x)
    if m > 0:
        a = x < p
        yc[a] = m / p**2 * (2 * p * x[a] - x[a]**2)
        dyc[a] = 2 * m / p**2 * (p - x[a])
        yc[~a] = m / (1 - p)**2 * ((1 - 2 * p) + 2 * p * x[~a] - x[~a]**2)
        dyc[~a] = 2 * m / (1 - p)**2 * (p - x[~a])
    th = np.arctan(dyc)
    xu, yu = x - yt * np.sin(th), yc + yt * np.cos(th)
    xl, yl = x + yt * np.sin(th), yc - yt * np.cos(th)
    XB = np.concatenate([xl[::-1], xu[1:]]) * corda     # b.u. -> ventre -> b.a. ->
    dorso
    YB = np.concatenate([yl[::-1], yu[1:]]) * corda
    return XB, YB


def pannelli_vortice(XB, YB, alfa):
    """Restituisce i punti di controllo, gli angoli dei pannelli, la velocita'
    tangenziale V/V_inf, C_p e i valori gamma' nei nodi."""
    M = len(XB) - 1
    X = 0.5 * (XB[:-1] + XB[1:]); Y = 0.5 * (YB[:-1] + YB[1:])
    S = np.hypot(np.diff(XB), np.diff(YB))
    TH = np.arctan2(np.diff(YB), np.diff(XB))
    sn, cs = np.sin(TH), np.cos(TH)
    # matrici dei coefficienti (i = punto di controllo, j = pannello)
    dx = X[:, None] - XB[None, :-1]
    dy = Y[:, None] - YB[None, :-1]
    Ai = -dx * cs[None, :] - dy * sn[None, :]
    Bi = dx**2 + dy**2
    Ci = np.sin(TH[:, None] - TH[None, :])
    Di = np.cos(TH[:, None] - TH[None, :])
    Ei = dx * sn[None, :] - dy * cs[None, :]
    Sj = S[None, :]
    with np.errstate(divide="ignore", invalid="ignore"):
```

```
        F = np.log(1.0 + Sj * (Sj + 2.0 * Ai) / Bi)
        G = np.arctan2(Ei * Sj, Bi + Ai * Sj)
        P = dx * np.sin(TH[:, None] - 2 * TH[None, :]) + dy * np.cos(TH[:, None] -
    2 * TH[None, :])
        Q = dx * np.cos(TH[:, None] - 2 * TH[None, :]) - dy * np.sin(TH[:, None] -
    2 * TH[None, :])
        CN2 = Di + 0.5 * Q * F / Sj - (Ai * Ci + Di * Ei) * G / Sj
        CN1 = 0.5 * Di * F + Ci * G - CN2
        CT2 = Ci + 0.5 * P * F / Sj + (Ai * Di - Ci * Ei) * G / Sj
        CT1 = 0.5 * Ci * F - Di * G - CT2
    k = np.arange(M)
    CN1[k, k], CN2[k, k] = -1.0, 1.0           # auto-influenza del pannello
    CT1[k, k], CT2[k, k] = 0.5 * np.pi, 0.5 * np.pi
    AN = np.zeros((M + 1, M + 1)); AT = np.zeros((M, M + 1))
    AN[:M, :M] += CN1; AN[:M, 1:] += CN2
    AT[:, :M] += CT1; AT[:, 1:] += CT2
    AN[M, 0] = AN[M, M] = 1.0                  # condizione di Kutta
    rhs = np.concatenate([np.sin(TH - alfa), [0.0]])
    gam = np.linalg.solve(AN, rhs)
    V = np.cos(TH - alfa) + AT @ gam
    cp = 1.0 - V**2
    return X, Y, TH, S, V, cp, gam


def coefficienti(XB, YB, X, Y, TH, S, cp, gam, alfa, corda=1.0):
    """C_L dall'integrale della pressione e dalla circolazione; C_m,c/4."""
    nx, ny = -np.sin(TH), np.cos(TH)            # normale esterna (contorno orario)
    fx = np.sum(-cp * nx * S) / corda
    fy = np.sum(-cp * ny * S) / corda
    CL = fy * np.cos(alfa) - fx * np.sin(alfa)
    CD = fx * np.cos(alfa) + fy * np.sin(alfa)
    Gam = 2 * np.pi * np.sum(0.5 * (gam[:-1] + gam[1:]) * S)   # in unita' di V_inf
    CL_gamma = 2 * Gam / corda
    xr = X - 0.25 * corda
    # momento rispetto a c/4, positivo se cabrante (a naso in su)
    Cm = np.sum(cp * (xr * ny - Y * nx) * S) / corda**2
    return CL, CD, CL_gamma, Cm


if __name__ == "__main__":
    # ------------------------------------------------ 1) validazione Joukowski
    print("1) Profilo di Joukowski (a=1, b=0,8, y_P=0,199): confronto con l'esatto")
    print("   N     C_L pannelli   C_L esatto   max|C_p - C_p,ex| per x/c < 0,9")
    for alfa_g in (0.0,):
        alfa = np.radians(alfa_g)
        for N in (40, 80, 160, 320):
            fi = -jk.BETA - np.linspace(0, 2 * np.pi, N + 1)       # senso orario
            zb = jk.joukowski(jk.A * np.exp(1j * fi) + jk.ZP)
            XB, YB = zb.real, zb.imag
            X, Y, TH, S, V, cp, gam = pannelli_vortice(XB, YB, alfa)
            corda = XB.max() - XB.min()
            CL, CD, CLg, Cm = coefficienti(XB, YB, X, Y, TH, S, cp, gam, alfa,
    corda)
            # soluzione esatta nei punti di controllo: anomalia media del pannello
            fim = 0.5 * (fi[:-1] + fi[1:])
            Vex = jk._velocita_cerchio(fim, alfa)
            cpex = 1 - Vex**2
            xs = (X - XB.min()) / (XB.max() - XB.min())
            err = np.max(np.abs(cp - cpex)[xs < 0.9])
            CLex = 2 * jk.circolazione(alfa) / corda
```

```
            print(f"  {N:4d}     {CL:8.5f}      {CLex:8.5f}      {err:.2e}")
            if N == 160:
                salvati = (X, cp, fim, cpex)
    X, cp, fim, cpex = salvati
    fi_f, z_f, V_f = jk.superficie(400, 0.0)
    fig, ax = plt.subplots(figsize=(5.4, 3.6))
    ax.plot(z_f.real, 1 - V_f**2, color=GRIGIO, lw=2.0, label="trasformazione conforme (esatta)")
    ax.plot(X, cp, "o", color=BLU, ms=3, mfc="none", label="pannelli di vortice, $N=160$")
    ax.invert_yaxis(); ax.set_xlabel("$x$ [m]"); ax.set_ylabel("$C_p$"); ax.legend()
    salva(fig, "prog2_4c_validazione")

    # ------------------------------------------------ 2) NACA 0009, alfa = 0
    tab_x = np.array([0.05, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9])
    tab_V = np.array([1.140, 1.144, 1.137, 1.119, 1.100, 1.082, 1.061, 1.043, 1.018, 0.982])
    XB, YB = naca4("0009", 160)
    X, Y, TH, S, V, cp, gam = pannelli_vortice(XB, YB, 0.0)
    dorso = Y > 0
    Vd = np.interp(tab_x, X[dorso], np.abs(V[dorso]))
    print("\n2) NACA 0009, alfa = 0: velocita' sul dorso")
    print("   x/c    V/U tabella   V/U pannelli   scarto")
    for xx, vt, vp in zip(tab_x, tab_V, Vd):
        print(f"  {xx:4.2f}     {vt:5.3f}        {vp:6.4f}      {vp-vt:+.4f}")

    # ------------------------------------------------ 3) NACA 2412 e 4415
    print("\n3) Curve C_L(alfa) e C_m,c/4(alfa) (N = 160)")
    alfas = np.radians(np.arange(-4, 11, 2))
    fig, ax = plt.subplots(1, 2, figsize=(6.6, 2.9))
    for codice, col in (("0012", GRIGIO), ("2412", BLU), ("4415", ARANCIO)):
        XB, YB = naca4(codice, 160)
        CLs, Cms = [], []
        for al in alfas:
            X, Y, TH, S, V, cp, gam = pannelli_vortice(XB, YB, al)
            CL, CD, CLg, Cm = coefficienti(XB, YB, X, Y, TH, S, cp, gam, al)
            CLs.append(CL); Cms.append(Cm)
        CLs = np.array(CLs)
        pend = np.polyfit(alfas, CLs, 1)
        a0L = -pend[1] / pend[0]
        print(f"  NACA {codice}: dC_L/dalfa = {pend[0]:.3f} /rad ({pend[0]/(2*np.pi):.3f} x 2 pi), "
              f"alfa_0L = {np.degrees(a0L):+.2f} gradi, C_L(4 gradi) = {CLs[4]:.3f}, "
              f"C_m,c/4 medio = {np.mean(Cms):+.4f}")
        ax[0].plot(np.degrees(alfas), CLs, "o-", ms=3, color=col, label=f"NACA {codice}")
        ax[1].plot(np.degrees(alfas), Cms, "o-", ms=3, color=col)
    ax[0].set_xlabel(r"$\alpha$ [gradi]"); ax[0].set_ylabel("$C_L$"); ax[0].legend()
    ax[1].set_xlabel(r"$\alpha$ [gradi]"); ax[1].set_ylabel(r"$C_{m,c/4}$")
    salva(fig, "prog2_4c_naca")

    # distribuzione di C_p per NACA 2412 a 4 gradi
    XB, YB = naca4("2412", 160)
    X, Y, TH, S, V, cp, gam = pannelli_vortice(XB, YB, np.radians(4.0))
    fig, ax = plt.subplots(2, 1, figsize=(5.4, 4.4), sharex=True,
                           gridspec_kw={"height_ratios": [1.2, 3]})
    ax[0].plot(XB, YB, color="k", lw=0.8)
    ax[0].set_ylim(-0.08, 0.16); ax[0].set_yticks([]); ax[0].grid(False)
    sup = np.arange(len(X)) >= len(X) // 2
```

```
177     ax[1].plot(X[sup], cp[sup], "-", color=BLU, label="dorso")
178     ax[1].plot(X[~sup], cp[~sup], "-", color=ARANCIO, label="ventre")
179     ax[1].invert_yaxis(); ax[1].set_xlabel("$x/c$"); ax[1].set_ylabel("$C_p$");
        ax[1].legend()
180     salva(fig, "prog2_4c_cp2412")
```

Listato 5.2: `prog2_4c_pannelli.py`: metodo dei pannelli di vortice a intensità lineare; validazione e profili NACA.

Capitolo 6

Classificazione delle equazioni alle derivate parziali e metodi per le equazioni ellittiche

6.1 Subsonico e supersonico: due modi di propagare i disturbi

Si consideri un corpo che si muove in un fluido compressibile non viscoso. I disturbi che esso genera si propagano in tutte le direzioni alla velocità del suono. Se il corpo è molto più lento del suono, i disturbi raggiungono l'infinito in ogni direzione e il campo di moto somiglia a quello potenziale incomprimibile: nel sistema di riferimento del corpo la corrente si deforma *sia a monte sia a valle*. Se invece il corpo è supersonico, sorpassa i disturbi che esso stesso emette in avanti e se li lascia tutti alle spalle. Attorno a un cuneo sottile (Fig. 6.1) la corrente a monte è indisturbata e uniforme; la regione indisturbata è separata da quella disturbata da due rette, le *linee di Mach* o, in linguaggio matematico, le **caratteristiche**. Nel flusso subsonico linee di questo tipo non esistono.

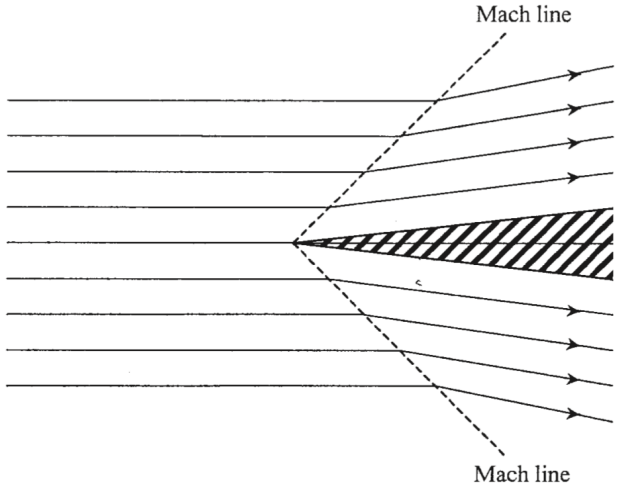


Figura 6.1: Corrente supersonica attorno a un cuneo sottile.

La differenza è scritta nelle equazioni. Attorno a un corpo sottile, che produce solo piccole perturbazioni, il potenziale di perturbazione di una corrente piana subsonica obbedisce

all'equazione linearizzata [11]

$$(1-M^2)\frac{\partial^2\phi}{\partial x^2}+\frac{\partial^2\phi}{\partial y^2}=0, \tag{6.1}$$

dove il numero di Mach M della corrente indisturbata è trattato come costante; per una corrente supersonica la stessa equazione si scrive

$$(M^2-1)\frac{\partial^2\phi}{\partial x^2}-\frac{\partial^2\phi}{\partial y^2}=0. \tag{6.2}$$

Nella (6.1) i due coefficienti hanno lo stesso segno, nella (6.2) segno opposto: le due equazioni sono di *tipo* diverso.

6.2 La classificazione delle equazioni del secondo ordine

Il tipo di un'equazione si determina studiando l'esistenza di caratteristiche, cioè di linee lungo le quali due soluzioni diverse della stessa equazione possono essere "raccordate" (come le linee di Mach della Fig. 6.1). Si consideri l'equazione rappresentativa

$$A\,\phi_{xx}+2B\,\phi_{xy}+C\,\phi_{yy}=D, \tag{6.3}$$

in cui i pedici indicano derivate parziali e A, B, C, D possono dipendere da x, y, ϕ, ϕ_x, ϕ_y.

Le caratteristiche e il discriminante B^2-AC

Se le derivate prime ϕ_x e ϕ_y sono continue, le loro variazioni passando da (x,y) al punto vicino $(x+\mathrm{d}x, y+\mathrm{d}y)$ sono

$$\mathrm{d}x\,\phi_{xx}+\mathrm{d}y\,\phi_{xy}=\mathrm{d}\phi_x, \qquad \mathrm{d}x\,\phi_{xy}+\mathrm{d}y\,\phi_{yy}=\mathrm{d}\phi_y.$$

Insieme alla (6.3) formano un sistema lineare nelle tre derivate seconde. Con la regola di Cramer, ad esempio,

$$\phi_{xy}=\frac{\begin{vmatrix} A & D & C \\ \mathrm{d}x & \mathrm{d}\phi_x & 0 \\ 0 & \mathrm{d}\phi_y & \mathrm{d}y \end{vmatrix}}{\begin{vmatrix} A & 2B & C \\ \mathrm{d}x & \mathrm{d}y & 0 \\ 0 & \mathrm{d}x & \mathrm{d}y \end{vmatrix}}.$$

Se $\mathrm{d}x$ e $\mathrm{d}y$ sono le componenti di uno spostamento *lungo una caratteristica*, le derivate della velocità nella direzione trasversale possono essere discontinue (lungo la linea si raccordano due soluzioni diverse) e le derivate seconde sono indeterminate: il denominatore si annulla. Sviluppandolo:

$$A\left(\frac{\mathrm{d}y}{\mathrm{d}x}\right)^2-2B\frac{\mathrm{d}y}{\mathrm{d}x}+C=0 \quad\Longrightarrow\quad \left(\frac{\mathrm{d}y}{\mathrm{d}x}\right)_{\text{car}}=\frac{B\pm\sqrt{B^2-AC}}{A}. \tag{6.4}$$

Il segno del discriminante decide tutto:

$B^2-AC>0$**, equazione iperbolica:** in ogni punto passano due caratteristiche reali. Vi appartengono le equazioni delle onde. Per la (6.2) $A=M^2-1$, $B=0$, $C=-1$, $B^2-AC=M^2-1>0$ e le pendenze delle caratteristiche sono $\pm1/\sqrt{M^2-1}=\pm\tan\mu$, con $\sin\mu=1/M$: sono le linee di Mach.

$B^2 - AC < 0$**, equazione ellittica:** le radici sono complesse e le caratteristiche non esistono. Sono ellittiche le equazioni di Laplace e di Poisson del Cap. 1 e la (6.1), per la quale $A = 1 - M^2$, $B = 0$, $C = 1$ e $B^2 - AC = -(1 - M^2) < 0$.

$B^2 - AC = 0$**, equazione parabolica:** esiste una sola famiglia di caratteristiche. L'esempio tipico è la conduzione del calore in un conduttore monodimensionale, $\partial T/\partial t = k\,\partial^2 T/\partial x^2$, che nella forma (6.3) (con t al posto di y) ha $A = k$, $B = C = 0$. Anche la diffusione della vorticità in un fluido viscoso, oggetto del prossimo modulo, è descritta da un'equazione parabolica.

Il tipo dell'equazione non è una curiosità matematica: stabilisce quali condizioni vanno assegnate e quale strategia numerica usare.

	ellittica	parabolica	iperbolica
esempio	Laplace, Poisson; flusso subsonico	calore, diffusione; strato limite	onde, flusso supersonico
dipendenza	ogni punto dipende da tutto il contorno	dal passato, non dal futuro	solo dal dominio di dipendenza delimitato dalle caratteristiche
condizioni	su tutto il contorno (problema ai limiti)	iniziali e al contorno	iniziali (posizione e velocità) e al contorno compatibili con le caratteristiche
strategia numerica	sistema algebrico globale, metodi iterativi (questo capitolo)	marcia nel tempo, spesso implicita	marcia nel tempo, esplicita con limite di stabilità (Cap. 8–11)

6.3 L'equazione di Poisson discretizzata

Si debba risolvere l'equazione di Poisson

$$\frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} = q(x, y) \tag{6.5}$$

in un dominio rettangolare, con i valori di f assegnati sul contorno. Il rettangolo si copre con la griglia di Fig. 6.2: il nodo (i, j) è l'intersezione della verticale per x_i e dell'orizzontale per y_j, e $f_{i,j} = f(x_i, y_j)$, $q_{i,j} = q(x_i, y_j)$.

Con la differenza centrata (2.9) per ciascuna derivata seconda,

$$\frac{\partial^2 f}{\partial x^2} = \frac{f_{i+1,j} - 2f_{i,j} + f_{i-1,j}}{(\Delta x)^2} + \mathcal{O}\left((\Delta x)^2\,\partial^4 f/\partial x^4\right),$$

e analogamente per y, la (6.5) è sostituita dall'equazione alle differenze

$$\frac{f_{i+1,j} - 2f_{i,j} + f_{i-1,j}}{(\Delta x)^2} + \frac{f_{i,j+1} - 2f_{i,j} + f_{i,j-1}}{(\Delta y)^2} = q_{i,j}, \tag{6.6}$$

con errore di troncamento dell'ordine di $(\Delta x)^2 + (\Delta y)^2$. Per una griglia quadrata, $\Delta x = \Delta y = h$:

$$f_{i,j} = \tfrac{1}{4}\left(f_{i-1,j} + f_{i+1,j} + f_{i,j-1} + f_{i,j+1}\right) - \tfrac{1}{4}h^2 q_{i,j}. \tag{6.7}$$

Per $q = 0$ (Laplace) la (6.7) afferma che il valore in un nodo interno è la media dei valori nei quattro nodi adiacenti: è la versione discreta del *teorema della media* per le funzioni armoniche, da cui discende il *principio del massimo*: una funzione armonica non ha massimi né minimi all'interno, e i suoi valori estremi stanno sul contorno.

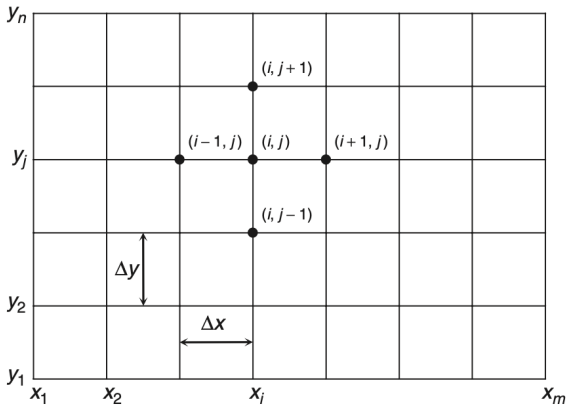


Figura 6.2: Griglia rettangolare e formula a cinque punti.

6.4 Soluzione diretta o iterativa?

Applicata a tutti i nodi interni di una griglia $m \times n$, la (6.7) dà $(m-2)(n-2)$ equazioni lineari, che includono automaticamente le condizioni al contorno. Risolverle simultaneamente con l'eliminazione di Gauss è possibile in teoria, ma la matrice è grande e *sparsa*: per una griglia di 100×100 nodi ci sono quasi diecimila incognite, e ogni riga ha al più cinque elementi non nulli su diecimila. Per problemi bidimensionali di dimensione moderata i solutori diretti per matrici sparse sono competitivi (useremo `scipy.sparse.linalg.spsolve` come riferimento); per griglie grandi e soprattutto in tre dimensioni si preferiscono i **metodi iterativi**, che non memorizzano la matrice e si programmano in poche righe.

6.5 I metodi di Jacobi e di Gauss–Seidel

Nel metodo di Liebmann si assegna un valore di primo tentativo $f_{i,j}^{(0)}$ in tutti i nodi interni (oltre ai valori al contorno, fissati), poi si applica la (6.7) nodo per nodo per ottenere i valori dell'iterazione successiva. L'ordine di visita parte dal nodo interno in basso a sinistra, sale lungo la prima verticale fino in cima, passa al fondo della verticale successiva, e così via fino al nodo in alto a destra. Quando si calcola $f_{i,j}$, i nodi a sinistra e in basso sono già stati aggiornati: è naturale usarne i valori nuovi. Si ottiene la **formula di Liebmann**, nota anche come **formula di Gauss–Seidel**,

$$f_{i,j}^{(n+1)} = \tfrac{1}{4}\left(f_{i-1,j}^{(n+1)} + f_{i+1,j}^{(n)} + f_{i,j-1}^{(n+1)} + f_{i,j+1}^{(n)}\right) - \tfrac{1}{4}h^2 q_{i,j}, \tag{6.8}$$

valida in ogni nodo interno, anche vicino al contorno (dove i valori al contorno non cambiano con l'iterazione). Se invece si usano soltanto i valori dell'iterazione precedente si ha la **formula di Richardson**, o **di Jacobi**,

$$f_{i,j}^{(n+1)} = \tfrac{1}{4}\left(f_{i-1,j}^{(n)} + f_{i+1,j}^{(n)} + f_{i,j-1}^{(n)} + f_{i,j+1}^{(n)}\right) - \tfrac{1}{4}h^2 q_{i,j}. \tag{6.9}$$

In un programma la differenza è sottile e decisiva: Jacobi ha bisogno di una copia dei valori vecchi, Gauss–Seidel sovrascrive i valori man mano.

```python
def jacobi_passo(psi, zeta, h, interni):
    vecchio = psi.copy()                    # si usano SOLO i valori precedenti
    err = 0.0
    for i, j in interni:
        nuovo = 0.25 * (vecchio[i - 1, j] + vecchio[i + 1, j] +
                        vecchio[i, j - 1] + vecchio[i, j + 1] + h * h * zeta[i, j])
        err += abs(nuovo - vecchio[i, j])
        psi[i, j] = nuovo
    return err

def gauss_seidel_passo(psi, zeta, h, interni):
    err = 0.0                               # metodo di Liebmann (Gauss-Seidel)
    for i, j in interni:
        nuovo = 0.25 * (psi[i - 1, j] + psi[i + 1, j] +
                        psi[i, j - 1] + psi[i, j + 1] + h * h * zeta[i, j])
        err += abs(nuovo - psi[i, j])
        psi[i, j] = nuovo
    return err
```

metodi_numerici.py

Le funzioni risolvono $\nabla^2\psi = -\zeta$ (l'equazione che useremo per la funzione di corrente), cioè la (6.5) con $q = -\zeta$; restituiscono la somma dei valori assoluti delle variazioni, usata come criterio di arresto.

Perché il metodo di Jacobi converge

Sottraendo la (6.7) (soddisfatta dalla soluzione esatta $f_{i,j}$ del sistema discreto) dalla (6.9) e definendo l'errore $e^{(n)}_{i,j} = f^{(n)}_{i,j} - f_{i,j}$:

$$e^{(n+1)}_{i,j} = \tfrac{1}{4}\left(e^{(n)}_{i-1,j} + e^{(n)}_{i+1,j} + e^{(n)}_{i,j-1} + e^{(n)}_{i,j+1}\right), \qquad \text{quindi} \qquad |e^{(n+1)}_{i,j}| \le \tfrac{1}{4} \sum_{4\text{ vicini}} |e^{(n)}|.$$

Se $E^{(n)}$ è il massimo errore assoluto sulla griglia all'iterazione n, si ha subito $E^{(n+1)} \le E^{(n)}$: l'errore non cresce. Per dimostrare che tende a zero si chiamano *nodi del primo strato* quelli adiacenti ad almeno un nodo di contorno, *del secondo strato* quelli adiacenti ad almeno un nodo del primo, e così via. In un nodo del primo strato uno dei vicini ha errore nullo (è sul contorno), quindi

$$|e^{(n+1)}_{i,j}| \le \tfrac{3}{4} E^{(n)} = \left(1 - \tfrac{1}{4}\right) E^{(n)}.$$

In un nodo del secondo strato almeno un vicino appartiene al primo:

$$|e^{(n+1)}_{i,j}| \le \tfrac{1}{4}\left[3E^{(n)} + \left(1 - \tfrac{1}{4}\right) E^{(n-1)}\right] \le \left(1 - \tfrac{1}{4^2}\right) E^{(n-1)},$$

e per induzione in un nodo dello strato m-esimo $|e^{(n+1)}| \le (1 - 4^{-m}) E^{(n-m+1)}$. Se la griglia ha M strati, l'errore massimo sta nello strato più interno e $E^{(n+1)} \le (1 - 4^{-M}) E^{(n-M+1)}$; dopo NM iterazioni

$$E^{(NM)} \le \left(1 - \frac{1}{4^M}\right)^N E^{(0)} \xrightarrow[N\to\infty]{} 0. \tag{6.10}$$

La stima è molto pessimistica, ma dimostra la convergenza; poiché usa valori più aggiornati, il metodo di Gauss–Seidel converge più rapidamente.

La velocità di convergenza: il raggio spettrale

Una stima realistica si ottiene scomponendo l'errore nei modi propri della griglia. Su un rettangolo diviso in $m \times n$ intervalli, con errore nullo sul contorno, l'errore si sviluppa nelle funzioni $\sin(k\pi i/m)\sin(l\pi j/n)$; sostituendo nella formula di Jacobi si trova che ogni modo viene moltiplicato a ogni iterazione per $\frac{1}{2}[\cos(k\pi/m)+\cos(l\pi/n)]$. Il modo più lento è quello con $k = l = 1$, e il fattore corrispondente è il **raggio spettrale** del metodo:

$$\varrho_J = \tfrac{1}{2}\left(\cos\frac{\pi}{m} + \cos\frac{\pi}{n}\right) \simeq 1 - \frac{\pi^2}{4}\left(\frac{1}{m^2}+\frac{1}{n^2}\right). \tag{6.11}$$

Per ridurre l'errore di un fattore 10^{-p} servono circa $p\ln 10/(-\ln\varrho)$ iterazioni. Per il metodo di Gauss–Seidel, sulla griglia a cinque punti con l'ordinamento naturale, si dimostra che $\varrho_{GS} = \varrho_J^2$: servono la metà delle iterazioni di Jacobi [8]. Il guaio è che ϱ tende a 1 quando la griglia si infittisce: dimezzando il passo, le iterazioni necessarie si quadruplicano, e il costo di ciascuna anche.

6.6 Il sovrarilassamento successivo (SOR)

Il metodo del **sovrarilassamento successivo** (SOR, *successive over-relaxation*) accelera Gauss–Seidel "spingendo" ogni correzione oltre il valore suggerito:

$$f_{i,j}^{(n+1)} = f_{i,j}^{(n)} + \frac{\omega}{4}\left(f_{i-1,j}^{(n+1)} + f_{i+1,j}^{(n)} + f_{i,j-1}^{(n+1)} + f_{i,j+1}^{(n)} - 4f_{i,j}^{(n)} - h^2 q_{i,j}\right), \tag{6.12}$$

cioè $f^{\text{nuovo}} = (1-\omega)f^{\text{vecchio}} + \omega f^{\text{GS}}$. Per $\omega = 1$ si ritrova la (6.8). Il metodo converge per $0 < \omega < 2$; per $1 < \omega < 2$ (sovrarilassamento) converge più rapidamente di Gauss–Seidel, e il guadagno è massimo per il valore ottimo trovato da Young [8]:

$$\omega_{\text{ott}} = \frac{2}{1+\sqrt{1-\varrho_J^2}}, \qquad \varrho_{SOR} = \omega_{\text{ott}} - 1. \tag{6.13}$$

Con la (6.11), posto $\alpha = \cos(\pi/m)+\cos(\pi/n) = 2\varrho_J$, la (6.13) si scrive nella forma, equivalente, spesso riportata nei programmi:

$$\omega_{\text{ott}} = \frac{8 - 4\sqrt{4-\alpha^2}}{\alpha^2}, \tag{6.14}$$

come si verifica moltiplicando numeratore e denominatore della (6.13) per $1-\sqrt{1-\varrho_J^2}$. Per griglie fitte $\varrho_{SOR} \simeq 1 - \pi\sqrt{2}\,\sqrt{1/m^2+1/n^2}$ (per un quadrato $1-2\pi/n$): il numero di iterazioni cresce come il numero di nodi per lato, non come il suo quadrato. Per geometrie non rettangolari o griglie non uniformi la formula vale solo come indicazione e l'ottimo si cerca per tentativi [7]; i metodi multigriglia [4, 9] fanno ancora meglio, con un numero di iterazioni indipendente dalla griglia.

```python
def sor_passo(psi, zeta, h, interni, omega):
    err = 0.0                              # sovrarilassamento successivo
    for i, j in interni:
        gs = 0.25 * (psi[i - 1, j] + psi[i + 1, j] +
                     psi[i, j - 1] + psi[i, j + 1] + h * h * zeta[i, j])
        nuovo = (1.0 - omega) * psi[i, j] + omega * gs
        err += abs(nuovo - psi[i, j])
        psi[i, j] = nuovo
    return err

def omega_ottimo(m, n):
    a = np.cos(np.pi / m) + np.cos(np.pi / n)
    return (8.0 - 4.0 * np.sqrt(4.0 - a * a)) / (a * a)
```

metodi_numerici.py

6.7 Programma 2.5: un vortice in un recinto rettangolare

Come prima applicazione consideriamo il flusso prodotto da una distribuzione di vorticità in un dominio rettangolare. La vorticità è il rotore della velocità, $\nabla \times \mathbf{V} = \boldsymbol{\imath}$; per un moto piano è un vettore diretto lungo z di modulo ζ, e con le (1.11) si scrive

$$\frac{\partial^2 \psi}{\partial x^2} + \frac{\partial^2 \psi}{\partial y^2} = -\zeta(x, y), \tag{6.15}$$

un'equazione di Poisson. Lo schema di Liebmann diventa

$$\psi_{i,j} = \tfrac{1}{4}\left(\psi_{i-1,j} + \psi_{i+1,j} + \psi_{i,j-1} + \psi_{i,j+1} + h^2 \zeta_{i,j}\right), \tag{6.16}$$

dove gli indici di iterazione sono omessi perché se ne occupa automaticamente l'ordine di visita dei nodi.

Il Programma 2.5 (Listato 6.1) studia come si deformano le linee di corrente circolari di un vortice quando esso è racchiuso nel rettangolo $-3 \le x \le 3$, $-2 \le y \le 2$. Poiché ψ è definita a meno di una costante, sulla parete si pone $\psi = 0$.

Dati del Programma 2.5

Passo $h = 0{,}25$: $m = 25$, $n = 17$ nodi, 425 in tutto, 345 interni. Vorticità $\zeta_0 = 100$ concentrata nel nodo $(x_0, y_0) = (1, 1)$, nulla altrove: equivale a un vortice di circolazione $\Gamma = \zeta_0 h^2 = 6{,}25$. Valore di primo tentativo $\psi = 0{,}5$ nei nodi interni. Criterio di arresto: somma dei valori assoluti delle variazioni in un'iterazione $\le$ `ERRMAX` $= 0{,}001$ (in media $3 \cdot 10^{-6}$ per nodo), oppure `ITMAX` $= 200$ iterazioni.

Le linee di corrente si cercano lungo le verticali *e* lungo le orizzontali della griglia (funzione `cerca_xy`), perché attorno al vortice sono in parte verticali.

Risultati

- Il criterio di arresto è soddisfatto dopo 153 iterazioni. Vicino al vortice le linee di corrente sono circolari; allontanandosi si adattano alla forma della parete (Fig. 6.3).
- La soluzione si può confrontare con quella esatta del problema *continuo*, un vortice puntiforme di circolazione Γ in un rettangolo con $\psi = 0$ sul contorno, che si scrive come serie doppia di Fourier (funzione `psi_serie`):

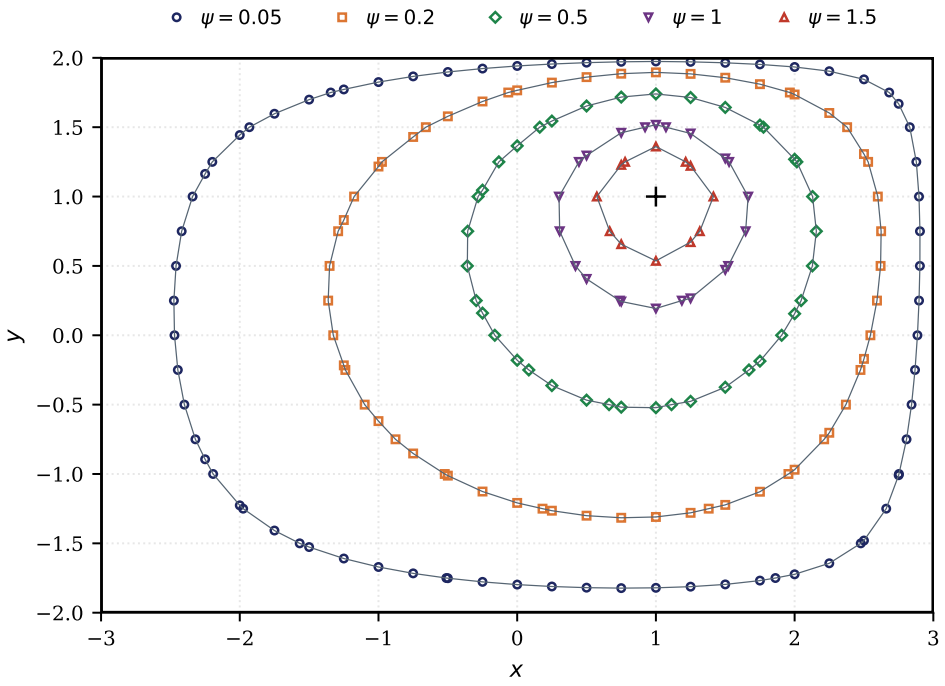


Figura 6.3: Programma 2.5: linee di corrente di un vortice racchiuso da una parete rettangolare (simboli: ricerca su verticali e orizzontali; linee grigie: `contour`).

punto	ψ numerico	ψ esatto
$(-2,\ 0)$	0,101 03	0,100 66
$(0,\ 0)$	0,556 23	0,559 15
$(1,\ -1)$	0,302 58	0,301 89
$(1,\ 0)$	0,818 66	0,813 54
$(2,\ 1)$	0,600 81	0,587 96

L'accordo è buono lontano dal vortice e peggiora avvicinandosi: a quattro passi di griglia dal vortice lo scarto è del 2%. Una vorticità concentrata in un nodo è la versione discreta di una singolarità, e la griglia non può risolvere il campo $\psi \propto \ln r$ nelle sue immediate vicinanze.

```
"""
prog2_5_vortice_rettangolo.py -- PROGRAMMA 2.5
Equazione di Poisson  psi_xx + psi_yy = -zeta  per il flusso indotto da un
vortice concentrato in (x0, y0) all'interno di una parete rettangolare
(-3 <= x <= 3, -2 <= y <= 2), risolta con la formula iterativa di Liebmann
(Gauss-Seidel). Condizione al contorno: psi = 0 sulla parete.

Verifica: la soluzione discreta si confronta con la soluzione esatta del
problema continuo (funzione di Green del rettangolo in serie doppia di Fourier)
per un vortice puntiforme di circolazione Gamma = zeta0 h^2.
"""
import numpy as np
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, VERDE, VIOLA, ROSSO, GRIGIO
from metodi_numerici import gauss_seidel_passo, nodi_interni_rettangolo, cerca_xy

XMIN, XMAX, YMIN, YMAX, H = -3.0, 3.0, -2.0, 2.0, 0.25
```

prog2_5_vortice_rettangolo.py

```
ZETA0, X0, Y0 = 100.0, 1.0, 1.0
ITMAX, ERRMAX = 200, 0.001


def griglia(h=H):
    x = np.arange(XMIN, XMAX + 1e-9, h)
    y = np.arange(YMIN, YMAX + 1e-9, h)
    return x, y


def psi_serie(xp, yp, gamma, mmax=200):
    """Soluzione esatta: vortice puntiforme di circolazione gamma in (X0, Y0)
    dentro il rettangolo con psi = 0 sul contorno (serie doppia di Fourier)."""
    Lx, Ly = XMAX - XMIN, YMAX - YMIN
    m = np.arange(1, mmax + 1)[:, None]
    n = np.arange(1, mmax + 1)[None, :]
    km, kn = m * np.pi / Lx, n * np.pi / Ly
    coef = 4.0 * gamma / (Lx * Ly) / (km**2 + kn**2) \
        * np.sin(km * (X0 - XMIN)) * np.sin(kn * (Y0 - YMIN))
    return np.sum(coef * np.sin(km * (xp - XMIN)) * np.sin(kn * (yp - YMIN)))


if __name__ == "__main__":
    x, y = griglia()
    M, N = len(x), len(y)
    psi = np.zeros((M, N))
    psi[1:-1, 1:-1] = 0.5                    # valori di primo tentativo
    zeta = np.zeros((M, N))
    i0, j0 = int(round((X0 - XMIN) / H)), int(round((Y0 - YMIN) / H))
    zeta[i0, j0] = ZETA0
    interni = nodi_interni_rettangolo(M, N)

    it, err = 0, 1.0
    storia = []
    while err > ERRMAX and it < ITMAX:
        it += 1
        err = gauss_seidel_passo(psi, zeta, H, interni)
        storia.append(err)
    print(f"M = {M}, N = {N}, iterazioni = {it}, somma degli errori = {err:.3e}")

    # tabella ridotta: una colonna ogni 3 e una riga ogni 2, dall'alto in basso
    print("\npsi(i,j) (righe dall'alto, una ogni 2; colonne una ogni 3):")
    for j in range(N - 1, -1, -2):
        print("  " + " ".join(f"{psi[i, j]:7.4f}" for i in range(0, M, 3)))

    # confronto con la soluzione esatta del problema continuo
    gamma = ZETA0 * H**2
    print(f"\ncircolazione equivalente Gamma = zeta0 h^2 = {gamma:.4f}")
    print("   punto (x, y)      psi numerico   psi esatto (serie)")
    for xp, yp in ((-2.0, 0.0), (0.0, 0.0), (1.0, -1.0), (2.0, 1.0), (1.0, 0.0)):
        i, j = int(round((xp - XMIN) / H)), int(round((yp - YMIN) / H))
        print(f"   ({xp:+.1f}, {yp:+.1f})       {psi[i, j]:8.5f}
    {psi_serie(xp, yp, gamma):8.5f}")

    # linee di corrente
    COST = [0.05, 0.2, 0.5, 1.0, 1.5]
    SIMB = ["o", "s", "D", "v", "^"]
    COL = [BLU, ARANCIO, VERDE, VIOLA, ROSSO]
    fig, ax = plt.subplots(figsize=(6.0, 4.2))
    for c, s, col in zip(COST, SIMB, COL):
```

```
77          xx, yy = cerca_xy(x, y, psi, c)
78          ax.plot(xx, yy, s, ms=3, color=col, mfc="none", label=fr"$\psi={c:g}$")
79      ax.contour(x, y, psi.T, levels=COST, colors=GRIGIO, linewidths=0.5,
        linestyles="solid")
80      ax.plot(X0, Y0, "+", color="k", ms=8)
81      ax.plot([XMIN, XMAX, XMAX, XMIN, XMIN], [YMIN, YMIN, YMAX, YMAX, YMIN], "k",
        lw=1.5)
82      ax.set_aspect("equal"); ax.set_xlabel("$x$"); ax.set_ylabel("$y$")
83      ax.legend(loc="upper center", ncol=5, bbox_to_anchor=(0.5, 1.12),
        handletextpad=0.2)
84      salva(fig, "prog2_5_vortice")
```

Listato 6.1: Programma 2.5 – vortice in un recinto rettangolare, metodo di Liebmann (`prog2_5_vortice_rettangolo.py`).

6.8 Jacobi, Gauss–Seidel e SOR a confronto

Quanto è lontana dalla soluzione del sistema discreto la ψ trovata dopo 153 iterazioni? E quanto conviene il sovrarilassamento? Il programma `prog2_5b_convergenza.py` (Listato 6.2) risponde calcolando prima la soluzione *esatta del sistema discreto*, ψ^*, con un solutore diretto per matrici sparse, e poi l'errore vero $\|\psi^{(k)} - \psi^*\|_\infty$ a ogni iterazione dei tre metodi.

Risultati: numero di iterazioni per ridurre l'errore di un fattore 10^6

Per la griglia di 24×16 intervalli, $\varrho_J = 0{,}98612$, $\varrho_{GS} = 0{,}97242$, $\omega_{\text{ott}} = 1{,}7152$, $\varrho_{SOR} = 0{,}7152$.

metodo	iterazioni	stima $\ln 10^6 / (-\ln \varrho)$
Jacobi	740	988
Gauss–Seidel	296	494
SOR, $\omega = 1{,}715$	49	41

- Il SOR è sei volte più veloce di Gauss–Seidel e quindici volte più di Jacobi (Fig. 6.4). Le stime basate sul raggio spettrale indicano il comportamento asintotico: Jacobi e Gauss–Seidel fanno un po' meglio perché l'errore iniziale ha sul modo più lento una componente piccola, il SOR un po' peggio perché all'ottimo i due autovalori dominanti coincidono e l'errore decresce come $k\,\varrho^k$ anziché ϱ^k.
- Il numero di iterazioni del SOR in funzione di ω (Fig. 6.5) ha un minimo netto vicino al valore teorico: con $\omega = 1{,}70$–$1{,}75$ bastano 54–56 iterazioni, con $\omega = 1{,}5$ ne servono 123, con $\omega = 1{,}95$ addirittura 251. Conviene sbagliare per difetto.
- Il criterio di arresto del Programma 2.5 (somma delle variazioni $\le 0{,}001$) si soddisfa all'iterazione 153, quando l'errore vero massimo è ancora $2{,}3 \cdot 10^{-4}$, cioè 80 volte la variazione media per nodo: un metodo che converge lentamente fa piccoli passi anche quando è lontano dalla soluzione.
- Al raffinarsi della griglia (sotto, riduzione dell'errore di 10^4) le iterazioni di Gauss–Seidel crescono circa come $1/h^2$, quelle del SOR come $1/h$:

h	nodi interni	Gauss–Seidel	SOR	ω_{ott}
0,500	77	53	18	1,5068
0,250	345	147	37	1,7152
0,125	1457	718	74	1,8462

- L'errore di *discretizzazione*, cioè la differenza fra ψ^* e la soluzione del problema continuo nel centro del rettangolo, si riduce di un fattore 4 a ogni dimezzamento del passo: da $-1{,}2 \cdot 10^{-2}$ ($h = 0{,}5$) a $-4{,}9 \cdot 10^{-5}$ ($h = 0{,}03125$). Per $h = 0{,}25$ vale $-3{,}1 \cdot 10^{-3}$, più di dieci volte l'errore di iterazione residuo dopo 153 iterazioni di Liebmann: in questo punto il calcolo era stato arrestato al momento giusto.

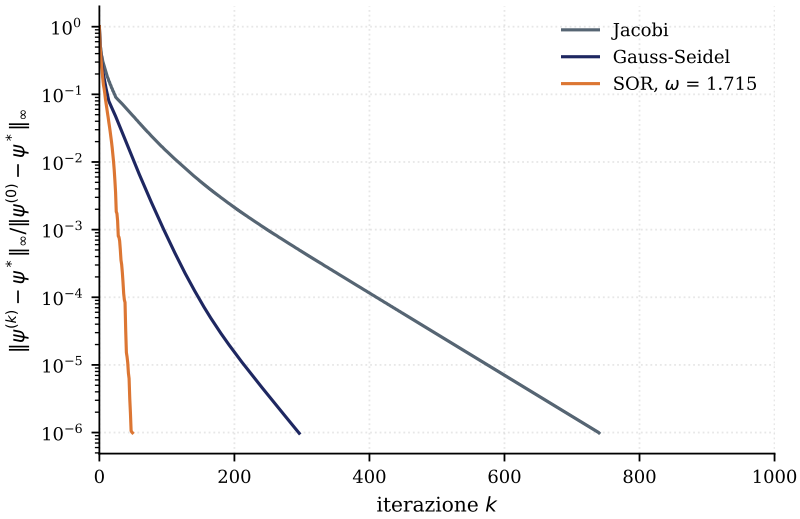


Figura 6.4: Storia dell'errore vero dei tre metodi iterativi sul problema del Programma 2.5.

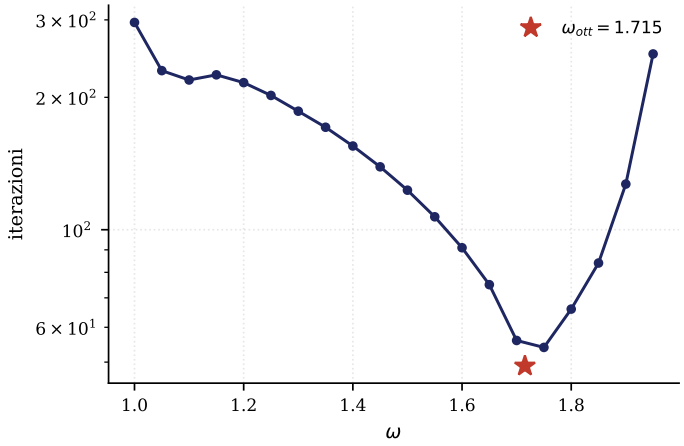


Figura 6.5: Iterazioni necessarie al SOR in funzione del fattore di rilassamento; la stella indica il valore teorico (6.13).

Errore di iterazione ed errore di discretizzazione

Un calcolo su griglia contiene due errori distinti: l'*errore di discretizzazione*, differenza fra la soluzione del sistema discreto e quella dell'equazione differenziale, che si riduce solo infittendo la griglia; e l'*errore di iterazione*, differenza fra la soluzione corrente e quella del sistema discreto, che si riduce iterando. È inutile spingere l'errore di iterazione molto al di sotto di quello di discretizzazione; è dannoso fermarsi prima. Un criterio di arresto basato sulla sola variazione fra due iterazioni sottostima l'errore vero di un fattore $1/(1-\varrho)$, che per Gauss–Seidel su una griglia fitta può essere di centinaia. Nei codici industriali si controlla invece il *residuo*, cioè quanto la soluzione corrente non soddisfa le equazioni discrete, normalizzato con il suo valore iniziale.

prog2_5b_convergenza.py

```
"""
prog2_5b_convergenza.py -- confronto fra i metodi iterativi di Jacobi
(Richardson), Gauss-Seidel (Liebmann) e del sovrarilassamento successivo (SOR)
sul problema del Programma 2.5.

1) storia dell'errore vero ||psi_k - psi*||_inf, con psi* soluzione esatta del
   sistema discreto (ottenuta con un solutore diretto per matrici sparse);
2) numero di iterazioni in funzione di omega (ricerca dell'ottimo);
3) numero di iterazioni al raffinarsi della griglia;
4) errore di discretizzazione rispetto alla soluzione del problema continuo.
"""
import numpy as np
import matplotlib.pyplot as plt
import scipy.sparse as sp
import scipy.sparse.linalg as spla
from stile_grafici import salva, BLU, ARANCIO, VERDE, ROSSO, GRIGIO, VIOLA
from metodi_numerici import (jacobi_passo, gauss_seidel_passo, sor_passo,
                             omega_ottimo, nodi_interni_rettangolo)
import prog2_5_vortice_rettangolo as p25

GAMMA = p25.ZETA0 * p25.H**2          # circolazione del vortice (fissa)


def problema(h):
    x, y = p25.griglia(h)
    M, N = len(x), len(y)
    zeta = np.zeros((M, N))
    i0 = int(round((p25.X0 - p25.XMIN) / h)); j0 = int(round((p25.Y0 - p25.YMIN) /
    h))
    zeta[i0, j0] = GAMMA / h**2       # stessa circolazione su ogni griglia
    return x, y, zeta


def soluzione_diretta(zeta, h):
    """Assembla la matrice a 5 punti e risolve con un metodo diretto sparso."""
    M, N = zeta.shape
    mi, ni = M - 2, N - 2
    Tx = sp.diags([1, -2, 1], [-1, 0, 1], shape=(mi, mi), dtype=float)
    Ty = sp.diags([1, -2, 1], [-1, 0, 1], shape=(ni, ni), dtype=float)
    L = (sp.kron(Tx, sp.identity(ni)) + sp.kron(sp.identity(mi), Ty)) / h**2
    b = -zeta[1:-1, 1:-1].ravel()
    psi = np.zeros((M, N))
    psi[1:-1, 1:-1] = spla.spsolve(L.tocsc(), b).reshape(mi, ni)
    return psi
```

```python
def itera(metodo, zeta, h, psi_star, tol=1e-6, omega=1.0, kmax=20000, guess=0.5):
    M, N = zeta.shape
    psi = np.zeros((M, N)); psi[1:-1, 1:-1] = guess
    interni = nodi_interni_rettangolo(M, N)
    e0 = np.max(np.abs(psi - psi_star))
    storia, somme = [e0], []
    for k in range(1, kmax + 1):
        if metodo == "jacobi":
            s = jacobi_passo(psi, zeta, h, interni)
        elif metodo == "gs":
            s = gauss_seidel_passo(psi, zeta, h, interni)
        else:
            s = sor_passo(psi, zeta, h, interni, omega)
        somme.append(s)
        storia.append(np.max(np.abs(psi - psi_star)))
        if storia[-1] < tol * e0:
            break
    return k, np.array(storia), np.array(somme)


if __name__ == "__main__":
    h = p25.H
    x, y, zeta = problema(h)
    psi_star = soluzione_diretta(zeta, h)
    m, n = len(x) - 1, len(y) - 1                  # numero di intervalli
    rJ = 0.5 * (np.cos(np.pi / m) + np.cos(np.pi / n))
    wopt = omega_ottimo(m, n)
    print(f"griglia {m} x {n} intervalli: raggio spettrale di Jacobi = {rJ:.5f}")
    print(f"  Gauss-Seidel: {rJ**2:.5f};   omega_ott = {wopt:.4f};  SOR:
    {wopt-1:.4f}")

    # -------- 1) storie dell'errore ---------------------------------------
    fig, ax = plt.subplots(figsize=(5.4, 3.6))
    print("\n metodo      iterazioni (errore / 1e6)   previste = ln(1e6)/(-ln rho)")
    for nome, met, om, rho, col in (("Jacobi", "jacobi", 1.0, rJ, GRIGIO),
                                    ("Gauss-Seidel", "gs", 1.0, rJ**2, BLU),
                                    (f"SOR, omega = {wopt:.3f}", "sor", wopt, wopt
    - 1, ARANCIO)):
        k, st, so = itera(met, zeta, h, psi_star, omega=om)
        prev = np.log(1e6) / (-np.log(rho))
        print(f" {nome:24s}  {k:5d}                {prev:7.0f}")
        ax.semilogy(st / st[0], color=col,
                    label=nome.replace("omega", r"$\omega$"))
        if met == "gs":
            # quante iterazioni avrebbe fatto il criterio del Programma 2.5?
            k25 = np.argmax(so <= p25.ERRMAX) + 1
            print(f"   (criterio 'somma |delta| <= 0,001' soddisfatto
    all'iterazione {k25}: "
                  f"errore vero = {st[k25]:.2e}, somma = {so[k25-1]:.2e})")
    ax.set_xlabel("iterazione $k$"); ax.set_ylabel(r"$\|\psi^{(k)}-\psi^*\|_\infty
    / \|\psi^{(0)}-\psi^*\|_\infty$")
    ax.set_xlim(0, 1000); ax.legend()
    salva(fig, "prog2_5b_storie")

    # -------- 2) ricerca dell'omega ottimo --------------------------------
    omegas = np.arange(1.0, 1.96, 0.05)
    its = [itera("sor", zeta, h, psi_star, omega=w)[0] for w in omegas]
    k_opt = itera("sor", zeta, h, psi_star, omega=wopt)[0]
    print("\n omega   iterazioni")
    for w, k in zip(omegas, its):
```

```
        print(f"  {w:.2f}    {k:5d}")
    fig, ax = plt.subplots(figsize=(4.8, 3.2))
    ax.plot(omegas, its, "o-", color=BLU, ms=3.5)
    ax.plot([wopt], [k_opt], "*", color=ROSSO, ms=10,
    label=fr"$\omega_{{ott}}={wopt:.3f}$")
    ax.set_yscale("log"); ax.set_xlabel(r"$\omega$"); ax.set_ylabel("iterazioni")
    ax.legend()
    salva(fig, "prog2_5b_omega")

    # -------- 3) iterazioni al raffinarsi della griglia ---------------------
    print("\n    h    nodi interni   Gauss-Seidel    SOR(omega_ott)   omega_ott")
    for hh in (0.5, 0.25, 0.125):
        xg, yg, zg = problema(hh)
        ps = soluzione_diretta(zg, hh)
        mg, ng = len(xg) - 1, len(yg) - 1
        w = omega_ottimo(mg, ng)
        kgs = itera("gs", zg, hh, ps, tol=1e-4)[0]
        ksor = itera("sor", zg, hh, ps, tol=1e-4, omega=w)[0]
        print(f"  {hh:5.3f}    {(mg-1)*(ng-1):6d}        {kgs:6d}
    {ksor:5d}         {w:.4f}")

    # -------- 4) errore di discretizzazione ---------------------------------
    print("\n    h     psi(0,0) discreto   psi(0,0) continuo   errore    rapporto")
    esatto = p25.psi_serie(0.0, 0.0, GAMMA, mmax=400)
    prec = None
    for hh in (0.5, 0.25, 0.125, 0.0625, 0.03125):
        xg, yg, zg = problema(hh)
        ps = soluzione_diretta(zg, hh)
        i, j = int(round(3.0 / hh)), int(round(2.0 / hh))
        e = ps[i, j] - esatto
        r = prec / e if prec else np.nan
        print(f"  {hh:7.5f}     {ps[i, j]:.6f}          {esatto:.6f}
    {e:+.2e}   {r:5.2f}")
        prec = e
```

Listato 6.2: `prog2_5b_convergenza.py`: confronto fra Jacobi, Gauss–Seidel e SOR; soluzione diretta sparsa; errore di discretizzazione.

Capitolo 7

Flussi in condotti e attorno a corpi: contorni irregolari e condizioni di derivata

7.1 Il problema dei contorni

Il flusso del Cap. 6 era racchiuso da una parete rettangolare lungo la quale la funzione di corrente aveva un valore costante: i nodi della griglia cadevano esattamente sul contorno e il valore al contorno era noto. Nei problemi reali le superfici di un corpo o di una parete hanno forme complicate, oppure esistono superfici lungo le quali è nota la *derivata* della funzione di corrente invece della funzione stessa. In questo capitolo due esempi mostrano le tecniche per trattare i contorni irregolari e le condizioni di derivata: il flusso in una camera con una parete inclinata e il flusso attorno a un cilindro circolare in un canale.

In assenza di vorticità la funzione di corrente soddisfa l'equazione di Laplace,

$$\frac{\partial^2 \psi}{\partial x^2} + \frac{\partial^2 \psi}{\partial y^2} = 0, \tag{7.1}$$

e la soluzione si ottiene con gli stessi metodi iterativi del capitolo precedente; cambiano solo il dominio e il modo di scrivere le equazioni nei nodi vicini al contorno.

7.2 Programma 2.6: flusso in una camera con una parete inclinata

Si consideri una camera rettangolare il cui angolo superiore destro è chiuso da una lastra inclinata di 45° sull'asse x (Fig. 7.1). Un fluido non viscoso incomprimibile attraversa stazionariamente la camera entrando da un'apertura sulla parete inferiore e uscendo da un'altra apertura sulla stessa parete. Con una griglia quadrata di passo $h = 0{,}25$ la parete inclinata interseca le linee della griglia esattamente in alcuni nodi, che sono naturalmente i nodi di contorno su quella parete; il caso più generale, in cui i punti di contorno non coincidono con i nodi, è il secondo esempio.

La funzione di corrente rappresenta la portata che scorre fra un punto e una linea di corrente di riferimento; la si normalizza in modo che la portata totale attraverso la camera sia unitaria. Il tratto AB della parete inferiore compreso fra l'ingresso e l'uscita si sceglie come linea di corrente di riferimento, $\psi = 0$; il resto della parete, che dall'ingresso fa il giro della

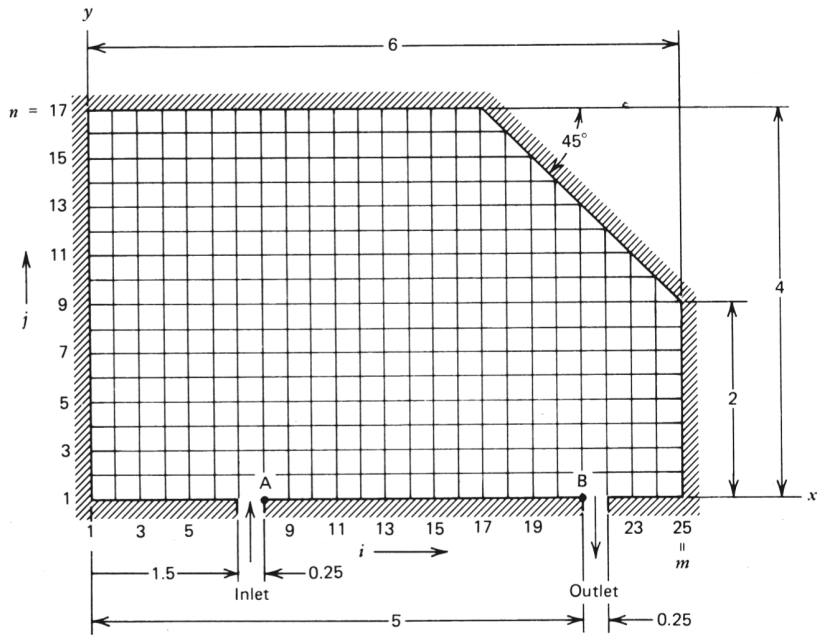


Figura 7.1: Camera con contorni irregolari.

camera fino all'uscita, è allora la linea $\psi = 1$. Le due aperture, larghe un passo di griglia, sono gli intervalli in cui ψ salta da 1 a 0 (ingresso, $1{,}5 < x < 1{,}75$) e da 0 a 1 (uscita, $5 < x < 5{,}25$).

L'equazione (7.1) si risolve con il SOR, scritto nella forma

$$\psi_{i,j} \leftarrow (1-\omega)\psi_{i,j} + \frac{\omega}{4}\left(\psi_{i-1,j} + \psi_{i+1,j} + \psi_{i,j-1} + \psi_{i,j+1}\right), \tag{7.2}$$

con il fattore (6.14) calcolato come per il rettangolo che contiene la camera. L'unica differenza rispetto al Programma 2.5 è la definizione del dominio: per ogni verticale i il programma conosce l'indice `jlast[i]` del nodo di parete superiore (orizzontale fino a $x = 4$, poi inclinata), e aggiorna solo i nodi compresi fra il fondo e quella parete. I nodi esterni alla camera restano marcati con `NaN` e vengono ignorati dalla ricerca delle linee di corrente.

Risultati

- Con $\omega_{\text{ott}} = 1{,}728$ la somma delle variazioni scende sotto 0,001 in 41 iterazioni; con il metodo di Liebmann ($\omega = 1$) ne servono 250. Il risparmio di calcolo è di un fattore sei. La ricerca dell'ottimo per tentativi conferma la formula:

ω	1,00	1,20	1,40	1,60	1,65	1,70	1,75	1,80	1,90
iterazioni	250	176	119	71	59	45	45	55	107

- La Fig. 7.2 è un diagramma di tubi di flusso: un tubo più largo indica una zona di velocità più bassa. Le velocità più alte si hanno nelle due aperture, dove tutte le linee di corrente convergono; nella parte alta della camera, e soprattutto nell'angolo chiuso dalla parete inclinata, il fluido è quasi fermo. La simmetria approssimata del disegno rispetto alla verticale per il centro di AB riflette la simmetria delle due aperture; la parete inclinata la rompe solo nella parte alta, dove la corrente è debole.

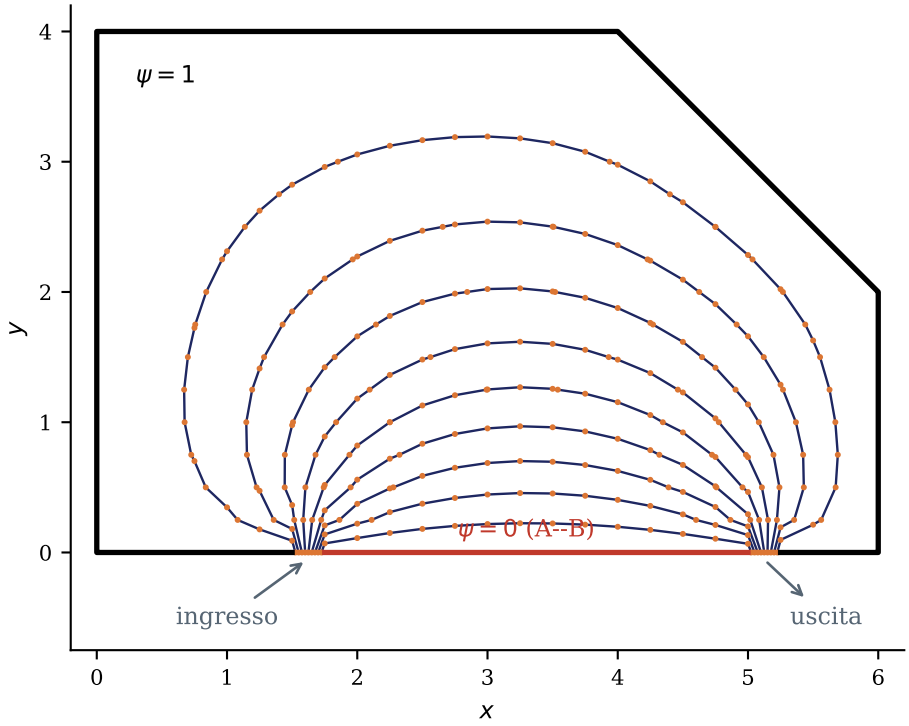


Figura 7.2: Programma 2.6: linee di corrente $\psi = 0{,}1, \ldots, 0{,}9$ nella camera (punti arancioni: ricerca lungo verticali e orizzontali). Fra due linee consecutive scorre un decimo della portata.

Ingressi e uscite d'aria

Una cabina pressurizzata, un vano avionico ventilato, la camera di calma di una galleria del vento: in tutti questi casi il progettista vuole sapere dove l'aria ristagna e dove corre. Il calcolo potenziale non vede i ricircoli che nella realtà si formano dietro gli spigoli (servono le equazioni viscose), ma indica già dove si concentrano le velocità e quali zone restano "morte", con una frazione minima del costo di un calcolo completo.

prog2_6_camera.py

```python
"""
prog2_6_camera.py -- PROGRAMMA 2.6
Flusso potenziale in una camera rettangolare (6 x 4) con l'angolo superiore
destro chiuso da una parete a 45 gradi; ingresso e uscita sono due aperture
sulla parete inferiore. Equazione di Laplace per psi risolta con il metodo
del sovrarilassamento successivo (SOR) e, per confronto, con Liebmann.

Condizioni al contorno: psi = 0 sul tratto AB della parete inferiore compreso
fra le due aperture, psi = 1 sul resto della parete (portata unitaria).
"""
import numpy as np
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, ROSSO, GRIGIO, VERDE
from metodi_numerici import sor_passo, cerca_xy

H = 0.25
M, N = 25, 17                       # nodi lungo x (0..6) e lungo y (0..4)


def prepara():
    """Valori al contorno, valori di tentativo e lista dei nodi interni."""
    jlast = np.full(M, N - 1)                       # indice (da 0) del nodo di
    parete superiore
    for i in range(17, M):                          # parete inclinata: da (4,4) a
    (6,2)
        jlast[i] = (N - 1) - (i - 16)
    psi = np.full((M, N), np.nan)                   # NaN = fuori dal dominio
    for i in range(M):
        psi[i, :jlast[i] + 1] = 0.5                 # tentativo nei nodi interni
        psi[i, jlast[i]] = 1.0                      # parete superiore / inclinata
    psi[0, :] = 1.0                                 # parete sinistra
    psi[M - 1, :jlast[M - 1] + 1] = 1.0             # parete destra (y <= 2)
    psi[:, 0] = 1.0                                 # parete inferiore ...
    psi[7:21, 0] = 0.0                              # ... tratto AB fra le aperture
    interni = [(i, j) for i in range(1, M - 1) for j in range(1, jlast[i])]
    return psi, interni, jlast


def risolvi(omega, tol=0.001, kmax=5000):
    psi, interni, jlast = prepara()
    zeta = np.zeros((M, N))
    it, err = 0, 1.0
    while err > tol and it < kmax:
        it += 1
        err = sor_passo(psi, zeta, H, interni, omega)
    return psi, it, jlast


if __name__ == "__main__":
    # omega ottimo con la formula del rettangolo (come nel programma originale
    # si usano i numeri di nodi M, N; con il numero di intervalli M-1, N-1 il
    # valore cambia di poco)
    alfa = np.cos(np.pi / M) + np.cos(np.pi / N)
    omega = (8 - 4 * np.sqrt(4 - alfa**2)) / alfa**2
    psi, it, jlast = risolvi(omega)
    _, it_gs, _ = risolvi(1.0)
    print(f"SOR  (omega = {omega:.4f}): {it} iterazioni")
    print(f"Liebmann (omega = 1)    : {it_gs} iterazioni")
    print("\n omega   iterazioni")
    for w in (1.0, 1.2, 1.4, 1.6, 1.65, 1.7, 1.75, 1.8, 1.9):
```

```
        print(f"  {w:.2f}    {risolvi(w)[1]:4d}")

    x = np.arange(M) * H
    y = np.arange(N) * H
    fig, ax = plt.subplots(figsize=(6.0, 4.4))
    livelli = np.round(np.arange(0.1, 1.0, 0.1), 2)
    ax.contour(x, y, psi.T, levels=livelli, colors=BLU, linewidths=0.9,
    linestyles="solid")
    for c in livelli:
        xx, yy = cerca_xy(x, y, psi, c)
        ax.plot(xx, yy, ".", ms=2.5, color=ARANCIO)
    # pareti
    ax.plot([1.5, 0, 0, 4, 6, 6, 5.25], [0, 0, 4, 4, 2, 0, 0], "k", lw=2)
    ax.plot([1.75, 5.0], [0, 0], color=ROSSO, lw=2)
    ax.annotate("ingresso", (1.62, -0.05), (1.0, -0.55), ha="center",
                arrowprops=dict(arrowstyle="->", color=GRIGIO), color=GRIGIO)
    ax.annotate("uscita", (5.12, -0.05), (5.6, -0.55), ha="center",
                arrowprops=dict(arrowstyle="<-", color=GRIGIO), color=GRIGIO)
    ax.text(3.3, 0.12, r"$\psi=0$ (A--B)", color=ROSSO, ha="center")
    ax.text(0.3, 3.6, r"$\psi=1$", color="k")
    ax.set_aspect("equal"); ax.set_xlim(-0.2, 6.2); ax.set_ylim(-0.75, 4.2)
    ax.set_xlabel("$x$"); ax.set_ylabel("$y$"); ax.grid(False)
    salva(fig, "prog2_6_camera")
```

Listato 7.1: Programma 2.6 – flusso in una camera con parete inclinata, metodo SOR (`prog2_6_camera.py`).

7.3 Contorni curvi: la formula a bracci disuguali

Il secondo esempio è il flusso in un canale contenente un cilindro circolare (Fig. 7.3). Il fluido entra nel canale da una piccola apertura centrale ed esce da una sezione a valle del cilindro; si suppone che all'uscita siano installati dei raddrizzatori che rendono la corrente orizzontale. Conviene porre l'origine nel centro del cilindro; per simmetria rispetto all'asse x si calcola solo la metà superiore del campo, con una griglia quadrata di passo $h = 0{,}25$, e si usa la formula di Liebmann

$$\psi_{i,j} = \tfrac{1}{4}\left(\psi_{i-1,j} + \psi_{i+1,j} + \psi_{i,j-1} + \psi_{i,j+1}\right). \tag{7.3}$$

Condizioni al contorno

1. $\psi = 0$ sulla superficie del cilindro e sull'asse x fuori dal corpo (linea di simmetria);
2. $\psi = 1$ sulla parete superiore del canale e sulla parete d'ingresso: l'ingresso è l'apertura fra l'asse e il primo nodo, $0 < y < h$;
3. $v = -\partial\psi/\partial x = 0$ nella sezione di uscita.

Nascono due problemi. I punti di contorno sul cilindro sono le intersezioni della superficie con le linee della griglia, e in generale non coincidono con i nodi: la formula (7.3), che presuppone il nodo (i, j) equidistante dai quattro vicini, non vale nei nodi più vicini al cilindro. Inoltre serve una tecnica per la condizione di derivata all'uscita.

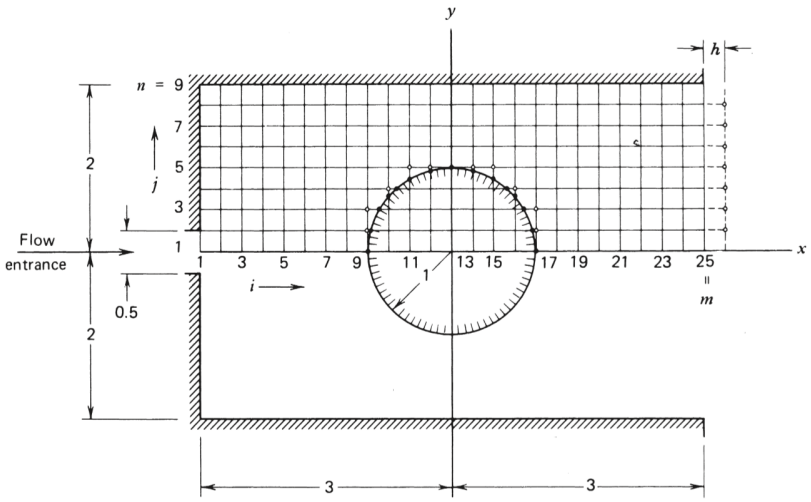


Figura 7.3: Cilindro circolare in un canale.

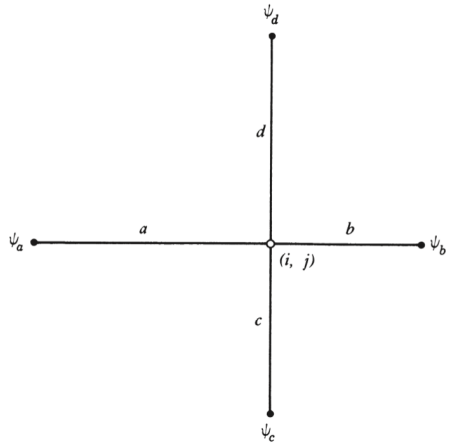


Figura 7.4: Nodo con bracci disuguali.

La formula a cinque punti con bracci disuguali

Siano a, b, c, d le distanze del nodo (i,j) dai quattro punti vicini (a sinistra, a destra, sotto, sopra), che possono essere nodi della griglia o punti di contorno, e ψ_a, ψ_b, ψ_c, ψ_d i valori di ψ in essi (Fig. 7.4). Si approssima il laplaciano con una combinazione lineare

$$\left(\frac{\partial^2\psi}{\partial x^2}+\frac{\partial^2\psi}{\partial y^2}\right)_{i,j} = \alpha_0\psi_{i,j}+\alpha_a\psi_a+\alpha_b\psi_b+\alpha_c\psi_c+\alpha_d\psi_d$$

e si determinano i coefficienti sviluppando ogni valore in serie di Taylor attorno al nodo, ad esempio

$$\psi_a = \psi_{i,j} - a\,\psi_x + \tfrac{1}{2}a^2\psi_{xx} - \cdots, \qquad \psi_d = \psi_{i,j} + d\,\psi_y + \tfrac{1}{2}d^2\psi_{yy} + \cdots.$$

Sostituendo e trascurando i termini del terzo ordine:

$$\alpha_0\psi + \sum\alpha_k\psi_k = (\alpha_0+\alpha_a+\alpha_b+\alpha_c+\alpha_d)\psi + (b\alpha_b - a\alpha_a)\psi_x + (d\alpha_d - c\alpha_c)\psi_y + \tfrac{1}{2}(a^2\alpha_a + b^2\alpha_b)\psi_{xx} + \tfrac{1}{2}(c^2\alpha_c + d^2\alpha_d)\psi_{yy}.$$

Perché il secondo membro coincida con $\psi_{xx} + \psi_{yy}$ si impongono cinque condizioni: coefficiente di ψ nullo, coefficienti delle derivate prime nulli, coefficienti delle derivate seconde uguali a uno. La soluzione è

$$\alpha_a = \frac{2}{a(a+b)}, \quad \alpha_b = \frac{2}{b(a+b)}, \quad \alpha_c = \frac{2}{c(c+d)}, \quad \alpha_d = \frac{2}{d(c+d)},$$

$$\alpha_0 = -2\left(\frac{1}{ab} + \frac{1}{cd}\right).$$

Annullando il laplaciano si ottiene la formula cercata:

$$\psi_{i,j} = \left[\frac{\psi_a}{a(a+b)} + \frac{\psi_b}{b(a+b)} + \frac{\psi_c}{c(c+d)} + \frac{\psi_d}{d(c+d)}\right] \Bigg/ \left(\frac{1}{ab} + \frac{1}{cd}\right). \tag{7.4}$$

Per $a = b = c = d = h$ si ritrova la (7.3). Con bracci disuguali i termini del terzo ordine non si cancellano più: l'errore di troncamento locale diventa del primo ordine, $\mathcal{O}(h)$; poiché riguarda una sola fila di nodi, l'accuratezza complessiva resta molto vicina a quella del secondo ordine.

Nel nodo (i, j) vicino al lato sinistro del cilindro (centro in $x = 3$, raggio 1; Fig. 7.5) si ha $\psi_a = \psi_{i-1,j}$, $\psi_d = \psi_{i,j+1}$, $a = d = h$. Se a destra, entro la distanza h, c'è un punto del cilindro, la sua distanza è

$$b = (x_c - x_i) - \sqrt{R^2 - y_j^2}, \qquad \psi_b = 0; \tag{7.5}$$

altrimenti $b = h$ e $\psi_b = \psi_{i+1,j}$. Per semplificare l'algoritmo si scrive comunque $\psi_b = \psi_{i+1,j}$ e si assegna il valore zero a tutti i nodi *interni* al cilindro: così il valore del vicino è automaticamente quello del punto di contorno. Allo stesso modo, se sotto il nodo c'è un punto del cilindro,

$$c = y_j - \sqrt{R^2 - (x_i - x_c)^2}, \tag{7.6}$$

e sul lato destro del cilindro, per il braccio verso sinistra, $a = (x_i - x_c) - \sqrt{R^2 - y_j^2}$.

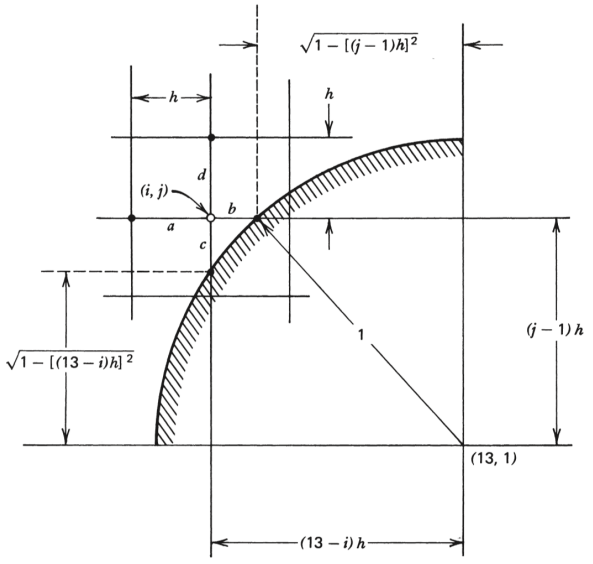


Figura 7.5: Un nodo vicino alla superficie sinistra del cilindro.

7.4 La condizione di derivata e i nodi fittizi

All'uscita si deve imporre $\partial\psi/\partial x = 0$. Con la differenza centrata (2.8) la condizione diventa

$$\psi_{i+1,j} = \psi_{i-1,j} \qquad \text{per } i = m \text{ (sezione di uscita)}, \tag{7.7}$$

che richiede una colonna di nodi fittizi a distanza h a destra dell'uscita (Fig. 7.3), esattamente come nel §2.6. I nodi fittizi non compaiono nel calcolo, perché sostituendo la (7.7) nella (7.3) si ottiene per i nodi di uscita

$$\psi_{m,j} = \tfrac{1}{4}\left(2\psi_{m-1,j} + \psi_{m,j-1} + \psi_{m,j+1}\right). \tag{7.8}$$

In conclusione i nodi interni sono di tre tipi, identificati nel programma da un indice: vicini al cilindro (formula (7.4)), sulla sezione di uscita (formula (7.8)), tutti gli altri (formula (7.3)).

7.5 Programma 2.7: il cilindro nel canale

Il programma originale classifica a mano i nodi e assegna a mano i bracci. La versione Python (Listato 7.2) lo fa automaticamente, per qualsiasi passo di griglia: per ogni nodo esterno al cilindro controlla se uno dei vicini è interno (o sulla superficie) e, in questo caso, calcola la distanza dalla superficie lungo quella direzione con le (7.5)–(7.6); il nodo viene trattato con la formula a bracci disuguali solo se almeno un braccio è più corto di h. Per $h = 0{,}25$ si ritrovano i dieci nodi della classificazione manuale:

nodo (i,j)	(9,2)	(9,3)	(10,4)	(11,5)	(12,5)	(14,5)	(15,5)	(16,4)	(17,2)	(17,3)
a	h	h	h	h	h	h	h	0,0886	0,0318	0,1340
b	0,0318	0,1340	0,0886	h	h	h	h	h	h	h
c	h	h	0,0886	0,1340	0,0318	0,0318	0,1340	0,0886	h	h

Il nodo $(13,6)$, sopra la sommità del cilindro, ha il vicino inferiore $(13,5)$ esattamente sulla superficie: i suoi bracci sono tutti uguali a h e si usa la formula ordinaria.

Risultati

- Il metodo di Liebmann converge (somma delle variazioni $\leq 0{,}0005$) in 67 iterazioni.
- La corrente entra dalla piccola apertura sull'asse, si allarga a ventaglio, aggira il cilindro e si dispone in modo uniforme verso l'uscita (Fig. 7.6). La velocità sul lato a monte del cilindro è maggiore che su quello a valle: la velocità media stimata nei nodi vicini alla superficie (portata fratto distanza dalla parete, $V \simeq \psi/\delta$) vale 0,834 sul lato sinistro e 0,789 su quello destro. Per il teorema di Bernoulli la pressione è più bassa a monte: la forza di pressione tende a spingere il cilindro *controcorrente*. Il paradosso di d'Alembert non vale, perché il flusso non è simmetrico fra monte e valle: l'ingresso concentrato e l'uscita uniforme rompono la simmetria.
- La soluzione è stabile rispetto al passo della griglia:

h	iterazioni	$\psi(3;\,1{,}5)$	V monte	V valle
0,2500	67	0,5439	0,834	0,789
0,1250	181	0,5447	0,830	0,789
0,0625	553	0,5449	0,825	0,788

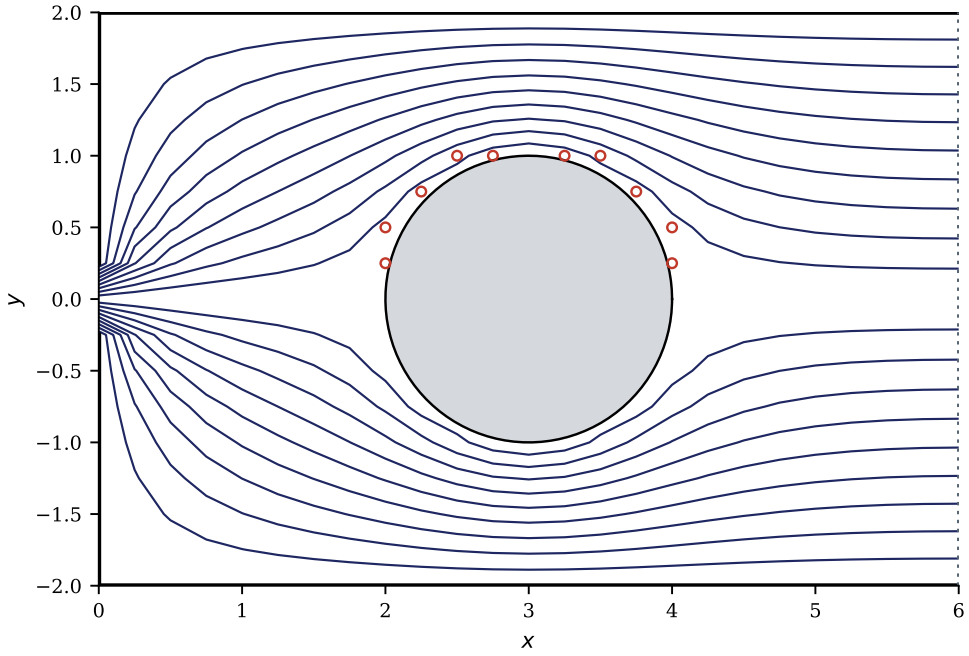


Figura 7.6: Programma 2.7: flusso in un canale con un cilindro circolare (la metà inferiore è l'immagine speculare di quella calcolata). Cerchietti rossi: nodi trattati con la formula a bracci disuguali.

(la tolleranza è stata ridotta come h^2, per mantenere costante la tolleranza per unità di area).

prog2_7_cilindro_canale.py

```
"""
prog2_7_cilindro_canale.py -- PROGRAMMA 2.7
Flusso potenziale in un canale con un cilindro circolare: contorno curvo che
non passa per i nodi della griglia (formula a cinque punti con bracci
disuguali) e condizione di derivata all'uscita (nodi fittizi).

Dominio (meta' superiore, per simmetria): 0 <= x <= 6, 0 <= y <= 2, cilindro di
raggio 1 centrato in (3, 0). Condizioni: psi = 0 sul cilindro e sull'asse,
psi = 1 sulla parete superiore e sulla parete d'ingresso (ingresso: piccola
apertura sull'asse, fra y = 0 e y = h), dpsi/dx = 0 all'uscita.
Metodo iterativo: Liebmann (Gauss-Seidel).
"""
import numpy as np
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, ROSSO, GRIGIO, VERDE

XC, R, LX, LY = 3.0, 1.0, 6.0, 2.0


def dentro(x, y):
    """Vero per i nodi interni al cilindro o sulla sua superficie (psi = 0)."""
    return (x - XC)**2 + y**2 <= R**2 + 1e-12


def prepara(h):
    M, N = int(round(LX / h)) + 1, int(round(LY / h)) + 1
    x, y = np.arange(M) * h, np.arange(N) * h
    psi = np.zeros((M, N))
```

```
    psi[:, N - 1] = 1.0                    # parete superiore
    psi[0, 1:] = 1.0                       # parete d'ingresso (apertura fra y=0 e
    y=h)
    tipo = np.zeros((M, N), dtype=int)     # 0 regolare, 1 vicino al cilindro, -1
    uscita, 9 contorno/solido
    tipo[0, :] = 9; tipo[:, 0] = 9; tipo[:, N - 1] = 9
    bracci = np.full((M, N, 4), h)         # a (sinistra), b (destra), c (sotto), d
    (sopra)
    for i in range(1, M):
        for j in range(1, N - 1):
            if dentro(x[i], y[j]):
                tipo[i, j] = 9             # nodo interno al cilindro: psi = 0
                continue
            if i == M - 1:
                tipo[i, j] = -1
                continue
            # distanza dal cilindro lungo le quattro direzioni, se il vicino e'
    interno
            if dentro(x[i + 1], y[j]):     # b: verso destra (lato sinistro del
    cilindro)
                bracci[i, j, 1] = (XC - np.sqrt(R**2 - y[j]**2)) - x[i]
            if dentro(x[i - 1], y[j]):     # a: verso sinistra (lato destro del
    cilindro)
                bracci[i, j, 0] = x[i] - (XC + np.sqrt(R**2 - y[j]**2))
            if dentro(x[i], y[j - 1]):     # c: verso il basso
                bracci[i, j, 2] = y[j] - np.sqrt(R**2 - (x[i] - XC)**2)
            # formula a bracci disuguali solo se almeno un braccio e' piu' corto di
    h
            if np.min(bracci[i, j]) < h - 1e-12:
                tipo[i, j] = 1
    interni = [(i, j) for i in range(1, M) for j in range(1, N - 1) if tipo[i, j]
    != 9]
    # valori di tentativo
    for i, j in interni:
        psi[i, j] = 0.5
    return x, y, psi, tipo, bracci, interni


def liebmann(h, tol=0.0005, kmax=20000):
    x, y, psi, tipo, br, interni = prepara(h)
    it, err = 0, 1.0
    while err > tol and it < kmax:
        it += 1
        err = 0.0
        for i, j in interni:
            vecchio = psi[i, j]
            if tipo[i, j] == 0:
                nuovo = 0.25 * (psi[i - 1, j] + psi[i + 1, j] + psi[i, j - 1] +
    psi[i, j + 1])
            elif tipo[i, j] == -1:         # uscita: psi_{M+1,j} = psi_{M-1,j}
                nuovo = 0.25 * (2.0 * psi[i - 1, j] + psi[i, j - 1] + psi[i, j + 1])
            else:                          # bracci disuguali: i nodi interni al
    cilindro
                a, b, c, d = br[i, j]      # valgono 0, come il punto di contorno
                nuovo = (psi[i - 1, j] / (a * (a + b)) + psi[i + 1, j] / (b * (a +
    b))
                         + psi[i, j - 1] / (c * (c + d)) + psi[i, j + 1] / (d * (c
    + d))) \
                    / (1.0 / (a * b) + 1.0 / (c * d))
            psi[i, j] = nuovo
```

```
            err += abs(nuovo - vecchio)
    return x, y, psi, tipo, br, it


def velocita_parete(x, y, psi, tipo, br, h):
    """Stima della velocita' sul cilindro nei nodi di tipo 1 piu' vicini alla
    superficie: V ~ psi / (distanza normale dalla parete)."""
    pts = []
    for i in range(len(x)):
        for j in range(len(y)):
            if tipo[i, j] == 1:
                dist = np.hypot(x[i] - XC, y[j]) - R
                th = np.degrees(np.arctan2(y[j], x[i] - XC))
                pts.append((th, psi[i, j] / dist))
    return np.array(sorted(pts))


if __name__ == "__main__":
    h = 0.25
    x, y, psi, tipo, br, it = liebmann(h)
    print(f"h = {h}: {it} iterazioni di Liebmann")
    print("nodi con formula a bracci disuguali (i, j da 1 come nel testo) e bracci
    a, b, c:")
    for i in range(len(x)):
        for j in range(len(y)):
            if tipo[i, j] == 1:
                a, b, c, d = br[i, j]
                print(f"   ({i+1:2d},{j+1}) x={x[i]:.2f} y={y[j]:.2f}:  a={a:.4f}
    b={b:.4f} c={c:.4f}")

    fig, ax = plt.subplots(figsize=(6.4, 4.4))
    livelli = np.round(np.arange(0.1, 1.0, 0.1), 2)
    # i nodi solidi valgono 0: le curve vicine alla parete, che entrerebbero
    # leggermente nel cilindro, vengono coperte dal disegno del corpo
    ax.contour(x, y, psi.T, levels=livelli, colors=BLU, linewidths=0.9,
    linestyles="solid")
    ax.contour(x, -y[::-1], psi[:, ::-1].T, levels=livelli, colors=BLU,
    linewidths=0.9,
               linestyles="solid")
    t = np.linspace(0, 2 * np.pi, 200)
    ax.fill(XC + R * np.cos(t), R * np.sin(t), color="#d5d8dc", zorder=3)
    ax.plot(XC + R * np.cos(t), R * np.sin(t), "k", lw=1, zorder=4)
    ii, jj = np.nonzero(tipo == 1)
    ax.plot(x[ii], y[jj], "o", mfc="none", color=ROSSO, ms=4, zorder=5)
    ax.plot([0, 6], [2, 2], "k", lw=2); ax.plot([0, 6], [-2, -2], "k", lw=2)
    ax.plot([0, 0], [h, 2], "k", lw=2); ax.plot([0, 0], [-h, -2], "k", lw=2)
    ax.plot([6, 6], [-2, 2], ":", color=GRIGIO)
    ax.set_aspect("equal"); ax.set_xlabel("$x$"); ax.set_ylabel("$y$");
    ax.grid(False)
    salva(fig, "prog2_7_cilindro")

    # asimmetria a monte/valle e convergenza con il passo
    print("\n   h     iterazioni   psi(3, 1.5)   V media lato monte / lato valle
    (nodi vicini)")
    for hh in (0.25, 0.125, 0.0625):
        xx, yy, ps, tp, bb, k = liebmann(hh, tol=0.0005 * (0.25 / hh)**2)
        i, j = int(round(3.0 / hh)), int(round(1.5 / hh))
        vp = velocita_parete(xx, yy, ps, tp, bb, hh)
        monte = vp[vp[:, 0] > 90, 1].mean(); valle = vp[vp[:, 0] < 90, 1].mean()
        print(f"  {hh:6.4f}    {k:5d}       {ps[i, j]:.4f}        {monte:.3f} /
```

```
    {valle:.3f}")
```

Listato 7.2: Programma 2.7 – canale con un cilindro circolare: contorno curvo e condizione di derivata (`prog2_7_cilindro_canale.py`).

Griglie cartesiane e griglie adattate al corpo

La formula a bracci disuguali permette di usare una griglia cartesiana per corpi di forma qualsiasi, ma ha due difetti: l'accuratezza locale scende al primo ordine, e i bracci molto corti (qui fino a 0,03, un ottavo del passo) peggiorano il condizionamento delle equazioni e rallentano la convergenza. I codici moderni seguono due strade: le griglie *curvilinee adattate al corpo*, in cui una famiglia di linee coordinate coincide con la superficie (con le equazioni trasformate in coordinate generalizzate), oppure le griglie *non strutturate* a volumi finiti, che si adattano a qualsiasi geometria. Il metodo dei contorni immersi, erede della formula a bracci disuguali, è tornato di moda per i corpi in moto e per le geometrie molto complesse.

7.6 Il flusso assialsimmetrico in un condotto

Le stesse tecniche si applicano all'equazione (4.2) della funzione di corrente di Stokes. Con le differenze centrate su una griglia quadrata di passo h, con r_j distanza del nodo dall'asse, si ottiene la formula iterativa

$$\psi_{i,j} = \tfrac{1}{4}\left[\psi_{i-1,j} + \psi_{i+1,j} + \left(1 + \frac{h}{2r_j}\right)\psi_{i,j-1} + \left(1 - \frac{h}{2r_j}\right)\psi_{i,j+1}\right], \tag{7.9}$$

che differisce dalla (7.3) per i pesi dei vicini in direzione radiale. L'esercizio 11 la ricava e la applica al flusso in un tubo con diaframmi forati ripetuti; l'esercizio 10 tratta un canale con un ostacolo rettangolare.

Capitolo 8

Equazioni iperboliche: propagazione e riflessione di onde di piccola ampiezza

8.1 L'equazione delle onde

I problemi di moto ondoso in fluidodinamica sono governati da equazioni iperboliche. Un esempio è il flusso supersonico attorno a un corpo sottile, eq. (6.2); un altro, più semplice e di grande importanza pratica, è la propagazione monodimensionale di un'onda sonora di piccola ampiezza,

$$\frac{\partial^2 u}{\partial t^2} = a^2 \frac{\partial^2 u}{\partial x^2}, \tag{8.1}$$

dove t è il tempo, x la coordinata nella direzione di propagazione, a la velocità del suono (costante nell'analisi linearizzata) e u la velocità del fluido. Equazioni della stessa forma governano le perturbazioni di densità, pressione e temperatura.

Da dove viene l'equazione delle onde

Un gas in quiete, di densità ρ_0 e pressione p_0, sia perturbato: $\rho = \rho_0 + \rho'$, $p = p_0 + p'$, con $\rho' \ll \rho_0$ e velocità u piccola rispetto ad a. Le equazioni di continuità e di Eulero monodimensionali,

$$\frac{\partial \rho}{\partial t} + \frac{\partial (\rho u)}{\partial x} = 0, \qquad \rho \left(\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} \right) = -\frac{\partial p}{\partial x},$$

linearizzate (si trascurano i prodotti di perturbazioni) diventano $\partial \rho'/\partial t + \rho_0 \, \partial u/\partial x = 0$ e $\rho_0 \, \partial u/\partial t = -\partial p'/\partial x$. Le perturbazioni sono isentropiche, quindi $p' = a^2 \rho'$ con $a^2 = (\mathrm{d}p/\mathrm{d}\rho)_s$. Derivando la seconda equazione rispetto a t e usando la prima:

$$\rho_0 \frac{\partial^2 u}{\partial t^2} = -a^2 \frac{\partial}{\partial x} \frac{\partial \rho'}{\partial t} = a^2 \rho_0 \frac{\partial^2 u}{\partial x^2},$$

cioè la (8.1). Derivando invece la prima rispetto a t e la seconda rispetto a x si ottiene la stessa equazione per ρ' (e quindi per p') [20].

Si vuole calcolare u per $t > 0$ nel tratto $0 \le x \le L$ di un tubo a sezione costante, note le condizioni iniziali

$$u(x,0) = f(x), \qquad \frac{\partial u}{\partial t}(x,0) = g(x), \tag{8.2}$$

con f e g da specificare. Le condizioni al contorno dipendono dalla natura degli estremi.

Estremo chiuso ed estremo aperto

Estremo chiuso da una parete rigida: il gas non può attraversarla, quindi $u = 0$ in ogni istante.

Estremo aperto verso l'atmosfera: la pressione è quella esterna, costante: $p' = 0$, quindi $\rho' = 0$ in ogni istante. Dalla continuità linearizzata, $\rho_0 \, \partial u/\partial x = -\partial \rho'/\partial t = 0$: la condizione equivalente per la velocità è $\partial u/\partial x = 0$.

8.2 Lo schema alle differenze centrate

Per risolvere il problema misto ai valori iniziali e al contorno si divide il tratto del tubo in intervalli di ampiezza h e l'asse dei tempi in intervalli τ; le linee verticali della griglia sono m, quelle orizzontali tante quante servono (Fig. 8.1). Con $u_{i,j} = u(x_i, t_j)$ e le differenze centrate (2.9) per le due derivate seconde,

$$\frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{\tau^2} = a^2 \, \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2},$$

e quindi

$$u_{i,j+1} = 2u_{i,j} + \mathrm{Co}^2 \left(u_{i-1,j} - 2u_{i,j} + u_{i+1,j} \right) - u_{i,j-1}, \tag{8.3}$$

dove compare il parametro adimensionale

$$\mathrm{Co} = \frac{a\tau}{h}, \tag{8.4}$$

il **numero di Courant**: il rapporto fra la distanza percorsa dall'onda in un passo temporale e il passo spaziale. La (8.3) calcola la soluzione su un livello di tempo a partire da quelle sui due livelli precedenti.

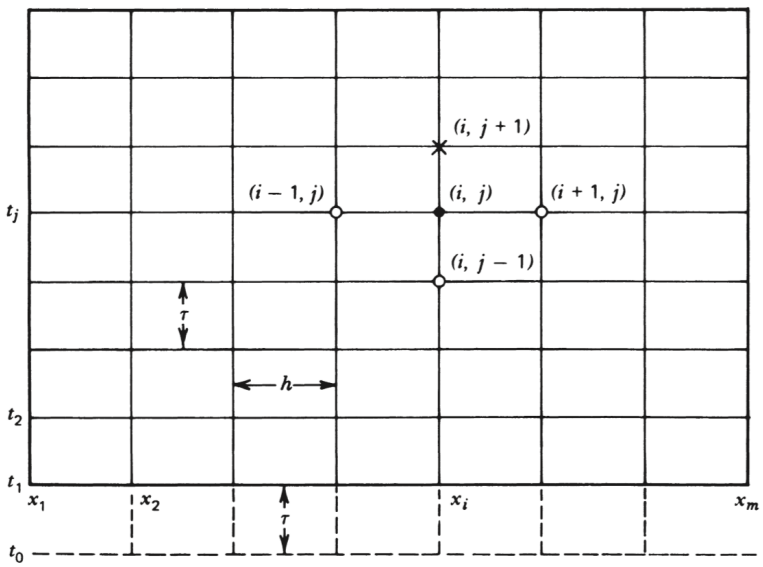


Figura 8.1: Griglia di calcolo per l'equazione delle onde.

8.3 La stabilità: l'analisi di von Neumann

Si possono costruire molti schemi per la stessa equazione; la loro utilità dipende innanzi tutto dalla **stabilità**, cioè dal fatto che la soluzione numerica non cresca senza limiti applicando ripetutamente lo schema. Nel Cap. 6 si è dimostrata, in un certo senso, la stabilità dei metodi iterativi; qui si segue un'altra via, l'analisi di von Neumann, e si scopre che la stabilità dello schema (8.3) dipende dal numero di Courant.

Il fattore di amplificazione e la condizione di stabilità

Si suppone che le variabili si separino e che la soluzione si sviluppi in serie di Fourier; una componente rappresentativa di numero d'onda k è

$$u_{i,j} = U_j\, e^{\mathrm{i}\, ikh},$$

dove U_j è l'ampiezza all'istante t_j (l'indice i all'esponente è quello del nodo, i l'unità immaginaria). Sostituendo nella (8.3) e semplificando il fattore comune e^{iikh}:

$$U_{j+1} = 2U_j + \mathrm{Co}^2 U_j \left(e^{-\mathrm{i}kh} + e^{\mathrm{i}kh} - 2\right) - U_{j-1} \quad\Longrightarrow\quad U_{j+1} = A\, U_j - U_{j-1},$$

con $A \equiv 2[1 - \mathrm{Co}^2(1 - \cos kh)]$, avendo usato $(e^{\mathrm{i}\theta} + e^{-\mathrm{i}\theta})/2 = \cos\theta$. Si introduce il **fattore di amplificazione** G, tale che $U_j = G\, U_{j-1}$ e $U_{j+1} = G^2 U_{j-1}$; l'equazione diventa

$$G^2 - AG + 1 = 0, \qquad G = \frac{A}{2} \pm \sqrt{\left(\frac{A}{2}\right)^2 - 1}. \tag{8.5}$$

Il prodotto delle due radici è 1. Se $|A| > 2$ le radici sono reali e una delle due ha modulo maggiore di 1: l'ampiezza cresce indefinitamente. Se $|A| \le 2$ le radici sono complesse coniugate (o coincidenti) e hanno modulo esattamente 1. La condizione di stabilità è quindi $|A| \le 2$, cioè

$$\left[1 - \mathrm{Co}^2(1 - \cos kh)\right]^2 \le 1 \quad\Longleftrightarrow\quad \mathrm{Co}^2 \le \frac{2}{1 - \cos kh}.$$

Al variare di $\cos kh$ fra -1 e $+1$ il secondo membro varia fra 1 e infinito; perché lo schema sia stabile per *tutte* le componenti si deve prendere il valore minimo, che corrisponde all'onda più corta rappresentabile sulla griglia ($kh = \pi$, lunghezza d'onda $2h$). Il criterio di stabilità è dunque

$$\mathrm{Co} = \frac{a\tau}{h} \le 1. \tag{8.6}$$

La (8.6) è la condizione di **Courant–Friedrichs–Lewy** (CFL): passo temporale e passo spaziale non si possono scegliere in modo indipendente. Se si infittisce la griglia, il passo temporale va ridotto nella stessa proporzione.

Stabile non vuol dire esatto: la dispersione numerica

Per $\mathrm{Co} \le 1$ si ha $|G| = 1$: lo schema non smorza nessuna componente, come l'equazione esatta. Ma ne altera la *fase*. Scritto $G = e^{-\mathrm{i}\theta}$, l'angolo di fase per passo è $\cos\theta = 1 - \mathrm{Co}^2(1 - \cos kh)$, mentre l'onda esatta avanza della fase $ak\tau = \mathrm{Co}\, kh$. Il rapporto fra velocità di fase numerica ed esatta, $c_{\mathrm{num}}/a = \theta/(\mathrm{Co}\, kh)$, è uguale a 1 solo per $\mathrm{Co} = 1$;

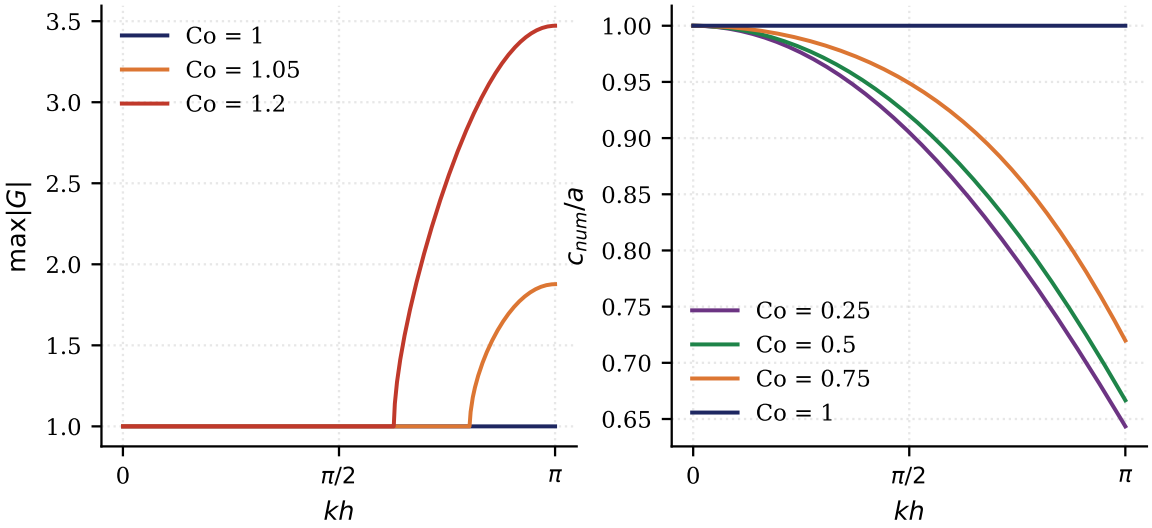


Figura 8.2: Analisi di von Neumann dello schema (8.3) (`prog2_8b_courant.py`). A sinistra: modulo massimo del fattore di amplificazione; per Co >1 supera 1 per le onde corte. A destra: rapporto fra velocità di fase numerica ed esatta per Co ≤ 1.

per Co <1 le onde corte viaggiano più lentamente di quelle lunghe (Fig. 8.2, a destra). Uno schema in cui la velocità di propagazione dipende dalla lunghezza d'onda si dice **dispersivo**: un impulso, che contiene molte lunghezze d'onda, si deforma e si lascia dietro una coda di oscillazioni.

8.4 La soluzione di d'Alembert e il significato della condizione CFL

Con Co $=1$ la (8.3) assume una forma particolarmente semplice:

$$u_{i,j+1} = u_{i-1,j} + u_{i+1,j} - u_{i,j-1}. \tag{8.7}$$

Il valore nel nodo segnato con la croce in Fig. 8.1 si calcola dai valori dei tre nodi cerchiati, sui due livelli precedenti, "scavalcando" il nodo centrale del livello intermedio: è lo schema detto *leapfrog* (salto della rana). Dimostriamo che fornisce la soluzione esatta dell'equazione differenziale.

Con Co $=1$ lo schema leapfrog è esatto

La soluzione della (8.1) con le condizioni iniziali (8.2) è la formula di d'Alembert,

$$u(x,t) = \tfrac{1}{2}\left[f(x+at) + f(x-at)\right] + \frac{1}{2a}\int_{x-at}^{x+at} g(s)\,\mathrm{d}s, \tag{8.8}$$

che si può scrivere nella forma

$$u(x,t) = F(x-at) + G_s(x+at)\,: \tag{8.9}$$

F e G_s sono onde semplici che si propagano senza cambiare forma, con velocità a, rispettivamente verso le x positive e negative. Le rette $\mathrm{d}x/\mathrm{d}t = \pm a$ del piano x–t, lungo le quali le onde viaggiano, sono le **caratteristiche** dell'equazione.

Nei nodi, con $x_i = x_1 + (i-1)h$ e $t_j = t_1 + (j-1)\tau$, si ha $u_{i,j} = F(\alpha + ih - ja\tau) + G_s(\beta + ih + ja\tau)$, con α e β costanti. Se $a\tau = h$:

$$u_{i,j} = F\big(\alpha + (i-j)h\big) + G_s\big(\beta + (i+j)h\big).$$

Il secondo membro della (8.7) è allora

$$\begin{aligned} u_{i-1,j} + u_{i+1,j} - u_{i,j-1} &= F\big(\alpha + (i-j-1)h\big) + G_s\big(\beta + (i+j-1)h\big) \\ &\quad + F\big(\alpha + (i-j+1)h\big) + G_s\big(\beta + (i+j+1)h\big) \\ &\quad - F\big(\alpha + (i-j+1)h\big) - G_s\big(\beta + (i+j-1)h\big) \\ &= F\big(\alpha + (i-j-1)h\big) + G_s\big(\beta + (i+j+1)h\big), \end{aligned}$$

che è esattamente $u_{i,j+1}$. Lo schema trasporta i valori lungo le caratteristiche, da nodo a nodo, senza alcun errore.

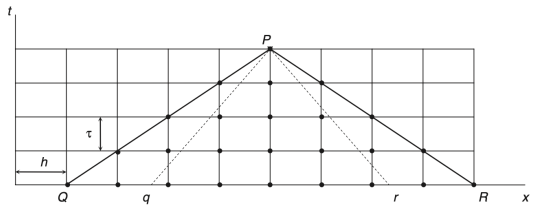


Figura 8.3: Interpretazione fisica del criterio di stabilità.

La condizione (8.6) ha un significato fisico chiaro (Fig. 8.3). La (8.3) mostra che la soluzione nel nodo P dipende dai nodi dei livelli precedenti compresi fra le due diagonali PQ e PR, di pendenza $\pm h/\tau$: la regione PQR è il *dominio di dipendenza numerico* di P. Per la teoria delle caratteristiche, invece, P può ricevere segnali solo dalla regione Pqr delimitata dalle caratteristiche di pendenza $\pm a$ uscenti all'indietro da P: è il *dominio di dipendenza fisico*.

- Se $a\tau/h < 1$ le caratteristiche cadono dentro PQR: il dominio numerico contiene quello fisico, il calcolo usa tutta l'informazione necessaria ed è stabile, ma usa anche informazione superflua (le regioni fra PQ e Pq e fra PR e Pr), e il risultato è approssimato.
- Se $a\tau/h > 1$ le caratteristiche cadono fuori: il calcolo ignora una parte dell'informazione che determina la soluzione in P ed è instabile.
- Se $a\tau/h = 1$ i due domini coincidono e la soluzione numerica è esatta.

8.5 La partenza del calcolo

La (8.7) richiede due livelli precedenti e non si può usare direttamente per calcolare la soluzione al tempo t_2, perché le condizioni sono assegnate solo all'istante iniziale $t_1 = 0$. Si introduce in Fig. 8.1 una riga di nodi fittizi all'istante $t_0 = t_1 - \tau$ e si riscrivono le condizioni iniziali in notazione indiciale, approssimando $\partial u/\partial t$ con la differenza centrata:

$$u_{i,1} = f_i, \qquad u_{i,0} = u_{i,2} - 2\tau g_i. \tag{8.10}$$

Per $j = 1$ la (8.7) diventa $u_{i,2} = f_{i-1} + f_{i+1} - u_{i,2} + 2\tau g_i$, cioè

$$u_{i,2} = \tfrac{1}{2}\left(f_{i-1} + f_{i+1}\right) + \tau g_i, \qquad i = 2, \ldots, m-1, \tag{8.11}$$

la *formula di partenza* dello schema. Per Co qualsiasi lo stesso procedimento applicato alla (8.3) dà $u_{i,2} = f_i + \frac{1}{2}\mathrm{Co}^2(f_{i-1} - 2f_i + f_{i+1}) + \tau g_i$.

8.6 Programma 2.8: un'ondina in un tubo chiuso e aperto

Pur conoscendo la soluzione esatta (8.8), scriverla esplicitamente è laborioso quando le onde si incontrano o si riflettono agli estremi. Studiamo questi fenomeni numericamente con lo schema leapfrog. Si consideri un tubo lungo 1 m, chiuso a sinistra e aperto a destra (Fig. 8.4). All'istante $t = 0$ viene generata in qualche modo nel tubo un'ondina sinusoidale di lunghezza d'onda 0,4 m, a 0,2 m dall'estremo chiuso, di ampiezza 1 m/s:

$$f(x) = \begin{cases} \sin\left(2\pi\dfrac{x - 0{,}2}{0{,}4}\right) & 0{,}2 \le x \le 0{,}6 \\ 0 & \text{altrove} \end{cases}, \qquad g(x) = 0. \tag{8.12}$$

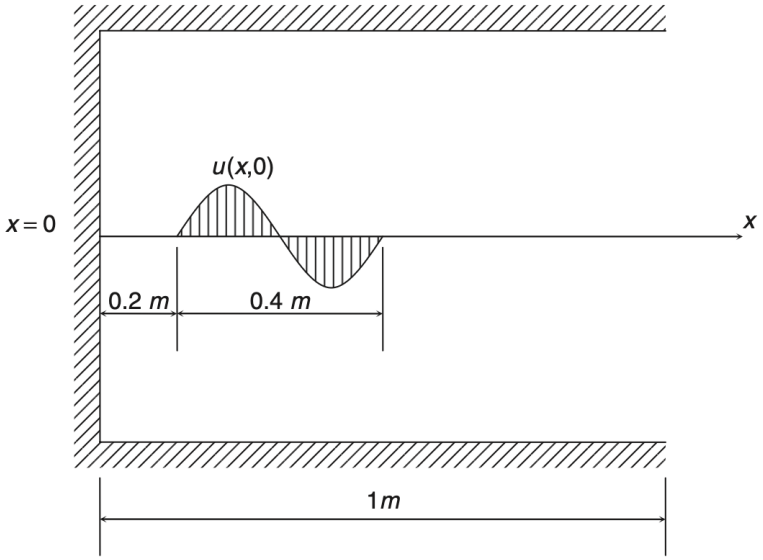


Figura 8.4: Ondina sinusoidale in un tubo monodimensionale.

Dati del Programma 2.8

Velocità del suono $a = 340$ m/s (livello del mare); $h = 0{,}02$ m, $m = 51$ nodi; l'ondina occupa i nodi $11 \le i \le 31$, dove $f_i = \sin\{5\pi[(i-1)h - 0{,}2]\}$; $\tau = h/a = 5{,}88 \cdot 10^{-5}$ s (Co $= 1$), il tempo che l'onda impiega a percorrere un passo; 115 passi temporali.

Le condizioni al contorno sono $u_{1,j} = 0$ all'estremo chiuso e, all'estremo aperto, $\partial u/\partial x = 0$. Quest'ultima si tratta con una colonna di nodi fittizi in $i = m + 1$, come per l'uscita del canale del Cap. 7: $u_{m+1,j} = u_{m-1,j}$, per cui nel nodo $i = m$ lo schema e la formula di partenza diventano

$$u_{m,j+1} = 2u_{m-1,j} - u_{m,j-1}, \qquad u_{m,2} = f_{m-1} + \tau g_m. \tag{8.13}$$

Il Programma 2.8 (Listato 8.1) è scritto per un Co qualsiasi: con Co $= 1$ le formule generali si riducono alle (8.7)–(8.13). Il programma confronta la soluzione numerica con quella di d'Alembert, in cui le riflessioni si ottengono con il metodo delle immagini: f si prolunga in

modo dispari rispetto all'estremo chiuso (dove $u = 0$) e pari rispetto all'estremo aperto (dove $\partial u/\partial x = 0$), con periodo $4L$.

Risultati: che cosa succede nel tubo

- Lo scarto massimo dalla soluzione di d'Alembert su tutti i 115 passi è $7 \cdot 10^{-15}$: l'arrotondamento della macchina. Lo schema con Co $= 1$ è esatto anche con le condizioni al contorno.
- L'ondina iniziale si scompone in due ondine identiche di ampiezza 0,5 m/s e della stessa lunghezza d'onda, che si propagano in versi opposti: la chiamiamo onda L quella verso sinistra e onda R quella verso destra (Fig. 8.5). A $t = 5\tau$, quando ciascuna ha percorso un quarto di lunghezza d'onda, le metà posteriori si sovrappongono e si cancellano, e si vedono solo le metà anteriori. A $t = 10\tau$ le due onde si sono appena separate e l'onda L comincia a investire l'estremo chiuso.
- **Estremo chiuso**: riflettendosi sulla parete rigida l'onda L inverte non solo il verso di propagazione ma anche il segno di u. L'onda incidente è guidata dai valori positivi di u, quella riflessa da valori negativi (fotogrammi a 10τ e 30τ); i tre fotogrammi intermedi mostrano l'interazione fra la parte riflessa e la parte ancora incidente.
- **Estremo aperto**: fra 20τ e 40τ l'onda R si riflette invertendo solo il verso di propagazione, non il segno di u: l'onda riflessa è ancora guidata da valori negativi, come prima di raggiungere l'estremo aperto.
- I quattro fotogrammi dopo 40τ mostrano l'incontro delle due onde riflesse. A $t = 50\tau$ si cancellano esattamente e il tubo è momentaneamente in quiete. Le riflessioni e le interazioni proseguono indefinitamente, in un fluido non viscoso; il fenomeno è periodico con periodo $4L/a = 200\tau$. Il diagramma x–t di Fig. 8.6 rende evidente che tutto avviene lungo le caratteristiche $x \pm at =$ cost.

Tubi aperti e chiusi in aeronautica e motoristica

Le regole di riflessione appena trovate governano i condotti di aspirazione e di scarico dei motori alternativi: un impulso di pressione che raggiunge l'estremo aperto di uno scarico si riflette come depressione e, se la lunghezza del condotto è scelta in modo che la depressione arrivi alla valvola mentre è aperta, aiuta a svuotare il cilindro (accordo dello scarico). Nei tubi delle prese di pressione degli strumenti di bordo e delle gallerie del vento gli stessi fenomeni limitano la risposta in frequenza: un tubo chiuso a un estremo e aperto all'altro ha la prima risonanza a $f = a/(4L)$, appena 85 Hz per un tubo di un metro.

```
"""
prog2_8_onda_tubo.py -- PROGRAMMA 2.8
Propagazione e riflessione di un'ondina sinusoidale di piccola ampiezza in un
tubo lungo 1 m, chiuso a sinistra (u = 0) e aperto a destra (du/dx = 0).
Equazione delle onde  u_tt = a^2 u_xx  risolta con lo schema "leapfrog"

   u_i^{j+1} = 2 u_i^j + Co^2 (u_{i-1}^j - 2 u_i^j + u_{i+1}^j) - u_i^{j-1},
   Co = a tau / h  (numero di Courant).

Con Co = 1 lo schema diventa u_i^{j+1} = u_{i-1}^j + u_{i+1}^j - u_i^{j-1} ed e'
esatto. La funzione 'leapfrog' accetta un Co qualsiasi (usata da prog2_8b).
"""
import numpy as np
```

prog2_8_onda_tubo.py

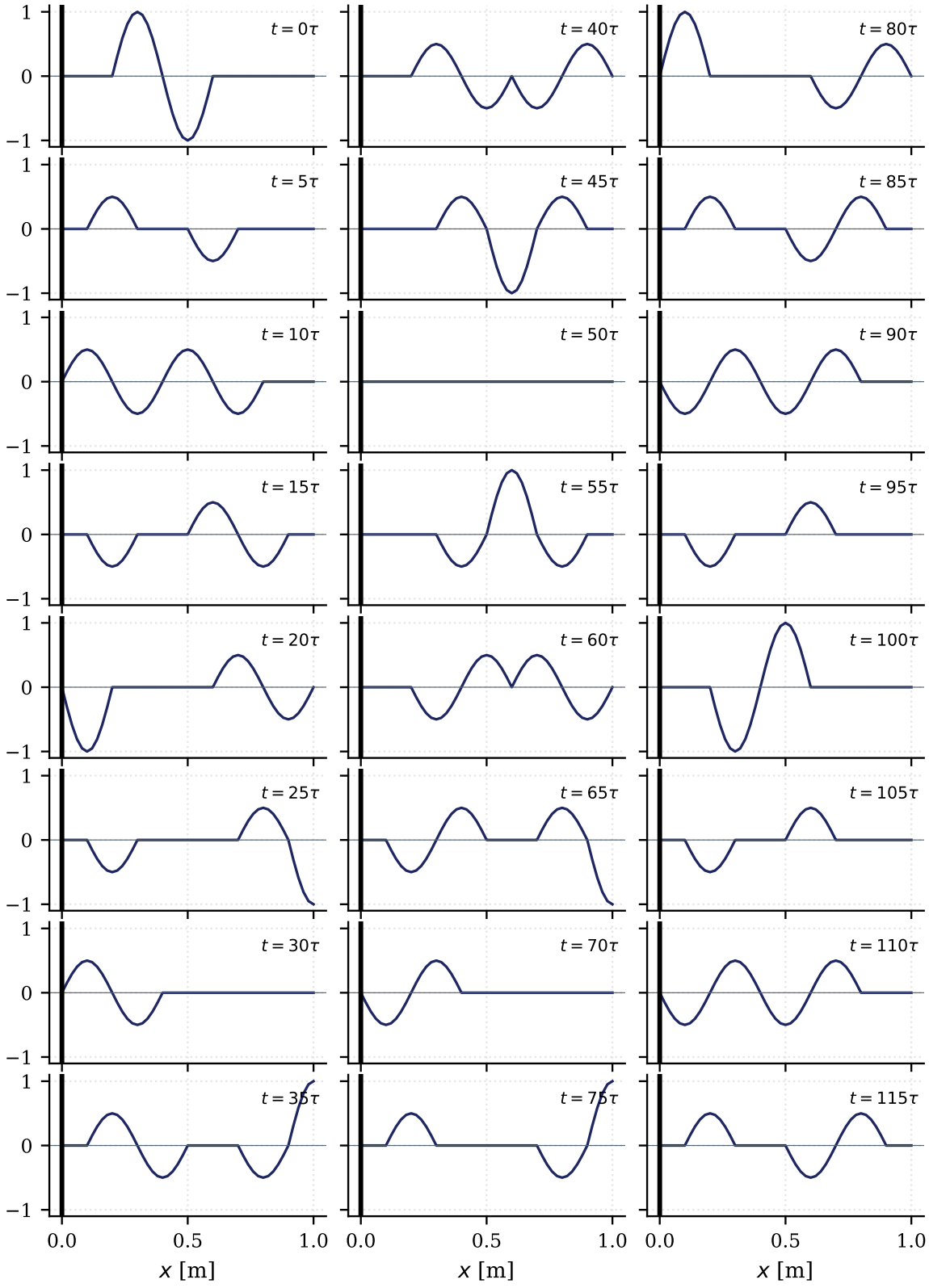


Figura 8.5: Programma 2.8: propagazione e riflessione dell'ondina, un fotogramma ogni cinque passi ($\tau = h/a$); le colonne si leggono dall'alto in basso. L'estremo chiuso è a sinistra (linea spessa), quello aperto a destra.

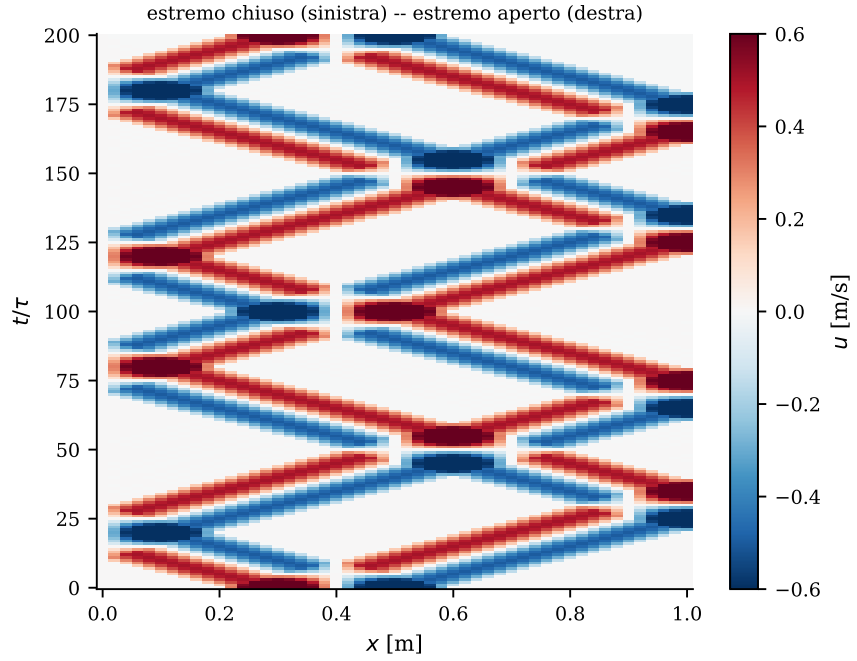


Figura 8.6: Diagramma x–t della velocità nel tubo: le onde viaggiano lungo le caratteristiche e si riflettono agli estremi; la colorazione dell'onda riflessa all'estremo chiuso (a sinistra) è invertita, quella riflessa all'estremo aperto no.

```
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, ROSSO, GRIGIO

A_SUONO = 340.0      # velocita' del suono [m/s]
L = 1.0              # lunghezza del tubo [m]


def f_iniziale(x):
    """Ondina di lunghezza d'onda 0,4 m e ampiezza 1 m/s fra x = 0,2 e 0,6 m."""
    return np.where((x >= 0.2) & (x <= 0.6), np.sin(5 * np.pi * (x - 0.2)), 0.0)


def g_iniziale(x):
    return np.zeros_like(x)


def leapfrog(h=0.02, Co=1.0, n_passi=115, f=f_iniziale, g=g_iniziale):
    """Restituisce x, i tempi e la matrice U[j, i] = u(x_i, t_j)."""
    m = int(round(L / h)) + 1
    x = np.linspace(0.0, L, m)
    tau = Co * h / A_SUONO
    C2 = Co**2
    U = np.zeros((n_passi + 1, m))
    U[0] = f(x)
    # formula di partenza (nodo fittizio u^{-1} = u^{1} - 2 tau g)
    U[1, 1:-1] = U[0, 1:-1] + 0.5 * C2 * (U[0, :-2] - 2 * U[0, 1:-1] + U[0, 2:]) \
        + tau * g(x)[1:-1]
    U[1, -1] = U[0, -1] + C2 * (U[0, -2] - U[0, -1]) + tau * g(x)[-1]    # u_{m+1} =
    u_{m-1}
    U[1, 0] = 0.0
    for j in range(1, n_passi):
        U[j + 1, 1:-1] = 2 * U[j, 1:-1] + C2 * (U[j, :-2] - 2 * U[j, 1:-1] + U[j,
```

```
    2:]) \
45              - U[j - 1, 1:-1]
46          U[j + 1, -1] = 2 * U[j, -1] + 2 * C2 * (U[j, -2] - U[j, -1]) - U[j - 1, -1]
47          U[j + 1, 0] = 0.0                          # estremo chiuso
48      t = tau * np.arange(n_passi + 1)
49      return x, t, U
50
51
52  def esatta(x, t, f=f_iniziale):
53      """d'Alembert con estensione di f dispari rispetto a x=0 (estremo chiuso),
54      pari rispetto a x=1 (estremo aperto), periodica di periodo 4L."""
55      def F(s):
56          s = np.mod(s + 2 * L, 4 * L) - 2 * L     # riporta in [-2L, 2L)
57          segno = np.sign(s); segno[segno == 0] = 1
58          sa = np.abs(s)
59          val = np.where(sa <= L, f(sa), f(2 * L - sa))
60          return segno * val
61      return 0.5 * (F(x - A_SUONO * t) + F(x + A_SUONO * t))
62
63
64  if __name__ == "__main__":
65      h = 0.02
66      x, t, U = leapfrog(h, 1.0, 115)
67      tau = t[1]
68      print(f"m = {len(x)} nodi, tau = h/a = {tau:.4e} s")
69      err = max(np.max(np.abs(U[j] - esatta(x, t[j]))) for j in range(len(t)))
70      print(f"Co = 1: massimo scarto dalla soluzione di d'Alembert = {err:.2e}")
71
72      # 24 fotogrammi, uno ogni 5 passi (serie temporale)
73      fig, axs = plt.subplots(8, 3, figsize=(6.6, 9.2), sharex=True, sharey=True)
74      for k, ax in enumerate(axs.T.flat):
75          j = 5 * k
76          ax.plot(x, U[j], color=BLU, lw=1.1)
77          ax.axhline(0, color=GRIGIO, lw=0.4)
78          ax.set_ylim(-1.1, 1.1); ax.set_yticks([-1, 0, 1]); ax.set_xticks([0, 0.5,
    1])
79          ax.text(0.98, 0.78, fr"$t={j}\tau$", transform=ax.transAxes, ha="right",
    fontsize=7)
80          ax.axvline(0, color="k", lw=2.0)
81      for ax in axs[-1]:
82          ax.set_xlabel("$x$ [m]")
83      fig.subplots_adjust(hspace=0.08, wspace=0.08)
84      salva(fig, "prog2_8_serie")
85
86      # diagramma x-t: le riflessioni seguono le caratteristiche x -+ a t = cost
87      x2, t2, U2 = leapfrog(h, 1.0, 200)
88      fig, ax = plt.subplots(figsize=(5.6, 4.2))
89      im = ax.pcolormesh(x2, t2 / tau, U2, cmap="RdBu_r", vmin=-0.6, vmax=0.6,
    shading="auto")
90      fig.colorbar(im, ax=ax, label="$u$ [m/s]")
91      ax.set_xlabel("$x$ [m]"); ax.set_ylabel(r"$t/\tau$")
92      ax.set_title("estremo chiuso (sinistra) -- estremo aperto (destra)", fontsize=8)
93      ax.grid(False)
94      salva(fig, "prog2_8_xt")
```

Listato 8.1: Programma 2.8 – schema leapfrog per la propagazione di un'ondina in un tubo chiuso–aperto (`prog2_8_onda_tubo.py`).

8.7 Il numero di Courant alla prova

Il programma `prog2_8b_courant.py` (Listato 8.2) ripete il calcolo dell'ondina allo stesso istante fisico $t = 40\,h/a$ con valori diversi del numero di Courant, e verifica le previsioni dell'analisi di von Neumann.

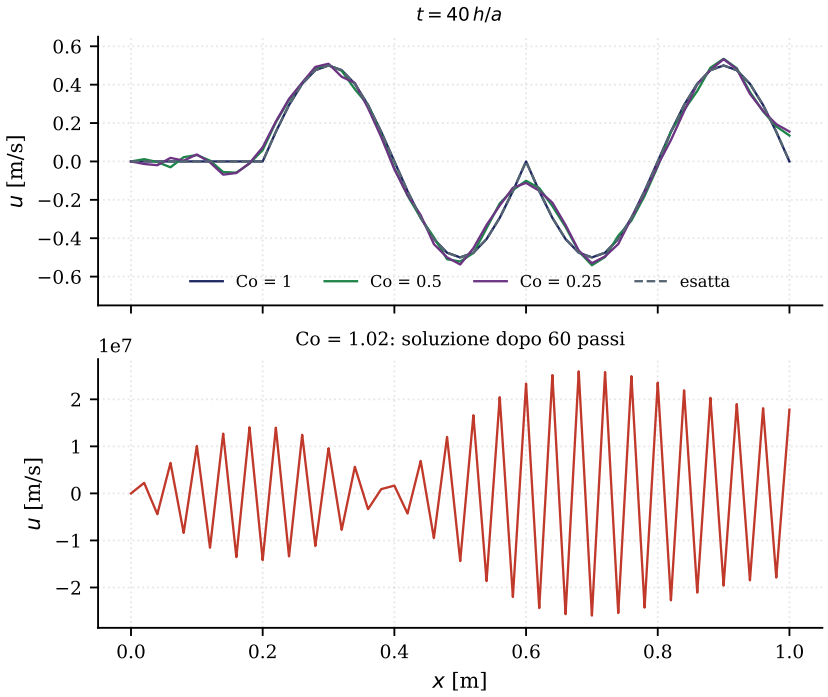


Figura 8.7: In alto: l'ondina a $t = 40\,h/a$ calcolata con Co $= 1$, 0,5 e 0,25, confrontata con la soluzione esatta. In basso: con Co $= 1{,}02$ la soluzione diventa un'oscillazione a dente di sega di ampiezza 10^7 dopo sessanta passi.

Risultati

- Con Co $= 1$ lo scarto dalla soluzione esatta è $3 \cdot 10^{-15}$; con Co $= 0{,}5$ e Co $= 0{,}25$ sale a 0,135 e 0,156 m/s, il 27–31% dell'ampiezza dell'onda (Fig. 8.7). Il passo temporale più piccolo non migliora il risultato, lo peggiora: l'errore dominante è la dispersione dovuta al passo spaziale, e l'analisi di von Neumann mostra che si riduce solo avvicinandosi a Co $= 1$. Gli spigoli dell'ondina, dove la derivata è discontinua, contengono onde corte che restano indietro e formano le oscillazioni visibili nella figura.
- Con Co $= 1{,}02$, appena oltre il limite, il modulo massimo del fattore di amplificazione (8.5) è $|G| = 1{,}491$ (per $kh = \pi$). Il disturbo cresce di un fattore 1,5 a ogni passo: l'ampiezza massima vale $1{,}2 \cdot 10^4$ dopo 40 passi, $6{,}5 \cdot 10^{10}$ dopo 80, $5{,}3 \cdot 10^{17}$ dopo 120, e la crescita segue proprio la legge $|G|^n$ (Fig. 8.8). La soluzione è dominata dall'onda più corta della griglia, a dente di sega: proprio la componente che il dominio di dipendenza numerico non riesce più a seguire.

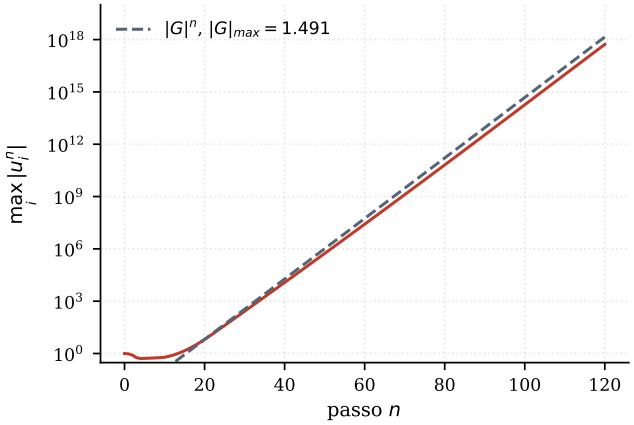


Figura 8.8: Crescita dell'ampiezza massima per Co $=$ 1,02 e confronto con la previsione di von Neumann.

La condizione CFL nei codici reali

Nei codici industriali il numero di Courant è il primo parametro che si controlla in un calcolo non stazionario. Negli schemi espliciti è un vincolo rigido di stabilità; negli schemi impliciti non lo è, ma un valore molto maggiore di uno degrada l'accuratezza nel tempo, e i manuali raccomandano in genere di non superare valori dell'ordine dell'unità [10]. Per i problemi multidimensionali la condizione coinvolge tutte le direzioni: con lo schema esplicito più semplice in due dimensioni diventa $a\tau\sqrt{1/\Delta x^2 + 1/\Delta y^2} \leq 1$.

prog2_8b_courant.py

```
"""
prog2_8b_courant.py -- il ruolo del numero di Courant nello schema leapfrog.

1) Analisi di von Neumann: fattore di amplificazione G(kh) e rapporto fra la
   velocita' di fase numerica e quella esatta, per diversi Co.
2) Esperimenti: la stessa ondina del Programma 2.8 integrata con Co = 1, 0,5
   e 0,25 (dispersione) e con Co = 1,02 (instabilita').
"""
import numpy as np
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, ROSSO, GRIGIO, VERDE, VIOLA
from prog2_8_onda_tubo import leapfrog, esatta, A_SUONO

# ------------------------------------------------------ 1) von Neumann
kh = np.linspace(1e-3, np.pi, 400)
fig, ax = plt.subplots(1, 2, figsize=(6.6, 2.8))
for Co, col in ((0.25, VIOLA), (0.5, VERDE), (0.75, ARANCIO), (1.0, BLU)):
    Ah = 1 - Co**2 * (1 - np.cos(kh))                # A/2
    theta = np.arccos(np.clip(Ah, -1, 1))            # fase numerica per passo
    ax[1].plot(kh, theta / (Co * kh), color=col, label=f"Co = {Co:g}")
for Co, col in ((1.0, BLU), (1.05, ARANCIO), (1.2, ROSSO)):
    Ah = 1 - Co**2 * (1 - np.cos(kh))
    G = np.where(np.abs(Ah) <= 1, 1.0, np.abs(Ah) + np.sqrt(np.maximum(Ah**2 - 1,
    0)))
    ax[0].plot(kh, G, color=col, label=f"Co = {Co:g}")
    print(f"Co = {Co:4.2f}: max |G| = {G.max():.4f} (a kh = pi)")
ax[0].set_xlabel("$kh$"); ax[0].set_ylabel(r"$\max|G|$"); ax[0].legend()
ax[0].set_xticks([0, np.pi / 2, np.pi]); ax[0].set_xticklabels(["0", r"$\pi/2$",
    r"$\pi$"])
ax[1].set_xlabel("$kh$"); ax[1].set_ylabel(r"$c_{num}/a$"); ax[1].legend()
ax[1].set_xticks([0, np.pi / 2, np.pi]); ax[1].set_xticklabels(["0", r"$\pi/2$",
    r"$\pi$"])
salva(fig, "prog2_8b_vonneumann")

# ------------------------------------------------------ 2) esperimenti
h = 0.02
t_fin = 40 * h / A_SUONO                          # stesso istante fisico per tutti
fig, ax = plt.subplots(2, 1, figsize=(5.6, 4.6), sharex=True)
print("\n  Co    passi   max|u - u_esatta| a t = 40 h/a")
for Co, col in ((1.0, BLU), (0.5, VERDE), (0.25, VIOLA)):
    n = int(round(40 / Co))
    x, t, U = leapfrog(h, Co, n)
    e = np.max(np.abs(U[-1] - esatta(x, t[-1])))
    print(f"  {Co:4.2f}    {n:4d}      {e:.3e}")
    ax[0].plot(x, U[-1], color=col, lw=1.0, label=f"Co = {Co:g}")
ax[0].plot(x, esatta(x, t_fin), "--", color=GRIGIO, lw=1.0, label="esatta")
ax[0].set_ylabel("$u$ [m/s]"); ax[0].legend(ncol=4, fontsize=7, loc="lower
    center"); ax[0].set_ylim(-0.75, 0.65)
ax[0].set_title(r"$t = 40\,h/a$", fontsize=8)
# instabilita': Co leggermente maggiore di 1
Co = 1.02
x, t, U = leapfrog(h, Co, 120)
amp = np.max(np.abs(U), axis=1)
print(f"\nCo = {Co}: ampiezza massima dopo 40, 80, 120 passi = "
      f"{amp[40]:.3g}, {amp[80]:.3g}, {amp[120]:.3g}")
ax[1].plot(x, U[60], color=ROSSO, lw=1.0)
ax[1].set_ylabel("$u$ [m/s]"); ax[1].set_xlabel("$x$ [m]")
ax[1].set_title(fr"Co = {Co}: soluzione dopo 60 passi", fontsize=8)
salva(fig, "prog2_8b_esperimenti")

```

```
fig, ax = plt.subplots(figsize=(4.4, 3.0))
ax.semilogy(amp, color=ROSSO)
Ah = 1 - 2 * Co**2
Gmax = abs(Ah) + np.sqrt(Ah**2 - 1)
ax.semilogy(np.arange(121), amp[20] * Gmax**(np.arange(121) - 20.0), "--",
    color=GRIGIO,
            label=fr"$|G|^n$, $|G|_{{max}}={Gmax:.3f}$")
ax.set_ylim(0.3, 1e20); ax.set_xlabel("passo $n$"); ax.set_ylabel(r"$\max_i
    |u_i^n|$")
ax.legend()
salva(fig, "prog2_8b_crescita")
```

Listato 8.2: `prog2_8b_courant.py`: analisi di von Neumann ed esperimenti con diversi numeri di Courant.

Capitolo 9

Onde di ampiezza finita e formazione dell'urto

9.1 Perché un'onda di ampiezza finita si deforma

Nell'esempio del capitolo precedente un'onda sonora si propaga a velocità costante senza cambiare forma. È una conseguenza della linearizzazione: si sono supposte perturbazioni piccole attorno a uno stato di equilibrio del gas. Per un'onda di ampiezza non piccola la situazione cambia [20]. Ogni porzione dell'onda si muove, rispetto al gas, alla velocità del suono *locale* a, e insieme è trasportata dal gas che si muove con velocità u: avanza quindi con velocità $u + a$. Nelle zone di compressione il gas è più caldo (la velocità del suono è maggiore) e si muove nel verso dell'onda; nelle zone di espansione accade il contrario. Le creste dell'onda viaggiano più veloci dei cavi: il fronte di compressione si irripidisce finché diventa praticamente una discontinuità, un'**onda d'urto**. È lo stesso meccanismo che fa frangere un'onda d'acqua sulla spiaggia.

9.2 Le equazioni per la velocità del gas e la velocità del suono

Si riscrivono l'equazione di continuità e l'equazione di Eulero per il moto non stazionario monodimensionale:

$$\frac{\partial \rho}{\partial t} + u\frac{\partial \rho}{\partial x} + \rho\frac{\partial u}{\partial x} = 0, \qquad \frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} + \frac{a^2}{\rho}\frac{\partial \rho}{\partial x} = 0, \tag{9.1}$$

dove nella seconda si è sostituito $\partial p/\partial x = a^2\,\partial\rho/\partial x$, con la velocità del suono a che ora dipende da x e da t. Si suppone che la velocità del gas sia inferiore a quella del suono.

Eliminazione della densità

In condizioni isentropiche, con il pedice 0 per lo stato indisturbato e γ rapporto dei calori specifici,

$$\frac{p}{p_0} = \left(\frac{\rho}{\rho_0}\right)^{\gamma}, \qquad a^2 = \frac{\mathrm{d}p}{\mathrm{d}\rho} = \gamma\frac{p}{\rho} = a_0^2\left(\frac{\rho}{\rho_0}\right)^{\gamma-1} \quad \Longrightarrow \quad \rho = \rho_0\left(\frac{a}{a_0}\right)^{2/(\gamma-1)}.$$

Ne segue $\mathrm{d}\rho/\rho = \frac{2}{\gamma-1}\,\mathrm{d}a/a$; sostituendo nelle (9.1):

$$\frac{2}{\gamma-1}\left(\frac{\partial a}{\partial t}+u\frac{\partial a}{\partial x}\right)+a\frac{\partial u}{\partial x}=0, \qquad \frac{\partial u}{\partial t}+u\frac{\partial u}{\partial x}+\frac{2a}{\gamma-1}\frac{\partial a}{\partial x}=0. \tag{9.2}$$

Sommando e sottraendo le due equazioni si ottengono le due forme notevoli

$$\left[\frac{\partial}{\partial t}+(u+a)\frac{\partial}{\partial x}\right]\left(u+\frac{2a}{\gamma-1}\right)=0, \qquad \left[\frac{\partial}{\partial t}+(u-a)\frac{\partial}{\partial x}\right]\left(u-\frac{2a}{\gamma-1}\right)=0. \tag{9.3}$$

9.3 Caratteristiche e invarianti di Riemann

La prima delle (9.3) afferma che la grandezza $J^+ = u + 2a/(\gamma-1)$ è costante lungo una curva del piano x–t. Infatti lungo una curva $x(t)$ si ha $\mathrm{d}J^+/\mathrm{d}t = \partial J^+/\partial t + (\mathrm{d}x/\mathrm{d}t)\,\partial J^+/\partial x$; confrontando con la (9.3), $\mathrm{d}J^+/\mathrm{d}t = 0$ sulla curva di pendenza $\mathrm{d}x/\mathrm{d}t = u + a$. Allo stesso modo la seconda equazione dice che $J^- = u - 2a/(\gamma-1)$ è costante lungo le curve di pendenza $\mathrm{d}x/\mathrm{d}t = u - a$.

Le proprietà fondamentali

1. Le curve di pendenza $\mathrm{d}x/\mathrm{d}t = u \pm a$ sono le **caratteristiche** del sistema (9.2): le famiglie C^+ e C^-.
2. Le grandezze $J^\pm = u \pm 2a/(\gamma-1)$ sono costanti lungo le rispettive caratteristiche: sono gli **invarianti di Riemann**.
3. Poiché u e a variano con x e t, le caratteristiche sono in generale curve, non rette come nel caso lineare (dove $u \simeq 0$, $a = a_0$ e le caratteristiche sono le rette $x \pm a_0 t =$ cost del capitolo precedente).

9.4 Il metodo delle caratteristiche

Su questa proprietà si fonda il **metodo delle caratteristiche** (Fig. 9.1). Noti i dati iniziali a $t = 0$, si vogliono le condizioni in un punto C di un istante successivo. Per C passano due caratteristiche, una di pendenza $u + a$ e una di pendenza $u - a$, che incontrano l'asse x nei punti A e B. Poiché $J^+_C = J^+_A$ e $J^-_C = J^-_B$, cioè

$$u_C + \frac{2a_C}{\gamma-1} = u_A + \frac{2a_A}{\gamma-1}, \qquad u_C - \frac{2a_C}{\gamma-1} = u_B - \frac{2a_B}{\gamma-1},$$

sommando e sottraendo si ottiene subito

$$u_C = \tfrac{1}{2}(u_A+u_B) + \frac{1}{\gamma-1}(a_A-a_B), \qquad a_C = \frac{\gamma-1}{4}(u_A-u_B) + \tfrac{1}{2}(a_A+a_B). \tag{9.4}$$

La forma esatta delle caratteristiche non si conosce finché u e a non sono noti nella regione sotto C, ma se A e B sono vicini le due caratteristiche si possono approssimare con rette di pendenze $u_A + a_A$ e $u_B - a_B$, e C è la loro intersezione. Scegliendo sull'asse x una fila di punti vicini, si costruisce passo dopo passo una rete di caratteristiche con le pendenze locali. Il metodo è elegante e fisicamente trasparente, ma poco adatto al calcolo automatico: le posizioni

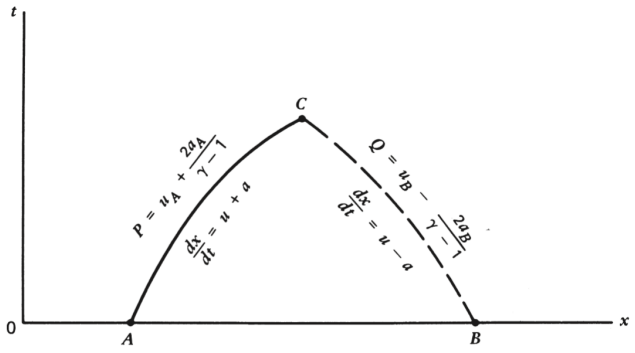


Figura 9.1: Il metodo delle caratteristiche.

dei nodi della rete non sono note in anticipo e dipendono dalla soluzione stessa. Sono stati quindi sviluppati metodi alle differenze finite che lavorano su una griglia rettangolare prefissata; quello che usiamo, proposto da Courant, Isaacson e Rees [22], è semplice da programmare e conserva un'interpretazione fisica immediata.

9.5 Lo schema di Courant, Isaacson e Rees

Si adotta la stessa griglia del capitolo precedente. Supponiamo noti u e a in tutti i nodi del livello t_j e calcoliamo le condizioni nel nodo C $= (i, j+1)$ del livello successivo (Fig. 9.2). Le caratteristiche che passano per C incontrano la riga precedente nei punti A e B; con le (9.4) si ottiene la soluzione in C da quelle in A e B, che a loro volta si ricavano per interpolazione dai nodi della riga j.

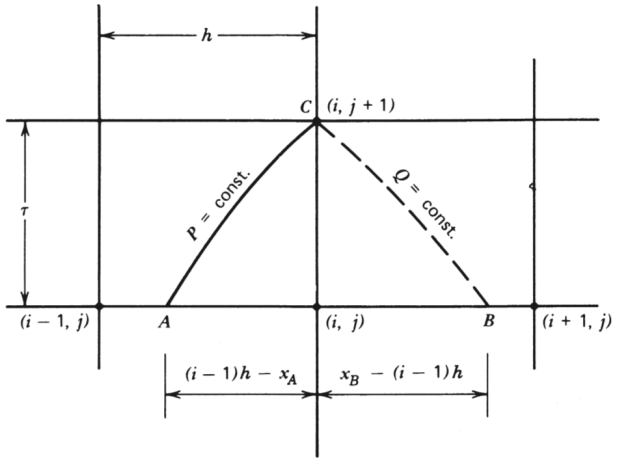


Figura 9.2: Lo schema di Courant, Isaacson e Rees.

Le formule dello schema

1. Si approssimano le caratteristiche per C con rette di pendenze $u_{i,j} + a_{i,j}$ e $u_{i,j} - a_{i,j}$; le loro intersezioni con la riga j sono

$$x_A = x_i - \tau\,(u_{i,j} + a_{i,j}), \qquad x_B = x_i - \tau\,(u_{i,j} - a_{i,j}).$$

Per un flusso subsonico $u + a > 0$ e $u - a < 0$: A sta a sinistra di x_i, B a destra.

2. Si interpola linearmente fra i nodi adiacenti, con $r = \tau/h$:

$$\begin{aligned} u_A &= u_{i,j} + r(u_{i,j} + a_{i,j})(u_{i-1,j} - u_{i,j}), \quad a_A = a_{i,j} + r(u_{i,j} + a_{i,j})(a_{i-1,j} - a_{i,j}), \\ u_B &= u_{i,j} - r(u_{i,j} - a_{i,j})(u_{i+1,j} - u_{i,j}), \quad a_B = a_{i,j} - r(u_{i,j} - a_{i,j})(a_{i+1,j} - a_{i,j}). \end{aligned} \tag{9.5}$$

3. Si calcolano $u_{i,j+1} = u_C$ e $a_{i,j+1} = a_C$ con le (9.4).

Lo schema è accurato al primo ordine in h. Per la stabilità il dominio di dipendenza numerico di ogni nodo non deve essere più piccolo di quello fisico, cioè i punti A e B devono cadere fra i nodi adiacenti:

$$\frac{\tau}{h}|u + a| \le 1 \qquad \text{e} \qquad \frac{\tau}{h}|u - a| \le 1 \tag{9.6}$$

ovunque nel dominio di calcolo. Hartree ha proposto una variante del secondo ordine in cui le pendenze delle caratteristiche sono le medie fra i valori in C e in A (o in B); la maggiore accuratezza si paga con un calcolo iterativo dei punti A e B.

Condizioni agli estremi del tubo

Estremo chiuso ($i = 1$). Per un nodo di contorno passa una sola caratteristica che viene dall'interno, la C^- uscita da B, lungo cui J^- è costante. Con la condizione $u = 0$:

$$u_{1,j+1} = 0, \qquad a_{1,j+1} = a_B - \frac{\gamma - 1}{2} u_B, \tag{9.7}$$

con u_B e a_B dalle (9.5) per $i = 1$.

Estremo aperto ($i = m$). La densità, e quindi la velocità del suono, sono quelle esterne; per il nodo passa la sola caratteristica C^+ uscita da A, lungo cui J^+ è costante:

$$a_{m,j+1} = a_0, \qquad u_{m,j+1} = u_A + \frac{2}{\gamma - 1}(a_A - a_0). \tag{9.8}$$

In ciascun estremo si impone una sola condizione fisica (la velocità o la pressione) e l'altra grandezza è fornita dalla caratteristica che arriva dall'interno: per un sistema iperbolico il numero di condizioni al contorno da assegnare è uguale al numero di caratteristiche *entranti* nel dominio.

9.6 Programma 2.9: nascita di un'onda d'urto

Si considera un tubo lungo 2 m chiuso a sinistra e aperto a destra, diviso in intervalli di 0,02 m (101 nodi). Sia a_0 la velocità del suono nel gas indisturbato. A $t = 0$ viene prodotta nella metà sinistra del tubo un'ondina di lunghezza d'onda 1 m che si propaga verso destra; per esaltare gli effetti dell'ampiezza finita e rendere evidente la deformazione, l'ampiezza iniziale è $a_0/2$ e

il profilo è formato da tratti rettilinei:

$$u_{i,1} = \begin{cases} \dfrac{a_0}{2}\dfrac{i-1}{12} & 1 \le i \le 13 \\ \dfrac{a_0}{2}\dfrac{26-i}{13} & 13 < i \le 39 \\ \dfrac{a_0}{2}\dfrac{i-51}{12} & 39 < i \le 51 \\ 0 & \text{altrove} \end{cases} \tag{9.9}$$

La condizione iniziale per a discende da quella per u:

$$a = a_0 \pm \frac{\gamma-1}{2}\,u, \tag{9.10}$$

con il segno positivo per un'onda che viaggia verso le x positive. È la relazione dell'**onda semplice**: davanti all'onda il gas è indisturbato, e lungo le caratteristiche C^-, che attraversano l'onda provenendo dalla regione indisturbata, $J^- = u - 2a/(\gamma-1)$ ha ovunque il valore $-2a_0/(\gamma-1)$; da qui la (9.10).

Dati del Programma 2.9

$a_0 = 340$ m/s, $\gamma = 1{,}4$ (aria), $h = 0{,}02$ m, $m = 101$; passo temporale di tentativo $\tau = 0{,}5\,h/a_0 = 2{,}94\cdot 10^{-5}$ s; 55 passi. Prima di calcolare ogni nodo il programma verifica la (9.6) e si arresta se è violata.

Il passo temporale dipende, attraverso la (9.6), dalle incognite u e a. Il valore di tentativo si rivela adeguato: il massimo di $(|u|+a)\tau/h$ nel corso del calcolo è 0,80.

L'istante di formazione dell'urto

Finché non si forma l'urto, la soluzione esatta dell'onda semplice è nota: lungo ciascuna caratteristica C^+ sono costanti sia J^+ sia J^-, quindi sono costanti u e a, e le C^+ sono *rette*, di pendenza

$$\frac{\mathrm{d}x}{\mathrm{d}t} = u + a = a_0 + \frac{\gamma+1}{2}\,u.$$

Ogni valore $u_0(\xi)$ del profilo iniziale viaggia senza cambiare lungo la retta $x = \xi + \big(a_0 + \frac{\gamma+1}{2}u_0(\xi)\big)t$. Dove $\mathrm{d}u_0/\mathrm{d}\xi < 0$ (compressione) le rette convergono; due rette vicine, uscite da ξ e da $\xi + \mathrm{d}\xi$, si incontrano all'istante

$$t_u = -\frac{1}{\dfrac{\gamma+1}{2}\dfrac{\mathrm{d}u_0}{\mathrm{d}\xi}}, \tag{9.11}$$

e l'urto nasce dove la pendenza negativa è massima. Nel tratto centrale dell'ondina (9.9) la pendenza è uniforme, $\mathrm{d}u_0/\mathrm{d}\xi = -(a_0/2)/(13h) = -653{,}8\ \mathrm{s}^{-1}$, e quindi

$$t_u = \frac{1}{1{,}2\cdot 653{,}8} = 1{,}27\cdot 10^{-3}\ \mathrm{s} = 43{,}3\,\tau.$$

Prima di t_u la pendenza massima cresce come $|\mathrm{d}u_0/\mathrm{d}\xi|/(1 - t/t_u)$: diverge a $t = t_u$.

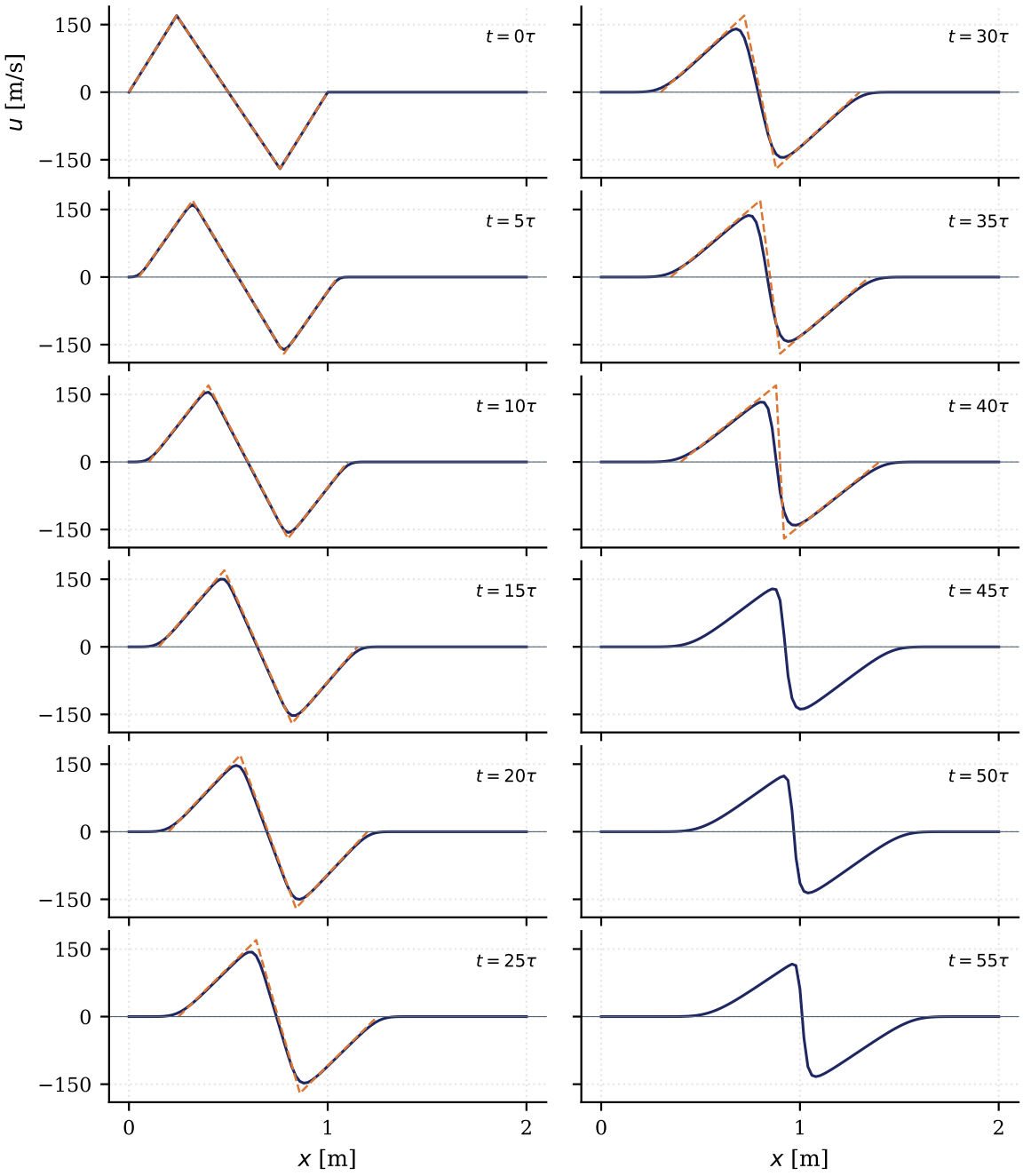


Figura 9.3: Programma 2.9: propagazione di un'onda di ampiezza finita, un fotogramma ogni cinque passi. Linea continua: schema di Courant, Isaacson e Rees; tratteggio arancione: soluzione esatta dell'onda semplice, tracciata finché non si forma l'urto ($t < 43{,}3\,\tau$).

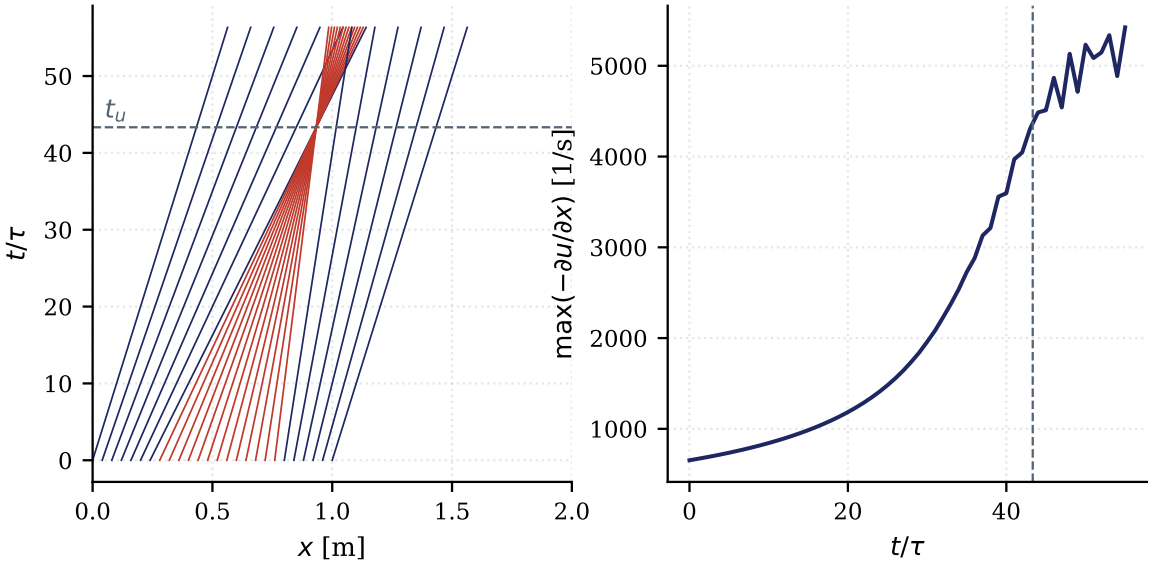


Figura 9.4: A sinistra: caratteristiche C^+ dell'onda semplice; quelle della zona di compressione (rosso) convergono e si incontrano all'istante t_u. A destra: pendenza massima calcolata del profilo di velocità.

Risultati: che cosa mostrano i fotogrammi

- Nella parte centrale dell'onda $\partial u/\partial x < 0$: ogni particella di gas si muove verso destra più velocemente di quella che la precede e la raggiunge. È una *regione di compressione*; i fotogrammi mostrano che si restringe nel tempo e il profilo si irripidisce (Fig. 9.3). Nella parte anteriore e in quella posteriore, dove $\partial u/\partial x > 0$, le particelle che precedono sono più veloci: sono regioni di *espansione*, che si allargano mentre il profilo si appiattisce.
- Il profilo perde la sua antisimmetria rispetto al nodo: la parte sinistra, con $u > 0$, si accorcia, quella destra, con $u < 0$, si allunga, mentre la lunghezza totale dell'onda resta costante. L'ampiezza a sinistra del nodo diminuisce più rapidamente che a destra.
- Verso $t = 40\tau$ il profilo nella regione di compressione è quasi verticale: una variazione molto forte di u in una distanza molto breve, cioè un'onda d'urto. La teoria prevede la formazione dell'urto a $43{,}3\,\tau$.
- La pendenza massima calcolata segue la previsione teorica finché la griglia può rappresentarla, poi si stabilizza attorno a 5400 s^{-1} perché lo schema, del primo ordine, è dissipativo e "spalma" il fronte su due o tre intervalli:

passo	0	10	20	30	35	40	50
max$(-\partial u/\partial x)$ calcolata [s^{-1}]	654	844	1185	1959	2728	3596	5232
$654/(1 - t/t_u)$ teorica [s^{-1}]	654	850	1214	2125	3400	8500	∞

Oltre l'istante dell'urto

Prima che il fronte diventi verticale il gradiente di velocità è così grande che gli effetti della viscosità e della conduzione del calore non sono più trascurabili, e le equazioni (9.3) perdono validità [20]. Anche se numericamente è possibile proseguire, il calcolo va arrestato prima che una parte dell'onda diventi troppo ripida: le equazioni isentropiche non contengono il salto di entropia attraverso l'urto, e uno schema scritto in forma non conservativa, come questo, propaga l'urto con una velocità sbagliata. Per seguire gli urti servono le equazioni in *forma conservativa* e schemi che ne catturino la velocità corretta (condizioni di Rankine–Hugoniot), come quelli del prossimo capitolo.

Il bang sonico e la "N-wave"

La stessa non linearità agisce sull'onda di pressione generata da un velivolo supersonico: lungo i chilometri di propagazione verso il suolo, le compressioni raggiungono le espansioni e il segnale si trasforma in un'onda a forma di N, con due urti, all'inizio e alla fine, separati da un'espansione lineare. Il doppio "bang" udito al suolo è il passaggio dei due urti. Il progetto dei velivoli a "bang ridotto" cerca di modellare la distribuzione delle aree in modo che il segnale non abbia il tempo di irripidirsi in urti prima di arrivare a terra.

prog2_9_onda_finita.py

```
"""
prog2_9_onda_finita.py -- PROGRAMMA 2.9
Propagazione di un'onda di ampiezza finita in un tubo di 2 m, chiuso a
sinistra e aperto a destra: formazione di un'onda d'urto.
Equazioni per u (velocita') e a (velocita' del suono) in forma caratteristica:
   [d/dt + (u+a) d/dx] (u + 2a/(gamma-1)) = 0,
   [d/dt + (u-a) d/dx] (u - 2a/(gamma-1)) = 0,
risolte con lo schema di Courant, Isaacson e Rees (caratteristiche rettilinee
all'indietro + interpolazione lineare sulla riga di tempo precedente).
Confronto con la soluzione esatta dell'onda semplice prima dell'urto.
"""
import numpy as np
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, ROSSO, GRIGIO, VERDE

A0, H, GAMMA = 340.0, 0.02, 1.4
M = 101                                  # nodi fra x = 0 e x = 2 m
TAU = 0.5 * H / A0
R = TAU / H                               # rapporto tau/h
K = (GAMMA - 1.0) / 2.0                   # COEFF del programma originale
AMP = A0 / 2.0


def u_iniziale():
    i = np.arange(1, M + 1)               # numerazione del testo (1..M)
    u = np.zeros(M)
    u[(i >= 1) & (i <= 13)] = AMP * (i[(i >= 1) & (i <= 13)] - 1) / 12
    s = (i > 13) & (i <= 39); u[s] = AMP * (26 - i[s]) / 13
    s = (i > 39) & (i <= 51); u[s] = AMP * (i[s] - 51) / 12
    return u


def cir(n_passi, u0=None, verso=+1):
    """Schema di Courant-Isaacson-Rees. verso = +1: onda verso destra
    (a = a0 + K u), verso = -1: onda verso sinistra (a = a0 - K u)."""
    u = u_iniziale() if u0 is None else u0.copy()
```

```
    a = A0 + verso * K * u
    U = [u.copy()]; Asuono = [a.copy()]
    for n in range(n_passi):
        upa, uma = R * (u + a), R * (u - a)
        if np.any(np.abs(upa) > 1) or np.any(np.abs(uma) > 1):
            print(f"condizione di stabilita' violata al passo {n}")
            break
        un, an = np.empty(M), np.empty(M)
        # nodi interni: punti A (a sinistra) e B (a destra) sulla riga precedente
        uA = u[1:-1] + upa[1:-1] * (u[:-2] - u[1:-1])
        aA = a[1:-1] + upa[1:-1] * (a[:-2] - a[1:-1])
        uB = u[1:-1] - uma[1:-1] * (u[2:] - u[1:-1])
        aB = a[1:-1] - uma[1:-1] * (a[2:] - a[1:-1])
        un[1:-1] = 0.5 * ((uA + uB) + (aA - aB) / K)
        an[1:-1] = 0.5 * (K * (uA - uB) + (aA + aB))
        # estremo chiuso (x = 0): u = 0 e invariante J- dal punto B
        uB0 = u[0] - uma[0] * (u[1] - u[0]); aB0 = a[0] - uma[0] * (a[1] - a[0])
        un[0] = 0.0
        an[0] = aB0 - K * uB0
        # estremo aperto (x = L): a = a0 e invariante J+ dal punto A
        uAm = u[-1] + upa[-1] * (u[-2] - u[-1]); aAm = a[-1] + upa[-1] * (a[-2] -
    a[-1])
        an[-1] = A0
        un[-1] = uAm + (aAm - A0) / K
        u, a = un, an
        U.append(u.copy()); Asuono.append(a.copy())
    return np.array(U), np.array(Asuono)


def onda_semplice(x0, u0, t):
    """Soluzione esatta prima dell'urto: ogni valore u0(xi) viaggia lungo la
    caratteristica rettilinea x = xi + (a0 + (gamma+1)/2 u0) t."""
    return x0 + (A0 + 0.5 * (GAMMA + 1) * u0) * t, u0


if __name__ == "__main__":
    x = H * np.arange(M)
    U, Asu = cir(55)
    print(f"tau = {TAU:.4e} s;  max Co = max(|u|+a) tau/h =
    {np.max((np.abs(U)+Asu))*R:.3f}")
    dudx0 = -AMP / (13 * H)
    t_urto = 1.0 / (0.5 * (GAMMA + 1) * abs(dudx0))
    print(f"pendenza iniziale nella zona di compressione = {dudx0:.1f} 1/s")
    print(f"istante di formazione dell'urto (teoria)     = {t_urto:.4e} s =
    {t_urto/TAU:.1f} tau")

    pend = np.max(-np.diff(U, axis=1) / H, axis=1)
    print("\n  passo   max(-du/dx) [1/s]")
    for n in (0, 10, 20, 30, 35, 40, 45, 50, 55):
        print(f"   {n:3d}     {pend[n]:9.1f}")

    # fotogrammi, con la soluzione esatta dell'onda semplice prima dell'urto
    u0 = u_iniziale()
    fig, axs = plt.subplots(6, 2, figsize=(6.6, 8.0), sharex=True, sharey=True)
    for k, ax in enumerate(axs.T.flat):
        n = 5 * k
        ax.plot(x, U[n], color=BLU, lw=1.1)
        if n * TAU < t_urto:
            xe, ue = onda_semplice(x[:51], u0[:51], n * TAU)
            ax.plot(xe, ue, "--", color=ARANCIO, lw=0.9)
```

```
        ax.axhline(0, color=GRIGIO, lw=0.4)
        ax.set_ylim(-190, 190); ax.set_yticks([-150, 0, 150]); ax.set_xticks([0, 1, 2])
        ax.text(0.98, 0.78, fr"$t={n}\tau$", transform=ax.transAxes, ha="right", fontsize=7)
    for ax in axs[-1]:
        ax.set_xlabel("$x$ [m]")
    axs[0, 0].set_ylabel("$u$ [m/s]")
    fig.subplots_adjust(hspace=0.08, wspace=0.08)
    salva(fig, "prog2_9_serie")

    # diagramma delle caratteristiche C+ (onda semplice) e pendenza massima
    fig, ax = plt.subplots(1, 2, figsize=(6.6, 3.0))
    tt = np.linspace(0, 1.3 * t_urto, 2)
    for i in range(0, 51, 2):
        xi, _ = onda_semplice(x[i], u0[i], tt)
        col = ROSSO if 13 <= i <= 39 else BLU
        ax[0].plot(xi, tt / TAU, color=col, lw=0.6)
    ax[0].axhline(t_urto / TAU, color=GRIGIO, ls="--", lw=0.8)
    ax[0].text(0.05, t_urto / TAU + 1, r"$t_u$", color=GRIGIO)
    ax[0].set_xlabel("$x$ [m]"); ax[0].set_ylabel(r"$t/\tau$"); ax[0].set_xlim(0, 2)
    ax[1].plot(np.arange(len(pend)), pend, color=BLU)
    ax[1].axvline(t_urto / TAU, color=GRIGIO, ls="--", lw=0.8)
    ax[1].set_xlabel(r"$t/\tau$"); ax[1].set_ylabel(r"$\max(-\partial u/\partial x)$ [1/s]")
    salva(fig, "prog2_9_caratteristiche")
```

Listato 9.1: Programma 2.9 – onda di ampiezza finita: schema di Courant, Isaacson e Rees (`prog2_9_onda_finita.py`).

Capitolo 10

Convezione non lineare: gli schemi di MacCormack e di Beam–Warming

10.1 L'equazione di convezione e la forma conservativa

L'esempio più semplice di equazione iperbolica è l'equazione di **convezione lineare**

$$\frac{\partial u}{\partial t} + c\frac{\partial u}{\partial x} = 0, \tag{10.1}$$

in cui una grandezza $u(x,t)$ è trasportata senza deformarsi alla velocità costante c: la soluzione è $u = u_0(x - ct)$, costante lungo le caratteristiche $\mathrm{d}x/\mathrm{d}t = c$. Il suo modello non lineare è l'equazione di **convezione non lineare** (o di Burgers non viscosa)

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = 0, \tag{10.2}$$

in cui ogni valore di u è trasportato con velocità uguale a se stesso. È la caricatura più semplice delle onde del capitolo precedente: i valori grandi raggiungono quelli piccoli, i profili di compressione si irripidiscono e si formano discontinuità. Conviene scriverla in **forma conservativa**,

$$\frac{\partial u}{\partial t} + \frac{\partial F}{\partial x} = 0, \qquad F = \frac{u^2}{2}, \tag{10.3}$$

che esprime il bilancio della grandezza u fra un intervallo e i suoi estremi: $\frac{\mathrm{d}}{\mathrm{d}t}\int_{x_1}^{x_2} u\,\mathrm{d}x = F(x_1) - F(x_2)$.

La velocità di un urto: la condizione di Rankine–Hugoniot

Sia u discontinua in un punto $x_s(t)$ che si muove con velocità $s = \mathrm{d}x_s/\mathrm{d}t$, con valori u_s a sinistra e u_d a destra. Si applica il bilancio integrale a un intervallo $[x_1, x_2]$ che contiene la discontinuità:

$$\frac{\mathrm{d}}{\mathrm{d}t}\left[\int_{x_1}^{x_s} u\,\mathrm{d}x + \int_{x_s}^{x_2} u\,\mathrm{d}x\right] = F(x_1) - F(x_2).$$

Facendo tendere x_1 e x_2 a x_s restano solo i termini dovuti allo spostamento degli estremi interni: $s\,u_s - s\,u_d = F(u_s) - F(u_d)$, cioè

$$s = \frac{F(u_s) - F(u_d)}{u_s - u_d} = \frac{u_s + u_d}{2} \qquad \text{per } F = \frac{u^2}{2}. \tag{10.4}$$

L'urto viaggia con la media delle velocità ai due lati. La forma conservativa è essenziale: le forme non conservative, equivalenti per le soluzioni regolari, darebbero velocità dell'urto diverse.

10.2 Lo schema esplicito di MacCormack

Lo schema di MacCormack [23] è un metodo a due livelli (predittore–correttore), esplicito e del secondo ordine, nato per le equazioni iperboliche non lineari e diventato uno dei cavalli di battaglia dell'aerodinamica computazionale degli anni Settanta e Ottanta. Applicato all'equazione lineare (10.1), con l'indice n per il livello di tempo e i per la posizione:

$$\begin{aligned} \bar{u}_i &= u_i^n - c\frac{\Delta t}{\Delta x}\left(u_{i+1}^n - u_i^n\right) \quad \text{(predittore, differenza in avanti)},\\ \bar{\bar{u}}_i &= u_i^n - c\frac{\Delta t}{\Delta x}\left(\bar{u}_i - \bar{u}_{i-1}\right) \quad \text{(correttore, differenza all'indietro)},\\ u_i^{n+1} &= \tfrac{1}{2}\left(\bar{u}_i + \bar{\bar{u}}_i\right). \end{aligned} \tag{10.5}$$

MacCormack è Lax–Wendroff: stabilità e dispersione

Sostituendo il predittore nel correttore, con $\mathrm{Co} = c\Delta t/\Delta x$:

$$u_i^{n+1} = u_i^n - \frac{\mathrm{Co}}{2}\left(u_{i+1}^n - u_{i-1}^n\right) + \frac{\mathrm{Co}^2}{2}\left(u_{i+1}^n - 2u_i^n + u_{i-1}^n\right):$$

per l'equazione lineare lo schema coincide con quello di Lax–Wendroff. Il primo termine è la differenza centrata della derivata spaziale; il secondo è la correzione che rende lo schema del secondo ordine anche nel tempo (deriva da $u_{tt} = c^2 u_{xx}$ nello sviluppo di Taylor). Con $u_i^n = U^n e^{\mathrm{i}ikh}$ il fattore di amplificazione è

$$G = 1 - \mathrm{i}\,\mathrm{Co}\sin kh - \mathrm{Co}^2(1-\cos kh), \qquad |G|^2 = 1 - 4\mathrm{Co}^2(1-\mathrm{Co}^2)\sin^4\frac{kh}{2}. \tag{10.6}$$

Lo schema è stabile ($|G| \le 1$ per ogni k) se e solo se $\mathrm{Co} \le 1$; per $\mathrm{Co} < 1$ smorza un poco le onde corte (dissipazione) e le rallenta (dispersione): la velocità di fase numerica è inferiore a quella esatta (Fig. 10.1). Per $\mathrm{Co} = 1$ si ha $u_i^{n+1} = u_{i-1}^n$: lo schema trasporta la soluzione esattamente lungo la caratteristica, come il leapfrog del Cap. 8.

Per l'equazione non lineare (10.3) lo schema si applica ai flussi:

$$\bar{u}_i = u_i^n - \frac{\Delta t}{\Delta x}\left(F_{i+1}^n - F_i^n\right), \qquad u_i^{n+1} = \tfrac{1}{2}\left[u_i^n + \bar{u}_i - \frac{\Delta t}{\Delta x}\left(\bar{F}_i - \bar{F}_{i-1}\right)\right], \tag{10.7}$$

con $F = u^2/2$ e $\bar{F} = \bar{u}^2/2$. Spesso si alternano, a passi successivi, l'ordine avanti–indietro e quello indietro–avanti, per eliminare la tendenza delle differenze non centrate a "preferire" una direzione. La condizione di stabilità è

$$\mathrm{Co} = \left|u_{\max}\frac{\Delta t}{\Delta x}\right| \le 1; \tag{10.8}$$

nei problemi non lineari, però, la velocità caratteristica varia nel dominio e nel tempo, e in genere conviene rinunciare a un po' di accuratezza per la stabilità e usare Co un po' minore di uno. Gottlieb e Turkel ne hanno ricavato una generalizzazione del quarto ordine nello spazio (schema due–quattro).

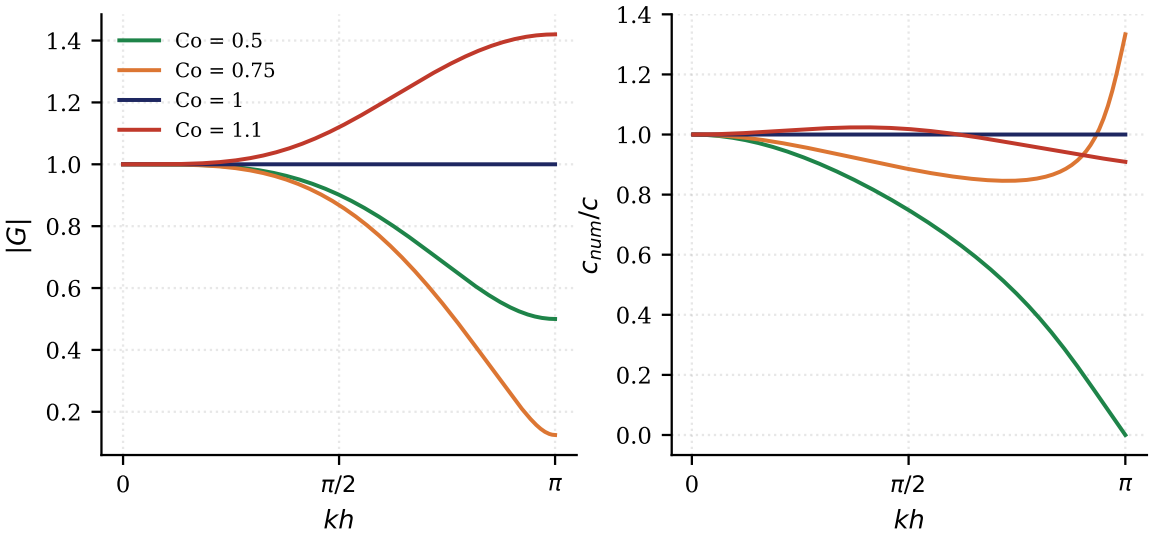


Figura 10.1: Analisi di von Neumann dello schema di MacCormack per l'equazione lineare: modulo del fattore di amplificazione e rapporto fra velocità di fase numerica ed esatta.

10.3 Programma 2.10: un salto che si propaga

Si risolve l'equazione (10.2) con le condizioni

$$u(x,0) = \begin{cases} 10 & 0 \le x < 20 \\ 0 & 20 \le x \le 40 \end{cases}, \qquad u(0,t) = 10, \qquad u(40,t) = 0. \tag{10.9}$$

La soluzione esatta è un urto che, per la (10.4), si muove con velocità $s = (10+0)/2 = 5$: si trova in $x_s = 20 + 5t$. Il Programma 2.10 (Listato 10.1) usa 200 nodi e lo schema (10.7) senza alternanza (per discontinuità che si muovono verso destra si raccomanda il predittore in avanti [5]); il passo temporale si ricalcola a ogni passo con la (10.8) a Co fissato.

Risultati

Co	t [s]	urto calcolato	urto esatto $20+5t$	$u_{\max}$
1,00	1,01	25,13	25,03	10,00
1,00	2,01	30,15	30,05	10,00
1,00	3,00	34,97	35,00	10,03
0,75	1,00	24,92	25,02	10,42
0,75	2,01	29,95	30,04	10,42
0,75	3,00	34,97	35,00	10,54

- Con Co $= 1$ il salto avanza senza oscillazioni e senza allargarsi: nella regione $u = 10$ lo schema si muove lungo le caratteristiche. La posizione dell'urto (dove $u = 5$) coincide con quella esatta entro mezzo intervallo di griglia ($\Delta x = 0{,}20$).
- Con Co $= 0{,}75$ la stabilità è mantenuta ma compare un'oscillazione a monte dell'urto, con un sorpasso del 4–5% ($u_{\max} = 10{,}4$–$10{,}5$): è l'errore di dispersione dello schema del secondo ordine, previsto dall'analisi di von Neumann. Le onde corte che compongono lo spigolo viaggiano più lentamente e restano indietro.
- Con Co $= 1{,}5$, oltre il limite di stabilità, la soluzione si deteriora in pochi decimi di secondo: $u_{\max} = 27$ a $t = 0{,}16$ s e la soluzione diverge subito dopo.

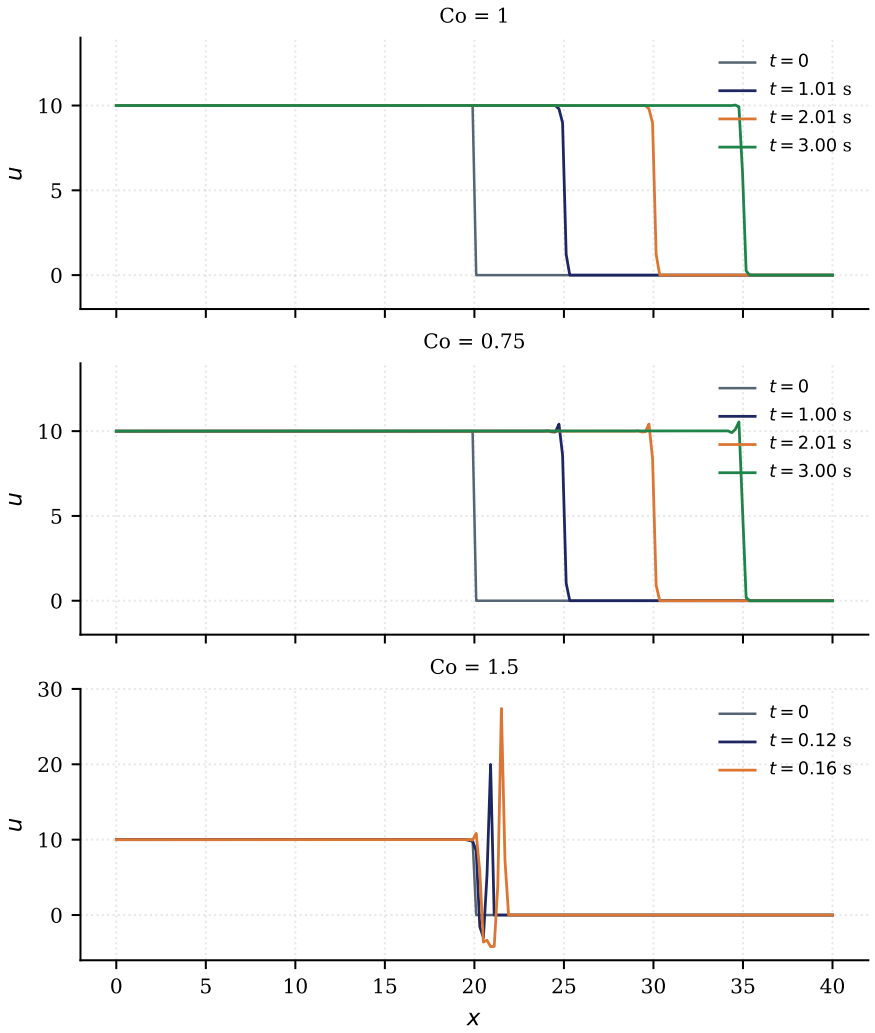


Figura 10.2: Programma 2.10: schema di MacCormack per la convezione non lineare con Co $= 1$, Co $= 0{,}75$ e Co $= 1{,}5$.

Quando l'instabilità non si vede

Con Co $= 1{,}1$ e $1{,}2$, pur oltre il limite lineare, la soluzione resta limitata fino a $t = 4$ s ($u_{\max} = 10{,}1$ e $13{,}2$). Il motivo è fisico: a monte dell'urto u è costante e lo schema ha a disposizione solo perturbazioni piccole, che vengono trasportate dalle caratteristiche *dentro* l'urto, dove tutte le caratteristiche convergono e si annullano. Un urto è un "pozzo" di caratteristiche e in questo esempio assorbe gli errori prima che crescano troppo. Non c'è da fidarsene: in un problema con espansioni, o con condizioni al contorno che riflettono le onde, la stessa instabilità esplode. L'analisi lineare dà la condizione *necessaria*; il fatto che un caso particolare non diverga non la smentisce.

prog2_10_maccormack.py

```python
"""
prog2_10_maccormack.py -- PROGRAMMA 2.10
Equazione di convezione non lineare  u_t + (u^2/2)_x = 0  risolta con lo
schema esplicito di MacCormack (predittore in avanti, correttore all'indietro).
Condizione iniziale a gradino: u = 10 per x < 20, u = 0 per 20 <= x <= 40;
condizioni al contorno di Dirichlet u(0) = 10, u(40) = 0.
La soluzione esatta e' un urto che si muove con la velocita' di
Rankine-Hugoniot  s = (u_s + u_d)/2 = 5.
Si confrontano Co = 1, 0,75 e 1,1, con Co = u_max dt/dx.
"""
import numpy as np
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, ROSSO, GRIGIO, VERDE, VIOLA

UMAX, L, NX = 10.0, 40.0, 200


def maccormack(Co, t_fin, nx=NX, uL=UMAX, uR=0.0, istanti=()):
    """Integra fino a t_fin; restituisce x, la soluzione finale e un dizionario
    con le soluzioni agli istanti richiesti (primo passo che li supera)."""
    x = np.linspace(0.0, L, nx)
    dx = x[1] - x[0]
    u = np.where(x < 20.0, uL, uR).astype(float)
    t, salvate = 0.0, {}
    richiesti = sorted(istanti)
    while t < t_fin:
        umax = np.max(np.abs(u))
        dt = min(Co * dx / umax, t_fin - t)
        r = dt / dx
        F = 0.5 * u**2
        us = u.copy()
        us[:-1] = u[:-1] - r * (F[1:] - F[:-1])          # predittore (in avanti)
        Fs = 0.5 * us**2
        un = u.copy()
        un[1:-1] = 0.5 * (u[1:-1] + us[1:-1] - r * (Fs[1:-1] - Fs[:-2]))   # correttore
        un[0], un[-1] = uL, uR                            # Dirichlet
        u, t = un, t + dt
        while richiesti and t >= richiesti[0] - 1e-12:
            salvate[richiesti.pop(0)] = (t, u.copy())
        if not np.all(np.isfinite(u)) or np.max(np.abs(u)) > 1e3:
            break
    return x, u, t, salvate


if __name__ == "__main__":
    u0 = lambda x: np.where(x < 20.0, UMAX, 0.0)
    fig, axs = plt.subplots(3, 1, figsize=(5.6, 6.6), sharex=True)
    print("  Co    t      posizione dell'urto (u = 5)   esatta 20 + 5t   max u")
    casi = ((1.0, (1.0, 2.0, 3.0)), (0.75, (1.0, 2.0, 3.0)), (1.5, (0.10, 0.16)))
    for ax, (Co, istanti) in zip(axs, casi):
        x, u, t, sal = maccormack(Co, max(istanti), istanti=istanti)
        ax.plot(x, u0(x), color=GRIGIO, lw=1.0, label="$t=0$")
        for (tr, (ts, us)), col in zip(sorted(sal.items()), (BLU, ARANCIO, VERDE)):
            ax.plot(x, us, color=col, lw=1.1, label=f"$t={ts:.2f}$ s")
            k = np.argmin(np.abs(us - 5.0) + 100 * (x < 15))
            print(f"  {Co:4.2f}  {ts:4.2f}         {x[k]:6.2f}         {20+5*ts:6.2f}"
                  f"       {us.max():6.2f}")
        ax.set_ylim(-2, 14) if Co <= 1 else ax.set_ylim(-6, 30)
```

```
        ax.set_ylabel("$u$")
        ax.set_title(f"Co = {Co:g}", fontsize=8); ax.legend(loc="upper right",
    fontsize=7)
    axs[-1].set_xlabel("$x$")
    salva(fig, "prog2_10_maccormack")

    # Co = 1,1 e 1,2: l'instabilita' lineare non esplode perche' le perturbazioni
    # vengono trasportate dentro l'urto
    for Co in (1.1, 1.2):
        x, u, t, _ = maccormack(Co, 4.0)
        print(f"  Co = {Co}: dopo 4 s max u = {u.max():.2f}, min u = {u.min():.2f}")

    # analisi di von Neumann (equazione lineare u_t + c u_x = 0): G di Lax-Wendroff
    kh = np.linspace(0, np.pi, 300)
    fig, ax = plt.subplots(1, 2, figsize=(6.6, 2.8))
    for Co, col in ((0.5, VERDE), (0.75, ARANCIO), (1.0, BLU), (1.1, ROSSO)):
        G = 1 - 1j * Co * np.sin(kh) - Co**2 * (1 - np.cos(kh))
        ax[0].plot(kh, np.abs(G), color=col, label=f"Co = {Co:g}")
        fase = -np.angle(G)
        with np.errstate(invalid="ignore", divide="ignore"):
            ax[1].plot(kh[1:], fase[1:] / (Co * kh[1:]), color=col)
    for a in ax:
        a.set_xticks([0, np.pi / 2, np.pi]); a.set_xticklabels(["0", r"$\pi/2$",
    r"$\pi$"])
        a.set_xlabel("$kh$")
    ax[0].set_ylabel("$|G|$"); ax[0].legend(fontsize=7)
    ax[1].set_ylabel(r"$c_{num}/c$")
    salva(fig, "prog2_10_vonneumann")
```

Listato 10.1: Programma 2.10 – convezione non lineare con lo schema esplicito di MacCormack (`prog2_10_maccormack.py`).

10.4 Lo schema implicito di Beam–Warming

Un altro metodo molto usato per integrare le equazioni di Navier–Stokes è lo schema implicito di Beam e Warming [24], basato sull'avanzamento nel tempo di Crank–Nicolson (regola dei trapezi):

$$u_i^{n+1} = u_i^n - \frac{\Delta t}{2}\left[\left(\frac{\partial F}{\partial x}\right)^n + \left(\frac{\partial F}{\partial x}\right)^{n+1}\right]. \tag{10.10}$$

L'equazione alle differenze è non lineare in u^{n+1}: bisognerebbe iterare. Beam e Warming la linearizzano sviluppando il flusso attorno al livello n,

$$F^{n+1} \simeq F^n + A^n\left(u^{n+1} - u^n\right), \qquad A = \frac{\partial F}{\partial u} = u, \tag{10.11}$$

dove A è lo *jacobiano* del flusso (per un sistema di equazioni è una matrice). Sostituendo nella (10.10) e approssimando le derivate spaziali con differenze centrate del secondo ordine,

$$\frac{\partial}{\partial x}\left[u^n\left(u^{n+1} - u^n\right)\right]_i \simeq \frac{u_{i+1}^n u_{i+1}^{n+1} - u_{i-1}^n u_{i-1}^{n+1}}{2\Delta x} - \frac{u_{i+1}^n u_{i+1}^n - u_{i-1}^n u_{i-1}^n}{2\Delta x},$$

si ottiene un'equazione **tridiagonale** nelle incognite del livello $n+1$:

$$-\frac{\Delta t\, u_{i-1}^n}{4\Delta x}\, u_{i-1}^{n+1} + u_i^{n+1} + \frac{\Delta t\, u_{i+1}^n}{4\Delta x}\, u_{i+1}^{n+1} = u_i^n - \frac{\Delta t}{2\Delta x}\left(F_{i+1}^n - F_{i-1}^n\right) + \frac{\Delta t}{4\Delta x}\left[(u_{i+1}^n)^2 - (u_{i-1}^n)^2\right]. \tag{10.12}$$

Per il flusso $F = u^2/2$ i due ultimi termini si cancellano, e il secondo membro si riduce a u_i^n. L'errore di troncamento è $\mathcal{O}(\Delta t^2, \Delta x^2)$. Le equazioni scritte in tutti i nodi interni formano un sistema tridiagonale che si risolve a ogni passo con l'algoritmo di Thomas del Cap. 2.

Stabilità e dissipazione artificiale

Per l'equazione lineare lo schema è *incondizionatamente stabile*: $|G| = 1$ per ogni Co. Ma $|G| = 1$ significa anche che non esiste alcuno smorzamento delle onde corte, che lo schema, dispersivo, genera vicino alle discontinuità; nel problema non lineare l'energia si accumula nelle armoniche più alte, che la griglia non risolve, e la soluzione diverge. Per mantenere la stabilità si aggiunge al secondo membro un termine di **dissipazione artificiale** del quarto ordine [5]:

$$D_i = -\frac{\mu_e}{8}\left(u_{i+2}^n - 4u_{i+1}^n + 6u_i^n - 4u_{i-1}^n + u_{i-2}^n\right), \qquad 0 < \mu_e \leq 1. \tag{10.13}$$

La quantità fra parentesi è la differenza quarta, cioè $(\Delta x)^4\,\partial^4 u/\partial x^4$: il termine è piccolo dove la soluzione è regolare e non riduce l'ordine formale dello schema del secondo ordine. Sulle onde corte agisce invece con decisione: con $u_i = e^{iikh}$ la differenza quarta vale $(2 - 2\cos kh)^2$, e da sola la (10.13) moltiplica ogni componente per

$$G_D = 1 - \frac{\mu_e}{2}(1 - \cos kh)^2,$$

che vale 1 per le onde lunghe e $1 - 2\mu_e$ per l'onda più corta, $kh = \pi$. Perché la dissipazione non diventi essa stessa instabile ($|G_D| \leq 1$) deve essere $0 < \mu_e \leq 1$: è la (10.13). Con $\mu_e = 0{,}5$ l'onda a dente di sega viene eliminata in un solo passo.

10.5 Programma 2.11: Beam–Warming con e senza dissipazione

Il Programma 2.11 (Listato 10.2) risolve lo stesso problema (10.9) con lo schema (10.12), 101 nodi, Co $= 1$ e dissipazione $\mu_e = 0{,}8$ (in tutti i nodi in cui è definita la differenza quarta); il sistema tridiagonale si risolve con la funzione `thomas` del modulo comune. Per confronto si ripete il calcolo con $\mu_e = 0$.

Risultati

- Con $\mu_e = 0{,}8$ la soluzione è stabile e non è affetta da dissipazione eccessiva: l'urto resta ripido e si trova al posto giusto ($x = 30{,}0$ e $37{,}6$ per $t = 2{,}03$ e $3{,}60$ s, contro 30,1 e 38,0 esatti). È però degradata da forti errori di dispersione vicino alla discontinuità: un sorpasso del 19% ($u_{\max} = 11{,}9$) a monte e un sottopasso ($u_{\min} = -0{,}8$) a valle, dove la soluzione esatta è nulla.
- Con $\mu_e = 0$ la soluzione diventa instabile e gli errori crescono rapidamente (Fig. 10.3, in basso): oscillazioni di ampiezza paragonabile alla soluzione stessa ($u_{\max} = 17$) riempiono tutta la regione a monte dell'urto.

Per l'equazione di convezione non lineare non viscosa lo schema implicito di Beam–Warming non è la scelta migliore: il vantaggio dell'implicito (passi temporali grandi) è inutile quando si vuole seguire con precisione un'onda, e la mancanza di dissipazione naturale va compensata con un parametro arbitrario. Gli schemi impliciti danno il meglio

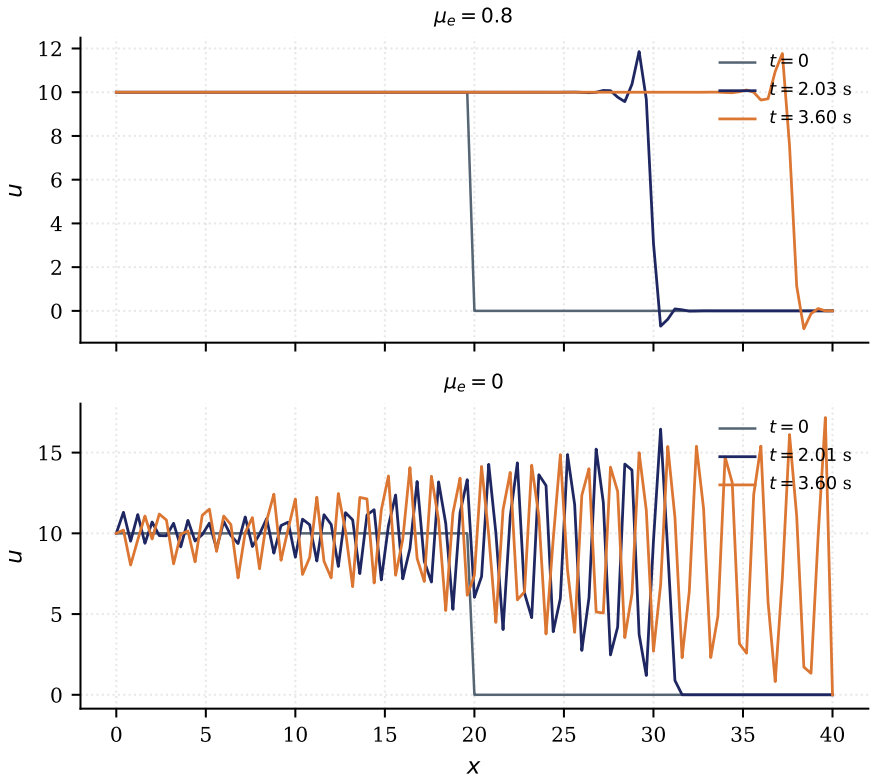


Figura 10.3: Programma 2.11: schema implicito di Beam–Warming con dissipazione artificiale $\mu_e = 0{,}8$ (in alto) e senza (in basso).

nei problemi stazionari, dove si vuole arrivare alla soluzione finale nel minor numero di passi, e in quelli con diffusione.

prog2_11_beam_warming.py

```
"""
prog2_11_beam_warming.py -- PROGRAMMA 2.11
Equazione di convezione non lineare u_t + (u^2/2)_x = 0 risolta con lo schema
implicito di Beam e Warming (Crank-Nicolson + linearizzazione del flusso):

  -r u_{i-1}^n u_{i-1}^{n+1} + u_i^{n+1} + r u_{i+1}^n u_{i+1}^{n+1} = u_i^n + D_i,
  r = dt/(4 dx),
  D_i = -(mu_e/8)(u_{i+2} - 4u_{i+1} + 6u_i - 4u_{i-1} + u_{i-2})^n
        (dissipazione artificiale del quarto ordine).
Il sistema tridiagonale si risolve a ogni passo con l'algoritmo di Thomas.
Stessi dati del Programma 2.10, 101 nodi, Co = 1; mu_e = 0,8 e mu_e = 0.
"""
import numpy as np
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, ROSSO, GRIGIO, VERDE
from metodi_numerici import thomas

UMAX, L = 10.0, 40.0


def beam_warming(Co, mu_e, t_fin, nx=101, istanti=(), uL=UMAX, uR=0.0):
    x = np.linspace(0.0, L, nx)
    dx = x[1] - x[0]
    u = np.where(x < 20.0, uL, uR).astype(float)
    t, salvate, richiesti = 0.0, {}, sorted(istanti)
    while t < t_fin - 1e-12:
```

```
        dt = min(Co * dx / np.max(np.abs(u)), t_fin - t)
        r = dt / (4.0 * dx)
        n = nx - 2                                  # incognite: nodi 1..nx-2
        a = np.zeros(n); b = np.ones(n); c = np.zeros(n)
        a[1:] = -r * u[1:-2]                        # coefficiente di u_{i-1}^{n+1}
        c[:-1] = r * u[2:-1]                        # coefficiente di u_{i+1}^{n+1}
        d = u[1:-1].copy()
        D = np.zeros(nx)
        D[2:-2] = -mu_e / 8.0 * (u[4:] - 4 * u[3:-1] + 6 * u[2:-2] - 4 * u[1:-3] +
    u[:-4])
        d += D[1:-1]
        d[0] += r * u[0] * uL                       # termini noti dei nodi di
    contorno
        d[-1] -= r * u[-1] * uR
        un = np.empty(nx)
        un[1:-1] = thomas(a, b, c, d)
        un[0], un[-1] = uL, uR
        u, t = un, t + dt
        while richiesti and t >= richiesti[0] - 1e-9:
            salvate[richiesti.pop(0)] = (t, u.copy())
        if not np.all(np.isfinite(u)) or np.max(np.abs(u)) > 1e4:
            break
    return x, u, t, salvate


if __name__ == "__main__":
    fig, axs = plt.subplots(2, 1, figsize=(5.6, 4.8), sharex=True)
    print("  mu_e    t     posizione dell'urto   esatta    max u    min u")
    for ax, mu in zip(axs, (0.8, 0.0)):
        x, u, t, sal = beam_warming(1.0, mu, 3.6, istanti=(2.0, 3.6))
        ax.plot(x, np.where(x < 20, UMAX, 0), color=GRIGIO, lw=1.0, label="$t=0$")
        for (tr, (ts, us)), col in zip(sorted(sal.items()), (BLU, ARANCIO)):
            ax.plot(x, us, color=col, lw=1.1, label=f"$t={ts:.2f}$ s")
            k = np.argmin(np.abs(us - 5.0) + 100 * (x < 15))
            print(f"  {mu:4.2f}  {ts:4.2f}       {x[k]:6.2f}
    {20+5*ts:6.2f}    "
                  f"{us.max():6.2f}   {us.min():6.2f}")
        ax.set_ylabel("$u$"); ax.legend(loc="upper right", fontsize=7)
        ax.set_title(fr"$\mu_e = {mu:g}$", fontsize=8)
    axs[1].set_xlabel("$x$")
    salva(fig, "prog2_11_beam_warming")
```

Listato 10.2: Programma 2.11 – convezione non lineare con lo schema implicito di Beam–Warming e dissipazione artificiale (`prog2_11_beam_warming.py`).

La dissipazione artificiale: un compromesso

Come la dissipazione artificiale modifichi i risultati è una questione discussa, ma nella pratica è stata usata moltissimo: i codici di Jameson per le equazioni di Eulero, che negli anni Ottanta hanno permesso il progetto aerodinamico di intere generazioni di velivoli da trasporto, usano una combinazione di dissipazione del secondo ordine (attivata solo vicino agli urti da un sensore di pressione) e del quarto ordine (ovunque, con coefficiente piccolo). Il coefficiente va scelto il più piccolo possibile compatibilmente con la stabilità, e un buon controllo è ripetere il calcolo con valori diversi e verificare che le grandezze di interesse non cambino [5, 6].

10.6 L'equazione di Burgers viscosa

Un'equazione modello più generale aggiunge alla convezione non lineare un termine di diffusione:

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = \nu\frac{\partial^2 u}{\partial x^2}, \tag{10.14}$$

dove ν è un coefficiente di trasporto (nei problemi fluidodinamici la viscosità cinematica). È l'**equazione di Burgers**, che contiene in una sola variabile i due meccanismi fondamentali delle equazioni di Navier–Stokes, il trasporto non lineare e la diffusione viscosa; serve come modello per fenomeni che vanno dagli urti forti alla dinamica spettrale della turbolenza. La convezione tende a irripidire i fronti, la diffusione a smussarli: invece di un urto discontinuo si forma uno strato di transizione di spessore proporzionale a $\nu/\Delta u$, come nella struttura interna di un'onda d'urto reale.

Per i flussi molto viscosi, a basso numero di Reynolds, conviene adimensionalizzare con una lunghezza L e con la scala viscosa: $x/L \to x$, $\nu t/L^2 \to t$, $uL/\nu \to u$. L'equazione diventa

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = \frac{\partial^2 u}{\partial x^2}. \tag{10.15}$$

Una soluzione esatta: la trasformazione di Cole–Hopf

La sostituzione $u = -2\varphi_x/\varphi$ trasforma la (10.15) nell'equazione del calore $\varphi_t = \varphi_{xx}$ [21]: l'equazione di Burgers è una delle pochissime equazioni non lineari risolubili esattamente per condizioni iniziali qualsiasi. Scegliendo la soluzione dell'equazione del calore $\varphi = e^t \cosh x - 1$ si ottiene una delle soluzioni catalogate da Benton e Platzman [25]:

$$u(x,t) = -\frac{2\sinh x}{\cosh x - e^{-t}}. \tag{10.16}$$

Per $t \to 0$ la soluzione tende a $u(x,0) = -2\sinh x/(\cosh x - 1)$, discontinua in $x = 0$ (dove va da $+\infty$ a $-\infty$); per $t \to \infty$ tende allo stato stazionario $u = -2\tanh x$, in cui convezione e diffusione si bilanciano esattamente. La pendenza nell'origine, $\partial u/\partial x = -2/(1 - e^{-t})$, mostra quanto rapidamente la diffusione smussa la discontinuità: $-11{,}0$ a $t = 0{,}2$, $-3{,}16$ a $t = 1$, $-2{,}31$ a $t = 2$. L'esercizio 14 verifica tutto ciò con il calcolo simbolico.

La soluzione esatta vale per il problema ai valori iniziali su tutta la retta; per usarla come banco di prova si sceglie il dominio $-6 \le x \le 6$, con condizioni al contorno di Dirichlet ($u = \pm 2$ in $x = \mp 6$) oppure di Neumann ($\partial u/\partial x = 0$), e la soluzione numerica sarà approssimata anche per l'effetto del troncamento del dominio.

Gli schemi per l'equazione di Burgers

MacCormack. Con $d = \nu\Delta t/(\Delta x)^2$, la diffusione si discretizza con la differenza centrata in entrambe le fasi:

$$\begin{aligned} \bar{u}_i &= u_i^n - \frac{\Delta t}{\Delta x}\left(F_{i+1}^n - F_i^n\right) + d\left(u_{i+1}^n - 2u_i^n + u_{i-1}^n\right), \\ u_i^{n+1} &= \tfrac{1}{2}\left[u_i^n + \bar{u}_i - \frac{\Delta t}{\Delta x}\left(\bar{F}_i - \bar{F}_{i-1}\right) + d\left(\bar{u}_{i+1} - 2\bar{u}_i + \bar{u}_{i-1}\right)\right]. \end{aligned} \tag{10.17}$$

Lo schema è esplicito e deve rispettare sia il limite convettivo sia quello viscoso; una

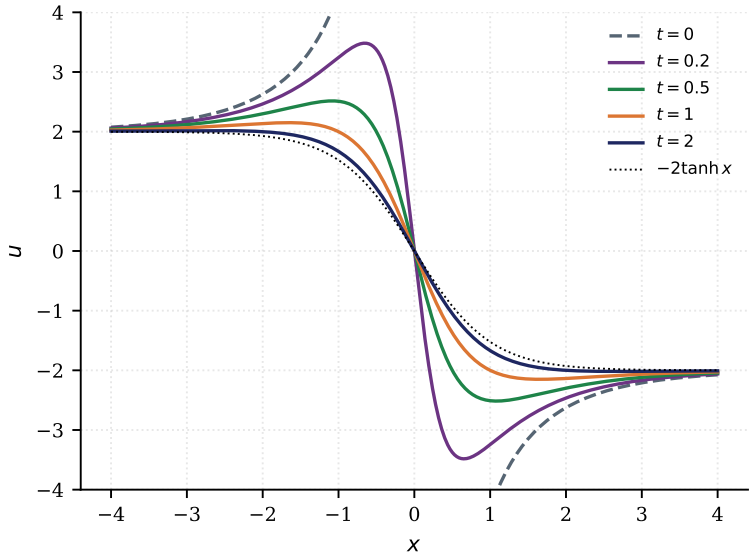


Figura 10.4: Soluzione esatta (10.16) dell'equazione di Burgers a vari istanti e stato stazionario $-2\tanh x$.

formula empirica [5] riassume i due vincoli:

$$\Delta t \leq \frac{(\Delta x)^2}{|u_{\max}|\Delta x + 2\nu}. \tag{10.18}$$

Beam–Warming. Con Crank–Nicolson anche per il termine diffusivo,

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} + \frac{F_{i+1}^n - F_{i-1}^n}{2\Delta x} + \frac{u_{i+1}^n(u_{i+1}^{n+1} - u_{i+1}^n) - u_{i-1}^n(u_{i-1}^{n+1} - u_{i-1}^n)}{4\Delta x} = \frac{\nu}{2(\Delta x)^2}\left(\delta_x^2 u_i^n + \delta_x^2 u_i^{n+1}\right), \tag{10.19}$$

con $\delta_x^2 u_i = u_{i+1} - 2u_i + u_{i-1}$: un'equazione ancora tridiagonale, del secondo ordine nel tempo e nello spazio.

I due schemi sono implementati nel programma `prog2_12_burgers.py` (Listato 10.3), rispettivamente nelle funzioni `maccormack_burgers` e `beam_warming_burgers`; gli esercizi 15 e 16 li confrontano con la soluzione esatta.

```
"""
prog2_12_burgers.py -- equazione di Burgers viscosa adimensionale
        u_t + u u_x = u_xx ,     -6 <= x <= 6
con la soluzione esatta di Benton e Platzman
        u(x, t) = -2 sinh x / (cosh x - e^{-t})
(discontinua in x = 0 per t = 0; per t -> infinito tende a -2 tanh x).

Funzioni:
  esatta(x, t)
  maccormack_burgers(...)   schema esplicito (condizioni di Neumann)
  beam_warming_burgers(...) schema implicito (condizioni di Dirichlet)
Il programma principale disegna la soluzione esatta; gli schemi sono messi
alla prova negli esercizi svolti (esercizi_svolti.py, es_13 ed es_14).
"""
import numpy as np
```

prog2_12_burgers.py

```
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, ROSSO, GRIGIO, VERDE, VIOLA
from metodi_numerici import thomas

XL, XR, NX = -6.0, 6.0, 100


def esatta(x, t):
    return -2.0 * np.sinh(x) / (np.cosh(x) - np.exp(-t))


def griglia(nx=NX):
    return np.linspace(XL, XR, nx)


def maccormack_burgers(t_fin, nx=NX, dt=None, nu=1.0, istanti=()):
    """MacCormack: predittore in avanti, correttore all'indietro, diffusione
    centrata in entrambe le fasi; du/dx = 0 agli estremi (primo ordine)."""
    x = griglia(nx); dx = x[1] - x[0]
    u = esatta(x, 0.0)
    if dt is None:      # entrambe le condizioni: convettiva e viscosa
        dt = 0.9 * min(dx / np.max(np.abs(u)), 0.5 * dx**2 / nu)
    r, d = dt / dx, nu * dt / dx**2
    t, sal, rich = 0.0, {}, sorted(istanti)
    while t < t_fin - 1e-12:
        F = 0.5 * u**2
        us = u.copy()
        us[1:-1] = u[1:-1] - r * (F[2:] - F[1:-1]) + d * (u[2:] - 2 * u[1:-1] +
    u[:-2])
        us[0], us[-1] = us[1], us[-2]
        Fs = 0.5 * us**2
        un = u.copy()
        un[1:-1] = 0.5 * (u[1:-1] + us[1:-1] - r * (Fs[1:-1] - Fs[:-2])
                          + d * (us[2:] - 2 * us[1:-1] + us[:-2]))
        un[0], un[-1] = un[1], un[-2]          # Neumann: du/dx = 0
        u, t = un, t + dt
        while rich and t >= rich[0] - 1e-9:
            sal[rich.pop(0)] = (t, u.copy())
    return x, u, dt, sal


def beam_warming_burgers(t_fin, dt, nx=NX, nu=1.0, mu_e=0.0, istanti=()):
    """Beam-Warming (Crank-Nicolson + linearizzazione del flusso) con
    condizioni di Dirichlet u(-6) = +2, u(+6) = -2; dissipazione opzionale."""
    x = griglia(nx); dx = x[1] - x[0]
    u = esatta(x, 0.0)
    uL, uR = 2.0, -2.0
    u[0], u[-1] = uL, uR
    r, d = dt / (4 * dx), nu * dt / dx**2
    t, sal, rich = 0.0, {}, sorted(istanti)
    while t < t_fin - 1e-12:
        n = nx - 2
        a = -r * u[:-2] - 0.5 * d            # coefficiente di u_{i-1}^{n+1}
        b = np.full(n, 1.0 + d)
        c = r * u[2:] - 0.5 * d              # coefficiente di u_{i+1}^{n+1}
        rhs = u[1:-1] + 0.5 * d * (u[2:] - 2 * u[1:-1] + u[:-2])
        if mu_e > 0:
            D = np.zeros(nx)
            D[2:-2] = -mu_e / 8 * (u[4:] - 4 * u[3:-1] + 6 * u[2:-2] - 4 * u[1:-3]
    + u[:-4])
```

```
            rhs += D[1:-1]
        rhs[0] -= a[0] * uL
        rhs[-1] -= c[-1] * uR
        un = u.copy()
        un[1:-1] = thomas(a, b, c, rhs)
        u, t = un, t + dt
        while rich and t >= rich[0] - 1e-9:
            sal[rich.pop(0)] = (t, u.copy())
    return x, u, sal


if __name__ == "__main__":
    x = np.linspace(-4, 4, 801)
    fig, ax = plt.subplots(figsize=(5.0, 3.6))
    with np.errstate(divide="ignore", invalid="ignore"):
        u0 = esatta(x, 0.0)
    u0[np.abs(u0) > 4.5] = np.nan
    ax.plot(x, u0, "--", color=GRIGIO, label="$t=0$")
    for t, col in ((0.2, VIOLA), (0.5, VERDE), (1.0, ARANCIO), (2.0, BLU)):
        ax.plot(x, esatta(x, t), color=col, label=f"$t={t:g}$")
    ax.plot(x, -2 * np.tanh(x), ":", color="k", lw=0.8, label=r"$-2\tanh x$")
    ax.set_ylim(-4, 4); ax.set_xlabel("$x$"); ax.set_ylabel("$u$");
    ax.legend(fontsize=7)
    salva(fig, "prog2_12_burgers_esatta")
    for t in (0.2, 0.5, 1.0, 2.0):
        print(f"t = {t}: pendenza in x = 0  du/dx = {-2/(1-np.exp(-t)):.4f}")
```

Listato 10.3: `prog2_12_burgers.py`: equazione di Burgers viscosa, soluzione esatta e schemi di MacCormack e di Beam–Warming.

Capitolo 11

Un'applicazione biomedica: il flusso pulsante in un tubo elastico

11.1 Il modello monodimensionale

Concludiamo il modulo con un sistema iperbolico non lineare completo, che riunisce gli strumenti dei capitoli precedenti: l'analisi monodimensionale del flusso in un tubo elastico, usata come modello del flusso sanguigno nelle arterie dell'uomo e degli animali. Il modello, dovuto ad Anliker, Rockwell e Ogden [26] ed esteso al flusso coronarico da Rumberger e Nerem [27] con il metodo delle caratteristiche, si fonda sulle equazioni di massa e quantità di moto mediate sulla sezione e su un'equazione che lega l'area della sezione alla pressione:

$$\frac{\partial S}{\partial t} + \frac{\partial (SV)}{\partial z} + \psi_f = 0, \tag{11.1}$$

$$\frac{\partial V}{\partial t} + V\frac{\partial V}{\partial z} + \frac{1}{\rho}\frac{\partial p}{\partial z} = f, \tag{11.2}$$

$$S = S(p, z). \tag{11.3}$$

$V(z,t)$ è la velocità media istantanea nella sezione, $p(z,t)$ la pressione locale, ρ la densità del fluido; l'effetto dell'attrito di parete è rappresentato dal termine f, forza per unità di massa nella direzione assiale z. La (11.3) è un'*equazione di stato* della parete, che esprime l'area S in funzione della pressione e della posizione. Poiché consideriamo un tubo senza diramazioni e quindi senza perdite laterali, il termine di efflusso volumetrico è $\psi_f = 0$. Assegnate l'equazione di stato e la forza d'attrito, il problema si riduce a integrare le (11.1)–(11.2) nelle incognite S e V; la pressione si ricava dalla (11.3).

Le ipotesi del modello

1. Flusso monodimensionale: si trascurano le variazioni di velocità nella sezione e si usano valori medi; il profilo di velocità reale entra solo attraverso il termine d'attrito.
2. Fluido incomprimibile, newtoniano, di proprietà costanti; parete elastica senza inerzia, la cui area dipende solo dalla pressione locale istantanea.
3. Attrito laminare calcolato con la formula di Poiseuille, assunta valida anche per il moto vario.

11.2 L'equazione di stato e la velocità delle onde

Le onde di pressione in un tubo elastico non viaggiano alla velocità del suono nel fluido (circa 1500 m/s nel sangue) ma a una velocità molto minore, determinata dalla cedevolezza della parete: dilatandosi, la parete "assorbe" parte della sovrapressione. Linearizzando le (11.1)–(11.2) attorno a uno stato di quiete, con $\partial S/\partial t = (\mathrm{d}S/\mathrm{d}p)\,\partial p/\partial t$, si ottiene un'equazione delle onde come quella del Cap. 8, con velocità di propagazione

$$c^2 = \frac{S}{\rho\,\mathrm{d}S/\mathrm{d}p}. \tag{11.4}$$

Per un tubo a parete sottile, di modulo elastico E, spessore s e diametro D, la (11.4) dà la formula di Moens–Korteweg $c^2 = Es/(\rho D)$: nell'aorta umana $c \simeq 4$–6 m/s. Il modello che adottiamo assegna la velocità delle onde come prodotto di funzioni lineari di z e di p (in cm/s, con p in mmHg) [26],

$$c(p,z) = (97 + 2{,}03\,p)(1 + 0{,}02\,z), \tag{11.5}$$

e un'equazione di stato coerente con essa,

$$S(p,z) = S_0\,e^{-kz}\exp\left[\frac{p - p_0}{\rho\,c(p,z)\,c(p_0,z)}\right], \qquad S_0 = 7{,}07\ \mathrm{cm}^2,\ k = 0{,}25\ \mathrm{cm}^{-1}: \tag{11.6}$$

l'area di riferimento si restringe lungo il tubo (rastremazione) e cresce con la pressione; derivando la (11.6) rispetto a p si ritrova, a meno della dipendenza di c da p, la (11.4). La rastremazione è forte: alla pressione di riferimento l'area passa da 7,07 cm^2 (un diametro di 3 cm) a 0,014 cm^2 all'estremità distale, e il tubo rappresenta un vaso che si ramifica in un letto vascolare ad alta resistenza.

Il moto pulsante si suppone laminare, e l'attrito si scrive con la formula di Poiseuille per i tubi circolari.

L'attrito laminare

Nel moto di Poiseuille in un tubo di raggio R la caduta di pressione per unità di lunghezza è $-\mathrm{d}p/\mathrm{d}z = 8\mu V/R^2$, con V velocità media. La forza d'attrito per unità di massa che equilibra questo gradiente è quindi

$$f_{\mathrm{lam}} = -\frac{8\mu V}{\rho R^2} = -8\pi\frac{\mu}{\rho}\frac{V}{S}, \tag{11.7}$$

avendo usato $S = \pi R^2$. Nei vasi più stretti l'attrito per unità di massa cresce come $1/S$: è il termine che, nella parte distale fortemente rastremata del nostro tubo, rappresenta la resistenza del letto vascolare. Per il moto turbolento si usa una formula empirica [26],

$$f_{\mathrm{turb}} = -0{,}1360\left(\frac{\mu}{\rho}\right)^{1/4}\frac{|V|^{7/4}}{S^{5/8}}\,\mathrm{sgn}\,V. \tag{11.8}$$

11.3 La forma conservativa e lo schema numerico

Le (11.1)–(11.2) si scrivono nella forma vettoriale conservativa

$$\frac{\partial \mathbf{U}}{\partial t} + \frac{\partial \mathbf{F}}{\partial z} = \mathbf{R}, \qquad \mathbf{U} = \begin{pmatrix} S \\ V \end{pmatrix}, \quad \mathbf{F} = \begin{pmatrix} SV \\ \frac{1}{2}V^2 \end{pmatrix}, \quad \mathbf{R} = \begin{pmatrix} -\psi_f \\ f - \frac{1}{\rho}\frac{\partial p}{\partial z} \end{pmatrix}, \tag{11.9}$$

e si integrano con lo schema di MacCormack, con un predittore alle differenze in avanti e un correttore all'indietro:

$$\begin{aligned}\bar{\mathbf{U}}_i &= \mathbf{U}_i^n - \frac{\Delta t}{\Delta z}\left(\mathbf{F}_{i+1}^n - \mathbf{F}_i^n\right) + \Delta t\,\mathbf{R}_i^n,\\ \bar{\bar{\mathbf{U}}}_i &= \mathbf{U}_i^n - \frac{\Delta t}{\Delta z}\left(\bar{\mathbf{F}}_i - \bar{\mathbf{F}}_{i-1}\right) + \Delta t\,\bar{\mathbf{R}}_i,\\ \mathbf{U}_i^{n+1} &= \tfrac{1}{2}\left(\bar{\mathbf{U}}_i + \bar{\bar{\mathbf{U}}}_i\right).\end{aligned} \tag{11.10}$$

Il gradiente di pressione in $\mathbf{R}$ si approssima nella stessa direzione delle differenze dei flussi (in avanti nel predittore, all'indietro nel correttore), e la pressione corrispondente a un'area si ottiene invertendo la (11.6) con il metodo di Newton, che converge in pochissime iterazioni partendo dalla pressione del passo precedente. Si raccomanda l'alternanza degli operatori: a passi successivi si scambiano l'ordine avanti–indietro e indietro–avanti. Sebbene l'algoritmo sia scritto in forma vettoriale, in pratica l'avanzamento nel tempo si esegue equazione per equazione.

Le velocità caratteristiche e la condizione di stabilità

Il sistema (11.9), scritto nella forma $\partial\mathbf{U}/\partial t + \mathsf{A}\,\partial\mathbf{U}/\partial z = \dots$ con $\partial p/\partial z = (\partial p/\partial S)\,\partial S/\partial z + \dots$, ha matrice

$$\mathsf{A} = \begin{pmatrix} V & S \\ \dfrac{1}{\rho}\dfrac{\partial p}{\partial S} & V \end{pmatrix},$$

i cui autovalori, le velocità caratteristiche, sono $w_\pm = V \pm \sqrt{(S/\rho)\,\partial p/\partial S} = V \pm c$. Come per le onde nei gas, le perturbazioni viaggiano con velocità $V \pm c$, e la condizione di stabilità dello schema esplicito è

$$\frac{\Delta t}{\Delta z}\max\left(|V| + c\right) \le 1. \tag{11.11}$$

Un numero di Courant basato sulla sola velocità del fluido, $\mathrm{Co}_V = V_{\max}\Delta t/\Delta z$, *non* garantisce la stabilità, perché c è qui alcune volte maggiore di V: con $c \simeq 300$–500 cm/s e $V \lesssim 100$ cm/s la (11.11) equivale a $\mathrm{Co}_V \lesssim 0{,}2$. È la ragione per cui, nei calcoli di questo tipo, si osserva empiricamente che occorre mantenere Co_V molto minore di uno.

Condizioni al contorno, condizioni iniziali, dissipazione

1. **Estremo prossimale** ($z = 0$): pressione assegnata $p_L(t)$, un impulso periodico di circa 86 battiti al minuto (periodo 0,7 s) fra 55 e 85 mmHg (Fig. 11.1); il programma usa una forma d'onda generica a quattro armoniche.
2. **Estremo distale** ($z = L = 25$ cm): pressione fissa 15 mmHg.
3. **Velocità agli estremi**: estrapolazione di ordine zero dal nodo interno adiacente.
4. **Condizioni iniziali**: poco importanti, perché dopo alcuni cicli la soluzione raggiunge uno stato quasi periodico indipendente da esse; si scelgono andamenti esponenziali fra i valori agli estremi,

$$V = V_L \exp\left(\frac{z}{L}\ln\frac{V_R}{V_L}\right), \qquad p = p_L \exp\left(\frac{z}{L}\ln\frac{p_R}{p_L}\right),$$

con $V_L = 3$ cm/s, $V_R = 0{,}1$ cm/s, $p_R = 15$ mmHg e p_L dalla condizione prossimale.

5. **Dissipazione artificiale**: i transitori iniziali possono avere fronti molto ripidi, perché il termine convettivo $V\,\partial V/\partial z$ trasferisce energia alle armoniche alte, non risolte dalla griglia. All'equazione della velocità si aggiunge la (10.13),

$$D = -\frac{\mu_e}{8}(\Delta z)^4 \frac{\partial^4 V}{\partial z^4} \simeq -\frac{\mu_e}{8}\left(V_{i+2} - 4V_{i+1} + 6V_i - 4V_{i-1} + V_{i-2}\right),$$

con il coefficiente piccolo $\mu_e = 0{,}04$, che non influisce in modo apprezzabile sull'accuratezza.

Dati del calcolo

Tubo generico lungo $L = 25$ cm; sangue: $\rho = 1{,}06$ g/cm^3, $\mu = 0{,}049$ poise; pressione di riferimento $p_0 = 80$ mmHg (circa la pressione massima nel sistema); 101 nodi, $\Delta z = 0{,}25$ cm; numero di Courant (11.11) pari a 0,8; sei cicli cardiaci (4,2 s). Unità CGS: le pressioni sono in dyn/cm^2 nelle equazioni (1 mmHg = 1333,2 dyn/cm^2) e in mmHg nei dati e nei risultati.

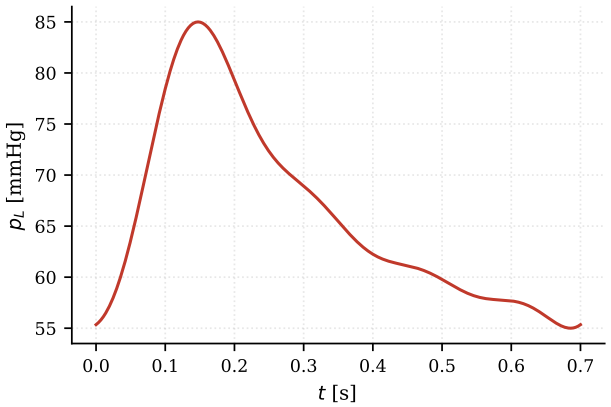


Figura 11.1: Condizione al contorno prossimale: un periodo della pressione imposta.

11.4 Risultati

Risultati: il segnale lungo il tubo

z [cm]	p [mmHg]	V [cm/s]	portata media [cm^3/s]
5,0	53,0 – 85,9	−16,7 – 37,5	1,332
12,5	49,0 – 91,4	−17,5 – 48,3	1,328
15,0	47,8 – 94,0	−12,5 – 53,5	1,328
20,0	44,8 – 97,2	15,6 – 73,3	1,327
22,0	41,9 – 93,9	33,9 – 94,3	1,327

- Dopo quattro cicli la soluzione è periodica: i valori estremi cambiano di meno di un decimo di mmHg da un ciclo all'altro.
- Nella parte prossimale la velocità cambia segno durante il ciclo: c'è *flusso inverso*, che scompare procedendo verso valle. Zone localizzate di flusso inverso sono state osservate anche in altri studi.

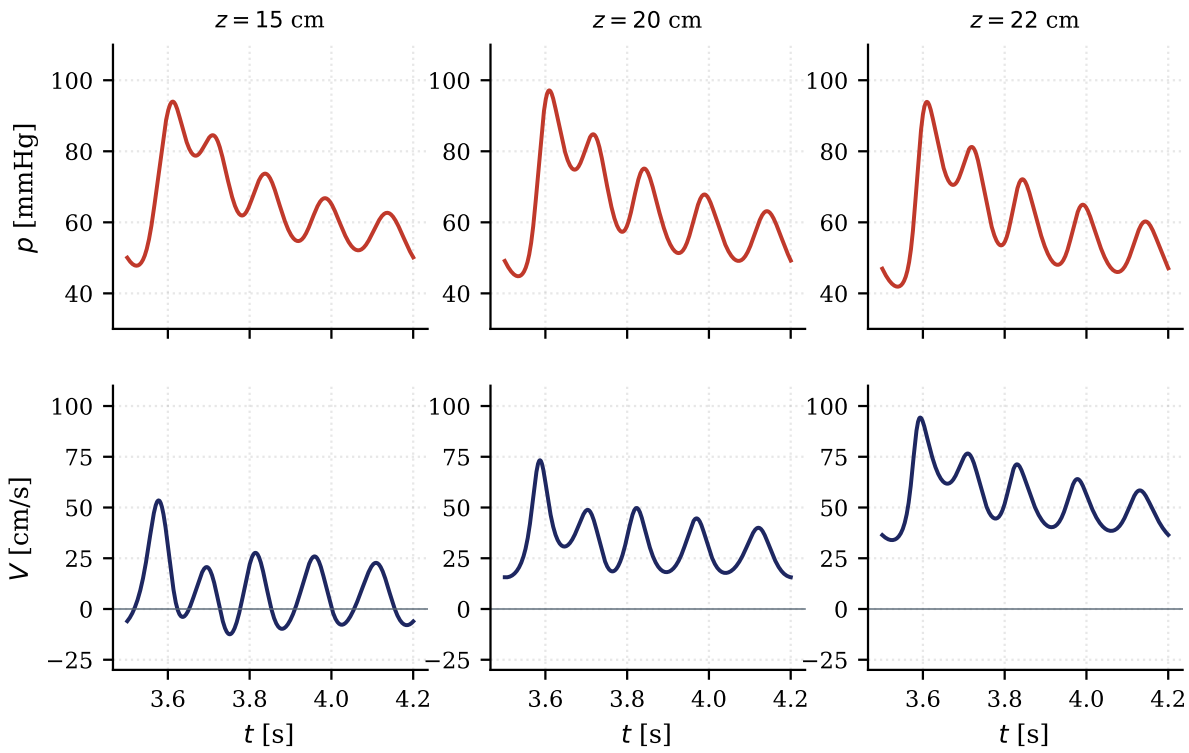


Figura 11.2: Pressione (in alto) e velocità (in basso) in tre sezioni del tubo durante l'ultimo ciclo calcolato.

- Procedendo verso valle crescono sia i massimi sia i minimi della velocità, e la velocità diventa sincrona con la pressione (Fig. 11.2). L'ampiezza dell'onda di pressione aumenta: da 33 mmHg a $z = 5$ cm a 52 mmHg a $z = 20$ cm, perché le onde riflesse dalla parte distale, rastremata e resistente, si sommano a quelle incidenti. Anche nelle arterie reali la pressione sistolica cresce allontanandosi dal cuore (amplificazione periferica).
- Entrambe le grandezze sono periodiche ma contengono armoniche più alte di quelle della forzante: il "treno" di oscillazioni dopo il picco sistolico è il segno delle riflessioni multiple fra le due estremità.
- La portata media sul ciclo è la stessa in tutte le sezioni interne entro il 4 per mille (1,33 cm^3/s): lo schema in forma conservativa rispetta il bilancio di massa. Nei nodi di estremità, invece, il prodotto SV non è rappresentativo, perché la velocità estrapolata e l'area calcolata dalla pressione imposta non sono coerenti fra loro: è il prezzo di condizioni al contorno così semplici.

Tutte queste osservazioni qualitative sono compatibili con altri modelli di simulazione arteriosa. Va però sottolineato che l'esempio riguarda un tubo elastico generico: per descrivere un'arteria reale tutte le costanti del modello vanno tarate su misure, e molti altri fattori vanno considerati. Qual è l'effetto della viscosità? È adeguato l'attrito di Poiseuille in un moto pulsante? Si deve considerare la turbolenza? L'elasticità della parete è descritta con sufficiente accuratezza? Le condizioni al contorno prossimale e distale rappresentano bene il problema fisico (nei modelli moderni l'uscita è un circuito resistivo–capacitivo, il *windkessel*)? L'esercizio 17 esplora alcune di queste domande.

prog2_13_tubo_elastico.py

```
"""
prog2_13_tubo_elastico.py -- flusso pulsante in un tubo elastico
(modello monodimensionale di un'arteria).

   S_t + (S V)_z = 0
   V_t + (V^2/2)_z = -(1/rho) p_z + f
   S = S(p, z) = S0 e^{-k z} exp[(p - p0)/(rho c(p,z) c(p0,z))]
   c(p, z) = (97 + 2,03 p)(1 + 0,02 z)  [cm/s, p in mmHg]
   f = -8 pi (mu/rho) V/S  (attrito laminare di Poiseuille)
Unita' CGS (cm, s, g; pressioni in dyn/cm^2 nelle equazioni, in mmHg nei dati).
Schema di MacCormack con alternanza delle differenze (avanti/indietro e
indietro/avanti a passi alterni) e dissipazione artificiale del quarto ordine
sull'equazione della velocita'. Condizioni al contorno: pressione prossimale
pulsante p_L(t), pressione distale fissa 15 mmHg, velocita' estrapolate.
"""
import numpy as np
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, ROSSO, GRIGIO, VERDE, VIOLA

MMHG = 1333.22                      # 1 mmHg in dyn/cm^2
RHO, MU = 1.06, 0.049               # sangue: g/cm^3, poise
L, NZ = 25.0, 101                   # lunghezza [cm] e numero di nodi (dz = 0,25 cm)
S0, K_RASTR = 7.07, 0.25            # area di riferimento [cm^2], rastremazione
    [1/cm]
P0, PR = 80.0, 15.0                 # pressione di riferimento e distale [mmHg]
PERIODO = 0.7                       # periodo cardiaco [s] (circa 86 battiti/min)


def _forma(t, periodo):
    """Forma d'onda generica (4 armoniche) della pressione prossimale."""
    w = 2 * np.pi / periodo
    return (10 * np.cos(w * t - 1.8) + 5 * np.cos(2 * w * t - 2.8)
            + 2.5 * np.cos(3 * w * t - 3.6) + 1.2 * np.cos(4 * w * t - 4.5))


def pressione_prossimale(t, periodo=PERIODO, p_min=55.0, p_max=85.0):
    tg = np.linspace(0, periodo, 2001)
    s = _forma(tg, periodo)
    return p_min + (p_max - p_min) * (_forma(t, periodo) - s.min()) / (s.max() -
    s.min())


def c_onda(p, z):
    return (97.0 + 2.03 * p) * (1.0 + 0.02 * z)


def area(p, z):
    return S0 * np.exp(-K_RASTR * z) * np.exp((p - P0) * MMHG / (RHO * c_onda(p, z)
    * c_onda(P0, z)))


def pressione(S, z, p):
    """Inverte l'equazione di stato S(p, z) con il metodo di Newton."""
    lnr = np.log(S / (S0 * np.exp(-K_RASTR * z)))
    k = MMHG / (RHO * c_onda(P0, z))
    for _ in range(30):
        cp = c_onda(p, z)
        g = lnr - k * (p - P0) / cp
        dg = -k * (1.0 / cp - (p - P0) * 2.03 * (1 + 0.02 * z) / cp**2)
        dp = -g / dg
```

```
        p = p + dp
        if np.max(np.abs(dp)) < 1e-10:
            break
    return p


def attrito(V, S, tipo="laminare"):
    if tipo == "laminare":
        return -8.0 * np.pi * MU / RHO * V / S
    return -0.1360 * (MU / RHO)**0.25 * np.abs(V)**1.75 / S**0.625 * np.sign(V)


def simula(t_fin=2.1, periodo=PERIODO, p_min=55.0, p_max=85.0, mu_e=0.04, cfl=0.8,
           tipo_attrito="laminare", z_reg=(15.0, 20.0, 22.0)):
    z = np.linspace(0.0, L, NZ); dz = z[1] - z[0]
    pL = lambda t: pressione_prossimale(t, periodo, p_min, p_max)
    VL, VR = 3.0, 0.1                                        # condizioni iniziali
    V = VL * np.exp(z / L * np.log(VR / VL))
    p = pL(0.0) * np.exp(z / L * np.log(PR / pL(0.0)))
    S = area(p, z)
    idx = [int(round(zz / dz)) for zz in z_reg]
    t, n, storia, CV = 0.0, 0, [], 0.0

    def derivate(S, V, p, verso):
        F1, F2 = S * V, 0.5 * V**2
        d1, d2, dp = np.zeros(NZ), np.zeros(NZ), np.zeros(NZ)
        if verso == "avanti":
            d1[:-1] = np.diff(F1); d2[:-1] = np.diff(F2); dp[:-1] = np.diff(p)
        else:
            d1[1:] = np.diff(F1); d2[1:] = np.diff(F2); dp[1:] = np.diff(p)
        return (-d1 / dz,
                -d2 / dz - dp * MMHG / (RHO * dz) + attrito(V, S, tipo_attrito))

    def contorno(S, V, p, tt):
        p[0], p[-1] = pL(tt), PR
        S[0], S[-1] = area(p[0], z[0]), area(p[-1], z[-1])
        V[0], V[-1] = V[1], V[-2]                            # estrapolazione di
    ordine zero

    while t < t_fin:
        dt = cfl * dz / np.max(np.abs(V) + c_onda(p, z))  # stabilita': |V| + c
        CV = max(CV, np.max(np.abs(V)) * dt / dz)
        v1, v2 = ("avanti", "indietro") if n % 2 == 0 else ("indietro", "avanti")
        r1, r2 = derivate(S, V, p, v1)                       # predittore
        Ss, Vs = S + dt * r1, V + dt * r2
        ps = pressione(Ss, z, p.copy())
        contorno(Ss, Vs, ps, t + dt)
        q1, q2 = derivate(Ss, Vs, ps, v2)                    # correttore
        Sn, Vn = 0.5 * (S + Ss + dt * q1), 0.5 * (V + Vs + dt * q2)
        D = np.zeros(NZ)
        D[2:-2] = -mu_e / 8 * (V[4:] - 4 * V[3:-1] + 6 * V[2:-2] - 4 * V[1:-3] +
    V[:-4])
        Vn += D
        pn = pressione(Sn, z, p.copy())
        t, n = t + dt, n + 1
        contorno(Sn, Vn, pn, t)
        S, V, p = Sn, Vn, pn
        storia.append((t, *[p[i] for i in idx], *[V[i] for i in idx],
                       *[S[i] * V[i] for i in idx]))
        if not np.all(np.isfinite(V)):
```

```
            print("soluzione divergente a t =", t)
            break
    # colonne di 'storia': t | p nelle sezioni z_reg | V | portata S V
    return z, np.array(storia), n, CV, (S, V, p)


if __name__ == "__main__":
    z_reg = (5.0, 12.5, 15.0, 20.0, 22.0)
    nr = len(z_reg)
    z, H, n, CV, (S, V, p) = simula(t_fin=4.2, z_reg=z_reg)      # 6 periodi
    t = H[:, 0]
    print(f"passi = {n}, massimo numero di Courant basato su V: {CV:.3f}")
    print(f"area: {S0:.2f} cm^2 a z = 0, {S0*np.exp(-K_RASTR*L):.4f} cm^2 a z = L
    (p = p0)")
    ult = t >= t[-1] - PERIODO                          # ultimo periodo
    for k, zz in enumerate(z_reg):
        P, Vz = H[ult, 1 + k], H[ult, 1 + nr + k]
        q = np.trapezoid(H[ult, 1 + 2 * nr + k], t[ult]) / PERIODO
        print(f"z = {zz:4.1f} cm: p in [{P.min():6.1f}, {P.max():6.1f}] mmHg, "
              f"V in [{Vz.min():6.1f}, {Vz.max():6.1f}] cm/s, portata media {q:.3f}
    cm^3/s")

    # condizione al contorno prossimale
    tt = np.linspace(0, PERIODO, 400)
    fig, ax = plt.subplots(figsize=(4.4, 2.8))
    ax.plot(tt, pressione_prossimale(tt), color=ROSSO)
    ax.set_xlabel("$t$ [s]"); ax.set_ylabel("$p_L$ [mmHg]")
    salva(fig, "prog2_13_pressione_prossimale")

    # pressione e velocita' in tre sezioni sull'ultimo periodo
    fig, axs = plt.subplots(2, 3, figsize=(6.8, 4.0), sharex=True)
    for k, zz in enumerate(z_reg[2:]):
        axs[0, k].plot(t[ult], H[ult, 3 + k], color=ROSSO)
        axs[1, k].plot(t[ult], H[ult, 3 + nr + k], color=BLU)
        axs[1, k].axhline(0, color=GRIGIO, lw=0.5)
        axs[0, k].set_title(f"$z = {zz:g}$ cm", fontsize=8)
        axs[1, k].set_xlabel("$t$ [s]")
        axs[0, k].set_ylim(30, 110); axs[1, k].set_ylim(-30, 110)
    axs[0, 0].set_ylabel("$p$ [mmHg]"); axs[1, 0].set_ylabel("$V$ [cm/s]")
    salva(fig, "prog2_13_segnali")
```

Listato 11.1: `prog2_13_tubo_elastico.py`: flusso pulsante in un tubo elastico, schema di MacCormack per un sistema iperbolico non lineare.

Tubi elastici in aeronautica: il colpo d'ariete e l'effetto POGO

Le stesse equazioni, con il fluido idraulico o il propellente al posto del sangue e la parete metallica al posto di quella arteriosa, descrivono il *colpo d'ariete* negli impianti idraulici di bordo: la chiusura rapida di una valvola genera un'onda di pressione che viaggia alla velocità (11.4) e si riflette agli estremi della linea. Nei lanciatori a propellenti liquidi l'accoppiamento fra le onde di pressione nelle linee di alimentazione, la spinta del motore e le vibrazioni longitudinali della struttura produce l'instabilità detta POGO, che ha afflitto diversi lanciatori del programma Apollo e si combatte con accumulatori che aumentano la cedevolezza delle linee, cioè ne abbassano la velocità d'onda.

Appendice A

Simboli, listati di supporto e istruzioni

A.1 Simboli principali

Tabella A.1: Principali simboli della monografia e nomi corrispondenti nei programmi.

Simbolo	Programma	Significato
Flussi potenziali (Cap. 1–5)		
ϕ, ψ	`phi`, `psi`, `PSI`	potenziale di velocità e funzione di corrente
$w = \phi + \mathrm{i}\psi$	`w_complesso`	potenziale complesso
U, α	`U`, `alfa`	velocità e incidenza della corrente uniforme
σ, Γ, κ	`SIGMA`, `GAMMA`, `c`	intensità di sorgente, vortice (orario > 0), doppietta
C_p	`cp`	coefficiente di pressione $1 - (V/U)^2$
C_L, C_D, $C_{m,c/4}$	`CL`, `CD`, `Cm`	coefficienti di portanza, resistenza, momento
q_j, $\tilde{q}_j = q_j/U$	`q`	intensità per unità di lunghezza dei segmenti di sorgenti
d_{ij}, Δd_{ij}	`d`, `M`	distanze punto–estremi dei segmenti, eq. (4.9)
a, b, ζ_P, β	`A`, `B`, `ZP`, `BETA`	raggio del cerchio, parametro di Joukowski, centro, angolo di Kutta
γ'_j	`gam`	intensità nodali dei pannelli di vortice, $\gamma/(2\pi U)$
Problemi ai limiti ed equazioni ellittiche (Cap. 2, 6, 7)		
$C_{i1}, \ldots, C_{i4}$	`C[i, 0:4]`	coefficienti del sistema tridiagonale, eq. (2.11)
h	`h`, `H`	passo della griglia
ζ	`zeta`	vorticità, $\nabla^2\psi = -\zeta$
ω, ω_{ott}	`omega`, `omega_ottimo`	fattore di sovrarilassamento e suo valore ottimo
ϱ	`rJ`	raggio spettrale del metodo iterativo
a, b, c, d	`bracci[i, j, 0:4]`	bracci disuguali verso sinistra, destra, basso, alto
Equazioni iperboliche (Cap. 8–11)		
a, a_0	`A_SUONO`, `A0`	velocità del suono (locale, indisturbata)
τ, Δt	`tau`, `TAU`, `dt`	passo temporale
Co	`Co`	numero di Courant
G	`G`	fattore di amplificazione di von Neumann
$J^{\pm}$	–	invarianti di Riemann $u \pm 2a/(\gamma - 1)$
u_A, a_A, u_B, a_B	`uA`, `aA`, `uB`, `aB`	valori interpolati nei piedi delle caratteristiche, eq. (9.5)
F, $\bar{u}$	`F`, `us`	flusso $u^2/2$ e valore predetto (MacCormack)
μ_e	`mu_e`	coefficiente di dissipazione artificiale
S, V, p, c	`S`, `V`, `p`, `c_onda`	area, velocità, pressione e velocità d'onda nel tubo elastico

A.2 Listati di supporto


metodi_numerici.py

```
"""
metodi_numerici.py -- strumenti comuni della monografia 2 (flussi non viscosi).

Contiene:
  * trid(C)            eliminazione di Gauss per sistemi tridiagonali (forma a 4
    colonne)
  * thomas(a, b, c, d) algoritmo di Thomas in memoria compatta
  * cerca_x(...)       ricerca dei punti di una linea di corrente lungo le verticali
  * cerca_xy(...)      ricerca lungo le verticali e lungo le orizzontali
  * jacobi_passo, gauss_seidel_passo, sor_passo: un'iterazione dei metodi
                       iterativi per l'equazione di Poisson su griglia quadrata
"""
import numpy as np


# ---------------------------------------------------------------------------
# 1. Sistemi tridiagonali
# ---------------------------------------------------------------------------
def trid(C):
    """Risolve il sistema tridiagonale scritto riga per riga come
         C[i,0]*f[i-1] + C[i,1]*f[i] + C[i,2]*f[i+1] = C[i,3],  i = 0..n-1
    (C[0,0] e C[n-1,2] devono essere gia' stati azzerati dalle condizioni al
    contorno). Eliminazione in avanti per combinazione di righe, poi
    sostituzione all'indietro. Restituisce il vettore f (C non viene alterata).
    """
    C = np.array(C, dtype=float)          # copia di lavoro
    n = C.shape[0]
    for i in range(1, n - 1):             # eliminazione di C[i,0], i = 1..n-2
        c2 = C[i, 1] * C[i - 1, 1] - C[i, 0] * C[i - 1, 2]
        c3 = C[i, 2] * C[i - 1, 1]
        c4 = C[i, 3] * C[i - 1, 1] - C[i, 0] * C[i - 1, 3]
        C[i, 1], C[i, 2], C[i, 3] = c2, c3, c4
    f = np.zeros(n)
    # ultime due equazioni risolte insieme
    f[n - 1] = ((C[n - 1, 3] * C[n - 2, 1] - C[n - 1, 0] * C[n - 2, 3]) /
                (C[n - 1, 1] * C[n - 2, 1] - C[n - 1, 0] * C[n - 2, 2]))
    for j in range(n - 2, -1, -1):        # sostituzione all'indietro
        f[j] = (C[j, 3] - C[j, 2] * f[j + 1]) / C[j, 1]
    return f


def thomas(a, b, c, d):
    """Algoritmo di Thomas per  a[i] x[i-1] + b[i] x[i] + c[i] x[i+1] = d[i].
    a[0] e c[-1] non vengono usati. Costo: 8n operazioni circa."""
    n = len(d)
    cp = np.zeros(n)
    dp = np.zeros(n)
    cp[0] = c[0] / b[0]
    dp[0] = d[0] / b[0]
    for i in range(1, n):                 # eliminazione in avanti
        den = b[i] - a[i] * cp[i - 1]
        cp[i] = c[i] / den if i < n - 1 else 0.0
        dp[i] = (d[i] - a[i] * dp[i - 1]) / den
    x = np.zeros(n)
    x[-1] = dp[-1]
    for i in range(n - 2, -1, -1):        # sostituzione all'indietro
        x[i] = dp[i] - cp[i] * x[i + 1]
    return x
```

```
58
59
60  # ------------------------------------------------------------------------
61  # 2. Ricerca dei punti di una linea psi = psia su una griglia rettangolare
62  #    psi[i, j] = valore nel nodo (x[i], y[j]); i nodi esterni al campo di moto
63  #    possono essere marcati con NaN e vengono ignorati.
64  # ------------------------------------------------------------------------
65  def _attraversamenti(s, v, va):
66      """Punti dell'intervallo [s_k, s_k+1] in cui v - va cambia segno:
67      interpolazione lineare  s = s_k+1 - ds*|Q|/(|P|+|Q|)."""
68      P = v[:-1] - va
69      Q = v[1:] - va
70      ok = np.isfinite(P) & np.isfinite(Q) & (P * Q <= 0.0) & ((P != 0) | (Q != 0))
71      k = np.nonzero(ok)[0]
72      ds = s[k + 1] - s[k]
73      return s[k + 1] - ds * np.abs(Q[k]) / (np.abs(P[k]) + np.abs(Q[k]))
74
75
76  def cerca_x(x, y, psi, psia):
77      """Ricerca lungo le sole linee verticali (subroutine SEARCH)."""
78      xx, yy = [], []
79      for i in range(len(x)):
80          ys = _attraversamenti(y, psi[i, :], psia)
81          xx.extend([x[i]] * len(ys))
82          yy.extend(ys)
83      return np.array(xx), np.array(yy)
84
85
86  def cerca_xy(x, y, psi, psia):
87      """Ricerca lungo le verticali e lungo le orizzontali (SEARCH2):
88      utile quando le linee di corrente sono quasi parallele all'asse y."""
89      xx, yy = cerca_x(x, y, psi, psia)
90      x2, y2 = [], []
91      for j in range(len(y)):
92          xs = _attraversamenti(x, psi[:, j], psia)
93          x2.extend(xs)
94          y2.extend([y[j]] * len(xs))
95      return np.concatenate([xx, x2]), np.concatenate([yy, y2])
96
97
98  # ------------------------------------------------------------------------
99  # 3. Metodi iterativi per  psi_xx + psi_yy = -zeta  su griglia quadrata di
100 #    passo h. 'interni' e' una lista di coppie (i, j) da aggiornare, nell'ordine
101 #    in cui vengono visitate (colonna per colonna, dal basso verso l'alto).
102 #    Ogni funzione esegue UNA iterazione e restituisce la somma dei |delta|.
103 # ------------------------------------------------------------------------
104 def jacobi_passo(psi, zeta, h, interni):
105     vecchio = psi.copy()                  # si usano SOLO i valori precedenti
106     err = 0.0
107     for i, j in interni:
108         nuovo = 0.25 * (vecchio[i - 1, j] + vecchio[i + 1, j] +
109                         vecchio[i, j - 1] + vecchio[i, j + 1] + h * h * zeta[i, j])
110         err += abs(nuovo - vecchio[i, j])
111         psi[i, j] = nuovo
112     return err
113
114
115 def gauss_seidel_passo(psi, zeta, h, interni):
116     err = 0.0                             # metodo di Liebmann (Gauss-Seidel)
117     for i, j in interni:
```

```
        nuovo = 0.25 * (psi[i - 1, j] + psi[i + 1, j] +
                        psi[i, j - 1] + psi[i, j + 1] + h * h * zeta[i, j])
        err += abs(nuovo - psi[i, j])
        psi[i, j] = nuovo
    return err


def sor_passo(psi, zeta, h, interni, omega):
    err = 0.0                               # sovrarilassamento successivo
    for i, j in interni:
        gs = 0.25 * (psi[i - 1, j] + psi[i + 1, j] +
                     psi[i, j - 1] + psi[i, j + 1] + h * h * zeta[i, j])
        nuovo = (1.0 - omega) * psi[i, j] + omega * gs
        err += abs(nuovo - psi[i, j])
        psi[i, j] = nuovo
    return err


def omega_ottimo(m, n):
    """Fattore di sovrarilassamento ottimo per un rettangolo diviso in m x n
    intervalli:  omega = (8 - 4 sqrt(4 - a^2)) / a^2,  a = cos(pi/m) + cos(pi/n)."""
    a = np.cos(np.pi / m) + np.cos(np.pi / n)
    return (8.0 - 4.0 * np.sqrt(4.0 - a * a)) / (a * a)


def nodi_interni_rettangolo(M, N):
    """Coppie (i, j) dei nodi interni di una griglia M x N, colonna per colonna."""
    return [(i, j) for i in range(1, M - 1) for j in range(1, N - 1)]
```

Listato A.1: `metodi_numerici.py`: sistemi tridiagonali, ricerca delle linee di corrente, metodi iterativi per l'equazione di Poisson.

```
"""
stile_grafici.py -- impostazioni grafiche comuni (palette "della casa").
Importato da tutti gli script della monografia.
"""
import os
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

# Palette coerente con raucci-aero.sty
BLU      = "#1E2761"
ARANCIO  = "#DC7633"
ROSSO    = "#C0392B"
VERDE    = "#1E8449"
GIALLO   = "#B7950B"
VIOLA    = "#6C3483"
GRIGIO   = "#566573"
PALETTE  = [BLU, ARANCIO, VERDE, VIOLA, ROSSO, GIALLO, GRIGIO]

plt.rcParams.update({
    "font.family": "serif",
    "font.size": 9,
    "axes.labelsize": 9,
    "axes.titlesize": 9,
    "legend.fontsize": 8,
    "xtick.labelsize": 8,
    "ytick.labelsize": 8,
    "axes.prop_cycle": matplotlib.cycler(color=PALETTE),
    "axes.grid": True,
```

stile_grafici.py

```
    "grid.alpha": 0.3,
    "grid.linestyle": ":",
    "lines.linewidth": 1.4,
    "figure.dpi": 120,
    "savefig.bbox": "tight",
    "savefig.pad_inches": 0.02,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "legend.frameon": False,
})

# Le figure vengono salvate nella cartella "figure" accanto a quella degli script
# (creata se manca): cosi' ogni programma funziona anche se scaricato da solo.
FIG_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "..", "figure")


def salva(fig, nome):
    """Salva la figura in PDF nella cartella delle figure del progetto LaTeX."""
    os.makedirs(FIG_DIR, exist_ok=True)
    fig.savefig(os.path.join(FIG_DIR, f"{nome}.pdf"))
    plt.close(fig)
    print(f"  -> figura salvata: {nome}.pdf")
```

Listato A.2: `stile_grafici.py`: impostazioni comuni di Matplotlib.

prog2_2b_flussi_elementari.py

```
"""
prog2_2b_flussi_elementari.py -- i quattro flussi elementari e il cilindro
con circolazione.

1) Linee di corrente di corrente uniforme, sorgente, vortice e doppietta.
2) Cilindro circolare: corrente uniforme + doppietta + vortice. Per diversi
   valori della circolazione si traccia il campo, si calcola C_p sulla
   superficie e si integra la pressione: la resistenza e' nulla (paradosso di
   d'Alembert) e la portanza coincide con quella di Kutta-Joukowski, L = rho U
    Gamma.
"""
import numpy as np
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, ROSSO, GRIGIO, VERDE

U = 1.0

# ------------------------------------------------------------------ 1) galleria
x = np.linspace(-2, 2, 401)
X, Y = np.meshgrid(x, x, indexing="ij")
R2 = X**2 + Y**2
with np.errstate(divide="ignore", invalid="ignore"):
    flussi = {
        r"(a) corrente uniforme, $\alpha=30^\circ$":
            (U * (Y * np.cos(np.pi / 6) - X * np.sin(np.pi / 6)), np.linspace(-3,
    3, 13)),
        r"(b) sorgente, $\sigma>0$":
            (np.arctan2(Y, X) / np.pi, np.linspace(-1, 1, 17)),
        r"(c) vortice, $\Gamma>0$ (orario)":
            (np.log(np.sqrt(R2)), np.linspace(-2.0, 0.6, 14)),
        r"(d) doppietta, $\kappa>0$":
            (-Y / R2, np.array([-8, -4, -2, -1, -0.5, -0.25, 0.25, 0.5, 1, 2, 4,
    8])),
    }
fig, axs = plt.subplots(2, 2, figsize=(6.0, 6.0))
for ax, (titolo, (campo, livelli)) in zip(axs.flat, flussi.items()):
```

```
    ax.contour(X, Y, campo, levels=livelli, colors=BLU, linewidths=0.8,
               linestyles="solid")
    ax.plot(0, 0, "o", color=ROSSO, ms=3) if "uniforme" not in titolo else None
    ax.set_aspect("equal"); ax.set_title(titolo)
    ax.set_xticks([-2, 0, 2]); ax.set_yticks([-2, 0, 2])
    ax.grid(False)
# frecce indicative del verso del moto
axs[0, 0].annotate("", (1.2, 0.69), (0.2, 0.115), arrowprops=dict(arrowstyle="->",
    color=ROSSO))
for ang in np.radians([0, 90, 180, 270]):
    axs[0, 1].annotate("", (1.6 * np.cos(ang), 1.6 * np.sin(ang)),
                       (0.9 * np.cos(ang), 0.9 * np.sin(ang)),
                       arrowprops=dict(arrowstyle="->", color=ROSSO))
axs[1, 0].annotate("", (0.0, -1.2), (-0.05, 1.2), arrowprops=dict(arrowstyle="-",
    color="none"))
t = np.linspace(np.pi * 0.6, np.pi * 0.1, 30)
axs[1, 0].plot(1.35 * np.cos(t), 1.35 * np.sin(t), color=ROSSO, lw=1)
axs[1, 0].annotate("", (1.35 * np.cos(t[-1]) + 0.05, 1.35 * np.sin(t[-1]) - 0.12),
                   (1.35 * np.cos(t[-2]), 1.35 * np.sin(t[-2])),
                   arrowprops=dict(arrowstyle="->", color=ROSSO))
salva(fig, "prog2_2b_elementari")


# ------------------------------------------------ 2) cilindro con circolazione
A = 1.0                                   # raggio del cilindro
GAMMA = [0.0, 2.0 * np.pi, 4.0 * np.pi, 5.0 * np.pi]   # circolazioni (orarie)


def psi_cilindro(X, Y, gam):
    r = np.hypot(X, Y)
    return U * Y * (1 - A**2 / r**2) + gam / (2 * np.pi) * np.log(r / A)


xg = np.linspace(-3, 3, 481)
XG, YG = np.meshgrid(xg, xg, indexing="ij")
fig, axs = plt.subplots(2, 2, figsize=(6.0, 6.0))
theta = np.linspace(0, 2 * np.pi, 2001)
print(" Gamma/(U a)  sin(theta_s)    C_D (integr.)   C_L (integr.)   C_L = Gamma/(U
    a)")
for ax, gam in zip(axs.flat, GAMMA):
    with np.errstate(divide="ignore", invalid="ignore"):
        P = psi_cilindro(XG, YG, gam)
    P[np.hypot(XG, YG) < A] = np.nan
    ax.contour(XG, YG, P, levels=np.linspace(-3, 3, 25), colors=BLU,
               linewidths=0.7, linestyles="solid")
    ax.fill(A * np.cos(theta), A * np.sin(theta), color=GRIGIO, alpha=0.35)
    # velocita' tangenziale sulla superficie (theta antiorario da x positivo):
    # u_theta = -2 U sin(theta) - Gamma / (2 pi a)   (circolazione oraria > 0)
    ut = -2 * U * np.sin(theta) - gam / (2 * np.pi * A)
    cp = 1 - (ut / U)**2
    # forza per unita' di apertura: F = -int p n ds, n = (cos, sin)
    dth = theta[1] - theta[0]
    cx = -np.trapezoid(cp * np.cos(theta), dx=dth) * A / (2 * A)   # riferita al
    diametro
    cy = -np.trapezoid(cp * np.sin(theta), dx=dth) * A / (2 * A)
    s = gam / (4 * np.pi * U * A)
    print(f"   {gam/(U*A):7.4f}      {s:6.3f}        {cx:+.2e}       {cy:8.5f}
      {gam/(U*A):8.5f}")
    if s <= 1:
        th_s = np.arcsin(-s)
```

```
        ax.plot(A * np.cos([th_s, np.pi - th_s]), A * np.sin([th_s, np.pi - th_s]),
                "o", color=ROSSO, ms=4)
    else:
        # punto di ristagno libero sull'asse y negativo: u_theta = 0 fuori dal cilindro
        rr = (gam / (4 * np.pi * U) + np.sqrt((gam / (4 * np.pi * U))**2 - A**2))
        ax.plot(0, -rr, "o", color=ROSSO, ms=4)
    ax.set_aspect("equal"); ax.set_xticks([-3, 0, 3]); ax.set_yticks([-3, 0, 3])
    ax.set_title(fr"$\Gamma/(4\pi U a) = {s:.2f}$"); ax.grid(False)
salva(fig, "prog2_2b_cilindro")
```

Listato A.3: `prog2_2b_flussi_elementari.py`: flussi elementari e cilindro con circolazione (Cap. 3).


prog2_4b_mappe.py

```
"""
prog2_4b_mappe.py -- esempi di trasformazioni conformi.

(a) z = zeta^2 : la corrente uniforme del semipiano superiore diventa il
    flusso nell'angolo retto (linee di corrente iperboliche 2xy = cost).
(b) z = zeta + b^2/zeta con b < a: il cerchio |zeta| = a diventa un'ellisse e
    il flusso attorno al cilindro diventa il flusso attorno al cilindro ellittico.
(c) la stessa trasformazione applicata a cerchi passanti per zeta = +-b:
    centro sull'asse immaginario -> arco di cerchio (lamina curva);
    centro fuori dagli assi -> profilo di Joukowski.
Le linee di corrente si tracciano nel piano zeta (dove il flusso e' noto) e
se ne trasformano i punti nel piano z.
"""
import numpy as np
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, ROSSO, GRIGIO, VERDE, VIOLA

U = 1.0


def linee_mappate(ax, zeta_grid, campo, livelli, mappa, colore, stile="-", lw=0.8):
    """Traccia nel piano z le curve di livello di 'campo' calcolate nel piano zeta."""
    fig_tmp, ax_tmp = plt.subplots()
    cs = ax_tmp.contour(zeta_grid.real, zeta_grid.imag, campo, levels=livelli)
    for segs in cs.allsegs:
        for seg in segs:
            if len(seg) < 2:
                continue
            zm = mappa(seg[:, 0] + 1j * seg[:, 1])
            ax.plot(zm.real, zm.imag, stile, color=colore, lw=lw)
    plt.close(fig_tmp)


fig, axs = plt.subplots(2, 2, figsize=(6.4, 6.0))

# ---------------- (a) angolo retto ----------------------------------------
s = np.linspace(0, 2.5, 301)
ZETA = s[:, None] + 1j * s[None, :]            # primo quadrante del piano zeta
W = U * ZETA**2                                # w = U zeta^2 = U(x^2-y^2) + i 2U xy
ax = axs[0, 0]
ax.contour(ZETA.real, ZETA.imag, W.imag, levels=np.arange(0.5, 6.1, 0.5),
           colors=BLU, linewidths=0.8, linestyles="solid")
ax.contour(ZETA.real, ZETA.imag, W.real, levels=np.arange(-6, 6.1, 0.5),
           colors=ARANCIO, linewidths=0.6, linestyles="--")
ax.plot([0, 0, 2.5], [2.5, 0, 0], color="k", lw=2)
ax.set_aspect("equal"); ax.set_title(r"(a) $w=U\zeta^2$: angolo di $90^\circ$")
```

```
ax.set_xlim(0, 2.5); ax.set_ylim(0, 2.5); ax.grid(False)

# ---------------- (b) ellisse ------------------------------------------------
a, b = 1.0, 0.8
r = np.linspace(a, 4.0, 300)
t = np.linspace(0, 2 * np.pi, 721)
ZC = r[:, None] * np.exp(1j * t[None, :])      # esterno del cerchio
PSI = (U * (ZC + a**2 / ZC)).imag
ax = axs[0, 1]
linee_mappate(ax, ZC, PSI, np.linspace(-3, 3, 13), lambda zz: zz + b**2 / zz, BLU)
ell = a * np.exp(1j * t) + b**2 / (a * np.exp(1j * t))
ax.fill(ell.real, ell.imag, color=GRIGIO, alpha=0.4); ax.plot(ell.real, ell.imag,
    "k", lw=0.8)
ax.set_aspect("equal"); ax.set_xlim(-3, 3); ax.set_ylim(-2.5, 2.5); ax.grid(False)
ax.set_title(f"(b) cilindro ellittico ($b/a={b/a:.1f}$)")
print(f"ellisse: semiassi {a + b**2/a:.3f} e {a - b**2/a:.3f}")

# ---------------- (c) arco di cerchio e profilo ------------------------------
b = 0.8
ax = axs[1, 0]
for centro, col, nome in ((0.0 + 0.3j, VERDE, "arco"),
                          (-0.15 + 0.3j, BLU, "profilo")):
    rag = abs(b - centro)                       # il cerchio passa per zeta = b
    cerchio = centro + rag * np.exp(1j * t)
    ax.plot(cerchio.real, cerchio.imag, color=col, lw=1.0)
    ax.plot(centro.real, centro.imag, "+", color=col)
ax.plot([-b, b], [0, 0], "o", color=ROSSO, ms=3)
ax.set_aspect("equal"); ax.set_xlim(-1.4, 1.4); ax.set_ylim(-1.2, 1.6);
    ax.grid(False)
ax.set_title(r"(c) piano $\zeta$: cerchi per $\zeta=\pm b$")
ax = axs[1, 1]
for centro, col in ((0.0 + 0.3j, VERDE), (-0.15 + 0.3j, BLU)):
    rag = abs(b - centro)
    cerchio = centro + rag * np.exp(1j * t)
    zz = cerchio + b**2 / cerchio
    ax.plot(zz.real, zz.imag, color=col, lw=1.2)
ax.plot([-2 * b, 2 * b], [0, 0], "o", color=ROSSO, ms=3)
ax.set_aspect("equal"); ax.set_xlim(-2.2, 2.2); ax.set_ylim(-0.6, 1.0);
    ax.grid(False)
ax.set_title(r"(d) piano $z$: arco e profilo di Joukowski")
salva(fig, "prog2_4b_mappe")
```

Listato A.4: `prog2_4b_mappe.py`: esempi di trasformazioni conformi (Cap. 5).

A.3 Come riprodurre i risultati

Istruzioni

1. Requisiti: Python $\geq$ 3.9 con `numpy`, `scipy`, `matplotlib` e, per l'esercizio 14, `sympy` (`pip install numpy scipy matplotlib sympy`); una distribuzione TEX con `pdflatex` e i pacchetti `tcolorbox`, `pgfplots`, `siunitx`, `listings`, `qrcode`, `marginnote`, `babel-italian` (tutti inclusi in TEX Live / MacTeX completo).
2. Dalla cartella `codice/`: `bash esegui_tutti.sh` esegue tutti i programmi e gli esercizi (circa un minuto e mezzo) e rigenera le figure in `figure/`. Ogni

programma si può anche eseguire da solo con `python3 nome.py`; per gli esercizi, `python3 esercizi_svolti.py 5 12` esegue solo il quinto e il dodicesimo.

3. Dalla cartella principale: `pdflatex main.tex` due volte (o `latexmk -pdf main.tex`).

A.4 Codici in rete e QR code

I programmi sono pubblicati all'indirizzo

`https://braucci.github.io/CFD/monografia2/codici/`

La pagina elenca tutti i file con le rispettive dipendenze, li mostra con l'evidenziazione della sintassi e permette di scaricarli singolarmente o in un unico archivio. Il QR code nel margine accanto a ogni listato codifica l'indirizzo del file corrispondente e, nel PDF, è anche un collegamento cliccabile.

Per l'autore: configurazione e pubblicazione

1. In `main.tex` la macro `\nomemonografia` (qui `monografia2`) determina tutti gli indirizzi: `\urlcodici` vale `https://braucci.github.io/CFD/<nome>/codici/` e `\urlfile{file.py}` aggiunge il nome del file.
2. Un listato completo si inserisce con `\listatopy[opzioni]{file.py}` (legge `codice/file.py` e mette il QR nel margine); prima di un estratto in `lstlisting` si scrive `\qrlistato{file.py}`.
3. `python3 pubblica_github.py` crea `github/monografia2/` con il PDF, la cartella `codici/`, la pagina `index.html` e l'archivio zip: va copiata così com'è nella cartella `CFD/` del repository, accanto a `monografia1/`.

A.5 Come sostituire le figure

Le figure del documento sono di due tipi, gestite da due macro definite in `main.tex`.

`\figuraesterna[larghezza]{nome}{testo}` figure schematiche (griglie, schemi geometrici, costruzioni con le caratteristiche). Se nella cartella `immagini/` esiste il file `nome.png` (oppure `.pdf` o `.jpg`) viene incluso con la larghezza indicata; altrimenti compare un *segnaposto* grigio con il nome del file atteso e una breve descrizione. Basta copiare in `immagini/` le immagini con i nomi della Tab. A.2 e ricompilare.

`\figuragenerata[larghezza]{nome}` grafici prodotti dagli script Python, che il documento legge dalla cartella `figure/`. Anche questi si possono sostituire: se esiste `immagini/nome.png`, ha la precedenza sul PDF generato.

Tabella A.2: Figure con segnaposto e nome del file atteso in `immagini/`.

Figura	File	Contenuto
1.1	`fig_2_1_1.png`	volume di controllo con i flussi attraverso le facce
2.1	`fig_2_2_1.png`	discretizzazione di un problema ai limiti monodimensionale
3.3	`fig_2_4_3.png`	valori di ψ lungo una verticale e ricerca di $\psi = \psi_a$
4.1	`fig_2_5_2.png`	corpo di rivoluzione, segmenti di sorgenti e punti di controllo
6.1	`fig_2_7_1.png`	corrente supersonica attorno a un cuneo, linee di Mach
6.2	`fig_2_8_1.png`	dominio rettangolare e formula a cinque punti
7.1	`fig_2_9_1.png`	camera con parete inclinata
7.3	`fig_2_9_3.png`	cilindro circolare in un canale
7.4	`fig_2_9_4.png`	nodo con bracci disuguali
7.5	`fig_2_9_5.png`	nodo vicino alla superficie del cilindro
B.10	`fig_2_9_7.png`	canale con cilindro a sezione rettangolare (esercizio 10)
B.12	`fig_2_9_8.png`	tubo con diaframmi ripetuti (esercizio 11)
8.1	`fig_2_10_1.png`	griglia nel piano x–t e schema leapfrog
8.3	`fig_2_10_2.png`	domini di dipendenza numerico e fisico
8.4	`fig_2_11_1.png`	ondina sinusoidale in un tubo chiuso–aperto
9.1	`fig_2_12_1.png`	metodo delle caratteristiche
9.2	`fig_2_12_3.png`	schema di Courant, Isaacson e Rees

Appendice B

Esercizi svolti

Gli esercizi completano i capitoli: alcuni verificano un'affermazione del testo, altri estendono i programmi a situazioni nuove, due sono veri e propri progetti. Ogni esercizio è svolto con lo stesso schema del primo modulo: *testo*, *impostazione* (che cosa ci si aspetta e perché, prima di calcolare), *svolgimento* passo per passo con il codice essenziale, *risultato* e *commento*. Il codice completo è nel file `esercizi_svolti.py` (Listato B.1), in cui ogni esercizio è la funzione `es_NN()`; i numeri riportati sono quelli stampati dal programma.

B.1 Esercizio 1 – Combinazioni di flussi elementari

Testo

Modificando il Programma 2.2, tracciare le linee di corrente dei flussi formati da: (a) una corrente uniforme e una sorgente; (b) una corrente uniforme e un pozzo; (c) una sorgente sul semiasse x negativo e una doppietta nell'origine; (d) una corrente uniforme orizzontale e due doppiette di uguale intensità sull'asse x, a distanza d. Verificare che nel caso (d) si genera un corpo ovale per d piccolo, a forma di manubrio per d maggiore e che, aumentando ancora d, il corpo si divide in due corpi separati.

Impostazione. Le funzioni di corrente si sommano. Bisogna fare attenzione alla funzione $\arctan[(y-y_0)/(x-x_0)]$ della sorgente: deve variare con continuità da 0 a π nel primo e nel secondo quadrante e da $-\pi$ a 0 nel terzo e nel quarto. In Python lo garantisce `np.arctan2(y - y0, x - x0)`, che restituisce l'angolo nel quadrante giusto in $(-\pi, \pi]$; conviene normalizzarlo dividendo per π. Il "taglio" in cui l'angolo salta da π a $-\pi$ cade sul semiasse a monte della sorgente, che nei casi (a) e (b) è la linea divisoria stessa: non disturba. Per sapere quali linee formano un corpo si esamina la funzione di corrente combinata nei punti di ristagno.

Svolgimento.

1. (a) e (b): $\psi = Uy \pm \frac{\sigma}{2}\theta/\pi$. Con la sorgente (+) si ritrova il semicorpo di Rankine, chiuso a monte: ristagno in $x_s = -\sigma/(2\pi U)$, linea divisoria $\psi = \pm\sigma/2$, semispessore asintotico $\sigma/(2U)$. Con il pozzo (−) la figura è la stessa ribaltata: il ristagno si trova a valle del pozzo, in $x = +\sigma/(2\pi U)$, e la linea divisoria racchiude il fluido che finisce nel pozzo, proveniente da una striscia di larghezza σ/U a monte.

2. (c): $\psi = \frac{\sigma}{2}\theta_1/\pi - \kappa' y/(x^2+y^2)$, con θ_1 angolo visto dalla sorgente in $(-1,0)$ e $\kappa' = \kappa/(2\pi)$. Il fluido emesso dalla sorgente si allontana in tutte le direzioni; vicino all'origine prevalgono le linee chiuse della doppietta.
3. (d): con due doppiette di intensità adimensionale c in $x = \pm d/2$,

$$\Psi = Y - \frac{cY}{(X+d/2)^2+Y^2} - \frac{cY}{(X-d/2)^2+Y^2}.$$

Nel punto medio $X = 0$ la linea $\Psi = 0$ ha ordinata Y_0 data da $1 = 2c/(d^2/4 + Y_0^2)$, cioè

$$Y_0 = \sqrt{2c - d^2/4}\,:$$

il corpo è unico (la linea $\Psi = 0$ attraversa la verticale $X = 0$ fuori dall'asse) solo se $d < 2\sqrt{2c}$.

Risultato

- (a) $\sigma/(Ul) = 2$: ristagno in $X = -0{,}318$, semispessore asintotico 1; (b) ristagno in $X = +0{,}318$; (c) con $\sigma = 2$ e $\kappa' = 0{,}6$ il ristagno sull'asse si trova in $X = 2{,}61$, dove la velocità radiale della sorgente bilancia quella della doppietta (Fig. B.1).
- (d) Con $c = 0{,}25$ il limite è $d = 2\sqrt{0{,}5} = 1{,}414$: per $d = 0{,}4$ il semispessore al centro è 0,678 (corpo ovale), per $d = 1{,}0$ scende a 0,500 (strozzatura: manubrio), per $d = 1{,}6$ la linea $\Psi = 0$ non attraversa più la verticale di mezzeria e il corpo si divide in due (Fig. B.2).

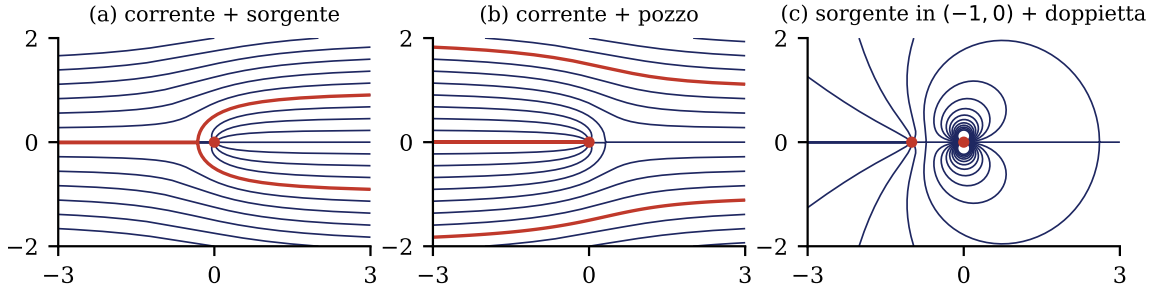


Figura B.1: Esercizio 1 (a)–(c): corrente e sorgente, corrente e pozzo, sorgente e doppietta. In rosso le linee divisorie.

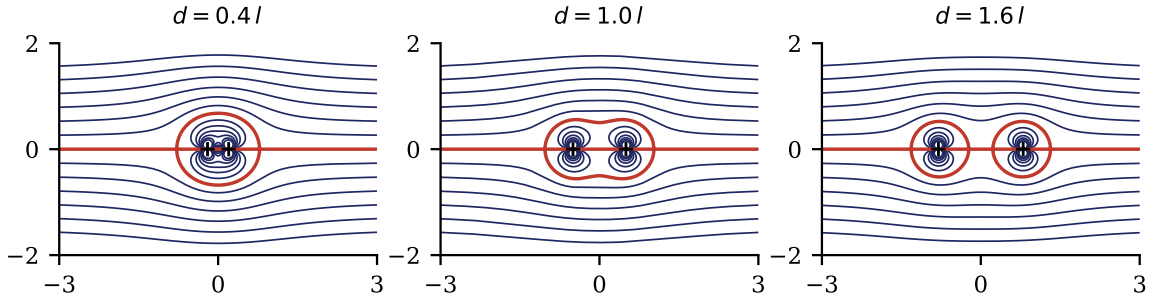


Figura B.2: Esercizio 1 (d): corrente uniforme e due doppiette uguali; corpo ovale, a manubrio, diviso in due.

Commento. Il caso (d) mostra la "logica" del metodo inverso: avvicinando le singolarità il corpo si arrotonda, allontanandole si strozza. Con più doppiette di intensità decrescente verso la coda (Programma 2.2) si ottengono forme affusolate; aggiungendo una coppia sorgente–pozzo si può allungare il corpo senza ispessirlo.

B.2 Esercizio 2 – Il metodo delle immagini

Testo

Tracciare le linee di corrente del flusso formato da un vortice di circolazione Γ in $(0, d)$ e da un vortice di circolazione $-\Gamma$ in $(0, -d)$, e verificare che la metà superiore descrive un vortice in presenza di una parete piana $y = 0$. Costruire allo stesso modo il flusso di una sorgente in $(-d, 0)$ in presenza della parete verticale $x = 0$.

Impostazione. La parete deve essere una linea di corrente. Per simmetria: i due vortici di segno opposto inducono sull'asse x velocità verticali uguali e opposte, che si cancellano; due sorgenti uguali inducono sull'asse y velocità orizzontali uguali e opposte. In entrambi i casi la linea di simmetria non è attraversata dal fluido e può essere sostituita da una parete.

Svolgimento.

1. Vortici: $\psi = \frac{\Gamma}{2\pi}\left[\ln\sqrt{x^2+(y-d)^2} - \ln\sqrt{x^2+(y+d)^2}\right]$; sull'asse x le due radici sono uguali e $\psi = 0$.
2. Sulla parete la velocità è tangente e vale $u(x,0) = \partial\psi/\partial y = \Gamma d/[\pi(x^2+d^2)]$: è massima sotto il vortice, $u = \Gamma/(\pi d)$, il doppio di quella che il vortice da solo indurrebbe a distanza d.
3. Il vortice reale è trasportato dalla velocità indotta dalla sua immagine, $\Gamma/(2\pi\cdot 2d) = \Gamma/(4\pi d)$, parallela alla parete.
4. Sorgente e parete: la funzione di corrente della sorgente è a più valori (il taglio di arctan attraverserebbe il campo), quindi le linee di corrente si tracciano integrando il campo di velocità con `streamplot`, a partire da punti disposti attorno alla sorgente.

Risultato

Con $\Gamma = 2\pi$ e $d = 1$: velocità sulla parete $u = 2$, 1,6, 1, 0,4 in $x = 0$, 0,5, 1, 2; velocità di trasporto del vortice $\Gamma/(4\pi d) = 0{,}5$. Le linee di corrente (Fig. B.3) sono circonferenze attorno a ciascun vortice, deformate in modo che quella che passa all'infinito coincida con l'asse x; per la sorgente, le linee che partono verso la parete si incurvano e la seguono, come se il fluido emesso "rimbalzasse".

Commento. Il moto del vortice parallelo alla parete è quello che si osserva nei vortici di estremità di un velivolo vicino al suolo: invece di scendere, si separano lateralmente e possono raggiungere una pista parallela, con conseguenze sulle separazioni fra velivoli in decollo e atterraggio.

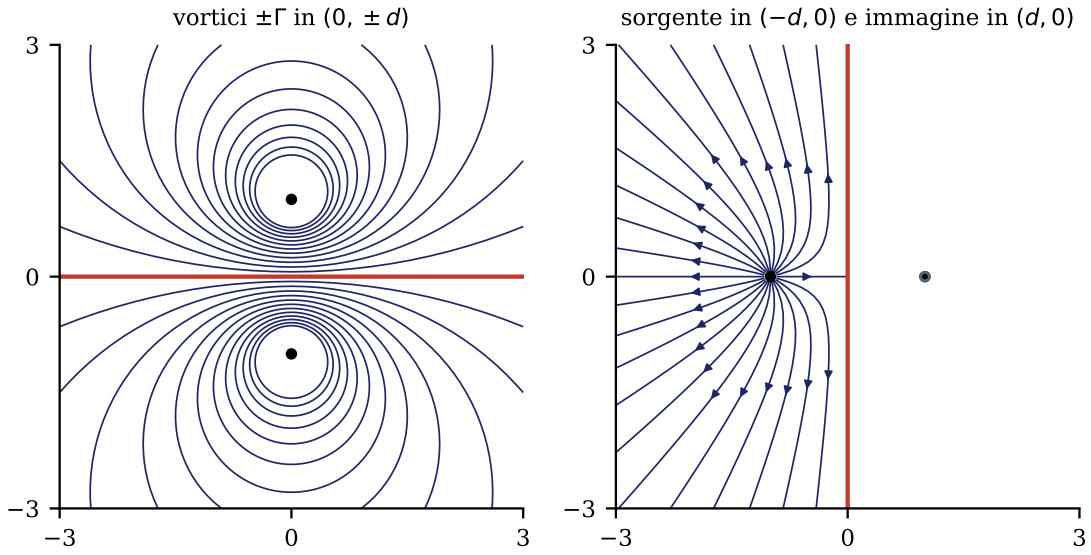


Figura B.3: Esercizio 2: vortice vicino a una parete orizzontale (a sinistra) e sorgente vicino a una parete verticale (a destra); in rosso la parete, in grigio l'immagine.

B.3 Esercizio 3 – Un corpo vicino a una parete

Testo

Tracciare la metà superiore del flusso formato da una corrente uniforme orizzontale, da una doppietta sull'asse y e dalla sua immagine rispetto al piano $y = 0$: si ottiene il flusso attorno a un corpo chiuso in presenza di una lastra parallela alla corrente. Calcolare la distribuzione di pressione sulla faccia superiore della lastra e mostrare che la lastra è soggetta a una forza netta diretta verso il corpo (e quindi il corpo a una forza uguale e opposta, verso la lastra).

Impostazione. Fra il corpo e la lastra il fluido è costretto a passare in uno spazio ristretto e accelera: la pressione sulla lastra, sotto il corpo, scende sotto quella della corrente indisturbata. Ci si aspetta una forza di "aspirazione" verso il corpo. Una sottigliezza: lontano dal corpo la perturbazione decade lentamente, e il segno della pressione cambia; bisogna integrare su tutta la lastra.

Svolgimento.

1. Con $\kappa' = \kappa/(2\pi)$ e la doppietta in $(0, h)$:

$$\psi = Uy - \kappa' \frac{y-h}{x^2 + (y-h)^2} - \kappa' \frac{y+h}{x^2 + (y+h)^2};$$

   sulla lastra $\psi = 0$.

2. Sulla lastra la velocità è $u = U + u'$, con $u' = 2\kappa'(h^2 - x^2)/(x^2 + h^2)^2$, e $-C_p = 2u'/U + (u'/U)^2$ (esatto, non linearizzato).

3. La forza per unità di apertura, positiva verso l'alto, è $F_y/(\frac{1}{2}\rho U^2) = -\int C_p \, \mathrm{d}x$. Il termine lineare ha integrale nullo sulla lastra infinita, perché $\int_{-\infty}^{\infty} (h^2 - x^2)/(x^2 + h^2)^2 \, \mathrm{d}x =$

$[x/(x^2+h^2)]_{-\infty}^{\infty} = 0$; ma decade solo come $1/x^2$, e integrato su un tratto finito $[-X, X]$ vale $8\kappa' X/(X^2+h^2)$, che tende a zero lentamente.

4. Resta il termine quadratico. Con $x = h\tan\vartheta$:

$$\int_{-\infty}^{\infty} u'^2\,\mathrm{d}x = \frac{4\kappa'^2}{h^3}\int_{-\pi/2}^{\pi/2}\cos^2(2\vartheta)\cos^2\vartheta\,\mathrm{d}\vartheta = \frac{\pi\kappa'^2}{h^3} \quad\Longrightarrow\quad \frac{F_y}{\frac{1}{2}\rho U^2} = \frac{\pi\kappa'^2}{U^2h^3}.$$

Risultato

Con $\kappa' = 0{,}5$ e $h = 1{,}5$ (in unità di $U = 1$ e di una lunghezza di riferimento): $C_p^{\min} = -1{,}086$ sotto il corpo (Fig. B.4); integrando numericamente $-C_p$ su $[-15, 15]$, $[-150, 150]$ e $[-1500, 1500]$ si ottiene 0,497, 0,259 e 0,235, che convergono lentamente al valore esatto

$$\frac{F_y}{\frac{1}{2}\rho U^2} = \frac{\pi\kappa'^2}{h^3} = 0{,}2327.$$

La forza è positiva: la lastra è attratta verso il corpo. Al variare della quota: 0,785, 0,233, 0,098, 0,029 per $h = 1$, 1,5, 2, 3, in accordo con la legge $1/h^3$.

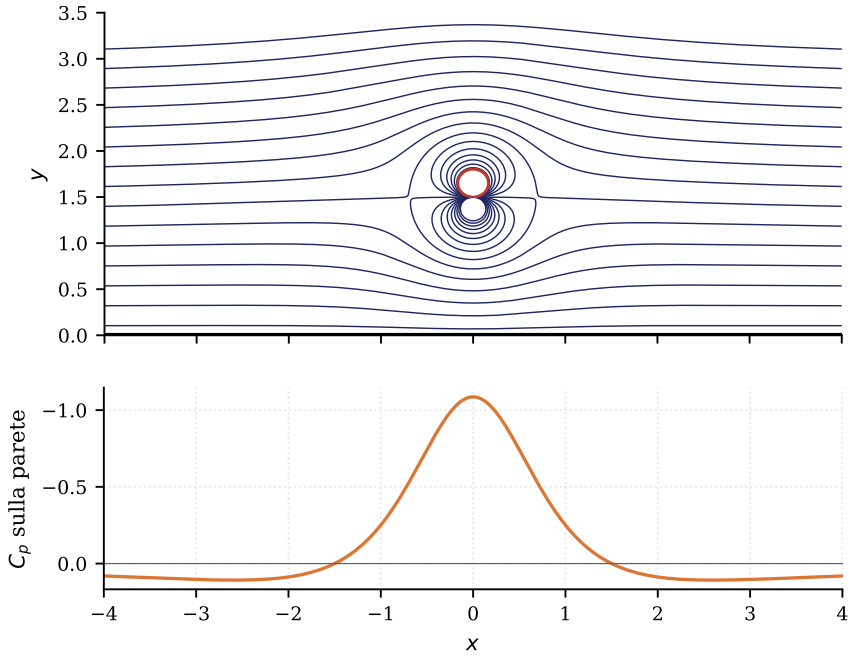


Figura B.4: Esercizio 3: corpo chiuso vicino a una lastra (in alto) e coefficiente di pressione sulla lastra (in basso).

Commento. Due insegnamenti. Fisico: un corpo che si muove parallelamente a una parete in un fluido ideale ne è *attratto* (è il motivo per cui due navi che procedono affiancate tendono ad avvicinarsi). Numerico: quando l'integrando decade lentamente, troncare l'intervallo di integrazione può dare un risultato sbagliato del 100% pur con una quadratura perfetta; bisogna separare analiticamente la parte che decade lentamente, come si è fatto qui.

B.4 Esercizio 4 – Il metodo di von Kármán con sorgenti puntiformi

Testo

Il metodo di von Kármán funziona anche se i segmenti di sorgenti sono sostituiti da sorgenti puntiformi discrete di intensità incognita. Scrivere un programma a questo scopo e usarlo per risolvere il problema della sfera del Programma 2.3.

Impostazione. Con n sorgenti puntiformi di intensità m_j in $z = z_j$ la funzione di corrente è $\psi = \frac{1}{2}Ur^2 - \sum_j m_j(z - z_j)/\sqrt{r^2 + (z - z_j)^2}$, eq. (4.4). Servono $n - 1$ punti di controllo sulla superficie più la chiusura $\sum m_j = 0$. Poiché la soluzione esatta è una doppietta puntiforme di momento $\frac{1}{2}Ua^3$, ci si aspetta che le intensità trovate abbiano momento $-\sum m_j z_j = \frac{1}{2}Ua^3$.

Svolgimento.

1. Sorgenti equispaziate fra $z = -0{,}6a$ e $0{,}6a$; punti di controllo equispaziati in angolo sulla sfera.
2. Sistema: $-\sum_j m_j(z_i - z_j)/R_{ij} = -\frac{1}{2}Ur_i^2$ per $i = 1, \ldots, n-1$ e $\sum_j m_j = 0$.
3. Velocità: $u_z = U + \sum_j m_j(z - z_j)/R_j^3$, $u_r = \sum_j m_j r/R_j^3$.

```
R = np.sqrt(r[:, None]**2 + (z[:, None] - zs[None, :])**2)
M = np.zeros((n, n))
M[:-1] = -(z[:, None] - zs[None, :]) / R        # psi di una sorgente: -m (z-zs)/R
M[-1] = 1.0                                       # somma delle intensita' nulla
b = np.concatenate([-0.5 * U * r**2, [0.0]])
m = np.linalg.solve(M, b)
```

esercizi_svolti.py

Risultato

n	$\max\lvert V - V_{\text{es}}\rvert/U$	$\text{cond}(M)$
4	$6{,}21 \times 10^{-2}$	$5{,}4 \times 10^{1}$
6	$4{,}84 \times 10^{-3}$	$1{,}4 \times 10^{3}$
10	$2{,}66 \times 10^{-5}$	$9{,}3 \times 10^{5}$

Con sei sorgenti, $m_j/U = 0{,}013;\ -0{,}151;\ 2{,}472;\ -2{,}472;\ 0{,}151;\ -0{,}013$, e il C_p è indistinguibile da quello esatto (Fig. B.5). Il momento di doppietta equivalente è $-\sum m_j z_j = 0{,}5000$, esattamente $\frac{1}{2}Ua^3$.

Commento. Le sorgenti puntiformi funzionano anche meglio dei segmenti, perché la soluzione esatta è essa stessa una singolarità concentrata; la struttura del risultato (coppia centrale intensa, correzioni alterne ai lati) e la crescita del numero di condizionamento sono le stesse del Programma 2.3, e hanno la stessa spiegazione (§4.5).

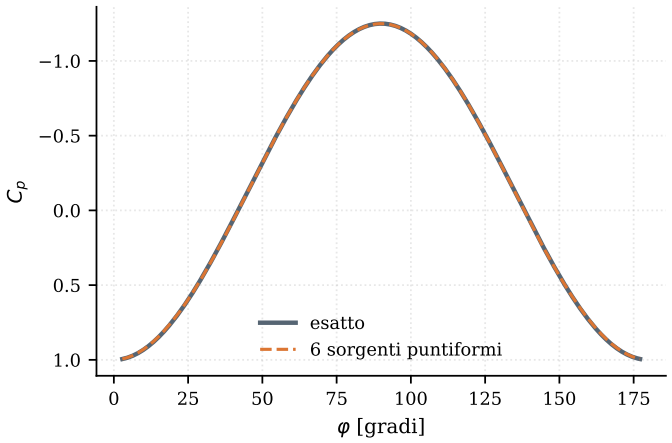


Figura B.5: Esercizio 4: coefficiente di pressione sulla sfera con sei sorgenti puntiformi.

B.5 Esercizio 5 – Progetto: il metodo di von Kármán per un profilo NACA 0009

Testo

Estendere il metodo di von Kármán al flusso uniforme attorno a un corpo piano chiuso, simmetrico rispetto all'asse parallelo alla corrente, distribuendo sorgenti e pozzi lineari su piccoli intervalli dell'asse. Applicarlo al calcolo della velocità V e del coefficiente di pressione sulla superficie di un'ala a incidenza nulla con profilo simmetrico NACA 0009, e confrontare la velocità con i valori tabulati (ottenuti con un altro metodo):

x/c	0	0,05	0,1	0,2	0,3	0,4	0,5	0,6	0,7	0,8	0,9	1
y/c	0	0,027	0,035	0,043	0,045	0,044	0,040	0,034	0,027	0,020	0,011	0
V/U	0	1,140	1,144	1,137	1,119	1,100	1,082	1,061	1,043	1,018	0,982	0

Impostazione. Nel piano la sorgente lineare ha $\psi = \frac{\sigma}{2\pi}\theta$; su un segmento $[\xi_j, \xi_{j+1}]$ con densità q_j bisogna integrare l'angolo $\theta = \arctan[y/(x-\xi)]$ rispetto a ξ. Conviene riscriverlo come $\operatorname{arccot}[(x-\xi)/y]$, perché per $y > 0$ varia con continuità in $(0, \pi)$. Il profilo è sottile: per la teoria dei profili sottili la densità di sorgenti sulla corda è circa $q \simeq 2U\,\mathrm{d}y_t/\mathrm{d}x$, un buon controllo per il risultato.

Svolgimento.

1. Con $s = x - \xi$, si verifica derivando che una primitiva di $\operatorname{arccot}(s/y)$ rispetto a s è $G(s) = s\,\theta + \frac{y}{2}\ln(s^2 + y^2)$. Poiché $\mathrm{d}\xi = -\mathrm{d}s$,

$$\psi_j(x,y) = \frac{q_j}{2\pi}\int_{\xi_j}^{\xi_{j+1}} \theta\,\mathrm{d}\xi = \frac{q_j}{2\pi}\big[G(x-\xi_j) - G(x-\xi_{j+1})\big].$$

2. Condizioni: $Uy_i + \sum_j \psi_j(x_i, y_i) = 0$ in $n-1$ punti del dorso (ascisse dei nodi interni dei segmenti, ordinate dalla formula dello spessore NACA) più la chiusura $\sum_j q_j s_j = 0$.

3. Velocità indotta da un segmento: $u = \frac{q_j}{4\pi}\ln\frac{(x-\xi_j)^2+y^2}{(x-\xi_{j+1})^2+y^2}$, $v = \frac{q_j}{2\pi}\big[\theta(x-\xi_{j+1}) - \theta(x-\xi_j)\big]$.

4. Si confrontano segmenti uniformi e segmenti addensati agli estremi con la spaziatura a coseno $\xi = \frac{1}{2}(1 - \cos\beta)$, $\beta \in [0, \pi]$.

Risultato

n	spaziatura	cond(M)	$\max\lvert V - V_{\text{tab}}\rvert / U$
10	uniforme	$7{,}0 \times 10^{1}$	0,0784
10	coseno	$1{,}2 \times 10^{2}$	0,0097
20	uniforme	$5{,}1 \times 10^{2}$	0,0268
20	coseno	$1{,}0 \times 10^{3}$	0,0041
40	uniforme	$1{,}6 \times 10^{4}$	0,0046
40	coseno	$4{,}0 \times 10^{4}$	0,0043

Con venti segmenti a coseno la velocità coincide con quella tabulata entro $0{,}004\,U$ in tutti i punti (ad esempio 1,136, 1,145, 1,135, 1,118, 1,099 contro 1,140, 1,144, 1,137, 1,119, 1,100 per $x/c = 0{,}05 \ldots 0{,}4$); le intensità seguono da vicino la stima dei profili sottili $2\,\mathrm{d}y_t/\mathrm{d}x$ (Fig. B.6). Il C_p si ottiene come $1 - (V/U)^2$: il minimo è $-0{,}31$ verso $x/c = 0{,}1$.

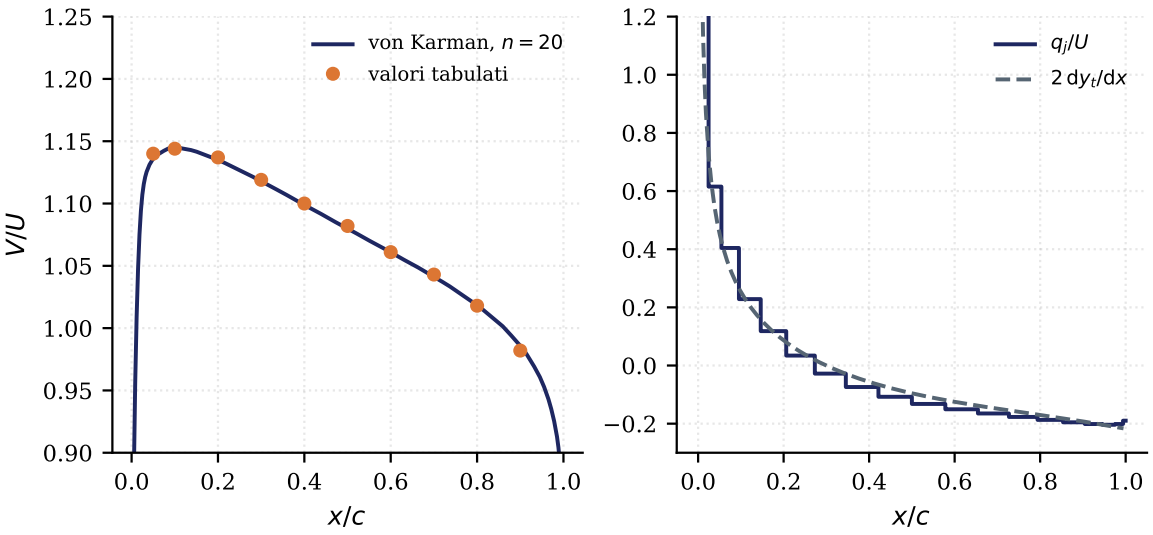


Figura B.6: Esercizio 5: velocità sul dorso del NACA 0009 e intensità dei segmenti di sorgenti confrontate con la teoria dei profili sottili.

Commento. Il metodo funziona bene perché il NACA 0009 è sottile e il suo bordo d'attacco arrotondato è "visto" dalle sorgenti di un segmento molto corto posto proprio lì (spaziatura a coseno). Lo scarto residuo, $0{,}004\,U$, è dello stesso ordine di quello del metodo dei pannelli (§5.6) e delle approssimazioni con cui sono stati calcolati i valori tabulati. Per profili spessi o con il bordo d'attacco molto arrotondato il metodo sull'asse perde accuratezza, per i motivi visti nel §4.5.

B.6 Esercizio 6 – La lastra piana senza circolazione

Testo

La trasformazione $z = \zeta + 1/\zeta$ porta il cerchio $|\zeta| = 1$ nel segmento dell'asse x fra $x = -2$ e $x = 2$, e l'esterno del cerchio in tutto il piano z. Il potenziale complesso $w = U\left(\zeta e^{-i\alpha} + e^{i\alpha}/\zeta\right)$ rappresenta una corrente uniforme di velocità U, inclinata di α, attorno a un cilindro di raggio unitario. Tracciare le linee di corrente del flusso trasformato, che è quello attorno a una lastra piana all'incidenza α, e la distribuzione di pressione sul dorso e sul ventre della lastra.

Impostazione. I punti critici della trasformazione sono $\zeta = \pm 1$, immagini dei due spigoli della lastra. Senza circolazione i punti di ristagno sul cerchio sono in $\varphi = \alpha$ e $\varphi = \pi + \alpha$, non nei punti critici: per la (5.8) la velocità sarà infinita in entrambi gli spigoli. Il flusso è antisimmetrico e la portanza nulla.

Svolgimento. Le linee di corrente si tracciano nel piano ζ (curve di livello di $\operatorname{Im} w$ con `contour`) e se ne trasformano i punti; la velocità sulla lastra è $|\mathrm{d}w/\mathrm{d}\zeta|/|1 - 1/\zeta^2|$ su $\zeta = e^{i\varphi}$.

Risultato

Con $\alpha = 10°$ i ristagni sul cerchio sono in $\varphi = 10°$ e $190°$, cioè in $x = \pm 1{,}970$ sulla lastra: quello posteriore sul dorso, a monte dello spigolo d'uscita, quello anteriore sul ventre (Fig. B.7). La corrente aggira gli spigoli con velocità infinita: il C_p tende a $-\infty$ a entrambe le estremità. La distribuzione è antisimmetrica e, in accordo con il teorema di Kutta–Joukowski con $\Gamma = 0$, la portanza è nulla.

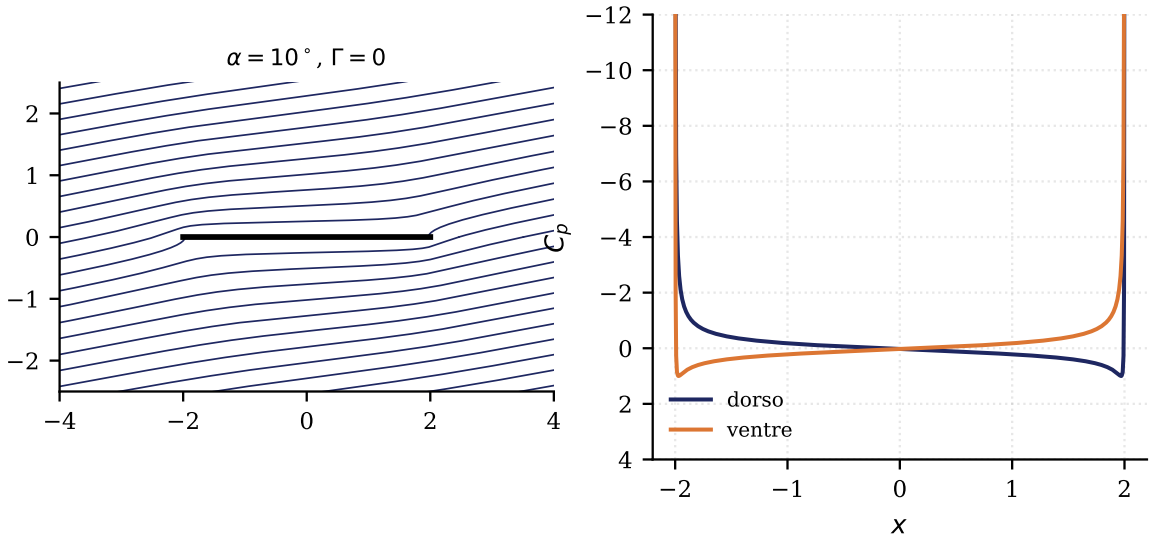


Figura B.7: Esercizio 6: lastra piana a $\alpha = 10°$ senza circolazione; linee di corrente e C_p.

Commento. È il flusso che si stabilisce nei primissimi istanti dopo la messa in moto di un'ala: la corrente gira attorno al bordo d'uscita, la velocità vi è altissima e lo strato limite si separa.

Il vortice di avviamento che ne nasce e si stacca lascia sull'ala la circolazione dell'esercizio successivo.

B.7 Esercizio 7 – La lastra piana con la condizione di Kutta

Testo

Per soddisfare la condizione di Kutta sulla lastra dell'esercizio precedente serve una circolazione che porti il ristagno posteriore del cerchio nel punto $(1, 0)$, immagine del bordo d'uscita. Tracciare linee di corrente e distribuzione di pressione e confrontarle con quelle senza circolazione.

Impostazione. La (5.13) con $a = 1$ e $\beta = 0$ dà $\Gamma = 4\pi U \sin\alpha$; con la corda $c = 4$, $C_L = 2\Gamma/(Uc) = 2\pi \sin\alpha$. Il bordo d'uscita ha velocità finita, il bordo d'attacco resta singolare. C'è però un'insidia: la pressione agisce perpendicolarmente alla lastra e produce solo una forza *normale*, mentre il teorema di Kutta–Joukowski dà una forza perpendicolare alla corrente. Dove si nasconde la differenza?

Svolgimento.

1. Si aggiunge al potenziale il vortice $\mathrm{i}\Gamma/(2\pi)\ln\zeta$ e si ripete il calcolo.
2. Coefficiente di forza normale dalla pressione: $C_N = \frac{1}{c}\int (C_p^{\text{ventre}} - C_p^{\text{dorso}})\,\mathrm{d}x$; con $x = 2\cos\varphi$ l'integrando ha al bordo d'attacco una singolarità integrabile.
3. Si confronta C_N con C_L.

Risultato

α [°]	$C_L = 2\pi\sin\alpha$	C_N (integrale)	$2\pi\sin\alpha\cos\alpha$	$C_S = 2\pi\sin^2\alpha$	$C_N\cos\alpha + C_S\sin\alpha$
2,0	0,219 28	0,219 14	0,219 15	0,007 65	0,219 28
5,0	0,547 62	0,545 53	0,545 53	0,047 73	0,547 61
10,0	1,091 06	1,074 48	1,074 49	0,189 46	1,091 05

La corrente lascia il bordo d'uscita tangente alla lastra (Fig. B.8); la pressione dà la forza normale $C_N = 2\pi\sin\alpha\cos\alpha$, e la portanza $2\pi\sin\alpha$ si ritrova solo aggiungendo la componente di portanza di una forza di **suzione** $C_S = 2\pi\sin^2\alpha$, diretta lungo la lastra verso il bordo d'attacco.

Commento. La suzione del bordo d'attacco è una forza concentrata, che nasce dalla depressione infinita (ma integrabile in una forza finita) attorno allo spigolo: nessuna integrazione della pressione su una lastra di spessore nullo la può "vedere". È lei a rendere nulla la resistenza della lastra ($C_N\sin\alpha = C_S\cos\alpha$), in accordo con il paradosso di d'Alembert. Su un profilo reale la suzione si distribuisce sul bordo d'attacco arrotondato; se il bordo è affilato e la corrente si separa, la suzione si perde e compare la resistenza: è uno dei motivi per cui le ali dei velivoli subsonici hanno il bordo d'attacco arrotondato, e quelle a delta dei velivoli supersonici sfruttano invece il vortice che si forma separandosi dallo spigolo.

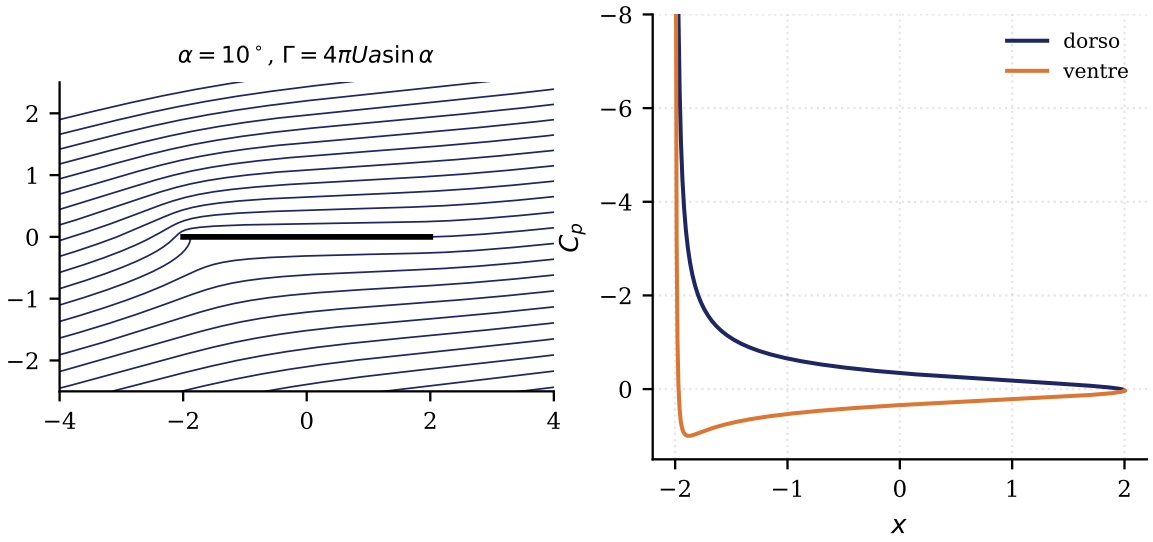


Figura B.8: Esercizio 7: lastra piana a $\alpha = 10^\circ$ con la circolazione di Kutta.

B.8 Esercizio 8 – SOR contro Liebmann

Testo

Ripetere il Programma 2.5 sostituendo la formula di Liebmann con quella del sovrarilassamento successivo, e confrontare l'efficienza dei due metodi in base al numero di iterazioni necessarie per una soluzione della stessa accuratezza.

Impostazione. Dal §6.8 ci si aspetta un fattore di guadagno di circa sei con $\omega_{\text{ott}} = 1{,}715$. Va però chiarito che cosa significa "stessa accuratezza": con il criterio del Programma 2.5 (somma delle variazioni), un metodo che converge più velocemente si ferma con un errore vero minore.

Svolgimento. Si usa `sor_passo` con il criterio `ERRMAX` $= 0{,}001$ e si misura l'errore vero rispetto alla soluzione diretta del sistema discreto.

Risultato

metodo	ω	iterazioni	errore vero massimo
Liebmann	1,000	153	$2{,}34 \times 10^{-4}$
SOR, formula (6.13)	1,715	42	$3{,}26 \times 10^{-5}$
SOR	1,500	87	$7{,}00 \times 10^{-5}$
SOR	1,900	112	$1{,}56 \times 10^{-5}$

Con il fattore ottimo il SOR richiede 42 iterazioni invece di 153 e si ferma con un errore sette volte più piccolo; a parità di errore vero il guadagno è il fattore sei della Fig. 6.4.

Commento. Il criterio basato sulla variazione fra due iterazioni è "generoso" con i metodi lenti e "severo" con quelli veloci, perché la variazione sottostima l'errore di un fattore $1/(1-\varrho)$: 36 per Liebmann, 3,5 per il SOR ottimo. Per confrontare metodi diversi conviene sempre misurare l'errore vero o il residuo.

B.9 Esercizio 9 – Due vortici nel rettangolo

Testo

Ripetere il Programma 2.5 aggiungendo nel rettangolo un'altra vorticità concentrata. Se le due vorticità hanno segni opposti, i due vortici risultano confinati in due regioni separate da un ramo della linea $\psi = 0$; se hanno lo stesso segno, le due regioni sono collegate da una linea di corrente a forma di otto, circondata dalle altre linee.

Impostazione. L'equazione è lineare: la soluzione è la somma delle soluzioni dei due vortici presi separatamente. Con i vortici in $(\mp 1, 0)$ e segni opposti, la soluzione è antisimmetrica rispetto all'asse $x = 0$, che è quindi la linea $\psi = 0$. Con lo stesso segno è simmetrica, e nel punto medio c'è un *punto di sella* di ψ, dove la velocità è nulla: la linea di corrente che vi passa è la separatrice a otto.

Svolgimento. Griglia di passo 0,125, vortici in $(-1, 0)$ e $(1, 0)$ con la stessa circolazione del Programma 2.5; soluzione con il solutore diretto sparso di `prog2_5b`. Nel caso di segno uguale si legge il valore di ψ nel punto di sella e lo si traccia in rosso.

Risultato

Segni opposti: la linea $\psi = 0$ comprende il contorno e la mezzeria $x = 0$, e separa due regioni di rotazione opposta. Stesso segno: nel punto di sella $(0, 0)$ si ha $\psi = 1{,}867$; la linea di corrente con questo valore ha la forma di un otto che racchiude i due vortici, e le linee esterne li circondano entrambi (Fig. B.9).

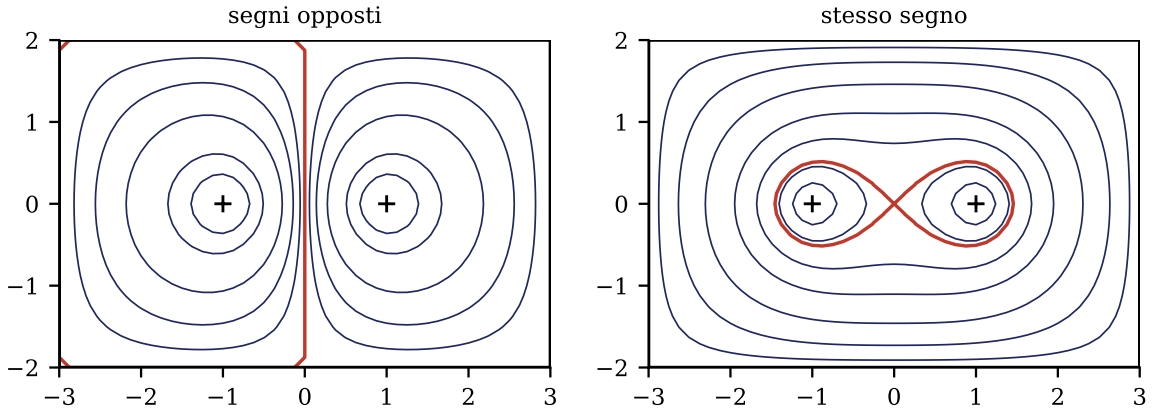


Figura B.9: Esercizio 9: due vortici di segno opposto (a sinistra) e dello stesso segno (a destra); in rosso le separatrici.

Commento. I due casi sono l'analogo confinato di due configurazioni classiche: la coppia di vortici controrotanti (come i vortici di estremità di un'ala, che scendono insieme) e la coppia di vortici corotanti (come quelli che si formano all'estremità di un flap e di un'ala vicini, che ruotano l'uno attorno all'altro e poi si fondono).

B.10 Esercizio 10 – Un cilindro a sezione rettangolare in un canale

Testo

Calcolare e tracciare il flusso in un canale attorno a un cilindro a sezione rettangolare (Fig. B.10). Il flusso potenziale si suppone simmetrico rispetto agli assi x e y, quindi basta calcolare il quarto superiore sinistro; si suppone inoltre che la corrente che entra dalla sezione AB sia uniforme. Condizioni al contorno: $\psi = 0$ lungo AFED, $\psi = 1$ lungo BC, $\psi = y/4$ lungo AB, $\partial\psi/\partial x = 0$ lungo CD. Passo $h = 0{,}25$.

Segnaposto figura

file atteso: `immagini/fig_2_9_7.png`

Canale di semialtezza 4 con un cilindro rettangolare 4×2 centrato nell'origine; quarto di calcolo ABCDEF: A$(-6,0)$, B$(-6,4)$, C$(0,4)$, D$(0,1)$, E$(-2,1)$, F$(-2,0)$ (Fig. 2.9.7 della fonte).

Figura B.10: Esercizio 10: canale con un cilindro a sezione rettangolare.

Impostazione. Il contorno solido coincide con le linee della griglia: non servono bracci disuguali. CD è la sezione di mezzeria, dove per simmetria la corrente è orizzontale: si tratta con i nodi fittizi, come l'uscita del Programma 2.7. La funzione di corrente è normalizzata con UH, con $H = 4$ semialtezza del canale: in ingresso $u/U = H\,\partial\psi/\partial y = 1$. In mezzeria la sezione libera si riduce da 4 a 3, quindi la velocità media deve salire a $4/3$; negli spigoli del corpo il flusso potenziale ha velocità infinita.

Svolgimento.

1. Griglia $x \in [-6,0]$, $y \in [0,4]$: 25×17 nodi; nodi solidi quelli con $x \geq -2$ e $y \leq 1$.
2. SOR con $\omega_{\text{ott}} = 1{,}715$; nei nodi di CD: $\psi = \frac{1}{4}(2\psi_{i-1,j} + \psi_{i,j-1} + \psi_{i,j+1})$.
3. Velocità sulla faccia superiore DE con la differenza unilaterale del secondo ordine $\partial\psi/\partial y \simeq (-3\psi_0 + 4\psi_1 - \psi_2)/(2h)$.
4. Disegno del canale intero per simmetria.

Risultato

Il SOR converge in 44 iterazioni ($\max|\Delta\psi| < 10^{-6}$). Sulla faccia superiore del corpo la velocità decresce dallo spigolo verso la mezzeria: $u/U = 2{,}09$ in $x = -2$ (spigolo E: il valore è limitato solo dal passo della griglia), 1,56 in $x = -1{,}5$, 1,44 in $x = -1$, 1,39 in $x = 0$. La velocità media nella sezione di mezzeria è $4/3 = 1{,}333$, come richiesto dalla continuità; il profilo è più veloce vicino al corpo (1,38) che vicino alla parete (1,29). Le linee di corrente (Fig. B.11) si addensano attorno agli spigoli.

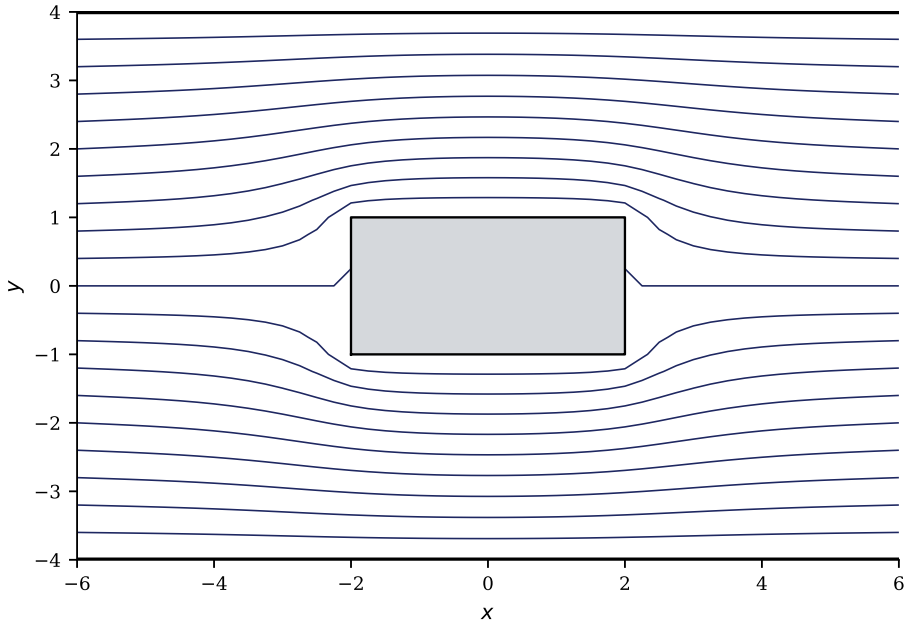


Figura B.11: Esercizio 10: flusso potenziale in un canale con un cilindro a sezione rettangolare (calcolato il quarto in alto a sinistra).

Commento. Il flusso potenziale aggira gli spigoli vivi con velocità infinita; un fluido reale non può farlo e si separa, formando ricircolazioni dietro lo spigolo anteriore e una scia a valle. Il calcolo non viscoso resta utile per stimare le velocità medie (il "bloccaggio" della sezione, che nelle gallerie del vento si corregge proprio con stime di questo tipo) ma non la distribuzione di pressione vicino agli spigoli.

B.11 Esercizio 11 – Flusso assialsimmetrico in un tubo con diaframmi

Testo

Mostrare che, su una griglia quadrata di passo h, l'equazione (4.2) della funzione di corrente assialsimmetrica si risolve con la formula iterativa (7.9), dove r_j è la distanza del nodo dall'asse. Con questo schema calcolare e tracciare il flusso nella regione OABCDO di un tubo assialsimmetrico con diaframmi ripetuti (Fig. B.12), con $h = 0{,}1$. Condizioni: $\psi = 0$ lungo OD, $\psi = 1$ lungo ABC, $\partial\psi/\partial z = 0$ lungo OA e CD, per la simmetria della funzione di corrente rispetto a queste due sezioni.

Impostazione. La regione è la metà di una cella periodica: OA è il foro del diaframma (piano di simmetria), AB il diaframma, BC la parete del tubo, CD il piano di mezzeria fra due diaframmi, OD l'asse. Ci si aspetta che il fluido acceleri per attraversare il foro, che ha un quarto della sezione del tubo, e che le linee di corrente si stringano verso il bordo del diaframma.

Svolgimento.

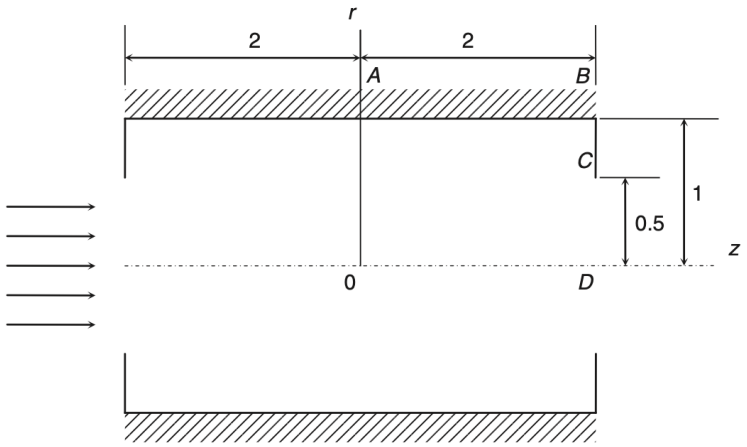


Figura B.12: Esercizio 11: flusso assialsimmetrico in un tubo con diaframmi ripetuti.

1. Con le differenze centrate (2.8)–(2.9) nel nodo (i, j) (i lungo z, j lungo r):

$$\frac{\psi_{i,j+1} - 2\psi_{i,j} + \psi_{i,j-1}}{h^2} - \frac{1}{r_j}\frac{\psi_{i,j+1} - \psi_{i,j-1}}{2h} + \frac{\psi_{i+1,j} - 2\psi_{i,j} + \psi_{i-1,j}}{h^2} = 0.$$

   Moltiplicando per h^2 e raccogliendo:

$$4\psi_{i,j} = \psi_{i-1,j} + \psi_{i+1,j} + \left(1 + \frac{h}{2r_j}\right)\psi_{i,j-1} + \left(1 - \frac{h}{2r_j}\right)\psi_{i,j+1},$$

   che è la (7.9). Per $r_j \to \infty$ si ritrova la formula piana.
2. Nodi fittizi nei piani di simmetria: in $z = 0$ (foro) $\psi_{-1,j} = \psi_{1,j}$, in $z = 2$ $\psi_{21,j} = \psi_{19,j}$.
3. SOR con $\omega = 1{,}8$, tolleranza $\max|\Delta\psi| < 10^{-7}$.
4. La velocità assiale sull'asse si ricava dal comportamento $\psi \simeq \frac{1}{2}u_z r^2$ per $r \to 0$: $u_z \simeq 2\psi_{i,1}/r_1^2$.

Risultato

Il SOR converge in 64 iterazioni. La velocità sull'asse vale 4,95 nel foro e 2,01 a metà cella, dove il flusso è quasi uniforme (la velocità media nel tubo pieno è $2\psi_{\text{parete}}/R^2 = 2$). Nel foro la velocità media è 8 (un quarto della sezione): sull'asse è minore della media perché, nel flusso potenziale, la velocità è massima, anzi infinita, sul bordo affilato del diaframma, dove le linee di corrente si addensano (Fig. B.13).

Commento. È il modello potenziale di un *diaframma tarato* (o di una serie di diaframmi), lo strumento più semplice per misurare la portata in un condotto o per creare una caduta di pressione controllata. Nel flusso reale il getto che esce dal foro si contrae (*vena contratta*) e si separa dal diaframma, e la caduta di pressione si stima con un coefficiente di efflusso sperimentale; il calcolo potenziale fornisce però il punto di partenza.

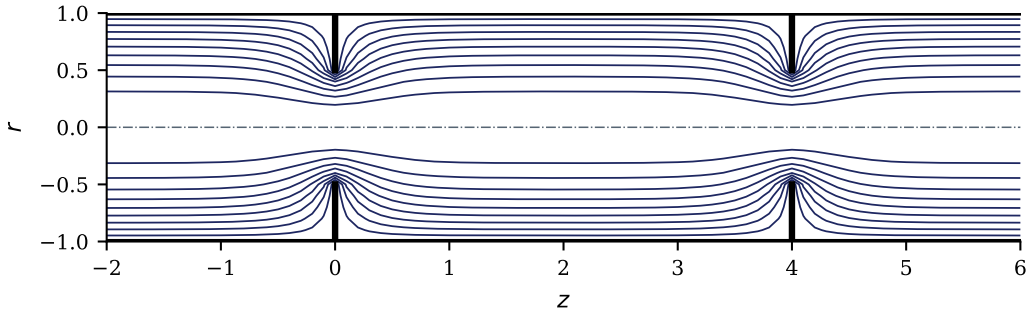


Figura B.13: Esercizio 11: linee di corrente nel tubo con diaframmi (due celle, riflesse rispetto all'asse).

B.12 Esercizio 12 – Progetto: il tubo d'urto linearizzato

Testo

Un tubo lungo 1 m, a sezione costante, chiuso a entrambi gli estremi, è diviso in due camere da un diaframma nella sezione di mezzo, attraverso il quale le densità sono leggermente diverse. La perturbazione di densità ρ' sia normalizzata in modo da valere 1 nel diaframma e alla sua destra, 0 alla sua sinistra. A $t = 0$, quando $\partial\rho'/\partial t = 0$, il diaframma viene rimosso. Risolvere il problema del tubo d'urto linearizzato calcolando $\rho'(x, t)$ con il metodo leapfrog e tracciare la distribuzione di densità in alcuni istanti rappresentativi. Mostrare che dopo la rottura del diaframma si generano un'onda di espansione che si propaga verso destra e un'onda di compressione verso sinistra; che su una parete rigida un'onda di espansione si riflette come espansione e una di compressione come compressione; che il tipo dell'onda cambia dopo la riflessione su un estremo aperto.

Impostazione. La perturbazione di densità obbedisce alla stessa equazione delle onde della velocità (§8). Le condizioni al contorno sono diverse: all'estremo chiuso $u = 0$ sempre, quindi $\partial u/\partial t = 0$ e, per l'equazione di Eulero linearizzata, $\partial p'/\partial x = 0$, cioè $\partial\rho'/\partial x = 0$; all'estremo aperto la densità è quella esterna, $\rho' = 0$. Le condizioni per ρ' sono dunque "scambiate" rispetto a quelle per u. Un'onda si dice di espansione o di compressione a seconda che viaggi verso una regione di densità maggiore o minore (abbassandola o alzandola).

Svolgimento.

1. $h = 0{,}01$ m, 101 nodi, Co $= 1$; partenza con la (8.11) ($g = 0$).
2. Estremi chiusi: nodi fittizi, $\rho'_{0,j+1} = 2\rho'_{1,j} - \rho'_{0,j-1}$ e analogamente a destra.
3. Variante (b): estremo destro aperto ($\rho' = 0$); per non introdurre un salto anche sull'estremo aperto, la camera ad alta densità è quella di sinistra.

Risultato

- (a) Estremi chiusi (Fig. B.14, a sinistra): il salto unitario si divide in due salti di ampiezza $\frac{1}{2}$. Quello che viaggia verso destra entra nella regione ad alta densità e la abbassa da 1 a $\frac{1}{2}$: è un'onda di *espansione*; quello verso sinistra entra nella regione a bassa densità e la alza a $\frac{1}{2}$: un'onda di *compressione*. Sulla parete destra

l'espansione si riflette come espansione (la densità scende a 0), su quella sinistra la compressione si riflette come compressione (sale a 1). A $t = 100\tau$ (cioè L/a) la distribuzione iniziale è invertita, a $t = 200\tau$ è tornata quella di partenza.

- (b) Estremo destro aperto: l'onda di compressione che raggiunge l'estremo aperto si riflette come onda di *espansione* (la densità torna verso 0 e poi scende sotto); dopo 200τ la distribuzione è quella iniziale cambiata di segno ($\rho'(0{,}25) = -1$), e il periodo diventa 400τ.

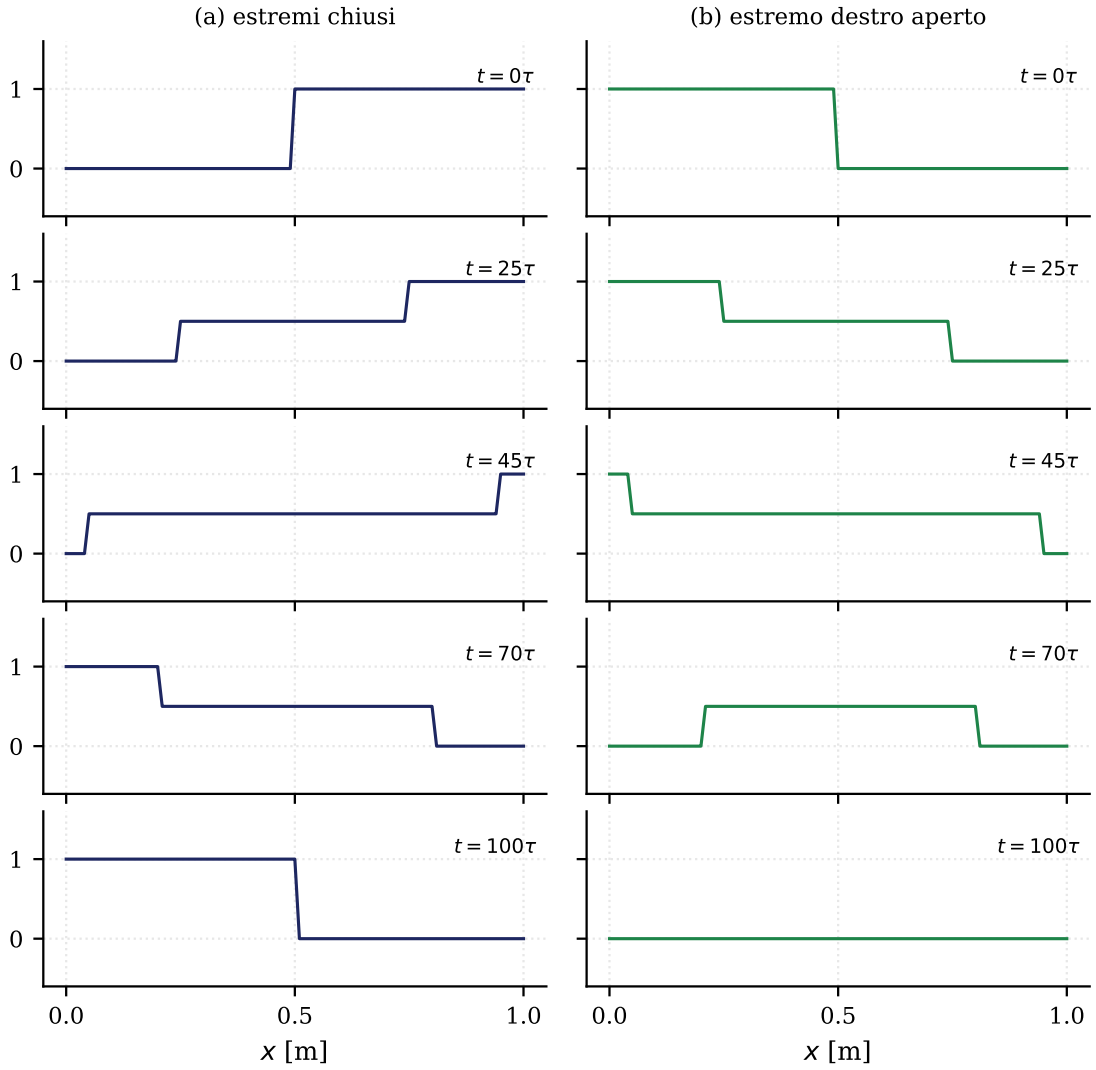


Figura B.14: Esercizio 12: tubo d'urto linearizzato con estremi chiusi (a sinistra) e con l'estremo destro aperto (a destra).

Commento. La conclusione sembra contraddire quella del Programma 2.8 (dove all'estremo chiuso la velocità cambia segno e all'estremo aperto no); la contraddizione scompare se il tipo di un'onda si definisce combinando il segno di u con il verso di propagazione [20]. In un'onda che viaggia verso destra compressione significa $u > 0$; dopo la riflessione su una parete l'onda viaggia verso sinistra e u ha cambiato segno: è ancora una compressione. Il problema del tubo d'urto "vero", non linearizzato, con l'urto e il ventaglio di espansione, è il banco di prova classico degli schemi per le equazioni di Eulero.

B.13 Esercizio 13 – Riflessione di onde di ampiezza finita

Testo

Nel calcolo del Programma 2.9 l'onda non risente delle condizioni al contorno, a meno di aggiungere altri passi. Per studiare gli effetti non lineari sulla riflessione, ripetere il Programma 2.9 per: (1) un'onda che si propaga verso sinistra con la stessa distribuzione iniziale di velocità; (2) un'onda che si propaga verso destra con lo stesso profilo, spostato di una lunghezza d'onda a destra.

Impostazione. Per un'onda verso sinistra la (9.10) va presa con il segno negativo, $a = a_0 - \frac{\gamma-1}{2}u$: dove $u > 0$ (il gas va verso destra, contro il verso dell'onda) il gas è rarefatto. Il caso (1) porta subito sulla parete chiusa la parte anteriore dell'onda, che è di espansione; nel caso (2) l'onda parte a ridosso dell'estremo aperto, con la parte anteriore ($u < 0$, anch'essa di espansione). In entrambi i casi l'urto si forma dopo circa 43τ nella zona centrale: si calcolano 40 passi.

Svolgimento. La funzione `cir` del Programma 2.9 accetta il profilo iniziale e il verso dell'onda (parametro `verso`); si registrano la velocità del suono sulla parete chiusa e la velocità del gas all'estremo aperto.

Risultato

- (1) Sulla parete chiusa $u = 0$ e la velocità del suono scende da 340 a 316, 299, 286, 285 m/s a $t = 10, 20, 30, 40\tau$: per l'isentropica la pressione sulla parete scende a 0,60, 0,40, 0,30, 0,29 volte quella iniziale. L'espansione riflessa si somma a quella incidente: l'abbassamento di a sulla parete, 55 m/s, supera già il massimo dell'onda incidente, 34 m/s (Fig. B.15, a sinistra).
- (2) All'estremo aperto $a = a_0$ e la velocità del gas passa da 0 a -110, -176, -216, -243 m/s, contro un minimo di -170 m/s nell'onda incidente: l'onda si riflette invertendo la pressione ma non la velocità, e le due velocità si sommano (Fig. B.15, a destra).

Commento. Nel caso lineare la riflessione su una parete raddoppia esattamente la perturbazione di pressione; nel caso non lineare le diverse parti dell'onda viaggiano a velocità diverse e la forma dell'onda riflessa è distorta. Per l'onda verso sinistra la cresta dell'espansione ($u = 170$ m/s, $a = 306$ m/s) si muove verso la parete con la velocità caratteristica $u - a = -136$ m/s, molto minore di a_0: dopo 40τ non l'ha ancora raggiunta, e il minimo di pressione sulla parete deve ancora arrivare. Per un'onda di compressione forte che colpisce una parete la sovrapressione riflessa supera il doppio di quella incidente: è il motivo per cui le onde d'urto riflesse sono così pericolose per le strutture.

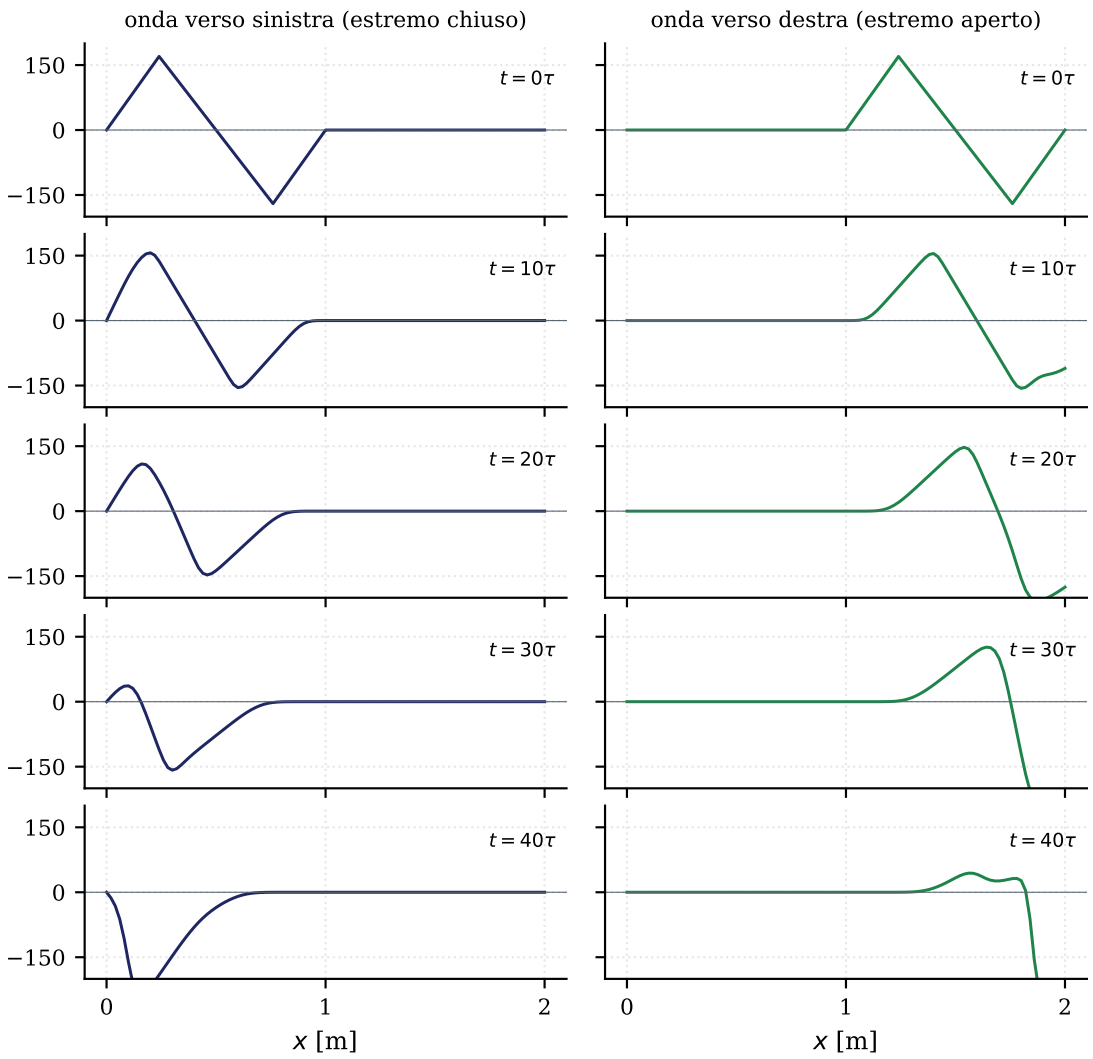


Figura B.15: Esercizio 13: riflessione di un'onda di ampiezza finita su un estremo chiuso (a sinistra) e su un estremo aperto (a destra).

B.14 Esercizio 14 – La soluzione esatta dell'equazione di Burgers

Testo

L'equazione di Burgers (10.14) è un modello per fenomeni che vanno dagli urti forti alla dinamica spettrale della turbolenza, e ha diverse soluzioni esatte. Verificare che la (10.16) è soluzione dell'equazione adimensionale (10.15), ricavarne il limite per $t \to 0$ e per $t \to \infty$ e la pendenza nell'origine.

Impostazione. Il calcolo a mano è lungo e soggetto a errori di segno: è un'occasione per usare il calcolo simbolico. La libreria `sympy` manipola espressioni matematiche esatte (derivate, semplificazioni, limiti).

Svolgimento.

```python
import sympy as sp
x, t = sp.symbols("x t", real=True)
u = -2 * sp.sinh(x) / (sp.cosh(x) - sp.exp(-t))
residuo = sp.simplify(sp.diff(u, t) + u * sp.diff(u, x) - sp.diff(u, x, 2))
print("u_t + u u_x - u_xx =", residuo)
print("limite per t -> infinito:", sp.limit(u, t, sp.oo))
print("pendenza in x = 0:  du/dx =", sp.simplify(sp.diff(u, x).subs(x, 0)))
phi = sp.exp(t) * sp.cosh(x) - 1              # trasformazione di Cole-Hopf
print(sp.simplify(sp.diff(phi, t) - sp.diff(phi, x, 2)))    # equazione del calore
print(sp.simplify(-2 * sp.diff(phi, x) / phi - u))          # u = -2 phi_x / phi
```

esercizi_svolti.py

Risultato

Il residuo $u_t + uu_x - u_{xx}$ si semplifica a 0: la (10.16) è soluzione esatta. Il limite per $t \to \infty$ è $(2 - 2e^{2x})/(e^{2x} + 1) = -2\tanh x$; per $t \to 0$ si ha $-2\sinh x/(\cosh x - 1)$, che diverge in $x = 0$. La pendenza nell'origine è $\partial u/\partial x = 2e^t/(1 - e^t) = -2/(1 - e^{-t})$. Infine $\varphi = e^t \cosh x - 1$ soddisfa l'equazione del calore e $-2\varphi_x/\varphi = u$: la soluzione nasce dalla trasformazione di Cole–Hopf.

Commento. Una soluzione esatta di un'equazione non lineare è preziosa per mettere alla prova i codici: negli esercizi seguenti la useremo per valutare due schemi. Il calcolo simbolico è il complemento naturale del calcolo numerico: serve a ricavare e verificare formule (coefficienti di uno schema, termini dell'errore di troncamento, soluzioni di riferimento) prima di programmarle.

B.15 Esercizio 15 – Burgers con lo schema di MacCormack

Testo

Integrare la (10.15) con lo schema esplicito di MacCormack (10.17), con la condizione iniziale ricavata dalla (10.16) per $t = 0$, 100 nodi equispaziati in $-6 \le x \le 6$ e condizioni di Neumann $\partial u/\partial x = 0$ in $x = \pm 6$, approssimate con differenze unilaterali del primo ordine. Scegliere il passo temporale in modo da rispettare sia il limite convettivo $u_{\max}\Delta t/\Delta x \le 1$ sia quello viscoso $\Delta t/\Delta x^2 \le \frac{1}{2}$, con $u_{\max}$ massima velocità a $t = 0$. Confrontare con la soluzione esatta a $t = 0{,}2$, $0{,}5$, 1, 2.

Impostazione. La condizione iniziale è singolare in $x = 0$; con 100 nodi l'origine non è un nodo (cade a metà fra due nodi) e $u_{\max}$ è finito ma grande, circa 66: il limite convettivo sarà il più severo. La diffusione smussa rapidamente la discontinuità: ci si aspetta un errore iniziale grande che si riduce nel tempo. Le condizioni di Neumann non sono quelle della soluzione esatta su un dominio finito, che ha in $x = \pm 6$ una pendenza piccola ma non nulla.

Svolgimento. Il passo è $\Delta t = 0{,}9 \min\left(\Delta x/u_{\max},\ \Delta x^2/2\right)$; nei nodi di estremità si pone $u_0 = u_1$ e $u_N = u_{N-1}$ dopo ciascuna fase. Il calcolo è svolto dalla funzione `maccormack_burgers` del programma `prog2_12_burgers.py`.

Risultato

$\Delta x = 0{,}121$, $u_{\max} = 66{,}0$: limite convettivo 0,00184, limite viscoso 0,00735, passo usato $\Delta t = 0{,}00165$.

t	$\max \lvert u - u_{\text{es}} \rvert$	$\max \lvert u - u_{\text{es}} \rvert$, $\lvert x \rvert < 5$
0,202	$2{,}555 \times 10^{-2}$	$2{,}555 \times 10^{-2}$
0,501	$1{,}174 \times 10^{-2}$	$6{,}671 \times 10^{-3}$
1,001	$1{,}573 \times 10^{-2}$	$1{,}079 \times 10^{-2}$
2,001	$1{,}908 \times 10^{-2}$	$1{,}707 \times 10^{-2}$

La soluzione numerica segue quella esatta in tutto il dominio (Fig. B.16). Raffinando la griglia, a $t = 1$, l'errore per $\lvert x \rvert < 2$ scende da $3{,}1 \cdot 10^{-3}$ (100 nodi) a $7{,}0 \cdot 10^{-4}$ (200 nodi), come atteso per uno schema del secondo ordine, poi si stabilizza ($6{,}5 \cdot 10^{-4}$ con 400 nodi); l'errore massimo, in $x = 6$, resta $1{,}5 \cdot 10^{-2}$ qualunque sia la griglia.

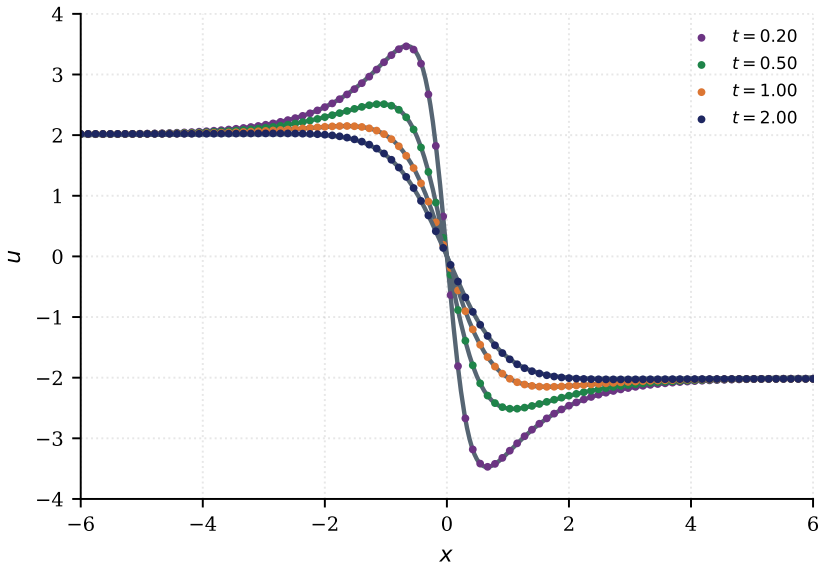


Figura B.16: Esercizio 15: equazione di Burgers con lo schema di MacCormack (punti) e soluzione esatta (linee grigie).

Commento. L'errore che non diminuisce con la griglia non è un errore dello schema ma del *modello*: le condizioni di Neumann sul dominio troncato non fissano l'ampiezza dello stato stazionario, che per le soluzioni del tipo $-2k\tanh(kx)$ resta libera, e la soluzione numerica tende a uno stato con k leggermente diverso da 1. È un esempio di come le condizioni al contorno di un dominio troncato possano dominare l'errore: prima di attribuire una discrepanza alla discretizzazione, conviene raffinare la griglia e vedere se l'errore cambia.

B.16 Esercizio 16 – Burgers con lo schema di Beam–Warming

Testo

Usare lo schema implicito di Beam–Warming (10.19) per risolvere la (10.15) con le stesse condizioni iniziali dell'esercizio 15 e 100 nodi, questa volta con condizioni di Dirichlet $u = \mp 2$ in $x = \pm 6$. Confrontare con la soluzione esatta a $t = 0{,}2,\ 0{,}5,\ 1,\ 2$. Qual è l'effetto di una viscosità artificiale come la (10.13)? E di Co > 1? Come si confronta la soluzione con quella dell'esercizio precedente?

Impostazione. Lo schema è implicito e non ha limiti di stabilità; l'accuratezza nei primi istanti, quando la soluzione cambia rapidamente, dipenderà però dal passo temporale. Le condizioni di Dirichlet fissano l'ampiezza dello stato stazionario, che con il valore ∓ 2 coincide con quello esatto ($-2\tanh 6 = -1{,}99998$): ci si aspetta di eliminare l'errore di contorno dell'esercizio 15.

Svolgimento. Funzione `beam_warming_burgers`: a ogni passo si assembla il sistema tridiagonale (coefficienti $-\Delta t\, u^n_{i-1}/(4\Delta x) - \frac{1}{2}d$, $1 + d$, $\Delta t\, u^n_{i+1}/(4\Delta x) - \frac{1}{2}d$; termine noto $u^n_i + \frac{1}{2}d\,\delta^2_x u^n_i$, più l'eventuale dissipazione) e lo si risolve con l'algoritmo di Thomas.

Risultato

Δt	Co iniziale	μ_e	errore massimo a $t =$			
			0,2	0,5	1	2
0,002	1,09	0,00	$1{,}74 \times 10^{-2}$	$6{,}20 \times 10^{-3}$	$4{,}81 \times 10^{-3}$	$3{,}74 \times 10^{-3}$
0,010	5,45	0,00	$9{,}57 \times 10^{-3}$	$6{,}20 \times 10^{-3}$	$4{,}81 \times 10^{-3}$	$3{,}67 \times 10^{-3}$
0,010	5,45	0,10	$1{,}33 \times 10^{-2}$	$6{,}20 \times 10^{-3}$	$4{,}81 \times 10^{-3}$	$4{,}03 \times 10^{-3}$
0,050	27,2	0,00	8,14	1,18	$7{,}75 \times 10^{-2}$	$6{,}20 \times 10^{-3}$
MacCormack ($\Delta t = 0{,}0017$)			$2{,}56 \times 10^{-2}$	$1{,}17 \times 10^{-2}$	$1{,}57 \times 10^{-2}$	$1{,}91 \times 10^{-2}$

- Con $\Delta t = 0{,}01$ (Co iniziale 5,5, ben oltre il limite di uno schema esplicito) lo schema è stabile e accurato quanto con un passo cinque volte più piccolo (Fig. B.17, a sinistra), e quattro volte più accurato di MacCormack a $t = 2$, grazie alle condizioni di Dirichlet.
- La dissipazione artificiale non serve (l'equazione ha già la sua diffusione fisica) e con $\mu_e = 0{,}1$ peggiora leggermente l'accuratezza.
- Con $\Delta t = 0{,}05$ (Co $= 27$) lo schema resta stabile, ma nei primi istanti l'errore è enorme (8 a $t = 0{,}2$, Fig. B.17, a destra): il passo è troppo lungo per seguire il rapido smussamento della discontinuità iniziale. Lo stato stazionario viene comunque raggiunto correttamente (errore $6 \cdot 10^{-3}$ a $t = 2$).

Commento. Il confronto chiarisce quando conviene uno schema implicito: quando c'è diffusione (che impone agli schemi espliciti il severo limite $\Delta t \propto \Delta x^2$) e quando interessa la soluzione a regime più che il transitorio. La stabilità incondizionata non è una licenza: con Co $= 27$ il transitorio è sbagliato, anche se la soluzione finale è giusta. Rispetto alla convezione pura del Programma 2.11, qui la diffusione fisica smorza le onde corte e rende inutile la dissipazione artificiale.

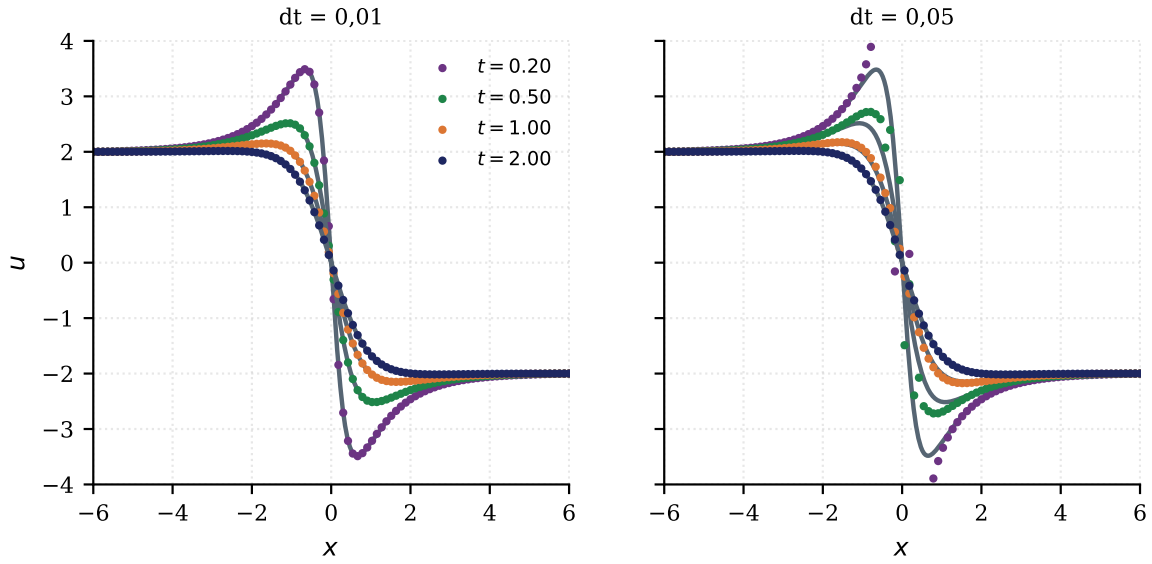


Figura B.17: Esercizio 16: schema di Beam–Warming con $\Delta t = 0{,}01$ e $\Delta t = 0{,}05$; linee grigie: soluzione esatta.

B.17 Esercizio 17 – Il tubo elastico: frequenza, ampiezza, attrito

Testo

Con il programma del Cap. 11, studiare l'influenza dei seguenti parametri: (a) frequenza della forzante (pressione prossimale) raddoppiata; (b) frequenza dimezzata; (c) ampiezza della forzante raddoppiata; (d) ampiezza dimezzata. Sostituire poi la formula laminare dell'attrito con quella turbolenta (11.8) e confrontare le risposte in ampiezza e frequenza di pressione e velocità. Calcolare almeno tre periodi della forzante.

Impostazione. Il tubo ha le sue frequenze proprie: un'onda impiega circa $25/350 \simeq 0{,}07$ s a percorrerlo, e per un tubo "aperto" a un'estremità (pressione imposta) e quasi chiuso all'altra (forte resistenza distale) la prima risonanza è circa $c/(4L) \simeq 3$–4 Hz. La frequenza di riferimento è 1,43 Hz; raddoppiandola ci si avvicina alla risonanza e ci si aspetta una forte amplificazione. Con l'ampiezza doppia gli effetti non lineari (convezione, rigidezza che cresce con la pressione) diventano importanti; con l'ampiezza dimezzata la risposta dovrebbe essere quasi lineare.

Svolgimento. La funzione `simula` accetta periodo, pressioni minima e massima della forzante e tipo di attrito; per ogni caso si calcolano sei periodi e si analizza l'ultimo. Le ampiezze si variano attorno alla stessa pressione media di 70 mmHg.

Risultato: pressione e velocità in $z = 20$ cm

caso	p [mmHg]	V [cm/s]	portata [cm^3/s]
riferimento ($T = 0{,}7$ s, 55–85 mmHg)	44,8 – 97,2	15,6 – 73,3	1,327
(a) frequenza doppia	26,6 – 170,7	−12,0 – 294,3	1,452
(b) frequenza dimezzata	52,3 – 83,7	25,5 – 43,1	1,313
(c) ampiezza doppia (40–100 mmHg)	20,9 – 192,5	−9,3 – 297,2	1,299
(d) ampiezza dimezzata (62,5–77,5 mmHg)	57,4 – 79,3	26,6 – 45,7	1,381
attrito turbolento	43,1 – 98,7	25,3 – 86,8	1,796

- (a) Con la frequenza doppia l'oscillazione di pressione in $z = 20$ cm passa da 52 a 144 mmHg e compaiono fronti ripidissimi, quasi degli urti (Fig. B.18): la forzante è vicina alla prima risonanza del tubo e le onde riflesse si sommano a quelle incidenti.
- (b) Con la frequenza dimezzata l'oscillazione di pressione in $z = 20$ cm è di 31 mmHg, praticamente uguale a quella imposta all'ingresso (30 mmHg): a bassa frequenza le onde riflesse non si sommano in fase con quelle incidenti e il tubo segue la forzante quasi staticamente; la velocità oscilla poco e non si inverte mai.
- (c)–(d) La risposta non è proporzionale all'ampiezza: dimezzandola l'oscillazione di pressione si riduce da 52 a 22 mmHg (meno della metà), raddoppiandola sale a 172 mmHg (più del triplo), con fronti d'urto. È l'effetto della non linearità: le creste di pressione viaggiano più veloci (la (11.5) cresce con p) e raggiungono i cavi, come nel Cap. 9.
- Con la formula turbolenta l'attrito nella parte distale, stretta, è minore di quello laminare, e la portata media sale del 35%.

Commento. La formula turbolenta (11.8) è di tipo Blasius, valida per numeri di Reynolds superiori a qualche migliaio; nella parte distale del tubo, con un diametro di circa 1 mm e velocità dell'ordine di 1 m/s, $Re = VD/\nu \simeq 300$ e il moto è certamente laminare: usarla lì significa sottostimare l'attrito. È un buon esempio di modello applicato fuori dal suo campo di validità, che dà numeri plausibili ma sbagliati. Quanto ai fronti ripidi dei casi (a) e (c), la loro esistenza nelle arterie reali è documentata (il "colpo di pistola" che si ausculta in alcune patologie valvolari), ma in un calcolo numerico vanno trattati con cautela: la dissipazione artificiale $\mu_e = 0{,}04$ ne controlla la stabilità, e la loro forma dipende dalla griglia.

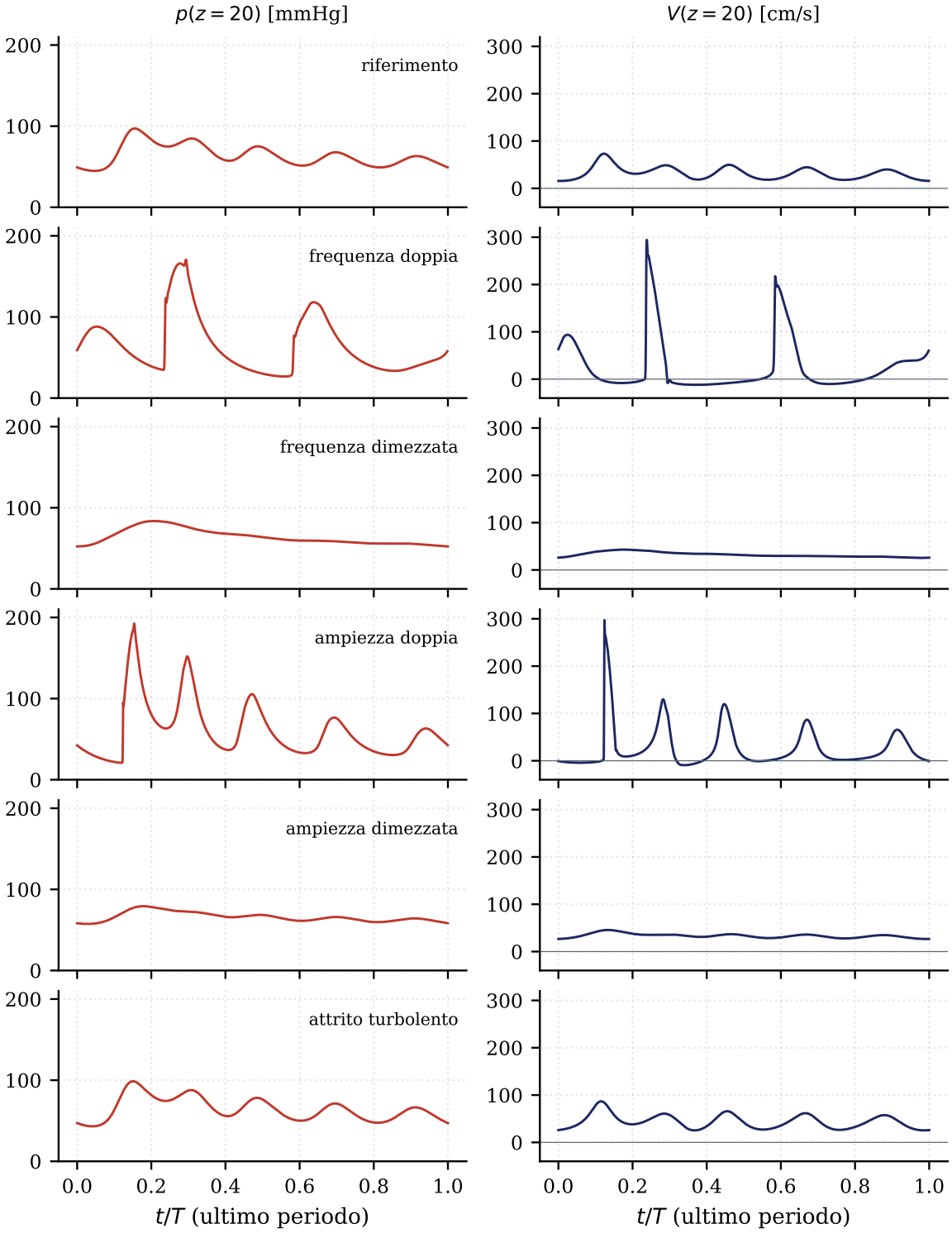


Figura B.18: Esercizio 17: pressione e velocità in $z = 20$ cm nell'ultimo periodo calcolato, per i sei casi.

B.18 Listato completo

esercizi_svolti.py

```
"""
esercizi_svolti.py -- codice degli esercizi svolti della monografia 2.
Ogni esercizio e' la funzione es_NN(); senza argomenti si eseguono tutti,
altrimenti solo quelli indicati:   python3 esercizi_svolti.py 3 7 12
Le figure vengono salvate come figure/es_NN*.pdf.
"""
import sys
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import brentq
from stile_grafici import salva, BLU, ARANCIO, ROSSO, GRIGIO, VERDE, VIOLA, GIALLO
from metodi_numerici import cerca_x, sor_passo, gauss_seidel_passo, omega_ottimo,
    thomas


def titolo(n, testo):
    print("\n" + "=" * 72 + f"\nEsercizio {n} -- {testo}\n" + "=" * 72)


# ============================================================ esercizio 1
def es_01():
    """Combinazioni di flussi elementari (metodo inverso)."""
    titolo(1, "combinazioni di flussi elementari")
    U = 1.0
    x = np.linspace(-3, 3, 601); y = np.linspace(-2, 2, 401)
    X, Y = np.meshgrid(x, y, indexing="ij")
    th = lambda X, Y, x0=0.0: np.arctan2(Y, X - x0) / np.pi      # theta/pi in (-1,
    1]
    sig = 2.0                                                    # sigma/(U l)
    kap = 0.6                                                    # kappa/(2 pi U l^2)
    with np.errstate(divide="ignore", invalid="ignore"):
        casi = {
            "(a) corrente + sorgente": U * Y + 0.5 * sig * th(X, Y),
            "(b) corrente + pozzo": U * Y - 0.5 * sig * th(X, Y),
            "(c) sorgente in $(-1,0)$ + doppietta": 0.5 * sig * th(X, Y, -1.0)
                                                    - kap * Y / (X**2 + Y**2),
        }
    fig, axs = plt.subplots(1, 3, figsize=(6.8, 2.6))
    for ax, (nome, P) in zip(axs, casi.items()):
        ax.contour(X, Y, P, levels=np.linspace(-2.5, 2.5, 21), colors=BLU,
                   linewidths=0.6, linestyles="solid")
        ax.set_aspect("equal"); ax.set_title(nome, fontsize=7.5); ax.grid(False)
        ax.set_xticks([-3, 0, 3]); ax.set_yticks([-2, 0, 2])
    # linee divisorie psi = +-sigma/2 (contorno del semicorpo; arctan normalizzato
    con pi)
    for ax, P in zip(axs[:2], list(casi.values())[:2]):
        ax.contour(X, Y, P, levels=[-1.0, 1.0], colors=ROSSO, linewidths=1.2,
                   linestyles="solid")
    axs[0].plot(0, 0, "o", color=ROSSO, ms=3); axs[1].plot(0, 0, "o", color=ROSSO,
    ms=3)
    axs[2].plot([-1, 0], [0, 0], "o", color=ROSSO, ms=3)
    salva(fig, "es_01a")
    xs = -sig / (2 * np.pi * U)
    print(f"(a) punto di ristagno x_s = -sigma/(2 pi U) = {xs:.4f}; semispessore a
    valle = "
          f"sigma/(2U) = {sig/2:.3f}")
    print(f"(b) ristagno in x = +{-xs:.4f} (a valle del pozzo)")
```

```
    # (c) ristagni sull'asse: u = sig/(2 pi (x+1)) + kap / x^2 ... (doppietta: u = -kap (x^2-y^2)/r^4)
    u_asse = lambda xx: sig / (2 * np.pi * (xx + 1.0)) - kap / xx**2
    xr = brentq(u_asse, 0.05, 5.0)
    print(f"(c) punto di ristagno sull'asse x > 0: x = {xr:.4f}")
    # (d) due doppiette uguali a distanza d
    k = 0.25
    fig, axs = plt.subplots(1, 3, figsize=(6.8, 2.4))
    print("(d) corrente + due doppiette di intensita' c = 0,25 in x = -+d/2:")
    for ax, d in zip(axs, (0.4, 1.0, 1.6)):
        with np.errstate(invalid="ignore"):
            P = Y - k * Y / ((X + d / 2)**2 + Y**2) - k * Y / ((X - d / 2)**2 + Y**2)
        ax.contour(X, Y, P, levels=np.linspace(-1.5, 1.5, 13), colors=BLU,
                   linewidths=0.6, linestyles="solid")
        ax.contour(X, Y, P, levels=[0.0], colors=ROSSO, linewidths=1.2)
        ax.plot([-d / 2, d / 2], [0, 0], "k+", ms=5)
        ax.set_aspect("equal"); ax.set_title(f"$d = {d}\\,l$", fontsize=8); ax.grid(False)
        ax.set_xticks([-3, 0, 3]); ax.set_yticks([-2, 0, 2])
        # spessore in x = 0: psi(0, y) = 0  ->  y^2 = 2c - d^2/4
        y0 = np.sqrt(max(2 * k - d**2 / 4, 0.0))
        print(f"   d = {d:.1f}: semispessore in x = 0 -> {y0:.4f}"
              f"   (corpo unico se d^2/4 < 2c, cioe' d < {2*np.sqrt(2*k):.3f})")
    salva(fig, "es_01b")


# ============================================================ esercizio 2
def es_02():
    """Metodo delle immagini: vortice vicino a una parete; sorgente e parete."""
    titolo(2, "metodo delle immagini")
    G, d = 2 * np.pi, 1.0
    x = np.linspace(-3, 3, 601); y = np.linspace(-3, 3, 601)
    X, Y = np.meshgrid(x, y, indexing="ij")
    with np.errstate(divide="ignore"):
        P1 = G / (2 * np.pi) * (np.log(np.hypot(X, Y - d)) - np.log(np.hypot(X, Y + d)))
    fig, axs = plt.subplots(1, 2, figsize=(6.4, 3.2))
    axs[0].contour(X, Y, P1, levels=np.linspace(-1.5, 1.5, 25), colors=BLU,
                   linewidths=0.6, linestyles="solid")
    axs[0].axhline(0, color=ROSSO, lw=1.5)
    axs[0].plot([0, 0], [d, -d], "o", color="k", ms=3)
    axs[0].set_title(r"vortici $\pm\Gamma$ in $(0,\pm d)$", fontsize=8)
    # la funzione di corrente della sorgente e' a piu' valori (taglio di arctan):
    # per il secondo caso si tracciano le linee di corrente dal campo di velocita'
    xs_ = np.linspace(-3, 0, 151); ys_ = np.linspace(-3, 3, 301)
    XS, YS = np.meshgrid(xs_, ys_)
    r1 = (XS + d)**2 + YS**2; r2 = (XS - d)**2 + YS**2
    with np.errstate(invalid="ignore", divide="ignore"):
        us = (XS + d) / r1 + (XS - d) / r2
        vs = YS / r1 + YS / r2
    semi = [(-d + 0.08 * np.cos(a), 0.08 * np.sin(a)) for a in np.linspace(0, 2 * np.pi, 24, endpoint=False)]
    axs[1].streamplot(XS, YS, us, vs, color=BLU, linewidth=0.6, density=1.2,
                      start_points=semi, arrowsize=0.6, broken_streamlines=False)
    axs[1].axvline(0, color=ROSSO, lw=1.5)
    axs[1].plot([-d, d], [0, 0], "o", color="k", ms=3)
    axs[1].plot([d], [0], "o", color=GRIGIO, ms=3, mfc="none")
    axs[1].set_xlim(-3, 3); axs[1].set_ylim(-3, 3)
    axs[1].set_title(r"sorgente in $(-d,0)$ e immagine in $(d,0)$", fontsize=8)
```

```
    for ax in axs:
        ax.set_aspect("equal"); ax.grid(False); ax.set_xticks([-3, 0, 3]); ax.set_yticks([-3, 0, 3])
    salva(fig, "es_02")
    # velocita' indotta dall'immagine sul vortice reale e velocita' sulla parete
    print(f"velocita' con cui il vortice si muove parallelamente alla parete: "
          f"Gamma/(4 pi d) = {G/(4*np.pi*d):.4f}")
    xw = np.array([0.0, 0.5, 1.0, 2.0])
    uw = G / np.pi * d / (xw**2 + d**2)
    print("velocita' sulla parete u(x,0) = Gamma d/(pi (x^2+d^2)):", np.round(uw, 4))
    print("sulla parete v = 0 per simmetria: la parete e' una linea di corrente (psi = 0)")


# ============================================================ esercizio 3
def es_03():
    """Corpo chiuso vicino a una lastra piana (doppietta + immagine)."""
    titolo(3, "doppietta e immagine: forza fra corpo e parete")
    U, kap, hd = 1.0, 0.5, 1.5        # kappa/(2 pi) e quota della doppietta
    x = np.linspace(-4, 4, 801); y = np.linspace(0, 3.5, 351)
    X, Y = np.meshgrid(x, y, indexing="ij")
    with np.errstate(invalid="ignore"):
        P = U * Y - kap * (Y - hd) / (X**2 + (Y - hd)**2) - kap * (Y + hd) / (X**2 + (Y + hd)**2)
    # velocita' sulla parete y = 0: u = dpsi/dy
    # velocita' sulla parete y = 0: u = U + u',  u' = 2 kap (h^2 - x^2)/(x^2 + h^2)^2
    up = lambda xx, hh: 2 * kap * (hh**2 - xx**2) / (xx**2 + hh**2)**2
    xw = np.linspace(-4, 4, 2001)
    cpw = 1 - (U + up(xw, hd))**2
    print(f"doppietta kappa/(2 pi) = {kap}, a quota h = {hd}")
    print(f"C_p minimo sulla parete = {cpw.min():.4f} in x = {xw[np.argmin(cpw)]:.3f}")
    # -C_p = 2u' + u'^2: il primo termine ha integrale nullo sulla lastra infinita
    # ma decade solo come 1/x^2: integrarlo su un tratto finito falsa il risultato
    for X_ in (15.0, 150.0, 1500.0):
        xx = np.linspace(-X_, X_, 600001)
        Fn = np.trapezoid(2 * up(xx, hd) + up(xx, hd)**2, xx)
        print(f"   integrale numerico di -C_p su [-{X_:g}, {X_:g}]: {Fn:.5f}")
    for hh in (1.0, 1.5, 2.0, 3.0):
        xx = np.linspace(-200, 200, 400001)
        F2 = np.trapezoid(up(xx, hh)**2, xx)
        print(f"   h = {hh:.1f}:  F_y/(rho U^2/2) = int u'^2 dx = {F2:.5f}"
              f"   (formula pi kap^2/h^3 = {np.pi*kap**2/hh**3:.5f})")
    print("=> la lastra e' attratta verso il corpo; sul corpo agisce la forza opposta")
    fig, axs = plt.subplots(2, 1, figsize=(5.6, 4.4), sharex=True,
                            gridspec_kw={"height_ratios": [1.6, 1]})
    axs[0].contour(X, Y, P, levels=np.linspace(0.1, 3.0, 15), colors=BLU,
                   linewidths=0.6, linestyles="solid")
    axs[0].contour(X, Y, P, levels=[0.0], colors=ROSSO, linewidths=1.2)
    axs[0].axhline(0, color="k", lw=2)
    axs[0].set_aspect("equal"); axs[0].set_ylim(0, 3.5); axs[0].grid(False)
    axs[0].set_ylabel("$y$")
    axs[1].plot(xw, cpw, color=ARANCIO); axs[1].invert_yaxis()
    axs[1].axhline(0, color=GRIGIO, lw=0.5)
    axs[1].set_xlim(-4, 4); axs[1].set_xlabel("$x$"); axs[1].set_ylabel("$C_p$ sulla parete")
    salva(fig, "es_03")
```

```
160
161
162 # ============================================================ esercizio 4
163 def es_04():
164     """von Karman con sorgenti puntiformi per la sfera."""
165     titolo(4, "sfera con sorgenti puntiformi sull'asse")
166     A, U = 1.0, 1.0
167     for n in (4, 6, 10):
168         zs = np.linspace(-0.6, 0.6, n)                    # posizioni delle sorgenti
169         fi = np.linspace(np.pi, 0, n + 1)[1:-1]           # n-1 punti di controllo
170         r, z = A * np.sin(fi), A * np.cos(fi)
171         R = np.sqrt(r[:, None]**2 + (z[:, None] - zs[None, :])**2)
172         M = np.zeros((n, n))
173         M[:-1] = -(z[:, None] - zs[None, :]) / R        # psi di una sorgente: -m
    (z-zs)/R
174         M[-1] = 1.0                                       # somma delle intensita'
    nulla
175         b = np.concatenate([-0.5 * U * r**2, [0.0]])
176         m = np.linalg.solve(M, b)
177         ft = np.linspace(0.05, np.pi - 0.05, 200)
178         rt, zt = A * np.sin(ft), A * np.cos(ft)
179         Rt = np.sqrt(rt[:, None]**2 + (zt[:, None] - zs[None, :])**2)
180         uz = U + ((zt[:, None] - zs[None, :]) / Rt**3) @ m
181         ur = (rt[:, None] / Rt**3) @ m
182         V = np.hypot(ur, uz)
183         err = np.max(np.abs(V - 1.5 * U * rt / A))
184         print(f"n = {n:2d}: intensita' m_j/U = {np.array2string(m, precision=4)}")
185         print(f"        max |V - V_esatta|/U sulla sfera = {err:.2e};  cond =
    {np.linalg.cond(M):.1e}")
186         if n == 6:
187             Vs, fs = V.copy(), ft.copy()
188     mu_eq = np.sum(m * zs) * -1
189     print(f"momento di doppietta equivalente -sum m_j z_j = {mu_eq:.4f}  (esatto U
    a^3/2 = 0,5)")
190     fig, ax = plt.subplots(figsize=(4.6, 3.0))
191     ax.plot(np.degrees(fs), 1 - 2.25 * np.sin(fs)**2, color=GRIGIO, lw=2,
    label="esatto")
192     ax.plot(np.degrees(fs), 1 - Vs**2, "--", color=ARANCIO, label="6 sorgenti
    puntiformi")
193     ax.invert_yaxis(); ax.set_xlabel(r"$\varphi$ [gradi]"); ax.set_ylabel("$C_p$");
    ax.legend()
194     salva(fig, "es_04")
195
196
197 # ============================================================ esercizio 5
198 def _naca00(x, t=0.09):
199     return 5 * t * (0.2969 * np.sqrt(x) - 0.1260 * x - 0.3516 * x**2
200                     + 0.2843 * x**3 - 0.1036 * x**4)
201
202
203 def _G2d(x, y, xi):
204     """Primitiva di int theta d(xi): s theta + (y/2) ln(s^2 + y^2), s = x - xi."""
205     s = x - xi
206     return s * np.arctan2(y, s) + 0.5 * y * np.log(s**2 + y**2)
207
208
209 def es_05():
210     """Progetto: metodo di von Karman piano per il profilo NACA 0009."""
211     titolo(5, "von Karman piano: NACA 0009 a incidenza nulla")
212     tab_x = np.array([0.05, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9])
```

```
    tab_V = np.array([1.140, 1.144, 1.137, 1.119, 1.100, 1.082, 1.061, 1.043, 1.018, 0.982])
    risultati = {}
    print("  n   spaziatura   cond(M)     max|V - V_tab|/U")
    for n in (10, 20, 40):
        for nome in ("uniforme", "coseno"):
            xx = np.linspace(0, 1, n + 1) if nome == "uniforme" \
                else 0.5 * (1 - np.cos(np.linspace(0, np.pi, n + 1)))
            xc = xx[1:-1]; yc = _naca00(xc)
            M = np.zeros((n, n))
            for j in range(n):
                M[:-1, j] = (_G2d(xc, yc, xx[j]) - _G2d(xc, yc, xx[j + 1])) / (2 * np.pi)
            M[-1] = np.diff(xx)
            q = np.linalg.solve(M, np.concatenate([-yc, [0.0]]))
            def velocita(x, y, xx=xx, q=q):
                u = np.ones_like(x); v = np.zeros_like(x)
                for j in range(len(q)):
                    u += q[j] / (4 * np.pi) * np.log(((x - xx[j])**2 + y**2) /
                                                     ((x - xx[j + 1])**2 + y**2))
                    v += q[j] / (2 * np.pi) * (np.arctan2(y, x - xx[j + 1]) - np.arctan2(y, x - xx[j]))
                return u, v
            u, v = velocita(tab_x, _naca00(tab_x))
            V = np.hypot(u, v)
            print(f"  {n:2d}   {nome:9s}   {np.linalg.cond(M):.1e}   {np.max(np.abs(V-tab_V)):.4f}")
            risultati[(n, nome)] = (xx, q, velocita)
    xx, q, velocita = risultati[(20, "coseno")]
    xf = np.linspace(0.005, 0.995, 300)
    u, v = velocita(xf, _naca00(xf)); Vf = np.hypot(u, v)
    u, v = velocita(tab_x, _naca00(tab_x)); Vt = np.hypot(u, v)
    print("\n  x/c    V/U tabella   V/U von Karman (n = 20, coseno)")
    for a, b, c in zip(tab_x, tab_V, Vt):
        print(f"  {a:4.2f}     {b:5.3f}          {c:6.4f}")
    fig, ax = plt.subplots(1, 2, figsize=(6.6, 2.8))
    ax[0].plot(xf, Vf, color=BLU, label="von Karman, $n=20$")
    ax[0].plot(tab_x, tab_V, "o", color=ARANCIO, ms=4, label="valori tabulati")
    ax[0].set_xlabel("$x/c$"); ax[0].set_ylabel("$V/U$"); ax[0].legend(fontsize=7)
    ax[0].set_ylim(0.9, 1.25)
    xm = 0.5 * (xx[1:] + xx[:-1])
    dydx = np.gradient(_naca00(xf), xf)
    ax[1].step(xx, np.concatenate([q, [q[-1]]]), where="post", color=BLU, label="$q_j/U$")
    ax[1].plot(xf, 2 * dydx, "--", color=GRIGIO, label=r"$2\,\mathrm{d}y_t/\mathrm{d}x$")
    ax[1].set_ylim(-0.3, 1.2); ax[1].set_xlabel("$x/c$"); ax[1].legend(fontsize=7)
    salva(fig, "es_05")


# ============================================================ esercizi 6 e 7
def _lastra(alfa, con_kutta, npunti=400):
    a, U = 1.0, 1.0
    gam = 4 * np.pi * a * U * np.sin(alfa) if con_kutta else 0.0
    fi = np.linspace(0, 2 * np.pi, npunti, endpoint=False) + 1e-4
    zeta = a * np.exp(1j * fi)
    dw = U * (np.exp(-1j * alfa) - a**2 * np.exp(1j * alfa) / zeta**2) + 1j * gam / (2 * np.pi * zeta)
    dz = 1 - 1 / zeta**2
    V = np.abs(dw / dz)
```

```
    x = (zeta + 1 / zeta).real
    return fi, x, V, gam


def _linee_lastra(ax, alfa, con_kutta):
    a, U = 1.0, 1.0
    gam = 4 * np.pi * a * U * np.sin(alfa) if con_kutta else 0.0
    r = np.linspace(1.0, 5.0, 300); t = np.linspace(0, 2 * np.pi, 721)
    Z = r[:, None] * np.exp(1j * t[None, :])
    with np.errstate(divide="ignore", invalid="ignore"):
        W = U * (Z * np.exp(-1j * alfa) + np.exp(1j * alfa) / Z) + 1j * gam / (2 * np.pi) * np.log(Z)
    figt, axt = plt.subplots()
    cs = axt.contour(Z.real, Z.imag, W.imag, levels=np.linspace(-3, 3, 25))
    for segs in cs.allsegs:
        for s in segs:
            zz = s[:, 0] + 1j * s[:, 1]
            zm = zz + 1 / zz
            ax.plot(zm.real, zm.imag, color=BLU, lw=0.6)
    plt.close(figt)
    ax.plot([-2, 2], [0, 0], color="k", lw=2.0)
    ax.set_xlim(-4, 4); ax.set_ylim(-2.5, 2.5); ax.set_aspect("equal"); ax.grid(False)


def es_06():
    """Lastra piana a incidenza, senza circolazione."""
    titolo(6, "lastra piana senza circolazione")
    alfa = np.radians(10.0)
    fi, x, V, gam = _lastra(alfa, False)
    cp = 1 - V**2
    dorso = np.sin(fi) > 0
    fig, ax = plt.subplots(1, 2, figsize=(6.8, 2.8))
    _linee_lastra(ax[0], alfa, False)
    ax[0].set_title(r"$\alpha = 10^\circ$, $\Gamma = 0$", fontsize=8)
    ax[1].plot(x[dorso], cp[dorso], color=BLU, label="dorso")
    ax[1].plot(x[~dorso], cp[~dorso], color=ARANCIO, label="ventre")
    ax[1].set_ylim(4, -12); ax[1].set_xlabel("$x$"); ax[1].set_ylabel("$C_p$"); ax[1].legend(fontsize=7)
    salva(fig, "es_06")
    # punti di ristagno sul cerchio: fi = alfa e fi = pi + alfa
    zs = np.exp(1j * np.array([alfa, np.pi + alfa]))
    print(f"ristagni sul cerchio in fi = {np.degrees(alfa):.1f} e {180+np.degrees(alfa):.1f} gradi")
    print(f"-> sulla lastra in x = {(zs + 1/zs).real.round(4)} (ristagno posteriore sul dorso)")
    print("velocita' infinita ai due spigoli (dz/dzeta = 0 in zeta = +-1)")
    # forza: per simmetria della distribuzione la risultante e' nulla
    print("Gamma = 0: per il teorema di Kutta-Joukowski la portanza e' nulla")


def es_07():
    """Lastra piana con condizione di Kutta: portanza e suzione al bordo d'attacco."""
    titolo(7, "lastra piana con condizione di Kutta")
    fig, axs = plt.subplots(1, 2, figsize=(6.8, 2.8))
    alfa = np.radians(10.0)
    _linee_lastra(axs[0], alfa, True)
    axs[0].set_title(r"$\alpha = 10^\circ$, $\Gamma = 4\pi U a\sin\alpha$", fontsize=8)
```

```
    fi, x, V, gam = _lastra(alfa, True, 4000)
    cp = 1 - V**2
    dorso = np.sin(fi) > 0
    axs[1].plot(x[dorso], cp[dorso], color=BLU, label="dorso")
    axs[1].plot(x[~dorso], cp[~dorso], color=ARANCIO, label="ventre")
    axs[1].set_ylim(1.5, -8); axs[1].set_xlabel("$x$"); axs[1].set_ylabel("$C_p$")
    axs[1].legend(fontsize=7)
    salva(fig, "es_07")
    c = 4.0                                  # corda della lastra (da -2a a 2a)
    print("  alfa   C_L=2pi sin(a)   C_N (integrale di C_p)   2pi sin(a)cos(a)   C_S=2pi sin^2(a)"
          "   C_N cos(a) + C_S sin(a)")
    for ag in (2.0, 5.0, 10.0):
        al = np.radians(ag)
        # C_N = (1/c) int (C_p,ventre - C_p,dorso) dx, con x = 2 cos(fi), dx = -2 sin(fi) dfi
        fi = np.linspace(0, np.pi, 200001)[1:-1]
        cp = lambda f: 1 - _V_lastra(f, al)**2
        CN = np.trapezoid((cp(2 * np.pi - fi) - cp(fi)) * 2 * np.sin(fi), fi) / c
        CL = 2 * np.pi * np.sin(al)
        CS = 2 * np.pi * np.sin(al)**2
        print(f"  {ag:4.1f}     {CL:.5f}            {CN:.5f}            {2*np.pi*np.sin(al)*np.cos(al):.5f}"
              f"           {CS:.5f}              {CN*np.cos(al) + CS*np.sin(al):.5f}")
    print("la pressione sulle due facce da' solo la forza normale C_N = 2 pi sin(a) cos(a);")
    print("la portanza 2 pi sin(a) richiede anche la suzione C_S concentrata nel bordo d'attacco")


def _V_lastra(fi, alfa):
    """Velocita' sulla lastra (z = zeta + 1/zeta, a = 1) con circolazione di Kutta."""
    zeta = np.exp(1j * fi)
    gam = 4 * np.pi * np.sin(alfa)
    dw = (np.exp(-1j * alfa) - np.exp(1j * alfa) / zeta**2) + 1j * gam / (2 * np.pi * zeta)
    return np.abs(dw / (1 - 1 / zeta**2))


# ============================================================ esercizio 8
def es_08():
    """SOR contro Liebmann sul problema del Programma 2.5."""
    titolo(8, "SOR e Liebmann a confronto")
    import prog2_5_vortice_rettangolo as p25
    import prog2_5b_convergenza as p25b
    x, y = p25.griglia(); M, N = len(x), len(y)
    zeta = np.zeros((M, N)); zeta[16, 12] = p25.ZETA0
    psi_star = p25b.soluzione_diretta(zeta, p25.H)
    interni = [(i, j) for i in range(1, M - 1) for j in range(1, N - 1)]
    print("  metodo          omega    iterazioni (somma|delta| <= 0,001)   errore vero max")
    for nome, w in (("Liebmann", 1.0), ("SOR, formula", omega_ottimo(M - 1, N - 1)),
                    ("SOR", 1.5), ("SOR", 1.9)):
        psi = np.zeros((M, N)); psi[1:-1, 1:-1] = 0.5
        it, err = 0, 1.0
        while err > 0.001 and it < 2000:
            it += 1
            err = sor_passo(psi, zeta, p25.H, interni, w)
```

```
        print(f"  {nome:14s}  {w:5.3f}        {it:4d}                                  "
              f"{np.max(np.abs(psi-psi_star)):.2e}")


# ============================================================ esercizio 9
def es_09():
    """Due vortici concentrati nel rettangolo."""
    titolo(9, "due vortici nel rettangolo")
    import prog2_5_vortice_rettangolo as p25
    import prog2_5b_convergenza as p25b
    x, y = p25.griglia(0.125); M, N = len(x), len(y); h = 0.125
    G = p25.ZETA0 * 0.25**2
    fig, axs = plt.subplots(1, 2, figsize=(6.8, 2.6))
    for ax, segno, nome in zip(axs, (-1, 1), ("segni opposti", "stesso segno")):
        zeta = np.zeros((M, N))
        for (xv, yv, s) in ((-1.0, 0.0, 1.0), (1.0, 0.0, segno)):
            zeta[int(round((xv + 3) / h)), int(round((yv + 2) / h))] = s * G / h**2
        psi = p25b.soluzione_diretta(zeta, h)
        if segno < 0:
            liv = np.concatenate([-np.array([1.5, 1.0, 0.5, 0.25, 0.1]), [0.1,
    0.25, 0.5, 1.0, 1.5]])
            ax.contour(x, y, psi.T, levels=np.sort(liv), colors=BLU,
    linewidths=0.6, linestyles="solid")
            ax.contour(x, y, psi.T, levels=[0.0], colors=ROSSO, linewidths=1.2)
        else:
            # separatrice a otto: livello di psi nel punto di sella (x = 0, y = 0)
            ps = psi[int(round(3 / h)), int(round(2 / h))]
            print(f"stesso segno: psi nel punto di sella = {ps:.4f}")
            liv = np.array([0.1, 0.3, 0.6, 1.0, 1.4, 2.0, 2.6])
            ax.contour(x, y, psi.T, levels=liv, colors=BLU, linewidths=0.6,
    linestyles="solid")
            ax.contour(x, y, psi.T, levels=[ps], colors=ROSSO, linewidths=1.2)
        ax.plot([-1, 1], [0, 0], "k+", ms=6)
        ax.plot([-3, 3, 3, -3, -3], [-2, -2, 2, 2, -2], "k", lw=1.2)
        ax.set_aspect("equal"); ax.set_title(nome, fontsize=8); ax.grid(False)
    salva(fig, "es_09")
    print("segni opposti: psi = 0 sul contorno e lungo l'asse x = 0, che separa i
    due vortici")


# ============================================================ esercizio 10
def es_10():
    """Canale con un cilindro a sezione rettangolare (un quarto del dominio)."""
    titolo(10, "cilindro rettangolare in un canale")
    h = 0.25
    x = np.arange(-6.0, 0.0 + 1e-9, h); y = np.arange(0.0, 4.0 + 1e-9, h)
    M, N = len(x), len(y)
    solido = (x[:, None] >= -2.0 - 1e-9) & (y[None, :] <= 1.0 + 1e-9)
    psi = np.zeros((M, N))
    psi[0, :] = y / 4.0                          # AB: corrente uniforme in ingresso
    psi[:, -1] = 1.0                             # BC: parete del canale
    interni = [(i, j) for i in range(1, M) for j in range(1, N - 1) if not
    solido[i, j]]
    for i, j in interni:
        psi[i, j] = y[j] / 4.0
    w = omega_ottimo(M - 1, N - 1)
    it, err = 0, 1.0
    while err > 1e-6 and it < 5000:
        it += 1; err = 0.0
        for i, j in interni:
```

```
            if i == M - 1:                    # CD: dpsi/dx = 0 (nodo fittizio)
                gs = 0.25 * (2 * psi[i - 1, j] + psi[i, j - 1] + psi[i, j + 1])
            else:
                gs = 0.25 * (psi[i - 1, j] + psi[i + 1, j] + psi[i, j - 1] + psi[i,
    j + 1])
            nuovo = (1 - w) * psi[i, j] + w * gs
            err = max(err, abs(nuovo - psi[i, j])); psi[i, j] = nuovo
    print(f"SOR con omega = {w:.4f}: {it} iterazioni (max|delta| < 1e-6)")
    # velocita' sulla faccia superiore DE (y = 1) e sulla faccia frontale EF (x =
    -2)
    j1 = int(round(1.0 / h)); i2 = int(round(4.0 / h))
    u_top = (-3 * psi[i2:, j1] + 4 * psi[i2:, j1 + 1] - psi[i2:, j1 + 2]) / (2 * h)
    # psi e' adimensionale con U H (H = 4): u/U = H dpsi/dy
    print("velocita' u/U sulla faccia superiore (y = 1):")
    for xx, uu in zip(x[i2:], 4 * u_top):
        print(f"   x = {xx:5.2f}   u/U = {uu:.4f}")
    print(f"velocita' media u/U nella sezione x = 0, 1 < y < 4 (continuita': 4/3 =
    1,333): "
          f"{4*(psi[-1, -1]-psi[-1, j1])/3:.4f}")
    print(f"u/U in x = 0 sull'asse di simmetria, appena sopra il corpo (y = 1,25): "
          f"{4*(psi[-1, j1+2]-psi[-1, j1])/(2*h):.4f};  sotto la parete (y = 3,75):
    "
          f"{4*(psi[-1, -1]-psi[-1, -3])/(2*h):.4f}")
    fig, ax = plt.subplots(figsize=(6.4, 4.4))
    P = psi.copy(); P[solido] = 0.0
    xf = np.concatenate([x, -x[::-1][1:]])
    Pf = np.concatenate([P, P[::-1][1:]], axis=0)
    yf = np.concatenate([-y[::-1], y[1:]])
    Pf = np.concatenate([-Pf[:, ::-1], Pf[:, 1:]], axis=1)
    ax.contour(xf, yf, Pf.T, levels=np.linspace(-0.9, 0.9, 19), colors=BLU,
               linewidths=0.7, linestyles="solid")
    ax.fill([-2, 2, 2, -2], [-1, -1, 1, 1], color="#d5d8dc", zorder=3)
    ax.plot([-2, 2, 2, -2, -2], [-1, -1, 1, 1, -1], "k", lw=1, zorder=4)
    ax.plot([-6, 6], [4, 4], "k", lw=2); ax.plot([-6, 6], [-4, -4], "k", lw=2)
    ax.set_aspect("equal"); ax.grid(False); ax.set_xlabel("$x$");
    ax.set_ylabel("$y$")
    salva(fig, "es_10")


# ============================================================== esercizio 11
def es_11():
    """Flusso assialsimmetrico in un tubo con diaframmi forati ripetuti."""
    titolo(11, "tubo con diaframmi (flusso assialsimmetrico)")
    h = 0.1
    z = np.arange(0.0, 2.0 + 1e-9, h); r = np.arange(0.0, 1.0 + 1e-9, h)
    M, N = len(z), len(r)
    psi = np.zeros((M, N))
    psi[:, -1] = 1.0                                  # BC: parete del tubo
    diaframma = r >= 0.5 - 1e-9                       # AB: diaframma in z = 0
    psi[0, diaframma] = 1.0
    interni = [(i, j) for i in range(M) for j in range(1, N - 1)
               if not (i == 0 and diaframma[j])]
    for i, j in interni:
        psi[i, j] = (r[j] / 1.0)**2
    w = 1.8
    it, err = 0, 1.0
    while err > 1e-7 and it < 20000:
        it += 1; err = 0.0
        for i, j in interni:
```

```
            sx = psi[i + 1, j] if i == 0 else psi[i - 1, j]       # nodi fittizi in z = 0
            dx_ = psi[i - 1, j] if i == M - 1 else psi[i + 1, j]  # ... e in z = 2
            gs = 0.25 * (sx + dx_ + (1 + h / (2 * r[j])) * psi[i, j - 1]
                         + (1 - h / (2 * r[j])) * psi[i, j + 1])
            nuovo = (1 - w) * psi[i, j] + w * gs
            err = max(err, abs(nuovo - psi[i, j])); psi[i, j] = nuovo
    print(f"SOR (omega = {w}): {it} iterazioni")
    # velocita' assiale sull'asse u_z = (1/r) dpsi/dr ~ 2 psi/r^2 per r -> 0
    uz_asse = 2 * psi[:, 1] / r[1]**2
    print(f"velocita' assiale sull'asse: nel foro (z = 0) {uz_asse[0]:.3f}, "
          f"a meta' cella (z = 2) {uz_asse[-1]:.3f};  media nel tubo pieno 2 psi_parete/R^2 = 2")
    fig, ax = plt.subplots(figsize=(6.4, 3.4))
    zz = np.concatenate([z - 2, z[1:]]); zz = np.concatenate([zz, zz[1:] + 4])
    P = np.concatenate([psi[::-1], psi[1:]], axis=0); P = np.concatenate([P, P[1:]], axis=0)
    rr = np.concatenate([-r[::-1], r[1:]])
    PP = np.concatenate([P[:, ::-1], P[:, 1:]], axis=1)
    ax.contour(zz, rr, PP.T, levels=np.linspace(0.1, 0.9, 9), colors=BLU,
               linewidths=0.7, linestyles="solid")
    for zd in (0.0, 4.0):
        ax.plot([zd, zd], [0.5, 1.0], "k", lw=2.5); ax.plot([zd, zd], [-0.5, -1.0], "k", lw=2.5)
    ax.plot([-2, 6], [1, 1], "k", lw=2); ax.plot([-2, 6], [-1, -1], "k", lw=2)
    ax.axhline(0, color=GRIGIO, lw=0.6, ls="-.")
    ax.set_aspect("equal"); ax.set_xlabel("$z$"); ax.set_ylabel("$r$");
    ax.grid(False)
    salva(fig, "es_11")


# ============================================================ esercizio 12
def es_12():
    """Progetto: tubo d'urto linearizzato (leapfrog per la densita')."""
    titolo(12, "tubo d'urto linearizzato")
    h, Co = 0.01, 1.0
    x = np.arange(0.0, 1.0 + 1e-9, h); m = len(x)
    def integra(n, f, aperto_a_destra=False):
        R = np.zeros((n + 1, m)); R[0] = f
        # partenza: rho^1 = (f_{i-1} + f_{i+1})/2 (g = 0); estremi chiusi: nodi fittizi
        R[1, 1:-1] = 0.5 * (f[:-2] + f[2:])
        R[1, 0] = f[1]
        R[1, -1] = 0.0 if aperto_a_destra else f[-2]
        for j in range(1, n):
            R[j + 1, 1:-1] = R[j, :-2] + R[j, 2:] - R[j - 1, 1:-1]
            R[j + 1, 0] = 2 * R[j, 1] - R[j - 1, 0]           # chiuso: drho/dx = 0
            R[j + 1, -1] = 0.0 if aperto_a_destra else 2 * R[j, -2] - R[j - 1, -1]
        return R
    # (a) tubo chiuso alle due estremita', rho' = 1 a destra del diaframma
    R = integra(200, np.where(x >= 0.5 - 1e-12, 1.0, 0.0))
    # (b) estremo destro aperto: il gas a destra ha la densita' esterna (rho' = 0)
    Ra = integra(200, np.where(x < 0.5 - 1e-12, 1.0, 0.0), aperto_a_destra=True)
    fig, axs = plt.subplots(5, 2, figsize=(6.6, 6.0), sharex=True, sharey=True)
    for k, n in enumerate((0, 25, 45, 70, 100)):
        for c, (RR, nome) in enumerate(((R, "(a) estremi chiusi"), (Ra, "(b) estremo destro aperto"))):
            ax = axs[k, c]
            ax.plot(x, RR[n], color=BLU if c == 0 else VERDE, lw=1.1)
```

```
            ax.set_ylim(-0.6, 1.6); ax.set_yticks([0, 1]); ax.set_xticks([0, 0.5, 1])
            ax.text(0.98, 0.75, fr"$t={n}\tau$", transform=ax.transAxes, ha="right", fontsize=7)
            if k == 0:
                ax.set_title(nome, fontsize=8)
    for ax in axs[-1]:
        ax.set_xlabel("$x$ [m]")
    fig.subplots_adjust(hspace=0.1, wspace=0.08)
    salva(fig, "es_12")
    print("(a) estremi chiusi: rho' in x = 0,25 e x = 0,75;   (b) estremo aperto: idem")
    for n in (0, 20, 40, 60, 80, 100, 150, 200):
        print(f"   t = {n:3d} tau:  (a) {R[n, 25]:5.2f}  {R[n, 75]:5.2f}     (b) {Ra[n, 25]:5.2f}  {Ra[n, 75]:5.2f}")


# ============================================================ esercizio 13
def es_13():
    """Riflessione di onde di ampiezza finita agli estremi del tubo."""
    titolo(13, "onde di ampiezza finita: riflessione")
    import prog2_9_onda_finita as p29
    x = p29.H * np.arange(p29.M)
    u0 = p29.u_iniziale()
    us = np.zeros_like(u0); us[50:] = u0[:p29.M - 50]          # spostata di 1 m
    U1, A1 = p29.cir(40, u0, verso=-1)                         # onda verso sinistra
    U2, A2 = p29.cir(40, us, verso=+1)                         # onda verso destra, estremo aperto
    fig, axs = plt.subplots(5, 2, figsize=(6.6, 6.2), sharex=True, sharey=True)
    for k, n in enumerate((0, 10, 20, 30, 40)):
        axs[k, 0].plot(x, U1[n], color=BLU, lw=1.1)
        axs[k, 1].plot(x, U2[n], color=VERDE, lw=1.1)
        for ax in axs[k]:
            ax.axhline(0, color=GRIGIO, lw=0.4); ax.set_ylim(-200, 200)
            ax.set_yticks([-150, 0, 150]); ax.set_xticks([0, 1, 2])
            ax.text(0.98, 0.75, fr"$t={n}\tau$", transform=ax.transAxes, ha="right", fontsize=7)
    axs[0, 0].set_title("onda verso sinistra (estremo chiuso)", fontsize=8)
    axs[0, 1].set_title("onda verso destra (estremo aperto)", fontsize=8)
    for ax in axs[-1]:
        ax.set_xlabel("$x$ [m]")
    fig.subplots_adjust(hspace=0.1, wspace=0.08)
    salva(fig, "es_13")
    g = p29.GAMMA
    print("estremo chiuso (x = 0): u = 0 sempre; velocita' del suono e pressione (isentropica)")
    for n in (0, 10, 20, 30, 40):
        print(f"   t = {n:2d} tau:  a(0) = {A1[n, 0]:6.1f} m/s,  p/p0 = {(A1[n, 0]/340)**(2*g/(g-1)):.3f}")
    print("estremo aperto (x = 2 m): a = a0 sempre; velocita' del gas")
    for n in (0, 10, 20, 30, 40):
        print(f"   t = {n:2d} tau:  u(2) = {U2[n, -1]:7.1f} m/s")
    print(f"   velocita' minima nell'onda incidente: {us.min():.1f} m/s")


# ============================================================ esercizio 14
def es_14():
    """Verifica simbolica della soluzione di Benton e Platzman (SymPy)."""
    titolo(14, "soluzione esatta dell'equazione di Burgers")
    import sympy as sp
```

```
583     x, t = sp.symbols("x t", real=True)
584     u = -2 * sp.sinh(x) / (sp.cosh(x) - sp.exp(-t))
585     residuo = sp.simplify(sp.diff(u, t) + u * sp.diff(u, x) - sp.diff(u, x, 2))
586     print("u_t + u u_x - u_xx =", residuo)
587     print("limite per t -> infinito:", sp.limit(u, t, sp.oo))
588     print("pendenza in x = 0:  du/dx =", sp.simplify(sp.diff(u, x).subs(x, 0)))
589     print("limite - (-2 tanh x) =", sp.simplify((sp.limit(u, t, sp.oo) + 2 *
        sp.tanh(x)).rewrite(sp.exp)))
590     # trasformazione di Cole-Hopf: u = -2 phi_x / phi con phi soluzione di phi_t =
        phi_xx
591     phi = sp.exp(t) * sp.cosh(x) - 1
592     print("phi = e^t cosh x - 1:  phi_t - phi_xx =",
593           sp.simplify(sp.diff(phi, t) - sp.diff(phi, x, 2)))
594     print("-2 phi_x/phi - u =", sp.simplify(-2 * sp.diff(phi, x) / phi - u))
595
596
597 # ============================================================ esercizi 15 e 16
598 def es_15():
599     """Burgers con MacCormack e condizioni di Neumann."""
600     titolo(15, "Burgers: MacCormack esplicito")
601     import prog2_12_burgers as pb
602     istanti = (0.2, 0.5, 1.0, 2.0)
603     x, u, dt, sal = pb.maccormack_burgers(2.0, istanti=istanti)
604     dx = x[1] - x[0]
605     u0 = pb.esatta(x, 0.0)
606     print(f"dx = {dx:.4f}; |u|max(t=0) = {np.max(np.abs(u0)):.2f}")
607     print(f"limiti: convettivo dx/|u|max = {dx/np.max(np.abs(u0)):.5f}, viscoso
        dx^2/2 = {dx**2/2:.5f};"
608           f" dt usato = {dt:.5f}")
609     fig, ax = plt.subplots(figsize=(5.4, 3.6))
610     xf = np.linspace(-6, 6, 600)
611     print("   t      max|u - u_esatta|   (|x| < 5)")
612     for (tr, (ts, us)), col in zip(sorted(sal.items()), (VIOLA, VERDE, ARANCIO,
        BLU)):
613         ax.plot(xf, pb.esatta(xf, ts), color=GRIGIO, lw=1.8)
614         ax.plot(x, us, "o", ms=2.2, color=col, label=f"$t={ts:.2f}$")
615         e = np.abs(us - pb.esatta(x, ts))
616         print(f"  {ts:5.3f}     {e.max():.3e}        {e[np.abs(x) < 5].max():.3e}")
617     ax.set_xlim(-6, 6); ax.set_ylim(-4, 4); ax.set_xlabel("$x$");
        ax.set_ylabel("$u$")
618     ax.legend(fontsize=7)
619     salva(fig, "es_15")
620     print("raffinamento della griglia (t = 1): errore massimo totale, per |x| < 2,
        e in x = 6")
621     for nx in (100, 200, 400):
622         xg, _, _, sg = pb.maccormack_burgers(1.0, nx=nx, istanti=(1.0,))
623         ts, us = sg[1.0]
624         e = us - pb.esatta(xg, ts)
625         print(f"   nx = {nx:3d}:  {np.max(np.abs(e)):.3e}
        {np.max(np.abs(e[np.abs(xg) < 2])):.3e}"
626               f"   {e[-1]:+.3e}")
627
628
629 def es_16():
630     """Burgers con Beam-Warming e condizioni di Dirichlet."""
631     titolo(16, "Burgers: Beam-Warming implicito")
632     import prog2_12_burgers as pb
633     istanti = (0.2, 0.5, 1.0, 2.0)
634     x = pb.griglia(); dx = x[1] - x[0]
635     umax = np.max(np.abs(pb.esatta(x, 0.0)))
```

```
    print("   dt      Co iniziale   mu_e     errore massimo a t = 0,2   0,5   1
    2")
    risultati = {}
    for dt, mu in ((0.002, 0.0), (0.01, 0.0), (0.01, 0.1), (0.05, 0.0)):
        _, _, sal = pb.beam_warming_burgers(2.0, dt, mu_e=mu, istanti=istanti)
        errs = [np.max(np.abs(us - pb.esatta(x, ts))) for _, (ts, us) in
    sorted(sal.items())]
        risultati[(dt, mu)] = sal
        print(f"  {dt:5.3f}     {umax*dt/dx:6.2f}      {mu:4.2f}      " +
              "  ".join(f"{e:.2e}" for e in errs))
    _, _, dtm, salm = pb.maccormack_burgers(2.0, istanti=istanti)
    errs = [np.max(np.abs(us - pb.esatta(x, ts))) for _, (ts, us) in
    sorted(salm.items())]
    print(f"  MacCormack (dt = {dtm:.4f}):                 " + "  ".join(f"{e:.2e}"
    for e in errs))
    fig, axs = plt.subplots(1, 2, figsize=(6.8, 2.9), sharey=True)
    xf = np.linspace(-6, 6, 600)
    for ax, chiave, tit in zip(axs, ((0.01, 0.0), (0.05, 0.0)), ("dt = 0,01", "dt =
    0,05")):
        for (tr, (ts, us)), col in zip(sorted(risultati[chiave].items()), (VIOLA,
    VERDE, ARANCIO, BLU)):
            ax.plot(xf, pb.esatta(xf, ts), color=GRIGIO, lw=1.6)
            ax.plot(x, us, "o", ms=2.0, color=col, label=f"$t={ts:.2f}$")
        ax.set_title(tit.replace(",", "{,}") if False else tit, fontsize=8)
        ax.set_xlim(-6, 6); ax.set_ylim(-4, 4); ax.set_xlabel("$x$")
    axs[0].set_ylabel("$u$"); axs[0].legend(fontsize=7)
    salva(fig, "es_16")


# ============================================================ esercizio 17
def es_17():
    """Tubo elastico: frequenza, ampiezza della forzante e attrito turbolento."""
    titolo(17, "tubo elastico: studio parametrico")
    import prog2_13_tubo_elastico as pt
    casi = (("riferimento", dict()),
            ("frequenza doppia", dict(periodo=0.35)),
            ("frequenza dimezzata", dict(periodo=1.4)),
            ("ampiezza doppia", dict(p_min=40.0, p_max=100.0)),
            ("ampiezza dimezzata", dict(p_min=62.5, p_max=77.5)),
            ("attrito turbolento", dict(tipo_attrito="turbolento")))
    fig, axs = plt.subplots(len(casi), 2, figsize=(6.6, 8.4), sharex=True)
    print("  caso                  p(z=20) [mmHg]      V(z=20) [cm/s]     portata
    media [cm^3/s]")
    for k, (nome, par) in enumerate(casi):
        T = par.get("periodo", pt.PERIODO)
        z, Hs, n, CV, _ = pt.simula(t_fin=6 * T, z_reg=(20.0,), **par)
        t = Hs[:, 0]; ult = t >= t[-1] - T
        P, V, Q = Hs[ult, 1], Hs[ult, 2], Hs[ult, 3]
        q = np.trapezoid(Q, t[ult]) / T
        print(f"  {nome:20s}  [{P.min():5.1f}, {P.max():5.1f}]     [{V.min():6.1f},
    {V.max():6.1f}]"
              f"         {q:.3f}")
        fase = (t[ult] - t[ult][0]) / T
        axs[k, 0].plot(fase, P, color=ROSSO, lw=1.0)
        axs[k, 1].plot(fase, V, color=BLU, lw=1.0)
        axs[k, 1].axhline(0, color=GRIGIO, lw=0.4)
        axs[k, 0].text(0.98, 0.8, nome, transform=axs[k, 0].transAxes, ha="right",
    fontsize=7)
        axs[k, 0].set_ylim(0, 210); axs[k, 1].set_ylim(-40, 320)
    axs[0, 0].set_title("$p(z=20)$ [mmHg]", fontsize=8)
```

```
    axs[0, 1].set_title("$V(z=20)$ [cm/s]", fontsize=8)
    for ax in axs[-1]:
        ax.set_xlabel("$t/T$ (ultimo periodo)")
    fig.subplots_adjust(hspace=0.12, wspace=0.18)
    salva(fig, "es_17")


if __name__ == "__main__":
    tutti = {k: v for k, v in globals().items() if k.startswith("es_")}
    scelti = [f"es_{int(a):02d}" for a in sys.argv[1:]] or sorted(tutti)
    for nome in scelti:
        tutti[nome]()
```

Listato B.1: `esercizi_svolti.py`: codice di tutti gli esercizi.

Bibliografia

[1] S. Biringen, C.-Y. Chow, *An Introduction to Computational Fluid Mechanics by Example*, Wiley, Hoboken, 2011. Capitolo 2: *Inviscid Fluid Flows*, pp. 50–144. Programmi MATLAB su `www.wiley.com/go/biringen`.

[2] C.-Y. Chow, *An Introduction to Computational Fluid Mechanics*, Wiley, New York, 1979 (edizione originale, con programmi FORTRAN).

[3] S. C. Chapra, R. P. Canale, *Numerical Methods for Engineers*, 7ª ed., McGraw-Hill, New York, 2015. Cap. 11: matrici a banda, algoritmo di Thomas, metodo di Gauss–Seidel e rilassamento; Cap. 27 e 29: problemi ai limiti ed equazioni ellittiche.

[4] J. H. Ferziger, M. Perić, *Computational Methods for Fluid Dynamics*, 3ª ed., Springer, Berlin, 2002. Cap. 3: differenze finite; Cap. 5: soluzione dei sistemi lineari e metodi iterativi; Cap. 6: problemi non stazionari.

[5] J. C. Tannehill, D. A. Anderson, R. H. Pletcher, *Computational Fluid Mechanics and Heat Transfer*, 2ª ed., Taylor & Francis, Washington, 1997. Cap. 4: equazioni modello (convezione, Burgers, Laplace), schemi di MacCormack e di Beam–Warming, dissipazione artificiale.

[6] R. J. LeVeque, *Finite Volume Methods for Hyperbolic Problems*, Cambridge University Press, 2002. Cap. 3: caratteristiche e invarianti di Riemann; Cap. 11: equazione di Burgers, formazione dell'urto, condizione di Rankine–Hugoniot.

[7] P. J. Roache, *Computational Fluid Dynamics*, Hermosa, Albuquerque, 1972 (Cap. III: metodi iterativi e sovrarilassamento).

[8] D. M. Young, *Iterative methods for solving partial difference equations of elliptic type*, Transactions of the American Mathematical Society **76** (1954) 92–111.

[9] B. Andersson, R. Andersson, L. Håkansson, M. Mortensen, R. Sudiyo, B. van Wachem, *Computational Fluid Dynamics for Engineers*, Cambridge University Press, 2012 (Cap. 3: aspetti numerici, rilassamento e multigrid).

[10] G. Caramia, E. Distaso, *A Practical Approach to Computational Fluid Dynamics Using OpenFOAM®*, Springer, Cham, 2025 (numero di Courant e condizione CFL nei codici industriali).

[11] A. M. Kuethe, C.-Y. Chow, *Foundations of Aerodynamics: Bases of Aerodynamic Design*, 5ª ed., Wiley, New York, 1998. Cap. 2–4: flussi potenziali, flussi elementari, trasformazione di Joukowski; Cap. 5: metodo dei pannelli di vortice.

[12] J. Katz, A. Plotkin, *Low-Speed Aerodynamics*, 2ª ed., Cambridge University Press, 2001 (metodi a pannelli).

[13] J. L. Hess, A. M. O. Smith, *Calculation of potential flow about arbitrary bodies*, Progress in Aerospace Sciences **8** (1967) 1–138.

[14] I. H. Abbott, A. E. von Doenhoff, *Theory of Wing Sections*, Dover, New York, 1959 (prima edizione McGraw-Hill, 1949).

[15] Th. von Kármán, *Berechnung der Druckverteilung an Luftschiffkörpern*, Abhandlungen aus dem Aerodynamischen Institut an der Technischen Hochschule Aachen **6** (1927) 3–17; trad. inglese *Calculation of pressure distribution on airship hulls*, NACA TM 574, 1930.

[16] H. Lamb, *Hydrodynamics*, 6ª ed., Cambridge University Press, 1932 (flusso attorno agli ellissoidi di rivoluzione, cap. V).

[17] L. M. Milne-Thomson, *Theoretical Hydrodynamics*, 5ª ed., Macmillan, London, 1968.

[18] R. V. Churchill, *Introduction to Complex Variables and Applications*, McGraw-Hill, New York, 1948.

[19] L. Prandtl, *Über Flüssigkeitsbewegung bei sehr kleiner Reibung*, Verhandlungen des III. Internationalen Mathematiker-Kongresses, Heidelberg, 1904, pp. 484–491.

[20] H. W. Liepmann, A. Roshko, *Elements of Gasdynamics*, Wiley, New York, 1957 (Cap. 3: onde in una dimensione, onde semplici, formazione dell'urto).

[21] G. B. Whitham, *Linear and Nonlinear Waves*, Wiley, New York, 1974 (equazione di Burgers e trasformazione di Cole–Hopf).

[22] R. Courant, E. Isaacson, M. Rees, *On the solution of nonlinear hyperbolic differential equations by finite differences*, Communications on Pure and Applied Mathematics **5** (1952) 243–255.

[23] R. W. MacCormack, *The effect of viscosity in hypervelocity impact cratering*, AIAA Paper 69-354, 1969.

[24] R. M. Beam, R. F. Warming, *An implicit finite-difference algorithm for hyperbolic systems in conservation-law form*, Journal of Computational Physics **22** (1976) 87–110.

[25] E. R. Benton, G. W. Platzman, *A table of solutions of the one-dimensional Burgers equation*, Quarterly of Applied Mathematics **30** (1972) 195–212.

[26] M. Anliker, R. L. Rockwell, E. Ogden, *Nonlinear analysis of flow pulses and shock waves in arteries*, Zeitschrift für angewandte Mathematik und Physik **22** (1971) 217–246.

[27] J. A. Rumberger, R. M. Nerem, *A method-of-characteristics calculation of coronary blood flow*, Journal of Fluid Mechanics **82** (1977) 429–448.